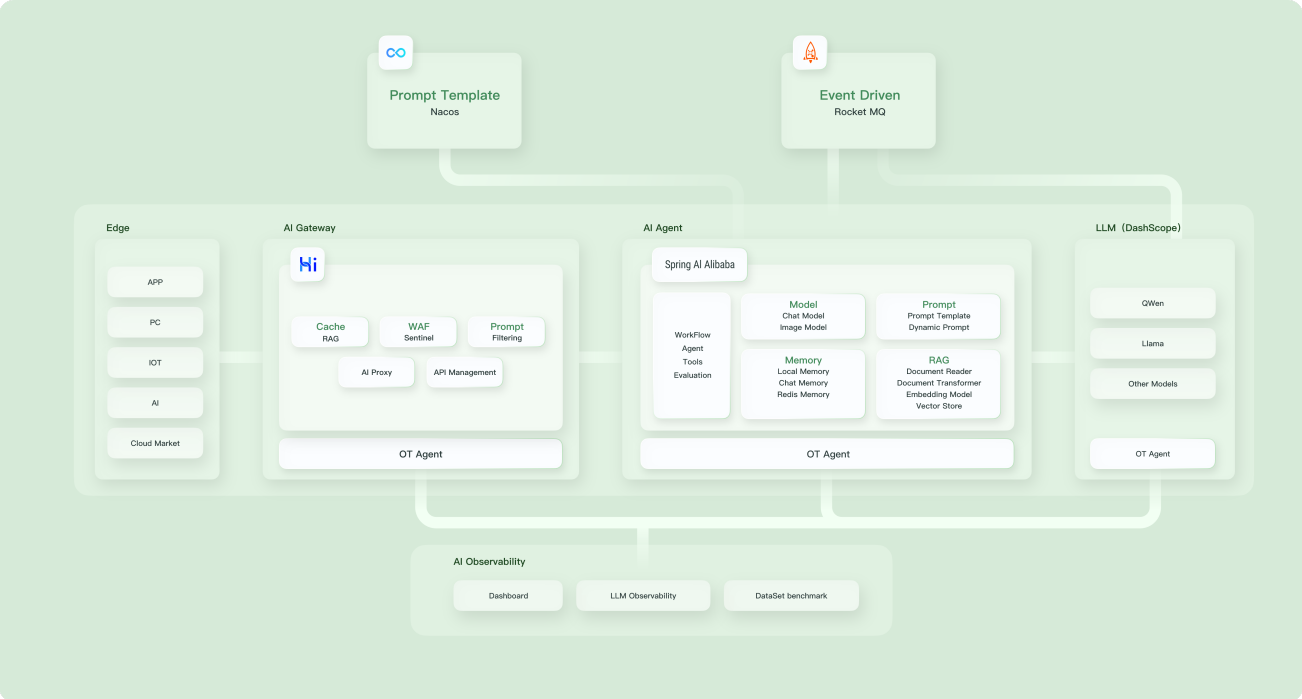
# Spring AI Alibaba

参考资料：

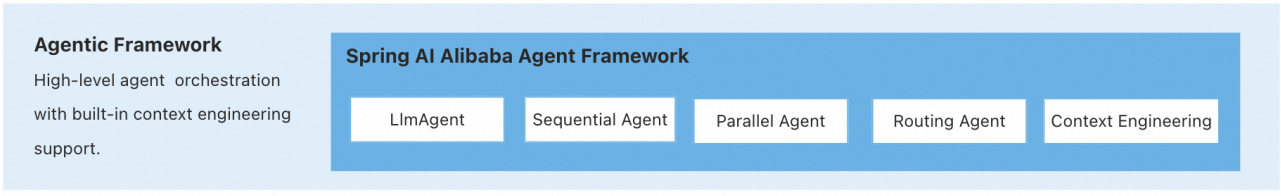
- https://java2ai.com/docs/overview/
- https://docs.spring.io/spring-ai/reference/

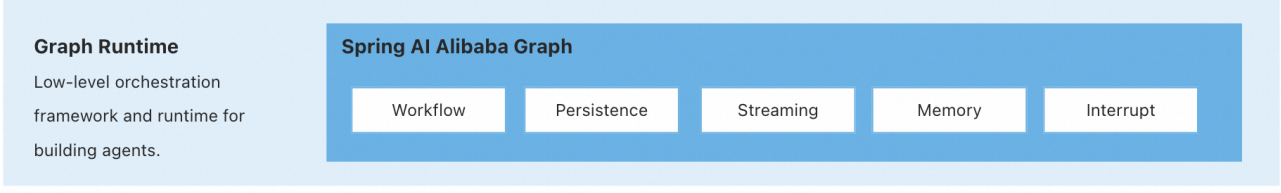
Java AI应用架构体系

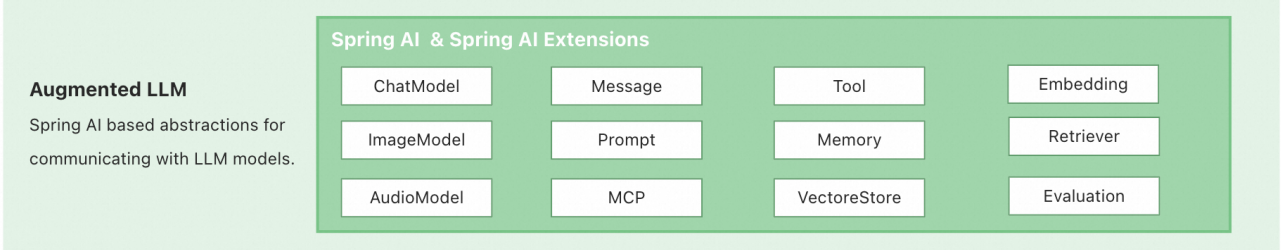



## 概述

Spring AI Alibaba 是构建 Agent 智能体应用最简单的方式，只需不到 10 行代码就可以构建您的智能体应用。



**注意！** Spring Ai Alibaba 深度集成 Spring AI 生态，它是一个专为多智能体系统和工作流编排设计的项目。因此，Spring Ai Alibaba 中的 `ReactAgent` 实际上运行在 Graph Runtime 之上，其设计目标主要是完成工作流和多智能体编排功能。

如果您正在寻找更高级的 ReactAgent 范式来构建模型驱动的智能体，请参阅我们近期发布的 **AgentScope** 项目。Spring Ai Alibaba 的ReactAgent部分将继续得到维护，并持续接收问题修复和关键安全补丁，我们将更专注于Spring Ai 集成与多智能体协同。

Spring AI Alibaba 项目从架构上包含如下三层：

- **Agent Framework**：是一个以 ReactAgent 设计理念为核心的 Agent 开发框架，使开发者能够构建具备自动上下文工程和人机交互等核心能力的Agent。
- **Graph**：是一个低级别的工作流和多代理协调框架，能够帮助开发者实现复杂的应用程序编排，它具备丰富的预置节点和简化的图状态定义，Graph 是 Agent Framework 的底层运行时基座。
- **Augmented LLM**：以 Spring AI 框架底层原子抽象为基础，为构建大型语言模型（LLM）应用提供基础抽象，例如模型（Model）、工具（Tool）、多模态组件（MCP）、消息（Message）、向量存储（Vector Store）等。

## 设计原则

我们推荐您使用 Agent Framework 内置的 `ReactAgent` 抽象快速构建 Agent 应用。

如果您需要组合 Multi-agent，Agent Framework 还预置了如 `SequentialAgent（顺序代理）`、`ParallelAgent（并行代理）`、`RoutingAgent（路由代理）` 和 `LoopAgent（循环代理）` 等基础多智能体工作流模式。

对于一些开发场景而言，直接使用 `Graph API` 也是可行的，它能为应用开发提供更灵活的编排、更直接的状态控制，适用于需要超高可靠性、大量自定义逻辑、需要精确控制延迟时的场景。

## 安装

```xml
<dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-agent-framework</artifactId>
    <version>1.1.2.2</version>
</dependency>
<dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
    <version>1.1.2.3</version>
</dependency>
```

## 创建Agent

```java
import com.alibaba.cloud.ai.dashscope.api.DashScopeApi;
import com.alibaba.cloud.ai.dashscope.chat.DashScopeChatModel;
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.alibaba.cloud.ai.graph.CompileConfig;

import org.springframework.ai.chat.model.ChatModel;

public class AgentExample {
```

```
9
10      public static void main(String[] args) throws Exception {
11          // 创建模型实例
12          DashScopeApi dashScopeApi = DashScopeApi.builder()
13              .apiKey(System.getenv("AI_DASHSCOPE_API_KEY"))
14              .build();
15          ChatModel chatModel = DashScopeChatModel.builder()
16              .dashScopeApi(dashScopeApi)
17              .build();
18
19          // 创建 Agent
20          ReactAgent agent = ReactAgent.builder()
21              .name("weather_agent")
22              .model(chatModel)
23              .instruction("You are a helpful weather forecast assistant.")
24              .build();
25
26          // 运行 Agent
27          agent.call("what is the weather in Hangzhou?");
28      }
29  }
```

## 核心功能

- **ReactAgent**：构建具有推理和行动能力的智能代理，遵循 ReAct（推理 + 行动）范式，用于迭代解决问题。
- **多代理编排**：使用内置模式（包括 `SequentialAgent`、ParallelAgent、`LlmRoutingAgent`和 `LoopAgent`）组合多个代理，以执行复杂的任务。
- **上下文工程**：内置快速工程、上下文管理和对话流控制的最佳实践，以提高代理的可靠性和性能。
- **人机协同**：将人工反馈和审批步骤无缝集成到代理工作流程中，从而实现关键工具和操作的监督执行。
- **流式传输支持**：代理响应的实时流式传输
- **错误处理**：强大的错误恢复和重试机制
- **基于图的工作流**：基于图的工作流运行时和 API，用于条件路由、嵌套图、并行执行和状态管理。可将工作流导出为 PlantUML 和 Mermaid 格式。
- **A2A 支持**：通过 Nacos 集成支持代理间通信，实现跨服务的分布式代理协调和协作。
- **丰富的模型、工具和 MCP 支持**：利用 Spring AI 的核心概念，支持多种 LLM 提供程序（DashScope、OpenAI 等）、工具调用和模型上下文协议 (MCP)。

# 快速开始

跟随快速开始，学习如何开发一个具备完整功能的 ReactAgent 智能体。

## 环境要求

- JDK 17+
- Maven 3.8+
- 选择你的 LLM 提供商并获取 API-KEY（如阿里云百炼的 DashScope）

## 添加依赖

在你的项目中添加以下 Maven 依赖：

```
<dependencies>
  <!-- Spring AI Alibaba Agent Framework -->
  <dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-agent-framework</artifactId>
    <version>1.1.2.2</version>
  </dependency>

  <!-- DashScope ChatModel 支持 -->
  <dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
    <version>1.1.2.3</version>
  </dependency>
</dependencies>
```

## 配置API Key

在使用之前，需要配置你的 API Key。推荐通过环境变量设置：

```
export AI_DASHSCOPE_API_KEY=your_api_key_here
```

你也可以在应用配置文件中设置（不推荐在生产环境中硬编码）：

```
# application.yml
spring:
  ai:
    dashscope:
      api-key: ${AI_DASHSCOPE_API_KEY}
```

**获取 API Key**：

- DashScope（阿里云百炼）：访问 https://bailian.console.aliyun.com/?apiKey=1&tab=api#/api
- 其他模型提供商请 参考文档

## 构建一个基础Agent

首先创建一个简单的 agent，它可以回答问题并调用工具。该agent将使用DashScope大模型、一个天气查询工具、以及简单的系统提示词。

### 1 创建查询对象

```java
@Data
public class WeatherRequest {
    @JsonProperty("city")
    @JsonPropertyDescription("城市的名称，比如：成都")
    private String city;
}
```

## 2 创建查询工具

```java
public class WeatherTool implements
    BiFunction<WeatherRequest, ToolContext, String> {
    @Override
    public String apply(WeatherRequest request,
                        ToolContext toolContext) {
        return "在" + request.getCity() + "的天气总是阳光明媚!";
    }
}
```

## 3 编写测试案例

```java
@Value("${spring.ai.dashscope.api-key}")
private String apiKey;

@Test
void testBaseAgent() throws GraphRunnerException {
    // 创建模型实例
    DashScopeApi dashScopeApi = DashScopeApi.builder()
        .apiKey(apiKey)
        .build();
    ChatModel chatModel = DashScopeChatModel.builder()
        .dashScopeApi(dashScopeApi)
        .build();
    // 创建工具
    ToolCallback weatherTool = FunctionToolCallback
        .builder("get_weather", new WeatherTool())
        .description("Get weather for a given city")
        .inputType(WeatherRequest.class)
        .build();
    // 创建Agent
    ReactAgent agent = ReactAgent.builder()
        .name("weather_agent")
        .model(chatModel)
        .tools(weatherTool)
        .saver(new MemorySaver())
        .systemPrompt("You are a helpful assistant.")
        .build();

    // 运行Agent
    AssistantMessage response = agent.call("成都的天气怎么样呢");
    System.out.println(response.getText());
}
```

# 构建一个真实Agent

接下来，构建一个实用的天气预报agent，演示关键的生产概念：

1. **详细的System Prom** - 获得更好的 agent 行为
2. **创建工具** - 与外部数据集成
3. **模型配置** - 获得一致的响应
4. **结构化输出** - 获得可预测的结果
5. **对话记忆** - 实现类似聊天的交互
6. **创建和运行 agent** - 创建一个功能完整的agent

让我们逐步完成每个步骤：

## 1. 定义系统提示

系统提示定义了agent的角色和行为。保持具体和可操作，这个在具体配置类中植入，这里先做说明

```
String SYSTEM_PROMPT = """
    你是一位精通天气预报的专家，并且非常喜欢幽默地用“谐音梗”或“双关语”来回答问题。

    你拥有以下两个工具的使用权限：
    - get_weather_for_location：用于获取特定地点的当前天气。
    - get_user_location：用于获取用户当前所在的地理位置。

    当用户向你查询天气时，请务必确认具体的地点：
    - 如果用户的提问中包含了明确的城市（例如“北京天气怎么样”），请直接调用 get_weather_for_location 工具。
    - 如果从用户的提问中可以看出他们指的是自己当前所在的任意地方（例如“我这里下雨了吗”、“外面冷不冷”），请先调用 get_user_location 工具来获取他们的位置。
    """;
```

## 2. 创建工具

工具让模型能够通过调用你定义的函数与外部系统交互。工具可以依赖运行时上下文，也可以与 agent 的记忆交互。

首先编写一个工具创建接口，方便管理工具创建与执行逻辑

```
import org.springframework.ai.chat.model.ToolContext;
import org.springframework.ai.tool.ToolCallback;
import java.util.function.BiFunction;
/**
 * <p>
 * 描述：Tool构建接口
 * </p>
 * <p>版权： &copy;01星球</p>
 * <p>地址： 01星球总部</p>
 * @author 阿伟学长
 * @version 1.0.0
 */
public interface Tool<I, O> extends BiFunction<I, ToolContext, O> {
    /**
     * 获取ToolCallback
     * @return ToolCallback 工具回调对象
```

```
     */
    ToolCallback toolCallback();
}
```

接下来实现两个工具，注意下面的`getUserLocation`工具如何使用运行时上下文（通过`ToolContext`）：

```
/**
 * <p>
 * 描述：天气查询工具，用于获取指定城市的天气信息
 * </p>
 * <p>版权：&copy;01星球</p>
 * <p>地址：01星球总部</p>
 * @author 阿伟学长
 * @version 1.0.0
 */
public class WeatherForLocationTool implements
    Tool<WeatherForLocationTool.weatherRequest, String> {
    @Override
    public ToolCallback toolCallback() {
        return FunctionToolCallback
            .builder("get_weather_for_location", this)
            .description("Get weather for a given city")
            .inputType(weatherRequest.class)
            .build();
    }
    @Override
    public String apply(weatherRequest weatherRequest,
                        ToolContext toolContext) {
        return weatherRequest.city() + "的天气是否总是阳光明媚!";
    }
    @JsonClassDescription("天气工具请求对象")
    public record weatherRequest(
        @JsonProperty("city")
        @JsonPropertyDescription("城市的名称，比如：成都")
        String city
    ) {}
}

/**
 * <p>
 * 描述：根据用户ID获取用户位置信息
 * </p>
 * <p>版权：&copy;01星球</p>
 * <p>地址：01星球总部</p>
 * @author 阿伟学长
 * @version 1.0.0
 */
public class UserLocationTool implements
    Tool<UserLocationTool.userLocationRequest, String> {
    @Override
    public ToolCallback toolCallback() {
        return FunctionToolCallback
            .builder("get_user_location", this)
            .description("Retrieve user location based on user ID")
            .inputType(userLocationRequest.class)
```

```
            .build();
    }
    @Override
    public String apply(userLocationRequest userLocationRequest,
                        ToolContext toolContext) {
        // 从上下文中获取用户信息
        String userId = "";
        if (toolContext != null && toolContext.getContext() != null) {
            RunnableConfig runnableConfig =
                (RunnableConfig) toolContext.getContext()
                .get(AGENT_CONFIG_CONTEXT_KEY);
            Optional<Object> userIdObjOptional =
                runnableConfig.metadata("user_id");
            if (userIdObjOptional.isPresent()) {
                userId = (String) userIdObjOptional.get();
            }
        }
        if (userId.isEmpty()) {
            userId = "1";
        }
        // 模拟用户位置信息查询
        return "1".equals(userId) ? "北京" : "成都";
    }

    @JsonClassDescription("用户位置查询工具请求对象")
    public record userLocationRequest(
        @JsonProperty("query")
        @JsonPropertyDescription("User query")
        String query
    ) {}
}
```

**提示**: 工具应该有良好的文档：它们的名称、描述和参数名称都会成为模型提示的一部分。

## 3. 配置模型

为你的用例配置合适的大语言模型参数：

```yaml
spring:
  ai:
    dashscope:
      # 配置dashscope的apikey
      api-key: sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
      chat:
        options:
          # 模型类型，如：qwen-max、qwen-turbo、qwen-plus
          model: qwen-plus
          # 核采样温度，取值范围[0.0,1.0]，
          # 越小越模型越严谨、保守、确定，
          # 越大模型越有创造力、越活泼、越容易天马行空
          temperature: 0.5
          # 限制大模型单次回答你时最多能吐出多少个字
          #（1000个Token在中文里大约相当于600-800个汉字）
          max_tokens: 1000
```

根据不同的模型选择，增加依赖

```
<!— DashScope—>
<dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
    <version>1.1.2.3</version>
</dependency>

<!— OpenAI—>
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-model-openai</artifactId>
    <version>1.1.2</version>
</dependency>
```

## 4. 定义响应格式

如果你需要agent响应匹配特定的模式，可以定义结构化响应格式。

```
/**
 * <p>
 * 描述：响应格式
 * </p>
 * <p>版权：&copy;01星球</p>
 * <p>地址：01星球总部</p>
 * @author 阿伟学长
 * @version 1.0.0
 */
@Data
public class ResponseFormat {
    /**
     * 一个双关语响应（始终必需）
     */
    private String punnyResponse;
    /**
     * 如果可用的话，关于天气的任何有趣信息
     */
    private String weatherConditions;
}
```

## 5. 添加记忆

为你的 agent 添加记忆以维持跨交互的状态。这允许 agent 记住之前的对话和上下文，在多次调用之间，使用同一个 `threadId` 即可加载之前的对话记录。

```
ReactAgent agent = ReactAgent.builder()
    .name("weather_agent")
    // ...
    .saver(new MemorySaver())
    .build();
```

在调用的时候，

```
RunnableConfig runnableConfig =
    RunnableConfig.builder().threadId(threadId).build();
// 第一次调用
AssistantMessage response1 =
    agent.call("what is the weather in San Francisco today.", runnableConfig);
// 第二次调用
AssistantMessage response2 =
    agent.call("How about the weather tomorrow", runnableConfig);
```

**注意**: 在生产环境中，使用持久化的 CheckPointer 将数据保存到数据库。更多详情参见 内存管理 文档。

## 6. 创建和运行Agent

编写配置类将Agent注入到容器

```
/**
 * <p>
 * 描述：  Agent配置类
 * </p>
 * <p>版权：&copy;01星球</p>
 * <p>地址：01星球总部</p>
 * @author 阿伟学长
 * @version 1.0.0
 */
@Configuration
public class AgentConfiguration {
    String SYSTEM_PROMPT = """
            你是一位精通天气预报的专家，并且非常喜欢幽默地用“谐音梗”或“双关语”来回答问题。

            你拥有以下两个工具的使用权限：
            - get_weather_for_location：用于获取特定地点的当前天气。
            - get_user_location：用于获取用户当前所在的地理位置。

            当用户向你查询天气时，请务必确认具体的地点：
            - 如果用户的提问中包含了明确的城市（例如“北京天气怎么样”），请直接调用 get_weather_for_location 工具。
            - 如果从用户的提问中可以看出他们指的是自己当前所在的任意地方（例如“我这里下雨了吗”、“外面冷不冷”），请先调用 get_user_location 工具来获取他们的位置。
            """;
    private final ChatModel chatModel;
    public AgentConfiguration(ChatModel chatModel) {
        this.chatModel = chatModel;
    }
    @Bean
    public ReactAgent reactAgent() {
        return ReactAgent.builder()
                .name("weather_pun_agent")
                .model(chatModel)
                .systemPrompt(SYSTEM_PROMPT)
                .tools(
                        new WeatherForLocationTool().toolCallback(),
                        new UserLocationTool().toolCallback()
                )
                .outputType(ResponseFormat.class)
```

```
                .saver(new MemorySaver())
                .build();
    }
}
```

测试调用

```
@Resource
private ReactAgent agent;

@Test
void testAgent() throws GraphRunnerException {
    RunnableConfig runnableConfig = RunnableConfig.builder()
        .threadId("1").addMetadata("user_id", "1").build();
    AssistantMessage response = agent.call("成都的天气怎么样呢", runnableConfig);
    System.out.println(response.getText());
    System.out.println("═══════════════════════════════");
    response = agent.call("我这儿明天的天气怎样呢", runnableConfig);
    System.out.println(response.getText());
}
```

# 更多示例代码

更多示例代码请查看仓库：

- https://github.com/alibaba/spring-ai-alibaba/tree/main/examples
- https://github.com/spring-ai-alibaba/examples
- https://github.com/spring-projects/spring-ai-examples

# 开启高级功能

## 定义输出格式

除了使用 `outputType`，你还可以使用 `outputSchema` 来定义自定义的 JSON 格式：

```
String customSchema = """
请按照以下JSON格式输出：
{
    "title": "标题",
    "content": "内容",
    "weather": "实时天气",
    "style": "风格"
}
""";

ReactAgent agent = ReactAgent.builder()
    .name("schema_agent")
    .model(chatModel)
    .saver(new MemorySaver())
    .outputSchema(customSchema)
    .build();

AssistantMessage message = agent.call("帮我写一首关于春天的诗歌。");
System.out.println(message.getText());
```

## 获取完整状态

如果需要访问完整的 agent 状态（不仅仅是最后的消息），可以使用 `invoke` 方法：

```
import com.alibaba.cloud.ai.graph.OverAllState;
import java.util.Optional;

Optional<OverAllState> result = agent.invoke("帮我写一首诗。");

if (result.isPresent()) {
    OverAllState state = result.get();
    // 访问消息历史
    List<Message> messages = state.value("messages", new ArrayList<>());
    // 访问其他状态信息
    System.out.println(state);
}
```

## 配置最大迭代次数

为防止无限循环，可以使用 `ModelCallLimitHook` 来限制模型调用次数：

```
import com.alibaba.cloud.ai.graph.agent.hook.modelcalllimit.ModelCallLimitHook;

ModelCallLimitHook hook = ModelCallLimitHook.builder()
    .runLimit(5)  // 限制最多调用 5 次
    .exitBehavior(ModelCallLimitHook.ExitBehavior.ERROR)  // 超出限制时抛出异常
    .build();

ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .hooks(hook)
    .saver(new MemorySaver())
    .build();
```

## 扩展功能

ReactAgent支持通过 Hooks 扩展功能，例如人机协同、工具注入等：

```
import com.alibaba.cloud.ai.graph.agent.hook.Hook;
import com.alibaba.cloud.ai.graph.agent.hook.hip.HumanInTheLoopHook;

// 创建hook
Hook humanInTheLoopHook = HumanInTheLoopHook.builder()
    .approvalOn("getWeatherTool", ToolConfig.builder()
                .description("Please confirm tool execution.")
                .build())
    .build();

ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
```

```
    .hooks(humanInTheLoopHook)
    .saver(new MemorySaver())
    .build();
```

# AgentsFramework基础

主要包含：Agents、Models、Tools、Hooks、Memory、Messages、Skills、Structured Output 等。

***提示：基础部分的官方提供的完整示例代码点击 链接 查看***

## Models

### 概述

ChatModel API 为开发者提供了将 AI 驱动的聊天补全功能集成到应用程序中的能力。它利用预训练的语言模型（如 GPT），根据用户的自然语言输入生成类似人类的响应。

该 API 通常通过向 AI 模型发送提示或部分对话来工作，然后模型根据其训练数据和对自然语言模式的理解生成对话的完成或延续。完成的响应随后返回给应用程序，应用程序可以将其呈现给用户或用于进一步处理。

`Spring AI ChatModel API` 被设计为一个简单且可移植的接口，用于与各种 AI 模型交互，允许开发者在不同模型之间切换时只需最少的代码更改。

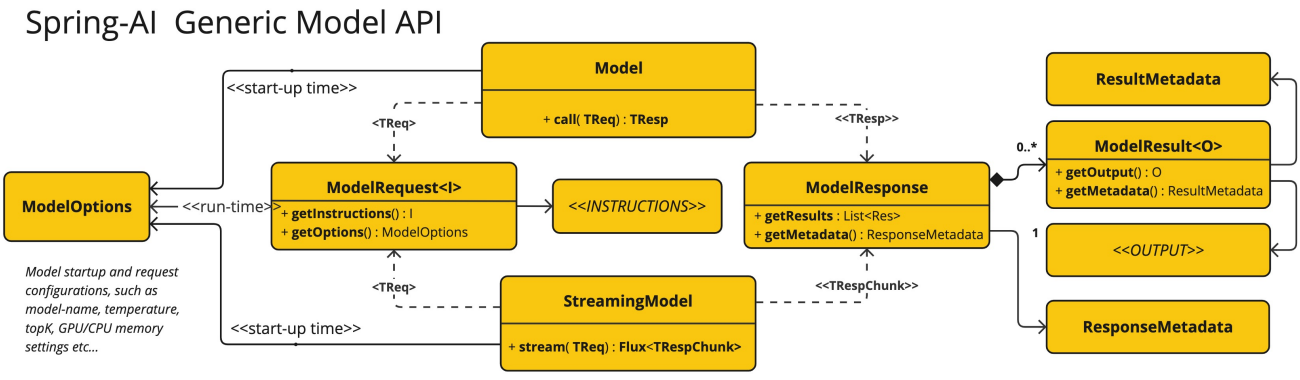
借助 `Prompt`（用于输入封装）和 `ChatResponse`（用于输出处理）等配套类，ChatModel API 统一了与 AI 模型的通信。它管理请求准备和响应解析的复杂性，提供直接且简化的 API 交互。

Spring AI ChatModel API 构建在 Spring AI `Generic Model API` 之上，提供 Chat 特定的抽象和实现。这允许轻松集成和在不同 AI 服务之间切换，同时为客户端应用程序维护一致的 API。

### Generic Model API

为了为所有 AI Models 提供基础，Spring AI 创建了 Generic Model API。这使得通过遵循通用模式轻松地为 Spring AI 贡献新的 AI Model 支持。

以下类图展示了 Generic Model API 的架构：



## Model

`Model` 接口提供了调用 AI model 的通用 API。它旨在通过抽象发送请求和接收响应的过程来处理与各种类型的 AI model 的交互。该接口使用 Java 泛型来容纳不同类型的请求和响应，增强了不同 AI model 实现的灵活性和适应性。

Model 接口定义

```
public interface Model
    <TReq extends ModelRequest<?>, TRes extends ModelResponse<?>> {
  /**
   * Executes a method call to the AI model.
   * @param request the request object to be sent to the AI model
   * @return the response from the AI model
   */
  TRes call(TReq request);
}
```

## StreamingModel

`StreamingModel` 接口提供了调用具有流式响应的 AI model 的通用 API。它抽象了发送请求和接收流式响应的过程。

StreamingModel 接口定义

```
public interface StreamingModel
    <TReq extends ModelRequest<?>, TResChunk extends ModelResponse<?>> {
  /**
   * Executes a method call to the AI model.
   * @param request the request object to be sent to the AI model
   * @return the streaming response from the AI model
   */
  Flux<TResChunk> stream(TReq request);
}
```

## ModelRequest

`ModelRequest` 接口表示对 AI model 的请求。它封装了与 AI model 交互所需的信息，包括指令或输入（泛型类型 `T`）和附加的 model 选项。

ModelRequest 接口定义

```java
public interface ModelRequest<T> {
  /**
   * Retrieves the instructions or input required by the AI model.
   * @return the instructions or input required by the AI model
   */
  T getInstructions(); // required input
  /**
   * Retrieves the customizable options for AI model interactions.
   * @return the customizable options for AI model interactions
   */
  ModelOptions getOptions();
}
```

## ModelOptions

`ModelOptions` 接口表示 AI model 交互的可自定义选项。此标记接口允许指定各种设置和参数，这些设置和参数可以影响 AI model 的行为和输出。

ModelOptions 接口定义

```java
public interface ModelOptions {}
```

## ModelResponse

`ModelResponse` 接口表示从 AI model 接收的响应。此接口提供访问 AI model 生成的主要结果或结果列表以及响应元数据的方法。

ModelResponse 接口定义

```java
public interface ModelResponse
    <T extends ModelResult<?>> {
  /**
   * Retrieves the result of the AI model.
   * @return the result generated by the AI model
   */
  T getResult();
  /**
   * Retrieves the list of generated outputs by the AI model.
   * @return the list of generated outputs
   */
  List<T> getResults();
  /**
   * Retrieves the response metadata associated with the AI model's response.
   * @return the response metadata
   */
  ResponseMetadata getMetadata();
}
```

### ModelResult

`ModelResult` 接口提供访问 AI model 的主要输出和与此结果相关的元数据的方法。它旨在提供一种标准化和全面的方式来处理和解释 AI model 生成的输出。

ModelResult 接口定义

```
public interface ModelResult<T> {
  /**
   * Retrieves the output generated by the AI model.
   * @return the output generated by the AI model
   */
  T getOutput();
  /**
   * Retrieves the metadata associated with the result of an AI model.
   * @return the metadata associated with the result
   */
  ResultMetadata getMetadata();
}
```

## Chat Model API

Chat Model API 构建在 Generic Model API 之上，提供 Chat 特定的抽象和实现。本节提供 Spring AI ChatModel API 接口和相关类的指南。

### ChatModel

`ChatModel` 接口扩展了 `Model<Prompt, ChatResponse>` 和 `StreamingChatModel`，提供了与 AI 模型交互的统一接口。

ChatModel 接口定义

```
public interface ChatModel extends
    Model<Prompt, ChatResponse>, StreamingChatModel {
  default String call(String message) { ... }
  @Override
  ChatResponse call(Prompt prompt);
}
```

带有 `String` 参数的 `call()` 方法简化了初始使用，避免了更复杂的 `Prompt` 和 `ChatResponse` 类的复杂性。在实际应用中，更常见的是使用接受 `Prompt` 实例并返回 `ChatResponse` 的 `call()` 方法。

### StreamingChatModel

`StreamingChatModel` 接口扩展了 `StreamingModel<Prompt, ChatResponse>`，提供了流式响应的能力。

StreamingChatModel 接口定义

```
public interface StreamingChatModel extends
    StreamingModel<Prompt, ChatResponse> {
  default Flux<String> stream(String message) { ... }
  @Override
  Flux<ChatResponse> stream(Prompt prompt);
}
```

`stream()` 方法接受 `String` 或 `Prompt` 参数，类似于 `ChatModel`，但使用响应式 Flux API 流式传输响应。

## Prompt

Prompt 是一个封装了 Message 对象列表和可选模型请求选项的 `ModelRequest`。以下是 `Prompt` 类的简化版本，排除了构造函数和其他实用方法：

Prompt 类定义

```
public class Prompt implements
    ModelRequest<List<Message>> {
  private final List<Message> messages;
  private ChatOptions modelOptions;
  @Override
  public ChatOptions getOptions() { ... }
  @Override
  public List<Message> getInstructions() { ... }
  // 构造函数和实用方法省略
}
```

## Message

`Message` 接口封装了 `Prompt` 文本内容、元数据属性集合以及称为 `MessageType` 的分类。

接口定义如下：

Message 接口定义

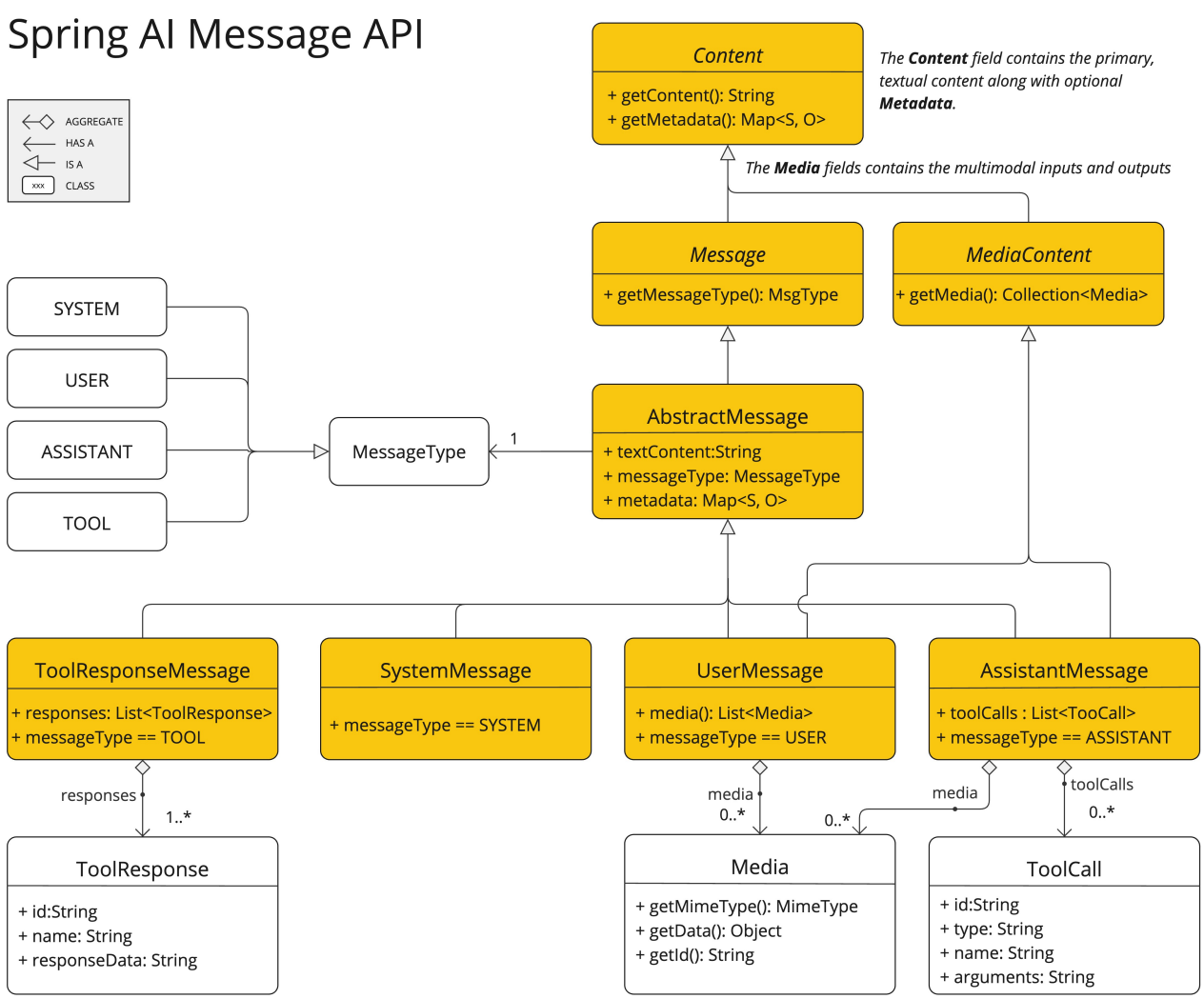
```
public interface Content {
  String getText();
  Map<String, Object> getMetadata();
}

public interface Message extends Content {
  MessageType getMessageType();
}
```

多模态消息类型还实现了 `MediaContent` 接口，提供 `Media` 内容对象列表。

MediaContent 接口定义

```
public interface MediaContent extends Content {
  Collection<Media> getMedia();
}
```

`Message` 接口有多种实现，对应于 AI 模型可以处理的消息类别：




主要消息类型包括：

- **UserMessage**: 用户消息
- **SystemMessage**: 系统消息
- **AssistantMessage**: 助手消息
- **FunctionMessage**: 函数消息
- **ToolResponseMessage**: 工具响应消息

聊天完成端点根据对话角色区分消息类别，由 `MessageType` 有效映射。

例如，OpenAI 识别不同对话角色的消息类别，如 `system`、`user`、`function` 或 `assistant`。

虽然术语 `MessageType` 可能暗示特定的消息格式，但在此上下文中，它有效地指定了消息在对话中扮演的角色。

对于不使用特定角色的 AI 模型，`UserMessage` 实现充当标准类别，通常表示用户生成的查询或指令。

## ChatOptions

表示可以传递给 AI 模型的选项。`ChatOptions` 类是 `ModelOptions` 的子类，用于定义可以传递给 AI 模型的少数可移植选项。`ChatOptions` 类定义如下：

ChatOptions接口定义

```java
public interface ChatOptions extends ModelOptions {
  String getModel();
  Float getFrequencyPenalty();
  Integer getMaxTokens();
  Float getPresencePenalty();
  List<String> getStopSequences();
  Float getTemperature();
  Integer getTopK();
  Float getTopP();
  ChatOptions copy();
}
```

**常用选项说明**:

- **model**: 要使用的模型 ID
- **frequencyPenalty**: 频率惩罚（-2.0 到 2.0），降低重复令牌的可能性
- **maxTokens**: 生成响应的最大令牌数
- **presencePenalty**: 存在惩罚（-2.0 到 2.0），鼓励谈论新主题
- **stopSequences**: 停止序列列表，遇到时停止生成
- **temperature**: 采样温度（0.0 到 2.0），控制随机性
- **topK**: Top-K 采样参数
- **topP**: Top-P（核采样）参数

此外，每个特定模型的 ChatModel/StreamingChatModel 实现都可以有自己的选项。例如，OpenAI Chat Completion 模型有自己的选项，如 `logitBias`、`seed` 和 `user`。

这是一个强大的功能，允许开发者在启动应用程序时使用特定于模型的选项，然后在运行时使用 `Prompt` 请求覆盖它们。

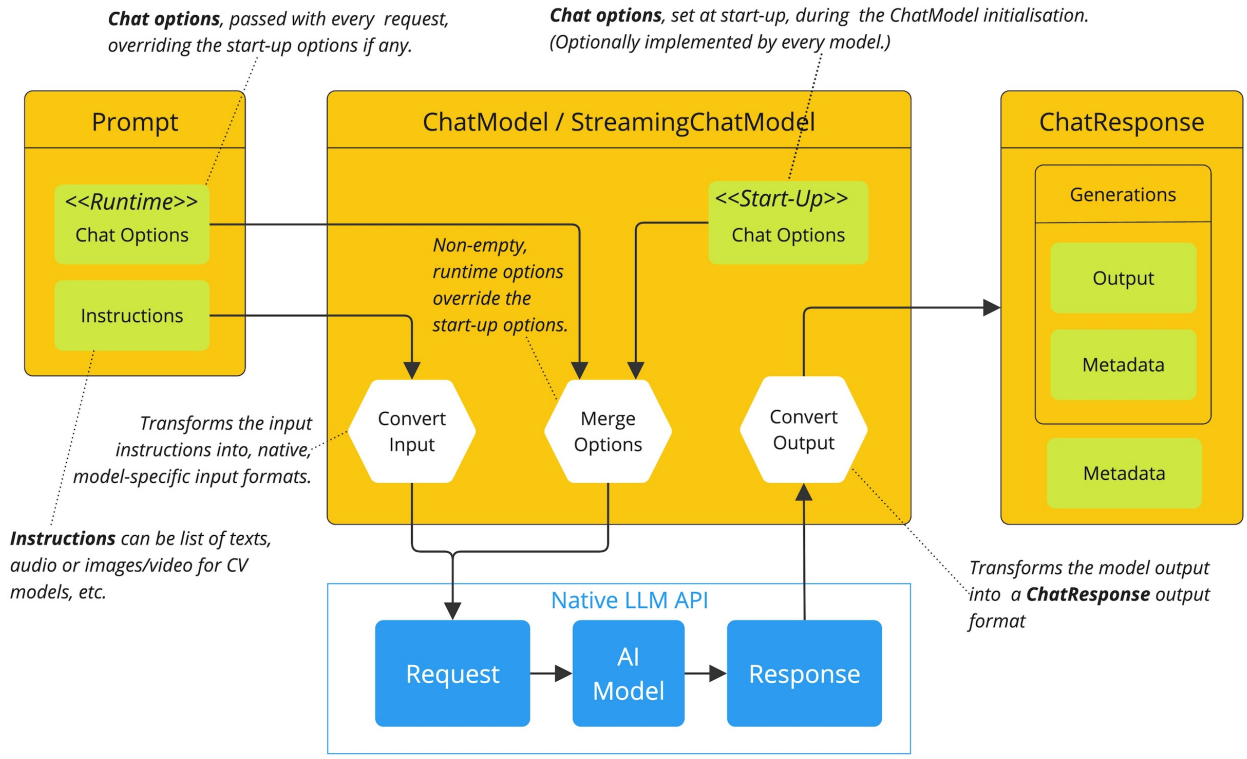
Spring AI 提供了一个复杂的系统来配置和使用 ChatModels。它允许在启动时设置默认配置，同时还提供了在每个请求基础上覆盖这些设置的灵活性。

**选项合并流程**:

1. **启动配置** - ChatModel/StreamingChatModel 使用"启动"ChatOptions 初始化。这些选项在 ChatModel 初始化期间设置，旨在提供默认配置。
2. **运行时配置** - 对于每个请求，Prompt 可以包含运行时 ChatOptions，这些可以覆盖启动选项。
3. **选项合并过程** - "合并选项"步骤结合启动和运行时选项。如果提供了运行时选项，它们优先于启动选项。
4. **输入处理** - "转换输入"步骤将输入指令转换为本地的、特定于模型的格式。
5. **输出处理** - "转换输出"步骤将模型的响应转换为标准化的 `ChatResponse` 格式。

启动和运行时选项的分离允许全局配置和特定于请求的调整。

以下流程图说明了 Spring AI 如何处理 Chat Models 的配置和执行，结合启动和运行时选项：




这种设计符合 Spring 的模块化和可互换性理念，使开发人员能够轻松地使用不同的 AI model 并根据需要调整参数，所有这些都在 Spring AI 框架提供的统一接口内。

## ChatResponse

`ChatResponse` 类的结构如下：

ChatResponse 类定义

```
public class ChatResponse implements
    ModelResponse<Generation> {
  private final ChatResponseMetadata chatResponseMetadata;
  private final List<Generation> generations;
  @Override
  public ChatResponseMetadata getMetadata() { ... }
  @Override
  public List<Generation> getResults() { ... }
  // 其他方法省略
}
```

ChatResponse 类保存 AI 模型的输出，每个 `Generation` 实例包含单个提示可能产生的多个输出之一。

`ChatResponse` 类还携带关于 AI 模型响应的 `ChatResponseMetadata` 元数据。

## Generation

最后，Generation 类从 `ModelResult` 扩展，表示模型输出（助手消息）和相关元数据：

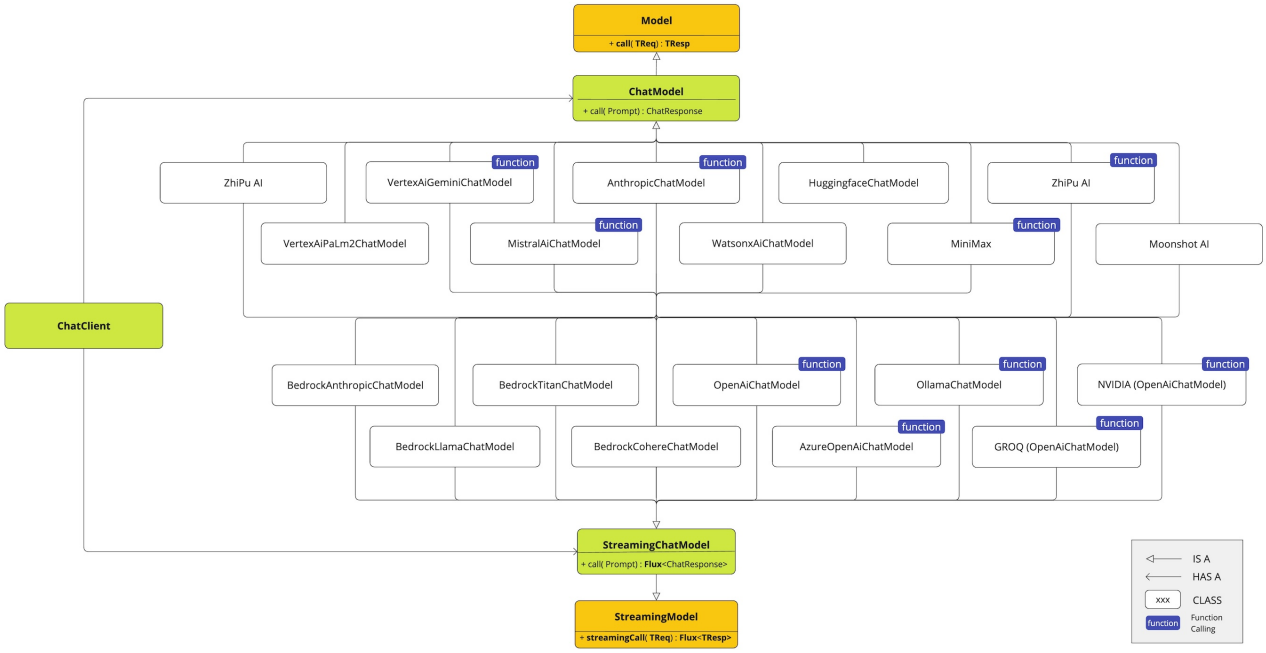
Generation 类定义

```java
public class Generation implements
    ModelResult<AssistantMessage> {
  private final AssistantMessage assistantMessage;
  private ChatGenerationMetadata chatGenerationMetadata;
  @Override
  public AssistantMessage getOutput() { ... }
  @Override
  public ChatGenerationMetadata getMetadata() { ... }
  // 其他方法省略
}
```

## 可用实现

Spring AI 提供了与多个 AI 服务提供商的集成，所有这些都通过统一的 `ChatModel` 和 `StreamingChatModel` 接口进行交互。这种设计允许轻松集成和在不同 AI 服务之间切换，同时为客户端应用程序维护一致的 API。

以下图表说明了统一的接口 `ChatModel` 和 `StreamingChatModel` 用于与来自不同提供商的各种 AI chat model 交互：



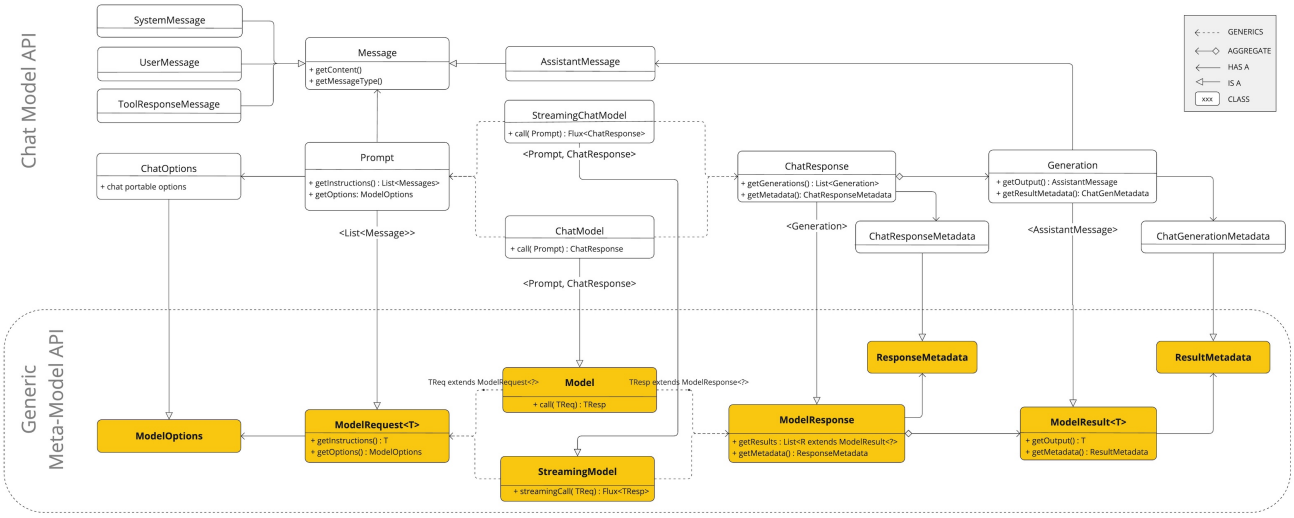

### 支持的模型提供商

- **OpenAI Chat Completion** (支持流式、多模态和函数调用)
- **Microsoft Azure OpenAI Chat Completion** (支持流式和函数调用)
- **Alibaba DashScope Chat Completion** (支持流式和函数调用)
- **Ollama Chat Completion** (支持流式、多模态和函数调用)
- **Hugging Face Chat Completion** (不支持流式)
- **Google Vertex AI Gemini Chat Completion** (支持流式、多模态和函数调用)
- **Amazon Bedrock**
- **Mistral AI Chat Completion** (支持流式和函数调用)
- **Anthropic Chat Completion** (支持流式和函数调用)

关于每个模型的具体用法与特性，请查看 Spring AI Alibaba 模型适配文档。

## API 架构

Spring AI Chat Model API 构建在 Spring AI `Generic Model API` 之上，提供 Chat 特定的抽象和实现。以下类图说明了 Spring AI Chat Model API 的主要类和接口：




# DashScopeChatModel

DashScope 是阿里云提供的大模型服务平台，提供通义千问等多个大语言模型。Spring AI Alibaba 提供了 DashScopeChatModel 的集成。

## 前置条件

在使用 DashScopeChatModel 之前，你需要：

1. 获取 DashScope API Key：访问 阿里云百炼
2. 设置环境变量：`export AI_DASHSCOPE_API_KEY=your_api_key`

## 添加依赖

```xml
<dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
    <version>1.1.2.3</version>
</dependency>
```

## 基础使用

**创建ChatModel**

创建 DashScopeChatModel

```java
import com.alibaba.cloud.ai.dashscope.api.DashScopeApi;
import com.alibaba.cloud.ai.dashscope.chat.DashScopeChatModel;
import org.springframework.ai.chat.model.ChatModel;

// 创建 DashScope API 实例
DashScopeApi dashScopeApi = DashScopeApi.builder()
  .apiKey(System.getenv("AI_DASHSCOPE_API_KEY"))
  .build();

// 创建 ChatModel
ChatModel chatModel = DashScopeChatModel.builder()
  .dashScopeApi(dashScopeApi)
  .build();
```

**简单调用**

简单调用示例

```java
// 使用字符串直接调用
String response = chatModel.call("介绍一下Spring框架");
System.out.println(response);
```

**使用Prompt**

使用Prompt调用示例

```java
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.model.ChatResponse;

// 创建 Prompt
Prompt prompt = new Prompt(new UserMessage("解释什么是微服务架构"));

// 调用并获取响应
ChatResponse chatResponse = chatModel.call(prompt);
String answer = response.getResult().getOutput().getText();
System.out.println(answer);
```

## 配置选项

**使用ChatOptions**

DashScopeChatOptions配置示例

```java
import com.alibaba.cloud.ai.dashscope.chat.DashScopeChatOptions;

// 创建配置选项
DashScopeChatOptions options = DashScopeChatOptions.builder()
    .model("qwen-plus")            // 模型名称
    .temperature(0.7)              // Temperature 参数
    .maxToken(2000)                // 最大令牌数
    .topP(0.9)                     // Top-P采样
```

```
    .build();

// 创建模型
ChatModel chatModel = DashScopeChatModel.builder()
    .dashScopeApi(dashScopeApi)
    .defaultOptions(options)
    .build();
```

**运行时覆盖选项**

运行时覆盖选项示例

```
// 创建带有特定选项的 Prompt
DashScopeChatOptions runtimeOptions = DashScopeChatOptions.builder()
    .temperature(0.3)  // 更低的温度，更确定的输出
    .maxToken(500)
    .build();
Prompt prompt = new Prompt(
    new UserMessage("用一句话总结Java的特点"),
    runtimeOptions
);

// 调用模型
ChatResponse response = chatModel.call(prompt);
System.out.println(response.getResult().getOutput().getText());
```

## 流式响应

流式响应示例

```
import reactor.core.publisher.Flux;

// 使用流式 API
Flux<ChatResponse> responseStream = chatModel.stream(
  new Prompt("详细解释Spring Boot的自动配置原理")
);

// 订阅并处理流式响应
responseStream.subscribe(
  chatResponse -> {
      String content = chatResponse.getResult()
          .getOutput()
          .getText();
      System.out.print(content);
  },
  error -> System.err.println("错误: " + error.getMessage()),
  () -> System.out.println("流式响应完成")
);
```

## 多轮对话

多轮对话示例

```
import org.springframework.ai.chat.messages.Message;
import org.springframework.ai.chat.messages.SystemMessage;
import org.springframework.ai.chat.messages.AssistantMessage;
import java.util.List;

// 创建对话历史
List<Message> messages = List.of(
    new SystemMessage("你是一个Java专家"),
    new UserMessage("什么是Spring Boot?"),
    new AssistantMessage("Spring Boot是 ..."),
    new UserMessage("它有什么优势?")
);
// 执行对话
Prompt prompt = new Prompt(messages);
ChatResponse response = chatModel.call(prompt);
// 打印对话结果
System.out.println(response.getResult().getOutput().getText());
```

提示关于消息类型：

- **System Message（系统消息）** - 告诉模型如何行为并为交互提供上下文
- **User Message（用户消息）** - 表示用户输入和与模型的交互
- **Assistant Message（助手消息）** - 模型生成的响应，包括文本内容、工具调用和元数据
- **Tool Response Message（工具响应消息）** - 表示工具调用的输出

消息相关的内容在Message章节细聊

## 支持的模型

DashScope 支持多个模型，包括：

- **qwen-turbo**: 通义千问超大规模语言模型，支持中文、英文等
- **qwen-plus**: 通义千问增强版
- **qwen-max**: 通义千问旗舰版
- **qwen-max-longcontext**: 支持长文本的通义千问

## 函数调用

DashScopeChatModel 支持函数调用（Function Calling），允许模型调用外部函数：

函数调用示例

```
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.function.FunctionToolCallback;

// 定义请求参数对象
record WeatherReq(String city) {}

// 定义函数工具
ToolCallback weatherFunction = FunctionToolCallback
```

```
    .builder("getWeather", (WeatherReq req) -> req.city() + "的实时天气预报")
    .description("获取指定城市的天气")
    .inputType(WeatherReq.class)
    .build();

// 使用函数
DashScopeChatOptions options = DashScopeChatOptions.builder()
    .toolCallbacks(List.of(weatherFunction))
    .build();
// 执行对话
Prompt prompt = new Prompt("成都的天气怎么样?", options);
ChatResponse response = chatModel.call(prompt);
System.out.println(response.getResult().getOutput().getText());
```

### 与ReactAgent集成

在 Spring AI Alibaba Agent Framework 中使用 DashScopeChatModel：

与 ReactAgent 集成示例

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;

// 创建 Agent
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .systemPrompt("你是一个有帮助的AI助手")
    .build();
// 调用 Agent
AssistantMessage response = agent.call("帮我分析1+1为什么等于2");
System.out.println(response.getText());
```

详细的 Agent 使用方法请参考 Agents 文档。

## DeepSeekChatModel

来自 DeepSeek 的各种 AI 语言模型。您可以与 DeepSeek 语言模型交互，并基于 DeepSeek 模型创建多语言对话助手。

官方参考：https://api-docs.deepseek.com/zh-cn/

### 前置条件

在使用 DeepSeekChatModel 之前，你需要：

1. 在 DeepSeek registration page 创建账户，并在 API Keys page 生成 token。
2. 设置环境变量：`export AI_DEEPSEEK_API_KEY=your-deepseek-api-key`

### 添加依赖

```xml
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-model-deepseek</artifactId>
    <version>1.1.2</version>
</dependency>
```

Spring AI 项目定义了一个名为 `spring.ai.deepseek.api-key` 的配置属性，您应将其设置为从 API Keys 页面获得的 `API Key` 值。

为了在处理敏感信息（如 API keys）时增强安全性，您可以使用 Spring Expression Language (SpEL) 来引用自定义环境变量：

```yaml
# In application.yml
spring:
  ai:
    deepseek:
      api-key: ${AI_DEEPSEEK_API_KEY}
```

您也可以在应用程序代码中以编程方式设置此配置：

```java
// Retrieve API key from a secure source or environment variable
String apiKey = System.getenv("AI_DEEPSEEK_API_KEY");
```

## 基础使用

### 创建ChatModel

```java
// 创建 DeepSeekApi 实例
DeepSeekApi api = DeepSeekApi.builder()
    .apiKey(System.getenv("AI_DEEPSEEK_API_KEY"))
    .build();
// 创建 ChatModel
ChatModel chatModel = DeepSeekChatModel.builder()
    .deepSeekApi(api)
    .build();
```

### 简单调用

```java
// 使用字符串直接调用
String response = chatModel.call("介绍一下你自己吧");
System.out.println(response);
```

### 使用Prompt

```java
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.model.ChatResponse;

// 创建 Prompt
Prompt prompt = new Prompt(new UserMessage("简单说说你能够做什么"));

// 调用并获取响应
ChatResponse chatResponse = chatModel.call(prompt);
String answer = response.getResult().getOutput().getText();
System.out.println(answer);
```

## 配置选项

### 使用ChatOptions

```java
import org.springframework.ai.deepseek.DeepSeekChatOptions;

// 创建配置选项
DeepSeekChatOptions options = DeepSeekChatOptions.builder()
    // 模型名称
    .model(DeepSeekApi.ChatModel.DEEPSEEK_CHAT.getValue())
    .temperature(0.7)              // Temperature 参数
    .maxTokens(2000)               // 最大令牌数
    .topP(0.9)                     // Top-P采样
    .build();

// 创建模型
ChatModel chatModel = DeepSeekChatModel.builder()
    .deepSeekApi(api)
    .defaultOptions(options)
    .build();
```

### 运行时覆盖选项

```java
// 调用模型
ChatResponse response = chatModel.call(
    new Prompt(
        "生成5位著名海盗的名字，再用诙谐幽默的风格，给他们5人串成一个小故事，适当加入表情包。",
        ChatOptions.builder()
        .model("deepseek-v4-pro")
        .temperature(1.0)
        .maxTokens(6000)
        .build()
    ));
// 输出模型响应结果
response.getResults().forEach(geo -> {
    System.out.println(geo.getOutput().getText());
});
```

## 流式响应

```
import reactor.core.publisher.Flux;

// 使用流式 API
Flux<ChatResponse> responseStream = chatModel.stream(
    new Prompt("详细解释水滴石穿是什么原理",
                ChatOptions.builder()
                .model("deepseek-v4-pro")
                .temperature(.5)
                .maxTokens(6000)
                .build())
);
// 打印标记数组
final boolean[] printTags = {true, true};
// 订阅并处理流式响应
responseStream.subscribe(
    chatResponse -> {
        // 获取输出消息
        Message output = chatResponse.getResult().getOutput();
        // 检查是否为Thinking消息
        if (output instanceof DeepSeekAssistantMessage dsMsg) {
            if (StringUtils.hasText(dsMsg.getReasoningContent())) {
                if (printTags[0]) {
                    System.out.println("[Thinking]");
                    printTags[0] = false;
                }
                System.out.print(dsMsg.getReasoningContent());
            }
        }
        // 普通模型响应（增量内容）
        if (StringUtils.hasText(output.getText())) {
            if (printTags[1]) {
                System.out.println("\n[Answer]");
                printTags[1] = false;
            }
            System.out.print(output.getText());
        }
    },
    error -> System.err.println("\n错误: " + error.getMessage()),
    () -> System.out.println("\n流式响应完成")
);
```

## 多轮对话

```
// 使用v4-pro模型，开启思考模式
ChatOptions promptOptions = ChatOptions.builder()
    .model("deepseek-v4-pro")
    .build();

// 存储消息列表
List<Message> messages = new ArrayList<>();

// 第一轮对话
messages.add(new UserMessage("9.11 与 9.8 谁大?"));
```

```
Prompt prompt = new Prompt(messages, promptOptions);
ChatResponse response = chatModel.call(prompt);
// 输出模型响应结果
DeepSeekAssistantMessage deepSeekAssistantMessage = (DeepSeekAssistantMessage)
response.getResult().getOutput();
String reasoningContent = deepSeekAssistantMessage.getReasoningContent();
System.out.println("\n[Thinking]\n" + reasoningContent);
String text = deepSeekAssistantMessage.getText();
System.out.println("\n[Answer]\n" + text);

// 第二轮对话
messages.add(AssistantMessage.builder().content(Objects.requireNonNull(text)).bui
ld());
messages.add(new UserMessage("单词 strawberry 中有几个字母R?"));
Prompt prompt2 = new Prompt(messages, promptOptions);
ChatResponse response2 = chatModel.call(prompt2);
// 输出模型响应结果
DeepSeekAssistantMessage deepSeekAssistantMessage2 = (DeepSeekAssistantMessage)
response2.getResult().getOutput();
String reasoningContent2 = deepSeekAssistantMessage2.getReasoningContent();
System.out.println("\n[Thinking]\n" + reasoningContent2);
String text2 = deepSeekAssistantMessage2.getText();
System.out.println("\n[Answer]\n" + text2);
```

## 支持的模型

DeepSeek支持多个模型，包括：

- **deepseek-v4-flash**: 快速经济版，高性价比、极速响应、低延迟
- **deepseek-v4-pro**: 专业旗舰版，最强推理、复杂任务、深度专业分析
- **deepseek-chat**: 普通通用对话，**即将停用**
- **deepseek-reasoner**: 深度推理/强化学习模型，**即将停用**

注意：新模型会默认开启思考模式，如果你都某些需求需要关闭思考模式，由于目前版本的JavaSDK不支持设置关闭它的思考模式，你可以通过自己书写一个请求拦截器来注入关闭思考模式的参数，参考代码如下：

```
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;
import org.springframework.boot.web.client.RestClientCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

import java.nio.charset.StandardCharsets;

/**
 * <p>
 * 描述：通过标准的RestClientCustomizer精准、安全地拦截DeepSeek的请求
 * </p>
 * <p>版权：&copy;01星球</p>
 * <p>地址：01星球总部</p>
 * @author 阿伟学长
 * @version 1.0.1
 */
```

```java
@Configuration
public class DeepSeekPayloadConfig {
    /**
     * 利用SpringBoot标准的定制器
     * 这样既能百分之百保留原来的超时时间（Timeout）、消息转换器（Converters）等系统全局配置
     * 又可以通过严格的域名过滤，确保非DeepSeek的业务流量受不到任何实质性的逻辑干扰
     */
    @Bean
    public RestClientCustomizer deepSeekRestClientCustomizer() {
        return new RestClientCustomizer() {
            private final ObjectMapper mapper = new ObjectMapper();

            @Override
            public void customize(RestClient.Builder restClientBuilder) {
                // 往全局的原Builder链条中，植入一个拦截器
                restClientBuilder.requestInterceptor((request, body, execution) -> {
                    // 只有当Host明确包含deepseek.com时，才会触发内存中的字节改写
                    if (request.getURI().getHost() != null
                            && request.getURI().getHost().contains("deepseek.com")) {
                        String rawJson = new String(body, StandardCharsets.UTF_8);
                        try {
                            JsonNode root = mapper.readTree(rawJson);
                            if (root instanceof ObjectNode objectNode) {
                                // 注入新一代V4架构所需的非标参数，关闭思考模式
                                ObjectNode thinkingNode = mapper.createObjectNode();
                                thinkingNode.put("type", "disabled");
                                objectNode.set("thinking", thinkingNode);
                                body = mapper.writeValueAsBytes(objectNode);
                            }
                        } catch (Exception e) {
                            // 转换失败则降级走原请求，确保不卡死任何哪怕格式异常的请求流程
                        }
                    }
                    // 非DeepSeek的普通三方业务HTTP请求走到这里，会由于不满足if直接越过，0延迟、0污染
                    return execution.execute(request, body);
                });
            }
        };
    }
}
```

## 函数调用

DeepSeekChatModel 支持函数调用（Function Calling），允许模型调用外部函数：

```java
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.function.FunctionToolCallback;

```

```java
// 定义请求参数对象
record WeatherReq(String city) {}

// 定义函数工具
ToolCallback weatherFunction = FunctionToolCallback
    .builder("getWeather", (WeatherReq req) -> req.city() + "的实时天气预报")
    .description("获取指定城市的天气")
    .inputType(WeatherReq.class)
    .build();

// 使用函数
ChatOptions options = DeepSeekChatOptions.builder()
    .toolCallbacks(List.of(weatherFunction))
    .build();

// 执行对话
Prompt prompt = new Prompt("成都的天气怎么样?", options);
ChatResponse response = chatModel.call(prompt);
System.out.println(response.getResult().getOutput().getText());
```

### 与ReactAgent集成

在 Spring AI Alibaba Agent Framework 中使用 DeepSeekChatModel:

```java
import com.alibaba.cloud.ai.graph.agent.ReactAgent;

// 创建 Agent
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .systemPrompt("你是一个有帮助的AI助手")
    .build();
// 调用 Agent
AssistantMessage response = agent.call("帮我分析1+1为什么等于2");
System.out.println(response.getText());
```

详细的 Agent 使用方法请参考 Agents 文档。

## 其它模型

官方还提供了许多其它的模型使用说明点击查看。

# Messages

Messages 是 Spring AI Alibaba 中模型交互的基本单元。它们代表模型的输入和输出，携带在与 LLM 交互时表示对话状态所需的内容和元数据。

Messages 是包含以下内容的对象:

- **Role（角色）** - 标识消息类型（如 `system`、`user`、`assistant`）
- **Content（内容）** - 表示消息的实际内容（如文本、图像、音频、文档等）
- **Metadata（元数据）** - 可选字段，如响应信息、消息 ID 和 token 使用情况

Spring AI Alibaba 提供了一个标准的消息类型系统，可在所有模型提供商之间工作，确保无论调用哪个模型都具有一致的行为。

# 基础使用

使用 messages 最简单的方式是创建消息对象并在调用模型时传递它们。

基础消息使用示例

```
import org.springframework.ai.chat.messages.Message;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.messages.SystemMessage;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.ai.chat.prompt.Prompt;
import java.util.List;

// 创建消息
SystemMessage systemMsg = new SystemMessage("你是一个有帮助的助手。");
UserMessage userMsg = new UserMessage("你好，你好吗？");

// 与聊天模型一起使用
List<Message> messages = List.of(systemMsg, userMsg);
Prompt prompt = new Prompt(messages);
ChatResponse response = chatModel.call(prompt);
// 返回ChatResponse，包含AssistantMessage
```

## 文本提示

文本提示是字符串 - 适用于简单的生成任务，不需要保留对话历史。

文本提示示例

```
// 使用字符串直接调用
String response = chatModel.call("写一首关于春天的俳句");
```

**使用文本提示的场景**:

- 有单个独立的请求
- 不需要对话历史
- 想要最小的代码复杂性

## 消息提示

或者，你可以通过提供消息对象列表向模型传递消息列表。

消息提示示例

```
import org.springframework.ai.chat.messages.Message;
import org.springframework.ai.chat.messages.SystemMessage;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.ai.chat.prompt.Prompt;
import java.util.List;
```

```
7
8  List<Message> messages = List.of(
9    new SystemMessage("你是一个诗歌专家"),
10   new UserMessage("写一首关于春天的俳句"),
11   new AssistantMessage("樱花盛开时 ... ")
12 );
13 Prompt prompt = new Prompt(messages);
14 ChatResponse response = chatModel.call(prompt);
```

**使用消息提示的场景**:

- 管理多轮对话
- 处理多模态内容（图像、音频、文件）
- 包含系统指令

# 消息类型

- **System Message（系统消息）** - 告诉模型如何行为并为交互提供上下文
- **User Message（用户消息）** - 表示用户输入和与模型的交互
- **Assistant Message（助手消息）** - 模型生成的响应，包括文本内容、工具调用和元数据
- **Tool Response Message（工具响应消息）** - 表示工具调用的输出

## System Message

`SystemMessage` 表示一组初始指令，用于引导模型的行为。你可以使用系统消息来设置语气、定义模型的角色并建立响应指南。

SystemMessage 基础指令示例

```
1  // 基础指令
2  SystemMessage systemMsg = new SystemMessage("你是一个有帮助的编程助手。");
3  List<Message> messages = List.of(
4    systemMsg,
5    new UserMessage("如何创建 REST API? ")
6  );
7  ChatResponse response = chatModel.call(new Prompt(messages));
```

SystemMessage 详细角色设定示例

```
1  // 详细的角色设定
2  SystemMessage systemMsg = new SystemMessage("""
3    你是一位资深的 Java 开发者，擅长 Web 框架。
4    始终提供代码示例并解释你的推理。
5    在解释中要简洁但透彻。
6    """);
7
8  List<Message> messages = List.of(
9    systemMsg,
10   new UserMessage("如何创建 REST API? ")
11 );
12 ChatResponse response = chatModel.call(new Prompt(messages));
```

## User Message

`UserMessage` 表示用户输入和交互。它们可以包含文本、图像、音频、文件和任何其他数量的多模态内容。

**文本内容**

UserMessage 文本内容示例

```
// 使用消息对象
ChatResponse response = chatModel.call(
  new Prompt(List.of(new UserMessage("什么是机器学习？ ")))
);

// 使用字符串快捷方式
// 使用字符串是单个 UserMessage 的快捷方式
String response = chatModel.call("什么是机器学习？ ");
```

**消息元数据**

UserMessage 消息元数据示例

```
import java.util.Map;

UserMessage userMsg = UserMessage.builder()
  .text("你好！ ")
  .metadata(Map.of(
      "user_id", "alice",       // 可选：识别不同用户
      "session_id", "sess_123"  // 可选：会话标识符
  ))
  .build();
```

**注意**：元数据字段的行为因提供商而异 - 有些用于用户识别，有些则忽略它。要检查，请参考模型提供商的文档。

**多模态内容**

`UserMessage` 可以包含多模态内容，如图像：

UserMessage 多模态内容示例

```
import org.springframework.ai.content.Media;
import org.springframework.util.MimeTypeUtils;

// 从URL创建图像
UserMessage userMsg = UserMessage.builder()
    .text("描述这张图片的内容。 ")
    .media(Media.builder()
            .mimeType(MimeTypeUtils.IMAGE_JPEG)
            .data("https://dashscope.oss-cn-beijing.aliyuncs.com/images/tiger.png")
            .build())
    .build();
```

```
// 创建Prompt对象
Prompt pr = Prompt.builder()
    .messages(userMsg)
    // 使用多模态支持的模型
    .chatOptions(DashScopeChatOptions.builder().model("qwen-vl-max").multiModel(true).build())
    .build();
// 调用模型
ChatResponse resp = chatModel.call(pr);
System.out.println(resp.getResult().getOutput().getText());
```

## Assistant Message

`AssistantMessage` 表示模型调用的输出。它们可以包括多模态数据、工具调用以及你稍后可以访问的提供商特定元数据。

AssistantMessage 基础使用示例

```
ChatResponse response = chatModel.call(new Prompt("解释 AI"));
AssistantMessage aiMessage = response.getResult().getOutput();
System.out.println(aiMessage.getText());
```

`AssistantMessage` 对象由模型调用返回，其中包含响应中的所有相关元数据。

提供商对消息类型的权重/上下文化方式不同，这意味着有时手动创建新的 `AssistantMessage` 对象并将其插入消息历史中（就像它来自模型一样）会很有帮助。

手动创建AssistantMessage示例

```
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.ai.chat.messages.SystemMessage;
import org.springframework.ai.chat.messages.UserMessage;

// 手动创建 AI 消息（例如，用于对话历史）
AssistantMessage aiMsg = new AssistantMessage("我很乐意帮助你回答这个问题！");

// 添加到对话历史
List<org.springframework.ai.chat.messages.Message> messages = List.of(
  new SystemMessage("你是一个有帮助的助手"),
  new UserMessage("你能帮我吗？"),
  aiMsg,  // 插入，就像它来自模型一样
  new UserMessage("太好了！2+2 等于多少？")
);
ChatResponse response = chatModel.call(new Prompt(messages));
```

**AssistantMessage 属性**：

- **text**: 消息的文本内容
- **metadata**: 消息的元数据映射
- **toolCalls**: 模型进行的工具调用列表
- **media**: 媒体内容列表（如果有）

**工具调用**

当模型进行工具调用时，它们包含在 `AssistantMessage` 中：

AssistantMessage工具调用示例

```
import org.springframework.ai.chat.messages.AssistantMessage.ToolCall;

ChatResponse response = chatModel.call(prompt);
AssistantMessage aiMessage = response.getResult().getOutput();
if (aiMessage.hasToolCalls()) {
  for (ToolCall toolCall : aiMessage.getToolCalls()) {
      System.out.println("Tool: " + toolCall.name());
      System.out.println("Args: " + toolCall.arguments());
      System.out.println("ID: " + toolCall.id());
  }
}
```

### Token使用

Spring AI Alibaba 的 `ChatResponse` 可以在其元数据中保存 token 计数和其他使用元数据：

Token 使用信息访问示例

```
ChatResponse response = chatModel.call(new Prompt("你好！"));
ChatResponseMetadata metadata = response.getMetadata();
// 访问使用信息
if (metadata != null && metadata.getUsage() != null) {
  System.out.println("Input tokens: " + metadata.getUsage().getPromptTokens());
  System.out.println("Output tokens: " +
metadata.getUsage().getCompletionTokens());
  System.out.println("Total tokens: " + metadata.getUsage().getTotalTokens());
}
```

### 流式和块

在流式传输期间，你将收到可以组合成完整消息对象的块：

流式输出示例

```
import reactor.core.publisher.Flux;

Flux<ChatResponse> responseStream = chatModel.stream(new Prompt("你好"));
StringBuilder fullResponse = new StringBuilder();
responseStream.subscribe(
    chunk -> {
        String content = chunk.getResult().getOutput().getText();
        fullResponse.append(content);
    },
    error -> System.out.println("\nError: " + error.getMessage()),
    () -> System.out.println(fullResponse)
);
```

**了解更多**：

- 从聊天模型流式传输 tokens
- 从 agents 流式传输 tokens 和/或步骤

### Tool Response Message

对于支持工具调用的模型，AI 消息可以包含工具调用。工具消息用于将单个工具执行的结果传回模型。

ToolResponseMessage 工具响应消息示例

```
// 在模型进行工具调用后
AssistantMessage aiMessage = AssistantMessage.builder()
    .content("")
    .toolCalls(List.of(
        new AssistantMessage.ToolCall(
            "call_123",
            "tool",
            "get_weather",
            "{\"location\": \"San Francisco\"}"
        )
    ))
    .build();

// 执行工具并创建结果消息
String weatherResult = "晴朗，22°C";
ToolResponseMessage toolMessage = ToolResponseMessage.builder()
    .responses(List.of(
        new ToolResponseMessage.ToolResponse("call_123", "get_weather",
weatherResult)
    ))
    .build();

// 继续对话
List<Message> messages = List.of(
    new UserMessage("旧金山的天气怎么样？"),
    aiMessage,      // 模型的工具调用
    toolMessage     // 工具执行结果
);
ChatResponse response = chatModel.call(new Prompt(messages));
System.out.println(response.getResult().getOutput().getText());
```

**ToolResponseMessage 属性**:

- responses

  : ToolResponse 对象列表，每个包含：

  - **id**: 工具调用 ID（必须与 AIMessage 中的工具调用 ID 匹配）
  - **name**: 调用的工具名称
  - **responseData**: 工具调用的字符串化输出

## 多模态内容

**多模态性**指的是处理不同形式数据的能力，如文本、音频、图像和视频。Spring AI Alibaba 包含这些数据的标准类型，可以跨提供商使用。

聊天模型可以接受多模态数据作为输入并生成它作为输出。下面我们展示包含多模态数据的输入消息的简短示例。

## 图像输入

图像输入示例

```
import org.springframework.ai.content.Media;
import org.springframework.util.MimeTypeUtils;

// 创建多模消息提示对象
Prompt createMultiPrompt(Message message) {
    return Prompt.builder()
        .messages(message)
        // 使用多模态支持的模型
        .chatOptions(DashScopeChatOptions.builder()
                    .model("qwen-vl-max").multiModel(true).build())
        .build();
}

// 从 URL
UserMessage message = UserMessage.builder()
    .text("描述这张图片的内容。")
    .media(Media.builder()
           .mimeType(MimeTypeUtils.IMAGE_JPEG)
           .data("https://dashscope.oss-cn-
beijing.aliyuncs.com/images/tiger.png")
           .build())
    .build();
ChatResponse resp1 = chatModel.call(createMultiPrompt(message));

// 从本地文件
import org.springframework.core.io.ClassPathResource;

UserMessage message = UserMessage.builder()
    .text("描述这张图片的内容。")
    .media(new Media(
        MimeTypeUtils.IMAGE_JPEG,
        new ClassPathResource("images/monkey.jpg")
    ))
    .build();
ChatResponse resp2 = chatModel.call(createMultiPrompt(message));
```

## 音频输入

音频输入示例

```
import org.springframework.ai.content.Media;
import org.springframework.util.MimeTypeUtils;

UserMessage message = UserMessage.builder()
    .text("描述这段音频的内容。")
    .media(Media.builder()
           .mimeType(MimeTypeUtils.parseMimeType("audio/wav"))
           .data(new ClassPathResource("audio/hh.wav"))
```

```
            .build())
    .metadata(Map.of(DashScopeApiConstants.MESSAGE_FORMAT, MessageFormat.AUDIO))
    .build();
ChatResponse resp = chatModel
    .call(Prompt.builder()
          .messages(message)
          // 使用多模态支持的语音模型
          .chatOptions(DashScopeChatOptions.builder()
                       .model("qwen-audio-turbo")
                       .maxToken(2048)
                       .multiModel(true)
                       .build())
          .build());
System.out.println(resp.getResult().getOutput().getText());
```

## 视频输入

视频输入示例

```
import org.springframework.ai.content.Media;
import org.springframework.util.MimeTypeUtils;

// 这种方式应该是有bug的，测试的时候提取不到视频帧
//UserMessage message = UserMessage.builder()
//    .text("描述这段视频的内容。")
//    .media(Media.builder().mimeType(MimeTypeUtils.parseMimeType("video/mp4"))
//           .data("https://help-static-aliyun-doc.aliyuncs.com/file-manage-files/zh-CN/20250307/ozcoym/hailuo_1060827889_RF.hd.mp4")
//           .build())
//    .metadata(Map.of(DashScopeApiConstants.MESSAGE_FORMAT, MessageFormat.VIDEO))
//    .build();
//ChatResponse resp = chatModel.call(createMultiPrompt(message));
//System.out.println(resp.getResult().getOutput().getText());

// 使用抽取视频帧的方式
List<Media> mediaList = FrameExtraHelper.createMediaList(10);
UserMessage message = UserMessage.builder()
    .text("描述这段视频的内容。")
    .media(mediaList)
    .metadata(Map.of(DashScopeApiConstants.MESSAGE_FORMAT, MessageFormat.VIDEO))
    .build();
ChatResponse resp = chatModel.call(createMultiPrompt(message));
System.out.println(resp.getResult().getOutput().getText());
```

**警告**：并非所有模型都支持所有文件类型。请查看模型提供商的文档以了解支持的格式和大小限制。

## 更多用法

关于多模态更多使用教程，点击查看官方说明。

# 与ChatModels一起使用

Chat models 接受消息对象序列作为输入并返回`ChatResponse`（包含`AssistantMessage`）作为输出。交互通常是无状态的，因此简单的对话循环涉及使用不断增长的消息列表调用模型。

## 基础对话示例

基础对话示例

```
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.messages.Message;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.ai.chat.prompt.Prompt;
import java.util.ArrayList;
import java.util.List;

ChatModel chatModel = // ... 初始化

List<Message> conversationHistory = new ArrayList<>();

// 第一轮对话
conversationHistory.add(new UserMessage("你好！"));
ChatResponse response1 = chatModel.call(new Prompt(conversationHistory));
conversationHistory.add(response1.getResult().getOutput());

// 第二轮对话
conversationHistory.add(new UserMessage("你能帮我学习 Java 吗？"));
ChatResponse response2 = chatModel.call(new Prompt(conversationHistory));
conversationHistory.add(response2.getResult().getOutput());

// 第三轮对话
conversationHistory.add(new UserMessage("从哪里开始？"));
ChatResponse response3 = chatModel.call(new Prompt(conversationHistory));
```

## 使用Builder模式

Spring AI Alibaba 的消息类提供了 builder 模式以便于构建：

Builder模式示例

```
// UserMessage with builder
UserMessage userMsg = UserMessage.builder()
  .text("你好，我想学习 Spring AI Alibaba")
  .metadata(Map.of("user_id", "user_123"))
  .build();

// SystemMessage with builder
SystemMessage systemMsg = SystemMessage.builder()
  .text("你是一个 Spring 框架专家")
  .metadata(Map.of("version", "1.0"))
  .build();

// AssistantMessage with builder
AssistantMessage assistantMsg = AssistantMessage.builder()
```

```
    .content("我很乐意帮助你学习 Spring AI Alibaba! ")
    .build();
```

### 消息复制和修改

消息复制和修改示例

```
// 复制消息
UserMessage original = new UserMessage("原始消息");
UserMessage copy = original.copy();

// 使用 mutate 创建修改的副本
UserMessage modified = original.mutate()
    .text("修改后的消息")
    .metadata(Map.of("modified", true))
    .build();
```

## 在ReactAgent中使用

ReactAgent 自动管理消息历史，但你也可以直接使用消息：

在 ReactAgent 中使用消息

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.messages.AssistantMessage;

ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .systemPrompt("你是一个有帮助的助手")
    .build();

// 使用字符串
AssistantMessage response1 = agent.call("你好");

// 使用 UserMessage
UserMessage userMsg = new UserMessage("帮我写一首诗");
AssistantMessage response2 = agent.call(userMsg);

// 使用消息列表
List<Message> messages = List.of(
    new UserMessage("我喜欢春天"),
    new UserMessage("写一首关于春天的诗")
);
AssistantMessage response3 = agent.call(messages);
```

详细的 Agent 使用方法请参考 Agents 文档。

## Agents

Agents 将大语言模型与工具结合，创建具备任务推理、工具使用决策、工具调用的自动化系统，系统具备持续推理、工具调用的循环迭代能力，直至问题解决。

Spring AI Alibaba 提供了基于 `ReactAgent` 的生产级 Agent 实现。

**一个 LLM Agent 在循环中通过运行工具来实现目标。**Agent 会一直运行直到满足停止条件 —— 即当模型输出最终答案或达到迭代限制时。

# ReactAgent理论基础

## 什么是ReAct

ReAct（Reasoning + Acting）是一种将推理和行动相结合的 Agent 范式。在这个范式中，Agent 会：

1. **思考（Reasoning）**：分析当前情况，决定下一步该做什么
2. **行动（Acting）**：执行工具调用或生成最终答案
3. **观察（Observation）**：接收工具执行的结果
4. **迭代**：基于观察结果继续思考和行动，直到完成任务

这个循环使 Agent 能够：

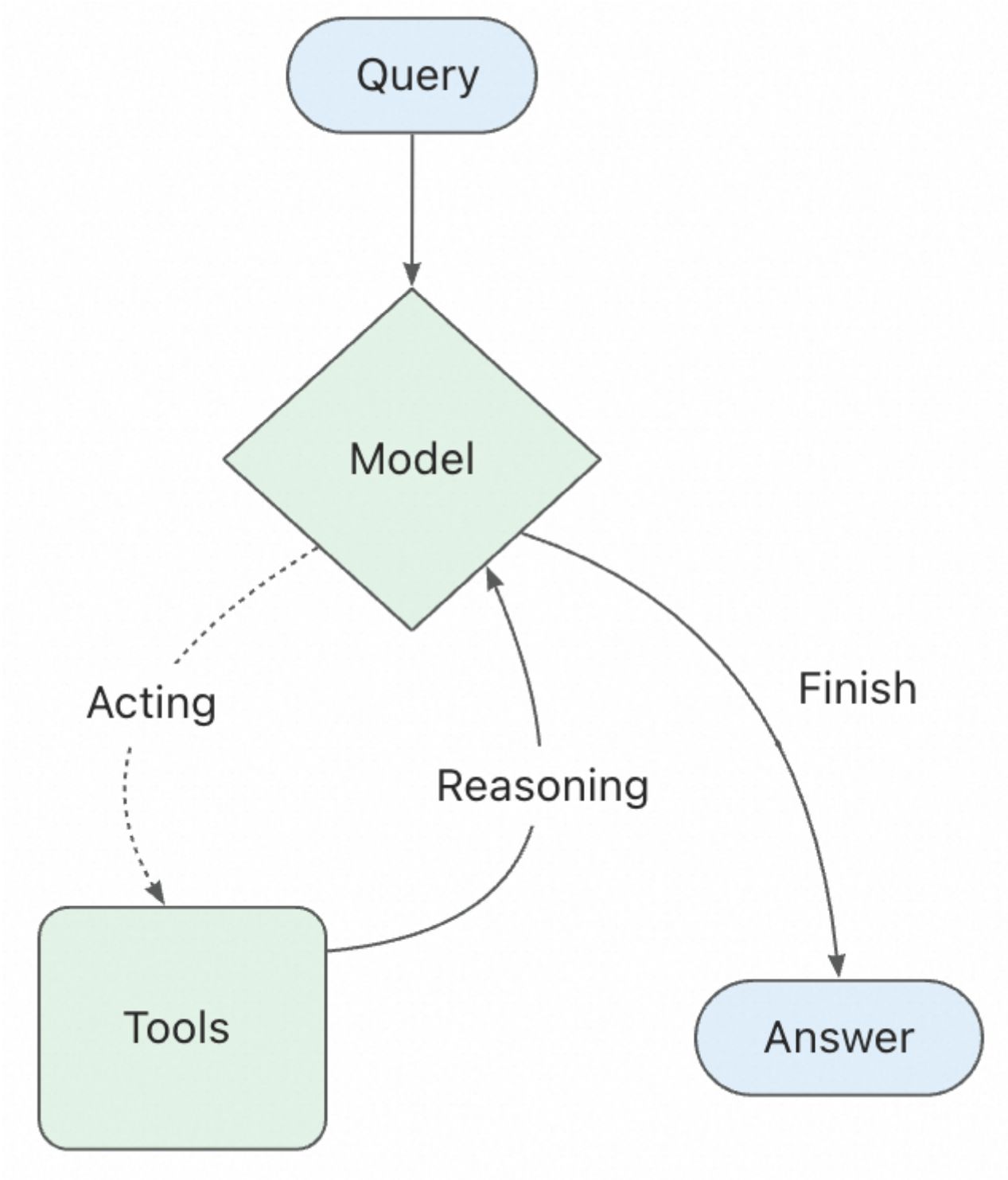
- 将复杂问题分解为多个步骤
- 动态调整策略基于中间结果
- 处理需要多次工具调用的任务
- 在不确定的环境中做出决策

## ReactAgent的工作原理

Spring AI Alibaba 中的`ReactAgent` 基于 **Graph 运行时**构建。Graph 由节点（steps）和边（connections）组成，定义了 Agent 如何处理信息。Agent 在这个 Graph 中移动，执行如下节点：

- **Model Node (模型节点)**：调用 LLM 进行推理和决策
- **Tool Node (工具节点)**：执行工具调用
- **Hook Nodes (钩子节点)**：在关键位置插入自定义逻辑

ReactAgent 的核心执行流程：




## 核心组件

### Model（模型）

Model 是 Agent 的推理引擎。Spring AI Alibaba 支持多种配置方式。

**基础模型配置**

最直接的方式是使用 `ChatModel` 实例：

ReactAgent 基础配置示例

```
import com.alibaba.cloud.ai.dashscope.api.DashScopeApi;
import com.alibaba.cloud.ai.dashscope.chat.DashScopeChatModel;
```

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;

// 创建 DashScope API 实例
DashScopeApi dashScopeApi = DashScopeApi.builder()
    .apiKey(System.getenv("AI_DASHSCOPE_API_KEY"))
    .build();

// 创建 ChatModel
ChatModel chatModel = DashScopeChatModel.builder()
    .dashScopeApi(dashScopeApi)
    .build();

// 创建 Agent
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .build();
```

**高级模型配置**

通过 `ChatOptions` 可以精细控制模型行为：

DashScopeChatModel 高级配置

```
import com.alibaba.cloud.ai.dashscope.chat.DashScopeChatOptions;

ChatModel chatModel = DashScopeChatModel.builder()
    .dashScopeApi(dashScopeApi)
    .defaultOptions(DashScopeChatOptions.builder()
                    .withModel(DashScopeChatModel.DEFAULT_MODEL_NAME)
                    .withTemperature(0.7)     // 控制随机性
                    .withMaxToken(2000)       // 最大输出长度
                    .withTopP(0.9)            // 核采样参数
                    .build())
    .build();
```

**常用参数说明**：

- `temperature`：控制输出的随机性（0.0-1.0），值越高越有创造性
- `maxTokens`：限制单次响应的最大token数
- `topP`：核采样，控制输出的多样性
- 更多参数请参考ChatModel适配

## Tools（工具）

***提示：这里只是简单引入，更详细介绍请查看Tools章节***

工具赋予 Agent 执行操作的能力，支持顺序执行、并行调用、动态选择和错误处理。

**定义和使用工具**

SearchTool 自定义工具示例

```
import org.springframework.ai.tool.ToolCallback;
```

```java
import org.springframework.ai.tool.function.FunctionToolCallback;
import org.springframework.ai.tool.annotation.ToolParam;
import org.springframework.ai.chat.model.ToolContext;
import java.util.function.BiFunction;

// 定义工具（示例：仅一个搜索工具）
public class SearchTool implements BiFunction<String, ToolContext, String> {
    @Override
    public String apply(String query, ToolContext context) {
        // 实现搜索逻辑
        return "搜索结果: " + query;
    }
}

// 创建工具回调
ToolCallback searchTool = FunctionToolCallback
    .builder("search", new SearchTool())
    .description("搜索工具")
    .build();

// 在Agent中使用
ReactAgent agent = ReactAgent.builder()
    .name("search_agent")
    .model(chatModel)
    .tools(searchTool)
    .build();
```

**工具错误处理**

ToolErrorInterceptor工具错误处理

```java
import com.alibaba.cloud.ai.graph.agent.interceptor.ToolInterceptor;
import com.alibaba.cloud.ai.graph.agent.interceptor.ToolCallRequest;
import com.alibaba.cloud.ai.graph.agent.interceptor.ToolCallResponse;
import com.alibaba.cloud.ai.graph.agent.interceptor.ToolCallHandler;

public class ToolErrorInterceptor extends ToolInterceptor {
    @Override
    public ToolCallResponse interceptToolCall
        (ToolCallRequest request, ToolCallHandler handler) {
        try {
            return handler.call(request);
        } catch (Exception e) {
            return ToolCallResponse.of(
                request.getToolCallId(),
                request.getToolName(),
                "Tool failed: " + e.getMessage());
        }
    }

    @Override
    public String getName() {
        return "ToolErrorInterceptor";
    }
}

```

```
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .interceptors(new ToolErrorInterceptor())
    .build();
```

**ReAct循环示例**：Agent 自动交替进行推理和工具调用，直到获得最终答案。

```
用户：查询杭州天气并推荐活动
→ [推理] 需要查天气 → [行动] get_weather("杭州") → [观察] 晴，25°C
→ [推理] 需要推荐活动 → [行动] search("户外活动") → [观察] 西湖游玩 ...
→ [推理] 信息充足 → [行动] 生成答案
```

## SystemPrompt（系统提示）

System Prompt 塑造 Agent 处理任务的方式。

**基础用法**

通过 `systemPrompt` 参数提供字符串：

系统提示基础配置

```
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .systemPrompt("你是一个专业的技术助手。请准确、简洁地回答问题。")
    .build();
```

**使用指令**

对于更详细的指令，使用 `instruction` 参数：

使用 instruction 提供详细指令

```
String instruction = """
    你是一个经验丰富的软件架构师。

    在回答问题时，请：
    1. 首先理解用户的核心需求
    2. 分析可能的技术方案
    3. 提供清晰的建议和理由
    4. 如果需要更多信息，主动询问

    保持专业、友好的语气。
    """;

ReactAgent agent = ReactAgent.builder()
    .name("architect_agent")
    .model(chatModel)
    .instruction(instruction)
    .build();
```

关于提示词和指令的核心区别：

- **systemPrompt（系统提示词）**
  - **最底层、最直接**的控制。
  - 作为大模型（LLM）API 的标准参数（`system` 角色），直接打包输入给模型。
  - 模型天然支持，优先级最高。
- **instruction（Agent 指令）**
  - **更上层、更具结构化**的概念。
  - 通常是开发框架（如 Spring AI、LangChain 或云平台）封装的 Agent 属性。
  - 框架最终会把 `instruction` **转化或拼接**到系统的 Prompt 中。

在编写复杂的Agent时，提示词往往会变得非常臃肿。

分开写可以帮你理清逻辑：

- **`instruction`（业务指令）**：定义 Agent 的**业务属性**。例如："你是一个架构师"、"你要语气友好"、"你必须先理解需求"。这部分通常由**业务开发人员**或产品经理来维护。
- **`systemPrompt` / 框架模板（运行逻辑）**：定义 Agent 的**技术行为**。例如：ReAct的思考步骤（Thought/Action/Observation）、工具调用的输入输出规范。这部分通常由**底层的开发框架**自动生成和控制。

**动态提示词**

使用 `ModelInterceptor` 实现基于上下文的动态提示：

DynamicPromptInterceptor动态提示拦截器

```
import com.alibaba.cloud.ai.graph.agent.interceptor.ModelInterceptor;
import com.alibaba.cloud.ai.graph.agent.interceptor.ModelRequest;
import com.alibaba.cloud.ai.graph.agent.interceptor.ModelResponse;
import com.alibaba.cloud.ai.graph.agent.interceptor.ModelCallHandler;
import org.springframework.ai.chat.messages.SystemMessage;

public class DynamicPromptInterceptor extends ModelInterceptor {
    @Override
    public ModelResponse interceptModel(ModelRequest request,
                                        ModelCallHandler handler) {
        // 基于上下文构建动态 system prompt
        String userRole = (String) request.getContext()
            .getOrDefault("user_role", "default");
        // 根据上下文构建动态提示词
        String dynamicPrompt = switch (userRole) {
            case "expert" -> """
                你正在与技术专家对话。
                - 使用专业术语
                - 深入技术细节
                - 提供高级建议
                """;
                case "beginner" -> """
                你正在与初学者对话。
                - 使用简单易懂的语言
                - 详细解释概念
                - 提供入门级建议
                """;
                default -> """
```

```
                你是一个专业的助手。
                - 根据问题复杂度调整回答
                - 保持友好和专业
                """;
        };
        // 增强 system message
        SystemMessage enhancedSystemMessage;
        if (request.getSystemMessage() == null) {
            enhancedSystemMessage = new SystemMessage(dynamicPrompt);
        } else {
            enhancedSystemMessage = new SystemMessage(
                request.getSystemMessage().getText() + "\n\n" + dynamicPrompt
            );
        }
        // 创建增强的请求
        ModelRequest modified = ModelRequest.builder(request)
            .systemMessage(enhancedSystemMessage)
            .build();
        return handler.call(modified);
    }

    @Override
    public String getName() {
        return "DynamicPromptInterceptor";
    }
}

// 创建智能体
ReactAgent agent = ReactAgent.builder()
    .name("adaptive_agent")
    .model(chatModel)
    .interceptors(new DynamicPromptInterceptor())
    .build();

// 创建消息
List<Message> messages = List.of(
    new UserMessage("我想了解Java多线程"),
    new UserMessage("特别是线程池的使用")
);

// 创建运行配置
RunnableConfig runnableConfig = RunnableConfig.builder().threadId("1")
    .addMetadata("user_role", "expert")
    .build();

// 调用智能体
AssistantMessage response = agent.call(messages, runnableConfig);
System.out.println(response.getText());
```

## 调用Agent

## 基础调用

使用 `call` 方法获取最终响应：

Agent 基础调用示例

```
import org.springframework.ai.chat.messages.AssistantMessage;

// 字符串输入
AssistantMessage response = agent.call("杭州的天气怎么样？ ");
System.out.println(response.getText());

// UserMessage输入
UserMessage userMessage = new UserMessage("帮我分析这个问题");
AssistantMessage response = agent.call(userMessage);

// 多个消息
List<Message> messages = List.of(
    new UserMessage("我想了解Java多线程"),
    new UserMessage("特别是线程池的使用")
);
AssistantMessage response = agent.call(messages);
```

## 获取完整状态

使用`invoke`方法获取完整的执行状态：

使用invoke获取完整状态

```
@Test
void testInvoke() throws GraphRunnerException {
    // 使用 StringTemplateRenderer.builder() 创建自定义分隔符的TemplateRenderer
    // 使用 {{ 和 }} 作为占位符分隔符
    TemplateRenderer customRenderer = SaaStTemplateRenderer.builder()
            .startDelimiter("{{")
            .endDelimiter("}}")
            .build();
    // 创建智能体
    ReactAgent agent = ReactAgent.builder()
            .name("my_agent")
            .model(chatModel)
            .templateRenderer(customRenderer)
            .systemPrompt("你是一个浪漫主义的诗人")
            .instruction("""
                    诗的主题是{{topic}}
                    请根据以下要求回答：
                    1 诗的风格是{{style}}
                    2 包含的城市是{{city}}
                    3 包含的季节是{{season}}
                    """)
            .build();

    // 初始化一个全局状态Map把你的自定义数据塞进去
    Map<String, Object> initialState = new HashMap<>(Map.of(
            "input", "帮我写一首的诗",
```

```
            "style", "七言绝句",
            "city", "成都",
            "season", "夏季",
            "topic", "麻将"
    ));

    // 调用智能体
    Optional<OverAllState> result = agent.invoke(initialState);
    if (result.isPresent()) {
        // 获取完整状态
        OverAllState state = result.get();
        System.out.println("完整状态：" + state + "\n");

        // 访问消息历史
        Optional<List<Message>> messages = state.value("messages");
        if (messages.isPresent()) {
            for (Message message : messages.get()) {
                System.out.println(message.getText());
            }
        }

        // 访问自定义状态
        Optional<Object> style = state.value("style");
        System.out.println("自定义状态：" + style);
    }
}
```

### 使用配置

通过 `RunnableConfig` 传递运行时配置：

使用 RunnableConfig 传递配置

```
import com.alibaba.cloud.ai.graph.RunnableConfig;

String threadId = "thread_123";
RunnableConfig runnableConfig = RunnableConfig.builder()
    .threadId(threadId)
    .addMetadata("key", "value")
    .build();

AssistantMessage response = agent.call("你的问题", runnableConfig);
```

## 高级特性

***提示：这里只是简单引入这些特性，更详细介绍请查看对应章节的介绍***

### 结构化输出

在某些情况下，你可能希望 Agent 以特定格式返回输出。ReactAgent 提供了两种策略。

**使用outputType**

通过 Java 类定义输出结构，Agent 会自动生成对应的JSON Schema：

PoemOutput 结构化输出示例

```java
public class PoemOutput {
    private String title;
    private String content;
    private String style;

    // Getters and Setters
    // ......
}

ReactAgent agent = ReactAgent.builder()
    .name("poem_agent")
    .model(chatModel)
    .outputType(PoemOutput.class)
    .saver(new MemorySaver())
    .build();

AssistantMessage response = agent.call("写一首关于春天的诗");
// 输出会遵循 PoemOutput 的结构
System.out.println(response.getText());
```

**使用outputSchema**

使用 `BeanOutputConverter` 生成输出Schema，提供类型安全和自动 schema 生成：

使用 outputSchema 自定义输出格式

```java
import org.springframework.ai.converter.BeanOutputConverter;

// 定义输出类型
public static class TextAnalysisResult {
    private String summary;
    private List<String> keywords;
    private String sentiment;
    private Double confidence;

    // Getters and Setters
    // .......
}

// 使用 BeanOutputConverter 生成 outputSchema
BeanOutputConverter<TextAnalysisResult> outputConverter =
    new BeanOutputConverter<>(TextAnalysisResult.class);
String format = outputConverter.getFormat();
ReactAgent agent = ReactAgent.builder()
    .name("analysis_agent")
    .model(chatModel)
    .outputSchema(format)
    .saver(new MemorySaver())
    .build();

AssistantMessage response = agent.call("分析这段文本：春天来了，万物复苏。");
```

**选择建议**：

- `outputType`：类型安全，适合结构固定的场景（**推荐**）
- `outputSchema`：使用 `BeanOutputConverter` 生成时提供类型安全，手动提供字符串时灵活性高，适合动态或复杂的输出格式

## Memory（记忆）

Agent 通过状态自动维护对话历史。使用 `MemorySaver` 配置持久化存储。

Memory 配置示例

```
import com.alibaba.cloud.ai.graph.checkpoint.config.SaverConfig;
import com.alibaba.cloud.ai.graph.checkpoint.constant.SaverEnum;
import com.alibaba.cloud.ai.graph.checkpoint.savers.MemorySaver;

// 配置内存存储
ReactAgent agent = ReactAgent.builder()
    .name("chat_agent")
    .model(chatModel)
    .saver(new MemorySaver())
    .build();

// 使用 thread_id 维护对话上下文
RunnableConfig config = RunnableConfig.builder()
    .threadId("user_123")
    .build();

agent.call("我叫张三", config);
agent.call("我叫什么名字？", config);  // 输出："你叫张三"
```

**生产环境**：使用 `RedisSaver`、`MongoSaver` 等持久化存储替代 `MemorySaver`。

## Hooks（钩子）

Hooks 允许在 Agent 执行的关键点插入自定义逻辑。

**Hook 类型与使用**

Hook 使用示例

```
import com.alibaba.cloud.ai.graph.agent.hook.*;
import com.alibaba.cloud.ai.graph.agent.hook.messages.MessagesModelHook;
import com.alibaba.cloud.ai.graph.agent.hook.messages.AgentCommand;
import com.alibaba.cloud.ai.graph.agent.hook.messages.UpdatePolicy;

// 1. AgentHook - 在 Agent 开始/结束时执行，每次Agent调用只会运行一次
@HookPositions({HookPosition.BEFORE_AGENT, HookPosition.AFTER_AGENT})
public class LoggingHook extends AgentHook {
  @Override
  public String getName() { return "logging"; }

  @Override
  public CompletableFuture<Map<String, Object>> beforeAgent(
      OverAllState state, RunnableConfig config) {
      System.out.println("Agent 开始执行");
      return CompletableFuture.completedFuture(Map.of());
```

```
  }

  @Override
  public CompletableFuture<Map<String, Object>> afterAgent(
      OverAllState state, RunnableConfig config) {
      System.out.println("Agent 执行完成");
      return CompletableFuture.completedFuture(Map.of());
  }
}

// 2. MessagesModelHook - 在模型调用前后执行（例如：消息修剪），
// 专门用于操作消息列表，使用更简单，更推荐。
// 区别于AgentHook，MessagesModelHook在一次agent调用中可能会调用多次，
// 也就是每次reasoning-acting迭代都会执行
@HookPositions({HookPosition.BEFORE_MODEL})
public class MessageTrimmingHook extends MessagesModelHook {
  private static final int MAX_MESSAGES = 10;

  @Override
  public String getName() {
      return "message_trimming";
  }

  @Override
  public AgentCommand beforeModel(
      List<Message> previousMessages, RunnableConfig config) {
      if (previousMessages.size() > MAX_MESSAGES) {
          // 只保留最后 MAX_MESSAGES 条消息
          List<Message> trimmedMessages = previousMessages.subList(
              previousMessages.size() - MAX_MESSAGES,
              previousMessages.size()
          );
          return new AgentCommand(trimmedMessages, UpdatePolicy.REPLACE);
      }
      // 消息数量未超过限制，直接返回原消息列表
      return new AgentCommand(previousMessages);
  }
}
```

**Hook 执行位置**：

- `BEFORE_AGENT` / `AFTER_AGENT`：Agent 整体执行前后
- `BEFORE_MODEL` / `AFTER_MODEL`：Agent Loop 循环过程中，每次模型调用前后

## Interceptors（拦截器）

Interceptors 提供更细粒度的控制，可以拦截和修改模型调用和工具执行。

**使用示例**

Interceptor 使用示例

```
import com.alibaba.cloud.ai.graph.agent.interceptor.*;

// ModelInterceptor - 内容安全检查
public class GuardrailInterceptor extends ModelInterceptor {
```

```
    @Override
    public ModelResponse interceptModel(
        ModelRequest request, ModelCallHandler handler) {
        // 前置：检查输入
        if (containsSensitiveContent(request.getMessages())) {
            return ModelResponse.of(AssistantMessage
                                    .builder()
                                    .content("检测到不适当的内容")
                                    .build());
        }
        // 执行调用
        ModelResponse response = handler.call(request);
        // 后置：检查输出
        return sanitizeIfNeeded(response);
    }
}

// ToolInterceptor - 监控和错误处理
public class ToolMonitoringInterceptor extends ToolInterceptor {
    @Override
    public ToolCallResponse interceptToolCall(
        ToolCallRequest request, ToolCallHandler handler) {
        long startTime = System.currentTimeMillis();
        try {
            ToolCallResponse response = handler.call(request);
            logSuccess(request, System.currentTimeMillis() - startTime);
            return response;
        } catch (Exception e) {
            logError(request, e, System.currentTimeMillis() - startTime);
            return ToolCallResponse.error(
                request.getToolCall(),
                "工具执行遇到问题，请稍后重试");
        }
    }
}

// 组合使用
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .interceptors(
    new GuardrailInterceptor(),
    new ToolMonitoringInterceptor())
    .saver(new MemorySaver())
    .build();
```

**常见用途**：

- **ModelInterceptor**：内容安全、动态提示、日志记录、性能监控
- **ToolInterceptor**：错误重试、权限检查、结果缓存、审计日志

## 控制与流式输出

### 迭代控制

通过 Hooks 控制 Agent 的执行迭代，防止无限循环或过度成本。

使用 ModelCallLimitHook 限制模型调用次数

```
import com.alibaba.cloud.ai.graph.agent.hook.modelcalllimit.ModelCallLimitHook;
import com.alibaba.cloud.ai.graph.checkpoint.savers.MemorySaver;

// 使用内置的 ModelCallLimitHook 限制模型调用次数
ReactAgent agent = ReactAgent.builder()
  .name("my_agent")
  .model(chatModel)
  .hooks(ModelCallLimitHook.builder().runLimit(5).build())  // 限制最多调用 5 次
  .saver(new MemorySaver())
  .build();
```

自定义停止条件 Hook

```
import com.alibaba.cloud.ai.graph.agent.hook.ModelHook;
import com.alibaba.cloud.ai.graph.agent.hook.HookPosition;
import com.alibaba.cloud.ai.graph.agent.hook.HookPositions;
import com.alibaba.cloud.ai.graph.agent.hook.JumpTo;
import org.springframework.ai.chat.messages.AssistantMessage;

// 自定义停止条件：基于状态判断是否继续
@HookPositions({HookPosition.BEFORE_MODEL})
public class CustomStopConditionHook extends ModelHook {

    @Override
    public String getName() {
        return "custom_stop_condition";
    }

    @Override
    public CompletableFuture<Map<String, Object>> beforeModel(
        OverAllState state, RunnableConfig config) {
        // 检查是否找到答案，展示使用 OverAllState
        boolean answerFound = (Boolean)
state.value("answer_found").orElse(false);
        // 检查错误次数，展示使用 RunnableConfig
        int errorCount = (Integer) config.context().get("error_count").orElse(0);

        // 找到答案或错误过多时停止
        if (answerFound || errorCount > 3) {
            List<Message> messages = new ArrayList<>();
            messages.add(new AssistantMessage(
                answerFound ? "已找到答案，Agent 执行完成。"
                : "错误次数过多 (" + errorCount + ")，Agent 执行终止。"
            ));
            // the messages will be appended to the original message list
context.
```

```
            return CompletableFuture.completedFuture(Map.of("messages",
messages));
        }
        return CompletableFuture.completedFuture(Map.of());
    }

}

// 使用自定义停止条件
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .hooks(new CustomStopConditionHook())
    .saver(new MemorySaver())
    .build();
```

**流式输出**

在 Agent 场景中，流式输出的核心是处理 `StreamingOutput` 类型。无论是模型推理、工具调用还是 Hook 节点，输出都统一为这个类型。

**使用 OutputType 区分输出类型**：

通过 `OutputType` 枚举可以区分不同节点的输出，以及判断是流式增量内容还是完成输出：

| OutputType | 说明 |
| --- | --- |
| `AGENT_MODEL_STREAMING` | 模型推理的流式增量内容 |
| `AGENT_MODEL_FINISHED` | 模型推理完成，可获取全量内容 |
| `AGENT_TOOL_STREAMING` | 工具调用的流式增量内容 |
| `AGENT_TOOL_FINISHED` | 工具调用完成 |
| `AGENT_HOOK_STREAMING` | Hook 节点的流式增量内容 |
| `AGENT_HOOK_FINISHED` | Hook 节点完成 |

提示

- 对于 Hook 等通常非流式的节点，直接使用 `AGENT_HOOK_FINISHED` 读取内容即可
- 并非所有节点输出都有意义，尤其是 Hook 节点可能产生无效输出，建议通过 `OutputType` 进行过滤

流式输出示例

```
import reactor.core.publisher.Flux;
import com.alibaba.cloud.ai.graph.NodeOutput;
import com.alibaba.cloud.ai.graph.streaming.StreamingOutput;
import com.alibaba.cloud.ai.graph.streaming.OutputType;

Flux<NodeOutput> stream = agent.stream("复杂任务");
stream.subscribe(
  output -> {
      // 检查是否为 StreamingOutput 类型
      if (output instanceof StreamingOutput streamingOutput) {
```

```
            OutputType type = streamingOutput.getOutputType();

            // 处理模型推理的流式输出
            if (type == OutputType.AGENT_MODEL_STREAMING) {
                // 流式增量内容，逐步显示
                System.out.print(streamingOutput.message().getText());
            } else if (type == OutputType.AGENT_MODEL_FINISHED) {
                // 模型推理完成，可获取完整响应
                System.out.println("\n模型输出完成");
            }

            // 处理工具调用完成（目前不支持 STREAMING）
            if (type == OutputType.AGENT_TOOL_FINISHED) {
                System.out.println("工具调用完成: " + output.node());
            }

            // 对于 Hook 节点，通常只关注完成事件（如果Hook没有有效输出可以忽略）
            if (type == OutputType.AGENT_HOOK_FINISHED) {
                System.out.println("Hook 执行完成: " + output.node());
            }
        }
    },
    error -> System.err.println("错误: " + error),
    () -> System.out.println("Agent 执行完成")
);
```

**消息类型识别**

`StreamingOutput.message()` 返回的消息有多种类型，需要结合 `OutputType` 和消息元数据进行区分处理：

| OutputType | 消息类型 | 判断条件 | 说明 |
| --- | --- | --- | --- |
| `AGENT_MODEL_STREAMING` / `AGENT_MODEL_FINISHED` | 模型普通响应 | `AssistantMessage` 且 `metadata.reasoningContent` 为空 | 模型的实际回复内容，通过 `getText()` 获取 |
| `AGENT_MODEL_STREAMING` / `AGENT_MODEL_FINISHED` | 模型 Thinking | `AssistantMessage` 且 `metadata.reasoningContent` 不为空 | 模型的思考过程（如 DeepSeek 等支持 Thinking 的模型） |
| `AGENT_MODEL_FINISHED` | 工具调用请求 | `AssistantMessage` 且 `hasToolCalls()` 为 `true` | 模型请求调用工具，包含工具名称和参数 |
| `AGENT_TOOL_FINISHED` | 工具响应结果 | `ToolResponseMessage` | 工具执行后的返回结果 |

消息类型识别示例

```
import org.springframework.ai.chat.messages.Message;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.ai.chat.messages.ToolResponseMessage;

// 结合 OutputType 和消息类型进行处理
if (output instanceof StreamingOutput streamingOutput) {
    OutputType type = streamingOutput.getOutputType();
```

```
    Message message = streamingOutput.message();
    // 处理模型流式输出
    if (type == OutputType.AGENT_MODEL_STREAMING) {
        if (message instanceof AssistantMessage assistantMessage) {
            // 检查是否为 Thinking 消息
            Object reasoningContent = assistantMessage
                .getMetadata().get("reasoningContent");
            if (reasoningContent != null &&
                !reasoningContent.toString().isEmpty()) {
                System.out.print("[Thinking] " + reasoningContent);
            } else {
                // 普通模型响应（增量内容）
                System.out.print(assistantMessage.getText());
            }
        }
    }
    // 处理模型输出完成
    else if (type == OutputType.AGENT_MODEL_FINISHED) {
        if (message instanceof AssistantMessage assistantMessage) {
            if (assistantMessage.hasToolCalls()) {
                // 工具调用请求
                assistantMessage.getToolCalls()
                    .forEach(toolCall -> {
                        System.out.println("[Tool Call] " +
                                           toolCall.name() + ": " +
                                           toolCall.arguments());
                    });
            } else {
                // 模型完整响应
                System.out.println("\n[Model Finished]");
            }
        }
    }
    // 处理工具执行结果
    else if (type == OutputType.AGENT_TOOL_FINISHED) {
        if (message instanceof ToolResponseMessage toolResponse) {
            toolResponse.getResponses()
                .forEach(response -> {
                    System.out.println("[Tool Result] " +
                                       response.name() + ": " +
                                       response.responseData());
                });
        }
    }
}
```

提示

- **先判断 OutputType**：通过 `OutputType` 先确定是模型输出、工具输出还是 Hook 输出，再处理具体消息
- **Thinking 消息**：部分模型（如 DeepSeek-R1）支持输出思考过程，通过 `metadata.reasoningContent` 获取
- **工具调用**：工具调用请求通常在 `AGENT_MODEL_FINISHED` 阶段出现，此时 `hasToolCalls()` 返回 `true`
- **流式 vs 完成**：`STREAMING` 阶段内容是增量的，`FINISHED` 阶段可获取完整消息或执行结果

更多关于Graph底层流式输出机制，请参考 Graph 流式输出。

## 更多智能体

官方还提供了一些其它的智能体，可以去了解一下。

- DeepResearch
- DataAgent
- JManus
- AssistantAgent

# Tools

许多 AI 应用程序通过自然语言与用户交互。然而，某些业务场景需要模型使用结构化输入直接与外部系统（如 API、数据库或文件系统）进行交互。

Tools 是 agents 调用来执行操作的组件。它们通过定义良好的输入和输出让模型与外部世界交互，从而扩展模型的能力。Tools 封装了一个可调用的函数及其输入模式。我们可以把工具定义传递给兼容的 models，允许模型决定是否调用工具以及使用什么参数。在这些场景中，工具调用使模型能够生成符合指定输入模式的请求。

> **注意：服务器端工具使用**
>
> 某些聊天模型（例如 OpenAI、Anthropic 和 Gemini）具有在服务器端执行的内置工具，如 Web 搜索和代码解释器。请参阅提供商概述以了解如何使用特定聊天模型访问这些工具。

> **TIP:** 迁移与更多 Tool Calling 说明请参考：Tool Calling 使用指南。

*Tool calling*（也称为 *function calling*）是 AI 应用程序中的常见模式，允许 model 与一组 API 或 *tools* 交互，增强其能力。

Tools 主要用于：

- **信息检索**。此类别中的 tools 可用于从外部源检索信息，例如数据库、Web 服务、文件系统或 Web 搜索引擎。目标是增强 model 的知识，使其能够回答原本无法回答的问题。因此，它们可以在 Retrieval Augmented Generation (RAG) 场景中使用。例如，可以使用 tool 检索给定位置的当前天气、检索最新新闻文章或查询数据库中的特定记录。
- **执行操作**。此类别中的 tools 可用于在软件系统中执行操作，例如发送电子邮件、在数据库中创建新记录、提交表单或触发工作流。目标是自动化原本需要人工干预或显式编程的任务。例如，可以使用 tool 为与聊天机器人交互的客户预订航班、填写网页上的表单，或在代码生成场景中基于自动化测试（TDD）实现 Java 类。

尽管我们通常将 *tool calling* 称为 model 能力，但实际上由客户端应用程序提供 tool calling 逻辑。Model 只能请求 tool call 并提供输入参数，而应用程序负责从输入参数执行 tool call 并返回结果。Model 永远无法访问作为 tools 提供的任何 API，这是一个关键的安全考虑。

Spring AI 提供了便捷的 API 来定义 tools、解析来自 model 的 tool call 请求并执行 tool calls。

# 快速开始

让我们看看如何在 Spring AI 中开始使用 tool calling。我们将实现两个简单的 tools：一个用于信息检索，一个用于执行操作。

## 信息检索

AI model 无法访问实时信息。任何假设了解当前日期或天气预报等信息的问题都无法由 model 回答。但是，我们可以提供一个可以检索此信息的 tool，并让 model 在需要访问实时信息时调用此 tool。

让我们在 `DateTimeTools` 类中实现一个 tool 来获取用户时区的当前日期和时间。该 tool 不接受任何参数。来自 Spring Framework 的 `LocaleContextHolder` 可以提供用户的时区。该 tool 将定义为用 `@Tool` 注解的方法。为了帮助 model 理解是否以及何时调用此 tool，我们将提供该 tool 功能的详细描述。

DateTimeTools 信息检索示例

```
import java.time.LocalDateTime;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.context.i18n.LocaleContextHolder;

public class DateTimeTools {

  @Tool(description = "根据当前用户所在时区获取当前日期")
  String getCurrentDateTime() {
      return LocalDateTime.now()
          .atZone(LocaleContextHolder.getTimeZone().toZoneId())
          .toString();
  }
}
```

接下来，让我们使 tool 可用于 model。在此示例中，我们将使用 `ChatClient` 与 model 交互。我们将通过 `tools()` 方法传递 `DateTimeTools` 的实例来向 model 提供 tool。当 model 需要知道当前日期和时间时，它将请求调用 tool。在内部，`ChatClient` 将调用 tool 并将结果返回给 model，然后 model 将使用 tool call 结果生成对原始问题的最终响应。

使用 DateTimeTools 示例

```
String response = ChatClient.create(chatModel)
      .prompt("明天是几号呢?")
      .tools(new DateTimeTools())
      .call()
      .content();
System.out.println(response);
```

***更多关于ChatClient使用说明，请参考官网说明，本教程不作赘述，它不作为重点。***

## 执行操作

AI model 可用于生成完成某些目标的计划。例如，model 可以生成预订前往丹麦旅行的计划。但是，model 无法执行计划。这就是 tools 的用武之地：它们可用于执行 model 生成的计划。

在前面的示例中，我们使用 tool 来确定当前日期和时间。在此示例中，我们将定义一个用于在特定时间设置闹钟的第二个 tool。目标是从现在起 10 分钟设置闹钟，因此我们需要向 model 提供两个 tools 来完成此任务。

我们将新 tool 添加到与之前相同的 `DateTimeTools` 类中。新 tool 将接受单个参数，即 ISO-8601 格式的时间。然后，tool 将向控制台打印一条消息，指示已为给定时间设置闹钟。与之前一样，tool 定义为用 `@Tool` 注解的方法，我们还使用它提供详细描述以帮助 model 理解何时以及如何使用 tool。

DateTimeTools 执行操作示例

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.context.i18n.LocaleContextHolder;

public class DateTimeTools {
    @Tool(description = "根据当前用户所在时区获取当前日期")
    String getCurrentDateTime() {
        return LocalDateTime.now()
            .atZone(LocaleContextHolder.getTimeZone().toZoneId())
            .toString();
    }

    @Tool(description = "为用户设置指定时间的闹钟，时间以 ISO-8601 格式提供")
    void setAlarm(String time) {
        LocalDateTime alarmTime = LocalDateTime
            .parse(time, DateTimeFormatter.ISO_DATE_TIME);
        System.out.println("闹钟设置时间为 " + alarmTime);
    }
}
```

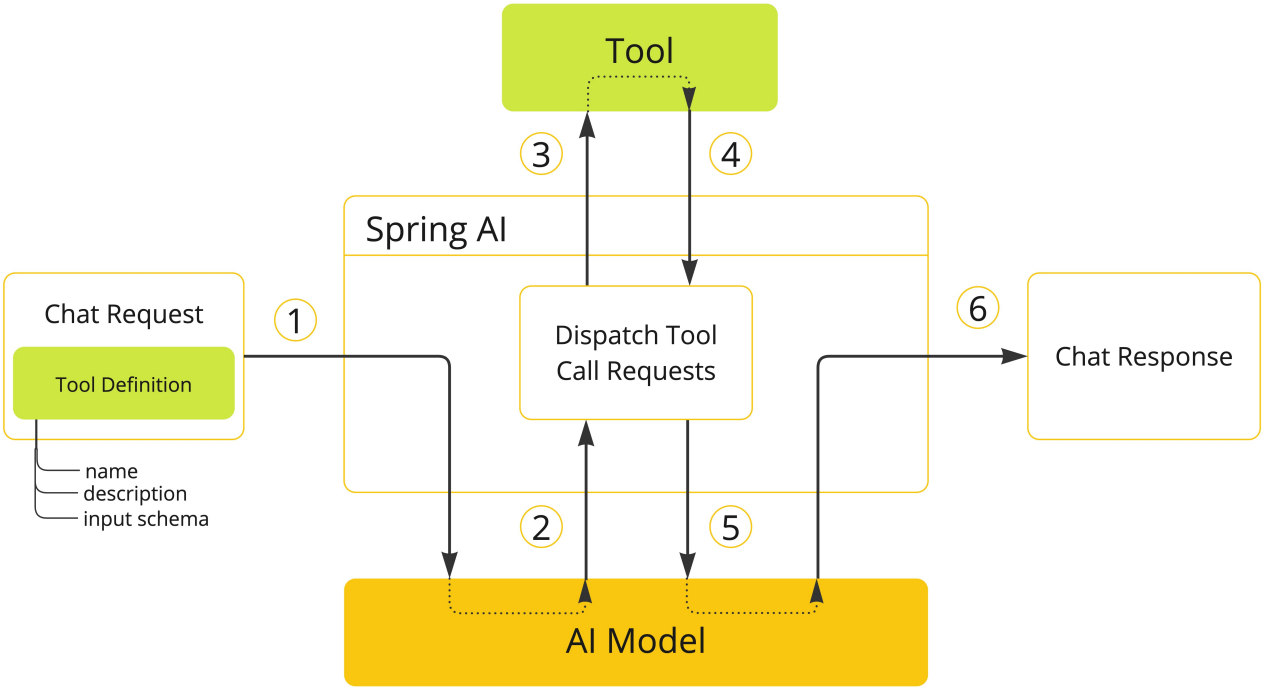
接下来，让我们使两个 tools 都可用于 model。当我们要求从现在起 10 分钟设置闹钟时，model 首先需要知道当前日期和时间。然后，它将使用当前日期和时间来计算闹钟时间。最后，它将使用闹钟 tool 来设置闹钟。

使用多个工具示例

```
String response = ChatClient.create(chatModel)
    .prompt("你能在10分钟后设置一个闹钟吗？ ")
    .tools(new DateTimeTools())
    .call()
    .content();
System.out.println(response);
```

# 概述

Spring AI 通过一组灵活的抽象支持 tool calling，允许您以一致的方式定义、解析和执行 tools。本节概述了 Spring AI 中 tool calling 的主要概念和组件。




1. 当我们想要使 tool 可用于 model 时，我们在聊天请求中包含其定义。每个 tool 定义包括名称、描述和输入参数的 schema。
2. 当 model 决定调用 tool 时，它发送带有 tool 名称和根据定义 schema 建模的输入参数的响应。
3. 应用程序负责使用 tool 名称来识别并使用提供的输入参数执行 tool。
4. Tool call 的结果由应用程序处理。
5. 应用程序将 tool call 结果发送回 model。
6. Model 使用 tool call 结果作为附加上下文生成最终响应。

Tools 是 tool calling 的构建块，它们由 `ToolCallback` 接口建模。Spring AI 提供了从方法和函数指定 `ToolCallback`(s) 的内置支持，但您始终可以定义自己的 `ToolCallback` 实现以支持更多用例。

`ChatModel` 实现透明地将 tool call 请求分派到相应的 `ToolCallback` 实现，并将 tool call 结果发送回 model，最终生成最终响应。它们使用 `ToolCallingManager` 接口来执行此操作，该接口负责管理 tool 执行生命周期。

`ChatClient` 和 `ChatModel` 都接受 `ToolCallback` 对象列表，以使 tools 可用于 model 和最终执行它们的 `ToolCallingManager`。

除了直接传递 `ToolCallback` 对象外，您还可以传递 tool 名称列表，这些名称将使用 `ToolCallbackResolver` 接口动态解析。

## 创建工具

Spring AI 提供了两种从方法指定 tools（即 `ToolCallback`(s)）的内置支持：

- 声明式，使用 `@Tool` 注解
- 编程式，使用低级 `MethodToolCallback` 实现。

### 方法作为Tools

**声明式规范**

您可以通过用 `@Tool` 注解方法来将方法转换为 tool。

@Tool 注解示例

```java
class DateTimeTools {
    @Tool(description = "根据当前用户所在时区获取当前日期时间")
    String getCurrentDateTime() {
        return LocalDateTime.now()
            .atZone(LocaleContextHolder.getTimeZone().toZoneId())
            .toString();
    }
}
```

`@Tool` 注解允许您提供有关 tool 的关键信息：

- `name`：tool 的名称。如果未提供，将使用方法名称。AI model 使用此名称在调用时识别 tool。因此，不允许在同一类中有两个同名 tools。名称必须在特定聊天请求中提供给 model 的所有 tools 中唯一。
- `description`：tool 的描述，model 可以使用它来理解何时以及如何调用 tool。如果未提供，方法名称将用作 tool 描述。但是，强烈建议提供详细描述，因为这对于 model 理解 tool 的用途以及如何使用它至关重要。未能提供良好的描述可能导致 model 在应该使用时不使用 tool 或错误使用它。
- `returnDirect`：tool 结果是否应直接返回给客户端或传递回 model。有关更多详细信息，请参阅返回直接。
- `resultConverter`：用于将 tool call 的结果转换为 `String object` 以发送回 AI model 的 `ToolCallResultConverter` 实现。有关更多详细信息，请参阅 结果转换。

方法可以是静态的或实例的，并且可以具有任何可见性（public、protected、package-private 或 private）。包含方法的类可以是顶级类或嵌套类，也可以具有任何可见性（只要在您计划实例化它的地方可以访问）。

**NOTE:** Spring AI 为 `@Tool` 注解方法的 AOT 编译提供内置支持，只要包含方法的类是 Spring bean（例如 `@Component`）。否则，您需要向 GraalVM 编译器提供必要的配置。例如，通过用 `@RegisterReflection(memberCategories = MemberCategory.INVOKE_DECLARED_METHODS)` 注解类。

您可以为方法定义任意数量的参数（包括无参数），支持大多数类型（基元、POJO、枚举、列表、数组、映射等）。同样，方法可以返回大多数类型，包括 `void`。如果方法返回值，返回类型必须是可序列化类型，因为结果将被序列化并发送回 model。

**NOTE:** 某些类型不受支持。有关更多详细信息，请参阅 方法 Tool 限制。

Spring AI 将自动为 `@Tool` 注解方法的输入参数生成 JSON schema。Schema 由 model 用于理解如何调用 tool 并准备 tool 请求。`@ToolParam` 注解可用于提供有关输入参数的附加信息，例如描述或参数是必需还是可选的。默认情况下，所有输入参数都被视为必需。

使用 @ToolParam 注解示例

```java
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.ai.tool.annotation.ToolParam;

class DateTimeTools {
    @Tool(description = "为用户设置指定时间的闹钟，时间以 ISO-8601 格式提供")
    void setAlarm(@ToolParam(description = "时间使用 ISO-8601 格式化") String time) {
        LocalDateTime alarmTime = LocalDateTime
            .parse(time, DateTimeFormatter.ISO_DATE_TIME);
        System.out.println("闹钟设置时间为 " + alarmTime);
    }
}
```

`@ToolParam` 注解允许您提供有关 tool 参数的关键信息：

- `description`：参数的描述，model 可以使用它来更好地理解如何使用它。例如，参数应该是什么格式，允许什么值等。
- `required`：参数是必需还是可选的。默认情况下，所有参数都被视为必需。

如果参数用 `@Nullable` 注解，除非使用 `@ToolParam` 注解明确标记为必需，否则将被视为可选。

除了 `@ToolParam` 注解外，您还可以使用来自 Swagger 的 `@Schema` 注解或来自 Jackson 的 `@JsonProperty`。有关更多详细信息，请参阅 JSON Schema。

**编程式规范**

您可以通过编程方式构建 `MethodToolCallback` 来将方法转换为 tool。

MethodToolCallback 示例

```java
import org.springframework.util.ReflectionUtils;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.method.MethodToolCallback;
import org.springframework.ai.tool.ToolDefinitions;

class DateTimeTools {
    String getCurrentDateTime() {
        return LocalDateTime.now()
            .atZone(LocaleContextHolder.getTimeZone().toZoneId())
            .toString();
    }
}

Method method = ReflectionUtils
    .findMethod(DateTimeTools.class, "getCurrentDateTime");
ToolCallback toolCallback = MethodToolCallback.builder()
    .toolDefinition(ToolDefinitions.builder(method)
                    .description("根据当前用户所在时区获取当前日期时间")
                    .build())
    .toolMethod(method)
    .toolObject(new DateTimeTools())
    .build();
```

`MethodToolCallback.Builder` 允许您构建 `MethodToolCallback` 实例并提供有关 tool 的关键信息：

- `toolDefinition`：定义 tool 名称、描述和输入 schema 的 `ToolDefinition` 实例。您可以使用 `ToolDefinition.Builder` 类构建它。必需。
- `toolMetadata`：定义附加设置的 `ToolMetadata` 实例，例如结果是否应直接返回给客户端，以及要使用的结果转换器。您可以使用 `ToolMetadata.Builder` 类构建它。
- `toolMethod`：表示 tool 方法的 `Method` 实例。必需。
- `toolObject`：包含 tool 方法的对象实例。如果方法是静态的，您可以省略此参数。
- `toolCallResultConverter`：用于将 tool call 的结果转换为 `String` 对象以发送回 AI model 的 `ToolCallResultConverter` 实例。如果未提供，将使用默认转换器（`DefaultToolCallResultConverter`）。

如果方法是静态的，您可以省略 `toolObject()` 方法，因为不需要它。

静态方法示例

```
class DateTimeTools {
    static String getCurrentDateTime() {
        return LocalDateTime.now()
            .atZone(LocaleContextHolder.getTimeZone().toZoneId())
            .toString();
    }
}

Method method = ReflectionUtils
    .findMethod(DateTimeTools.class, "getCurrentDateTime");
ToolCallback toolCallback = MethodToolCallback.builder()
    .toolDefinition(ToolDefinitions.builder(method)
                    .description("根据当前用户所在时区获取当前日期时间")
                    .build())
    .toolMethod(method)
    .build();
```

**方法Tool限制**

以下类型目前不支持作为用作 tools 的方法的参数或返回类型：

- `Optional`
- 异步类型（例如 `CompletableFuture`、`Future`）
- 响应式类型（例如 `Flow`、`Mono`、`Flux`）
- 函数类型（例如 `Function`、`Supplier`、`Consumer`）。

函数类型使用基于函数的 tool 规范方法支持。有关更多详细信息，请参阅函数作为Tools。

## 函数作为Tools

Spring AI 提供了对从函数指定工具的内置支持，可以通过编程方式使用低级 `FunctionToolCallback` 实现，也可以动态地作为在运行时解析的 `@Bean`。

**编程方式规范**

你可以通过编程方式构建 `FunctionToolCallback`，将函数类型（`Function`、`Supplier`、`Consumer` 或 `BiFunction`）转换为工具。

WeatherService 工具定义示例

```
import java.util.function.Function;

public class WeatherService implements
    Function<WeatherRequest, WeatherResponse> {
  public WeatherResponse apply(WeatherRequest request) {
      return new WeatherResponse(30.0, Unit.C);
  }
}

public enum Unit { C, F }
public record WeatherRequest(String location, Unit unit) {}
public record WeatherResponse(double temp, Unit unit) {}
```

`FunctionToolCallback.Builder` 允许你构建 `FunctionToolCallback` 实例并提供有关工具的关键信息：

- **name**: 工具的名称。AI 模型使用此名称在调用时识别工具。因此，在同一上下文中不允许有两个同名的工具。对于特定的聊天请求，名称在模型可用的所有工具中必须是唯一的。**必需。**
- **toolFunction**: 表示工具方法的函数对象（`Function`、`Supplier`、`Consumer` 或 `BiFunction`）。**必需。**
- **description**: 工具的描述，模型可以使用它来了解何时以及如何调用工具。如果未提供，将使用方法名称作为工具描述。但是，强烈建议提供详细描述，因为这对于模型理解工具的目的和使用方式至关重要。如果未提供良好的描述，可能导致模型在应该使用工具时不使用，或者使用不正确。
- **inputType**: 函数输入的类型。**必需。**
- **inputSchema**: 工具输入参数的 JSON schema。如果未提供，将根据 `inputType` 自动生成 schema。你可以使用 `@ToolParam` 注解提供有关输入参数的附加信息，例如描述或参数是必需还是可选。默认情况下，所有输入参数都被视为必需。
- **toolMetadata**: 定义附加设置的 `ToolMetadata` 实例，例如是否应将结果直接返回给客户端，以及要使用的结果转换器。你可以使用 `ToolMetadata.Builder` 类构建它。
- **toolCallResultConverter**: 用于将工具调用结果转换为字符串对象以发送回 AI 模型的 `ToolCallResultConverter` 实例。如果未提供，将使用默认转换器（`DefaultToolCallResultConverter`）。

FunctionToolCallback构建示例

```
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.function.FunctionToolCallback;

ToolCallback toolCallback = FunctionToolCallback
  .builder("currentWeather", new WeatherService())
  .description("获取指定位置的天气")
  .inputType(WeatherRequest.class)
  .build();
```

函数的输入和输出可以是 `Void` 或 POJO。输入和输出 POJO 必须是可序列化的，因为结果将被序列化并发送回模型。函数以及输入和输出类型必须是公共的。

**重要提示**：某些类型不受支持。有关更多详细信息，请参阅函数工具限制。

**添加工具到ChatClient**：

使用编程规范方法时，你可以将 `FunctionToolCallback` 实例传递给 `ChatClient` 的 `toolCallbacks()` 方法。该工具仅对添加到的特定聊天请求可用。

添加工具到 ChatClient 示例

```
import org.springframework.ai.chat.client.ChatClient;

ToolCallback toolCallback = ...;

ChatClient.create(chatModel)
  .prompt("成都的天气怎么样?")
  .toolCallbacks(toolCallback)
  .call()
  .content();
```

**添加默认工具到ChatClient**:

使用编程规范方法时，你可以通过将 `FunctionToolCallback` 实例传递给 `defaultToolCallbacks()` 方法将默认工具添加到 `ChatClient.Builder`。如果同时提供默认工具和运行时工具，运行时工具将完全覆盖默认工具。

添加默认工具到 ChatClient 示例

```
ChatModel chatModel = ...;
ToolCallback toolCallback = ...;

ChatClient chatClient = ChatClient.builder(chatModel)
  .defaultToolCallbacks(toolCallback)
  .build();
```

**注意**：默认工具在从同一 `ChatClient.Builder` 构建的所有 `ChatClient` 实例执行的所有聊天请求中共享。它们对于跨不同聊天请求常用的工具很有用，但如果不小心使用也可能很危险，可能在不应该使用时使它们可用。

**添加工具到ChatModel**:

使用编程规范方法时，你可以将 `FunctionToolCallback` 实例传递给 `ToolCallingChatOptions` 的 `toolCallbacks()` 方法。该工具仅对添加到的特定聊天请求可用。

添加工具到 ChatModel 示例

```java
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.model.tool.ToolCallingChatOptions;

ChatModel chatModel = ...;
ToolCallback toolCallback = ...;

ChatOptions chatOptions = ToolCallingChatOptions.builder()
  .toolCallbacks(toolCallback)
  .build();

Prompt prompt = new Prompt("成都的天气怎么样?", chatOptions);
chatModel.call(prompt);
```

**动态规范**

你可以将工具定义为 Spring beans，让 Spring AI 使用 `ToolCallbackResolver` 接口（通过 `SpringBeanToolCallbackResolver` 实现）在运行时动态解析它们，而不是以编程方式指定工具。此选项使你可以将任何 `Function`、`Supplier`、`Consumer` 或 `BiFunction` bean 用作工具。bean 名称将用作工具名称，Spring Framework 的 `@Description` 注解可用于为工具提供描述，供模型用来了解何时以及如何调用工具。

使用 @Bean 定义工具示例

```java
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Description;
import java.util.function.Function;

@Configuration(proxyBeanMethods = false)
class WeatherTools {
  @Bean
  @Description("获取指定位置的天气")
  Function<WeatherRequest, WeatherResponse> currentWeather() {
      return new WeatherService();
  }
}
```

**重要提示**：某些类型不受支持。有关更多详细信息，请参阅函数工具限制。

工具输入参数的 JSON schema 将自动生成。你可以使用 `@ToolParam` 注解提供有关输入参数的附加信息，例如描述或参数是必需还是可选。默认情况下，所有输入参数都被视为必需。

使用 @ToolParam 注解示例

```java
import org.springframework.ai.tool.annotation.ToolParam;

record WeatherRequest(
  @ToolParam(description = "城市或坐标") String location,
  Unit unit
) {}
```

此工具规范方法的缺点是不保证类型安全，因为工具解析是在运行时完成的。为了缓解这一问题，你可以使用 `@Bean` 注解显式指定工具名称并将值存储在常量中，以便你可以在聊天请求中使用它而不是硬编码工具名称。

使用常量定义工具名称示例

```
@Configuration(proxyBeanMethods = false)
class WeatherTools {
  public static final String CURRENT_WEATHER_TOOL = "currentWeather";
  @Bean(CURRENT_WEATHER_TOOL)
  @Description("获取指定位置的天气")
  Function<WeatherRequest, WeatherResponse> currentWeather() {
      return new WeatherService();
  }
}
```

**添加工具到ChatClient**（使用动态规范）：

使用动态规范添加工具到 ChatClient

```
ChatClient.create(chatModel)
  .prompt("成都的天气怎么样?")
  .toolNames(WeatherTools.CURRENT_WEATHER_TOOL)
  .call()
  .content();
```

**添加默认工具到ChatClient**（使用动态规范）：

使用动态规范添加默认工具到 ChatClient

```
ChatModel chatModel = ...;

ChatClient chatClient = ChatClient.builder(chatModel)
  .defaultToolNames(WeatherTools.CURRENT_WEATHER_TOOL)
  .build();
```

**函数工具限制**

以下类型目前不支持作为用作工具的函数的输入或输出类型：

- 原始类型
- `Optional`
- 集合类型（例如 `List`、`Map`、`Array`、`Set`）
- 异步类型（例如 `CompletableFuture`、`Future`）
- 响应式类型（例如 `Flow`、`Mono`、`Flux`）

使用基于方法的工具规范方法支持原始类型和集合。

# Tool 规范

在 Spring AI 中，tools 通过 `ToolCallback` 接口建模。在前面的部分中，我们已经看到了如何使用 Spring AI 提供的内置支持从方法和函数定义 tools。本节将更深入地介绍 tool 规范以及如何自定义和扩展它以支持更多用例。

## Tool Callback

`ToolCallback` 接口提供了一种定义可由 AI model 调用的 tool 的方法，包括定义和执行逻辑。当您想从头开始定义 tool 时，这是要实现的主要接口。

接口提供以下方法：

ToolCallback 接口

```
public interface ToolCallback {
  /**
   * Definition used by the AI model to determine when and how to call the tool.
   */
  ToolDefinition getToolDefinition();
  /**
   * Metadata providing additional information on how to handle the tool.
   */
  ToolMetadata getToolMetadata();
  /**
   * Execute tool with the given input and return the result to send back to the AI model.
   */
  String call(String toolInput);
  /**
   * Execute tool with the given input and context, and return the result to send back to the AI model.
   */
  String call(String toolInput, ToolContext toolContext);
}
```

Spring AI 为 tool 方法（`MethodToolCallback`）和 tool 函数（`FunctionToolCallback`）提供内置实现。

## Tool Definition

`ToolDefinition` 接口提供 AI model 了解 tool 可用性所需的信息，包括 tool 名称、描述和输入 schema。每个 `ToolCallback` 实现必须提供 `ToolDefinition` 实例来定义 tool。

接口提供以下方法：

ToolDefinition 接口

```
public interface ToolDefinition {
  /**
   * The tool name. Unique within the tool set provided to a model.
   */
  String name();
  /**
```

```
     * The tool description, used by the AI model to determine what the tool does.
     */
    String description();
    /**
     * The schema of the parameters used to call the tool.
     */
    String inputSchema();
}
```

`ToolDefinition.Builder` 允许您使用默认实现（`DefaultToolDefinition`）构建 `ToolDefinition` 实例。

构建 ToolDefinition 示例

```
ToolDefinition toolDefinition = ToolDefinition.builder()
    .name("currentWeather")
    .description("Get the weather in location")
    .inputSchema("""
        {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string"
                },
                "unit": {
                    "type": "string",
                    "enum": ["C", "F"]
                }
            },
            "required": ["location", "unit"]
        }
    """)
    .build();
```

## JSON Schema

向 AI model 提供 tool 时，model 需要知道调用 tool 的输入类型的 schema。Schema 用于理解如何调用 tool 并准备 tool 请求。Spring AI 通过 `JsonSchemaGenerator` 类为生成 tool 输入类型的 JSON Schema 提供内置支持。Schema 作为 `ToolDefinition` 的一部分提供。

`JsonSchemaGenerator` 类在底层用于为方法或函数的输入参数生成 JSON schema。JSON schema 生成逻辑支持一系列注解，您可以在方法和函数的输入参数上使用这些注解来自定义生成的 schema。

本节描述在生成 tool 输入参数的 JSON schema 时可以自定义的两个主要选项：描述和必需状态。

**描述**

除了为 tool 本身提供描述外，您还可以为 tool 的输入参数提供描述。描述可用于提供有关输入参数的关键信息，例如参数应该是什么格式、允许什么值等。这对于帮助 model 理解输入 schema 以及如何使用它很有用。Spring AI 使用以下注解之一为输入参数生成描述提供内置支持：

- Spring AI 的 `@ToolParam(description = "...")`
- Jackson 的 `@JsonClassDescription(description = "...")`
- Jackson 的 `@JsonPropertyDescription(description = "...")`

- Swagger 的 `@Schema(description = "...")`

此方法适用于方法和函数，并且可以递归地用于嵌套类型。

**必需/可选**

默认情况下，每个输入参数都被视为必需，这强制 AI model 在调用 tool 时为其提供值。但是，您可以使用以下注解之一使输入参数可选，按此优先级顺序：

- Spring AI 的 `@ToolParam(required = false)`
- Jackson 的 `@JsonProperty(required = false)`
- Swagger 的 `@Schema(required = false)`
- Spring Framework 的 `@Nullable`

此方法适用于方法和函数，并且可以递归地用于嵌套类型。

可选参数示例

```
class CustomerTools {
  @Tool(description = "Update customer information")
  void updateCustomerInfo(Long id, String name,
                          @ToolParam(required = false) String email) {
      System.out.println("Updated info for customer with id: " + id);
  }
}
```

**WARNING:** 为输入参数定义正确的必需状态对于减轻幻觉风险并确保 model 在调用 tool 时提供正确的输入至关重要。在前面的示例中，`email` 参数是可选的，这意味着 model 可以在不为其提供值的情况下调用 tool。如果参数是必需的，model 在调用 tool 时必须为其提供值。如果不存在值，model 可能会编造一个，导致幻觉。

## 结果转换

Tool call 的结果使用 `ToolCallResultConverter` 序列化，然后发送回 AI model。`ToolCallResultConverter` 接口提供了一种将 tool call 的结果转换为 `String` 对象的方法。

ToolCallResultConverter 接口

```
@FunctionalInterface
public interface ToolCallResultConverter {
  /**
   * Given an Object returned by a tool, convert it to a String compatible with the
   * given class type.
   */
  String convert(@Nullable Object result, @Nullable Type returnType);
}
```

结果必须是可序列化类型。默认情况下，结果使用 Jackson（`DefaultToolCallResultConverter`）序列化为 JSON，但您可以通过提供自己的 `ToolCallResultConverter` 实现来自定义序列化过程。

Spring AI 在方法和函数 tools 中都依赖 `ToolCallResultConverter`。

**方法ToolCall结果转换**

使用声明式方法从方法构建 tools 时，您可以通过设置 `@Tool` 注解的 `resultConverter()` 属性来为 tool 提供自定义 `ToolCallResultConverter`。

自定义结果转换器示例

```
class CustomerTools {
  @Tool(description = "Retrieve customer information",
        resultConverter = CustomToolCallResultConverter.class)
  Customer getCustomerInfo(Long id) {
      return customerRepository.findById(id);
  }
}
```

如果使用编程式方法，您可以通过设置 `MethodToolCallback.Builder` 的 `resultConverter()` 属性来为 tool 提供自定义 `ToolCallResultConverter`。

**函数ToolCall结果转换**

使用编程式方法从函数构建 tools 时，您可以通过设置 `FunctionToolCallback.Builder` 的 `resultConverter()` 属性来为 tool 提供自定义 `ToolCallResultConverter`。

## 返回直接

默认情况下，tool call 的结果作为响应发送回 model。然后，model 可以使用结果继续对话。

在某些情况下，您可能希望将结果直接返回给调用者，而不是将其发送回 model。例如，如果您构建一个依赖 RAG tool 的 agent，您可能希望将结果直接返回给调用者，而不是将其发送回 model 进行不必要的后处理。或者，您可能有某些 tools 应该结束 agent 的推理循环。

每个 `ToolCallback` 实现可以定义 tool call 的结果是否应直接返回给调用者或发送回 model。默认情况下，结果发送回 model。但您可以按 tool 更改此行为。

负责管理 tool 执行生命周期的 `ToolCallingManager` 负责处理与 tool 关联的 `returnDirect` 属性。如果属性设置为 `true`，tool call 的结果将直接返回给调用者。否则，结果将发送回 model。

**NOTE:** 如果同时请求多个 tool calls，所有 tools 的 `returnDirect` 属性必须设置为 `true` 才能将结果直接返回给调用者。否则，结果将发送回 model。

**方法返回直接**

使用声明式方法从方法构建 tools 时，您可以通过将 `@Tool` 注解的 `returnDirect` 属性设置为 `true` 来标记 tool 将结果直接返回给调用者。

returnDirect 示例

```
class CustomerTools {
  @Tool(description = "Retrieve customer information",
        returnDirect = true)
  Customer getCustomerInfo(Long id) {
      return customerRepository.findById(id);
  }
}
```

如果使用编程式方法，您可以通过 `ToolMetadata` 接口设置 `returnDirect` 属性，并将其传递给 `MethodToolCallback.Builder`。

使用 ToolMetadata 设置 returnDirect

```
ToolMetadata toolMetadata = ToolMetadata.builder()
  .returnDirect(true)
  .build();
```

**函数返回直接**

使用编程式方法从函数构建 tools 时，您可以通过 `ToolMetadata` 接口设置 `returnDirect` 属性，并将其传递给 `FunctionToolCallback.Builder`。

## Tool 执行

Tool 执行是使用提供的输入参数调用 tool 并返回结果的过程。Tool 执行由 `ToolCallingManager` 接口处理，该接口负责管理 tool 执行生命周期。

ToolCallingManager 接口

```
public interface ToolCallingManager {
  /**
   * Resolve the tool definitions from the model's tool calling options.
   */
  List<ToolDefinition> resolveToolDefinitions(ToolCallingChatOptions chatOptions);
  /**
   * Execute the tool calls requested by the model.
   */
  ToolExecutionResult executeToolCalls(Prompt prompt, ChatResponse chatResponse);
}
```

如果您使用任何 Spring AI Spring Boot Starters，`DefaultToolCallingManager` 是 `ToolCallingManager` 接口的自动配置实现。您可以通过提供自己的 `ToolCallingManager` bean 来自定义 tool 执行行为。

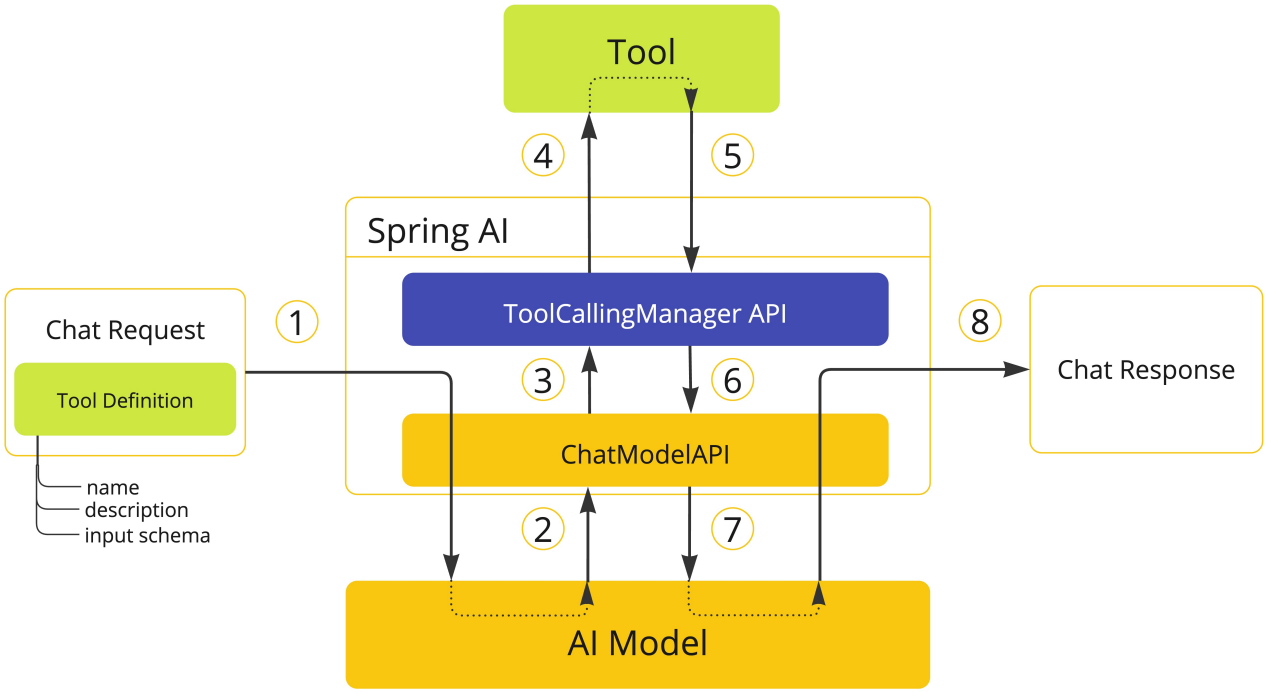
自定义 ToolCallingManager

```
@Bean
ToolCallingManager toolCallingManager() {
  return ToolCallingManager.builder().build();
}
```

默认情况下，Spring AI 在每个 `ChatModel` 实现内部透明地为您管理 tool 执行生命周期。但您可以选择退出此行为并自己控制 tool 执行。本节描述这两种场景。

## 框架控制的Tool执行

使用默认行为时，Spring AI 将自动拦截来自 model 的任何 tool call 请求，调用 tool 并将结果返回给 model。所有这些都由每个使用 `ToolCallingManager` 的 `ChatModel` 实现透明地为您完成。




1. 当我们想要使 tool 可用于 model 时，我们在聊天请求（`Prompt`）中包含其定义，并调用将请求发送到 AI model 的 `ChatModel` API。
2. 当 model 决定调用 tool 时，它发送带有 tool 名称和根据定义 schema 建模的输入参数的响应（`ChatResponse`）。
3. `ChatModel` 将 tool call 请求发送到 `ToolCallingManager` API。
4. `ToolCallingManager` 负责识别要调用的 tool 并使用提供的输入参数执行它。
5. Tool call 的结果返回到 `ToolCallingManager`。
6. `ToolCallingManager` 将 tool 执行结果返回给 `ChatModel`。
7. `ChatModel` 将 tool 执行结果发送回 AI model（`ToolResponseMessage`）。
8. AI model 使用 tool call 结果作为附加上下文生成最终响应，并通过 `ChatClient` 将其发送回调用者（`ChatResponse`）。

**WARNING:** 目前，与 model 交换的关于 tool 执行的内部消息不会暴露给用户。如果您需要访问这些消息，应该使用用户控制的 tool 执行方法。

确定 tool call 是否有资格执行的逻辑由 `ToolExecutionEligibilityPredicate` 接口处理。默认情况下，tool 执行资格通过检查 `ToolCallingChatOptions` 的 `internalToolExecutionEnabled` 属性是否设置为 `true`（默认值），以及 `ChatResponse` 是否包含任何 tool calls 来确定。

您可以在创建 `ChatModel` bean 时提供 `ToolExecutionEligibilityPredicate` 的自定义实现。

## 用户控制的Tool执行

在某些情况下，您可能希望自己控制 tool 执行生命周期。您可以通过将 `ToolCallingChatOptions` 的 `internalToolExecutionEnabled` 属性设置为 `false` 来执行此操作。

当您使用此选项调用 `ChatModel` 时，tool 执行将委托给调用者，让您完全控制 tool 执行生命周期。您有责任检查 `ChatResponse` 中的 tool calls 并使用 `ToolCallingManager` 执行它们。

以下示例演示了用户控制的 tool 执行方法的最小实现：

用户控制的 Tool 执行示例

```
ChatModel chatModel = ...;
ToolCallingManager toolCallingManager = ToolCallingManager.builder().build();

ChatOptions chatOptions = ToolCallingChatOptions.builder()
  .toolCallbacks(new CustomerTools())
  .internalToolExecutionEnabled(false)
  .build();
Prompt prompt = new Prompt("Tell me more about the customer with ID 42",
chatOptions);

ChatResponse chatResponse = chatModel.call(prompt);

while (chatResponse.hasToolCalls()) {
  ToolExecutionResult toolExecutionResult = toolCallingManager
      .executeToolCalls(prompt, chatResponse);
  prompt = new Prompt(toolExecutionResult.conversationHistory(), chatOptions);
  chatResponse = chatModel.call(prompt);
}

System.out.println(chatResponse.getResult().getOutput().getText());
```

**NOTE:** 选择用户控制的 tool 执行方法时，我们建议使用 `ToolCallingManager` 来管理 tool calling 操作。这样，您可以受益于 Spring AI 为 tool 执行提供的内置支持。但是，没有什么可以阻止您实现自己的 tool 执行逻辑。

## 异常处理

当 tool call 失败时，异常作为 `ToolExecutionException` 传播，可以捕获以处理错误。`ToolExecutionExceptionProcessor` 可用于处理 `ToolExecutionException`，有两种结果：要么生成要发送回 AI model 的错误消息，要么抛出异常供调用者处理。

ToolExecutionExceptionProcessor 接口

```java
@FunctionalInterface
public interface ToolExecutionExceptionProcessor {
  /**
   * Convert an exception thrown by a tool to a String that can be sent back to the AI
   * model or throw an exception to be handled by the caller.
   */
  String process(ToolExecutionException exception);
}
```

如果您使用任何 Spring AI Spring Boot Starters，`DefaultToolExecutionExceptionProcessor` 是 `ToolExecutionExceptionProcessor` 接口的自动配置实现。默认情况下，`RuntimeException` 的错误消息会发送回 model，而检查异常和错误（例如，`IOException`、`OutOfMemoryError`）总是被抛出。`DefaultToolExecutionExceptionProcessor` 构造函数允许您将 `alwaysThrow` 属性设置为 `true` 或 `false`。如果为 `true`，将抛出异常而不是将错误消息发送回 model。

您可以使用 `spring.ai.tools.throw-exception-on-error` 属性来控制 `DefaultToolExecutionExceptionProcessor` bean 的行为：

| 属性 | 描述 | 默认值 |
| --- | --- | --- |
| `spring.ai.tools.throw-exception-on-error` | 如果为 `true`，tool calling 错误将作为异常抛出供调用者处理。如果为 `false`，错误将转换为消息并发送回 AI model，允许它处理并响应错误。 | `false` |

自定义异常处理器

```java
@Bean
ToolExecutionExceptionProcessor toolExecutionExceptionProcessor() {
  return new DefaultToolExecutionExceptionProcessor(true);
}
```

**NOTE:** 如果您定义了自己的 `ToolCallback` 实现，请确保在 `call()` 方法中作为 tool 执行逻辑的一部分发生错误时抛出 `ToolExecutionException`。

`ToolExecutionExceptionProcessor` 由默认 `ToolCallingManager`（`DefaultToolCallingManager`）内部使用，以处理 tool 执行期间的异常。

## Tool 解析

向 model 传递 tools 的主要方法是在调用 `ChatClient` 或 `ChatModel` 时提供 `ToolCallback`(s)，使用前面描述的策略之一。

但是，Spring AI 还支持使用 `ToolCallbackResolver` 接口在运行时动态解析 tools。

ToolCallbackResolver 接口

```java
public interface ToolCallbackResolver {
  /**
   * Resolve the {@link ToolCallback} for the given tool name.
   */
  @Nullable
  ToolCallback resolve(String toolName);
}
```

使用此方法时：

- 在客户端，您向 `ChatClient` 或 `ChatModel` 提供 tool 名称，而不是 `ToolCallback`(s)。
- 在服务器端，`ToolCallbackResolver` 实现负责将 tool 名称解析为相应的 `ToolCallback` 实例。

默认情况下，Spring AI 依赖于 `DelegatingToolCallbackResolver`，它将 tool 解析委托给 `ToolCallbackResolver` 实例列表：

- `SpringBeanToolCallbackResolver` 从类型为 `Function`、`Supplier`、`Consumer` 或 `BiFunction` 的 Spring bean 解析 tools。
- `StaticToolCallbackResolver` 从 `ToolCallback` 实例的静态列表解析 tools。使用 Spring Boot 自动配置时，此解析器会自动配置应用程序上下文中定义的所有类型为 `ToolCallback` 的 bean。

如果您依赖 Spring Boot 自动配置，可以通过提供自定义 `ToolCallbackResolver` bean 来自定义解析逻辑。

自定义 ToolCallbackResolver

```java
@Bean
ToolCallbackResolver toolCallbackResolver(List<ToolCallback> toolCallbacks) {
  StaticToolCallbackResolver staticToolCallbackResolver = new StaticToolCallbackResolver(toolCallbacks);
  return new DelegatingToolCallbackResolver(List.of(staticToolCallbackResolver));
}
```

`ToolCallbackResolver` 由 `ToolCallingManager` 内部使用，以在运行时动态解析 tools，支持框架控制的 Tool 执行和用户控制的 Tool 执行。

## 可观测性

Tool calling 包括可观测性支持，使用 spring.ai.tool 观察来测量完成时间并传播跟踪信息。请参阅 Tool Calling 可观测性。

可选地，Spring AI 可以将 tool call 参数和结果导出为 span 属性，默认情况下出于敏感性原因禁用。详细信息：Tool Call 参数和结果数据。

### 日志记录

Tool calling 功能的所有主要操作都在 `DEBUG` 级别记录。您可以通过将 `org.springframework.ai` 包的日志级别设置为 `DEBUG` 来启用日志记录。

## 自定义工具属性

### 自定义工具名称

默认情况下，工具名称来自函数名称。当你需要更具描述性的内容时，可以覆盖它：

自定义工具名称示例

```
ToolCallback searchTool = FunctionToolCallback
  .builder("web_search", new SearchFunction())  // 自定义名称
  .description("Search the web for information")
  .inputType(String.class)
  .build();

System.out.println(searchTool.getName());  // web_search
// 推荐：使用 ToolDefinition 提取名称
System.out.println(searchTool.getToolDefinition().name());  // web_search
```

### 自定义工具描述

覆盖自动生成的工具描述以提供更清晰的模型指导：

自定义工具描述示例

```
ToolCallback calculatorTool = FunctionToolCallback
  .builder("calculator", new CalculatorFunction())
  .description("Performs arithmetic calculations. Use this for any math
problems.")
  .inputType(String.class)
  .build();
```

## 高级模式定义

使用 Java 类或 JSON schemas 定义复杂的输入：

**使用 Java 记录类（Record）**：

使用 Java Record 定义复杂输入示例

```
import org.springframework.ai.tool.annotation.ToolParam;

public class WeatherFunction implements Function<WeatherFunction.WeatherInput,
String> {
    public enum Unit {
        /**
         * 摄氏度
         */
        CELSIUS,
        /**
         * 华氏度
         */
        FAHRENHEIT
    }

```

```
    @SuppressWarnings("all")
    public record WeatherInput(
            @ToolParam(description = "城市名称或坐标") String location,
            @ToolParam(description = "温度单位偏好") Unit units,
            @ToolParam(description = "是否包含5天天气预报") boolean
includeForecast
    ) {}

    @Override
    public String apply(WeatherInput input) {
        double temp = input.units() == Unit.CELSIUS ? 22 : 72;
        String result = String.format(
                "%s当前天气：%.0f度%s",
                input.location(),
                temp,
                input.units().toString().substring(0, 1).toUpperCase()
        );
        if (input.includeForecast()) {
            result += "未来5天：晴";
        }
        return result;
    }
}

ToolCallback weatherTool = FunctionToolCallback
    .builder("get_weather", new WeatherFunction())
    .description("获取当前天气和可选预报")
    .inputType(WeatherFunction.WeatherInput.class)
    .build();
```

# 访问上下文

**为什么这很重要**：当工具可以访问 Agent 状态、运行时上下文和长期记忆时，它们最强大。这使工具能够做出上下文感知的决策、个性化响应并在对话中维护信息。

工具可以通过 `ToolContext` 参数访问运行时信息，该参数提供：

- **State（状态）** - 通过执行流动的可变数据（消息、计数器、自定义字段）
- **Context（上下文）** - 不可变配置，如用户 ID、会话详细信息或应用程序特定配置
- **Store（存储）** - 跨对话的持久长期记忆
- **Config（配置）** - 执行的 RunnableConfig
- **Tool Call ID** - 当前工具调用的 ID

## ToolContext

使用 `ToolContext` 在单个参数中访问所有运行时信息。只需将 `ToolContext` 添加到你的工具签名中，它将自动注入而不会暴露给 LLM。

**访问状态**：

工具可以使用 `ToolContext` 访问当前的 Graph 状态：

使用 ToolContext 访问状态示例

```
// 访问当前对话状态
```

```
public class ConversationSummaryTool implements
    BiFunction<ConversationSummaryTool.StateRequest, ToolContext, String> {
    @SuppressWarnings("all")
    public record StateRequest(String input) {
    }

    @Override
    public String apply(ConversationSummaryTool.StateRequest request, ToolContext
toolContext) {
        OverAllState state = (OverAllState)
toolContext.getContext().get(AGENT_STATE_CONTEXT_KEY);
        // 从state中获取消息
        Optional<List<Message>> messagesOpt = state.value("messages");
        List<Message> messages = messagesOpt.orElseGet(ArrayList::new);
        if (messages.isEmpty()) {
            return "No conversation history available";
        }
        // 统计消息类型
        long countUserMsg = messages.stream()
                .filter(m -> "user".equals(m.getMessageType().getValue()))
                .count();
        long countAiMsg = messages.stream()
                .filter(m -> "assistant".equals(m.getMessageType().getValue()))
                .count();
        long countToolMsg = messages.stream()
                .filter(m -> "tool".equals(m.getMessageType().getValue()))
                .count();

        return String.format(
                "Conversation has %d user messages, %d AI responses, and %d tool
results",
                countUserMsg, countAiMsg, countToolMsg
        );
    }
}

// 创建工具
ToolCallback summaryTool = FunctionToolCallback
  .builder("summarize_conversation", new ConversationSummaryTool())
  .description("总结目前的对话内容")
  .inputType(String.class)
  .build();
```

**警告**：`toolContext` 参数对模型是隐藏的。对于上面的示例，模型只看到 `request` 在工具模式中 - `toolContext` **不**包含在请求中。

**更新状态**：

在 Spring AI Alibaba 中，你可以通过 Hook 或在工具执行后返回的信息来更新 Agent 的状态。

在 Hook 中更新状态示例

```
public class UpdateStateHook extends ModelHook {

  @Override
  public String getName() {
```

```
        return "update_state";
    }

    @Override
    public HookPosition[] getHookPositions() {
        return new HookPosition[]{HookPosition.AFTER_MODEL};
    }

    @Override
    public CompletableFuture<Map<String, Object>>
        afterModel(OverAllState state, RunnableConfig config) {
        // 更新状态
        return CompletableFuture.completedFuture(Map.of(
            "user_name", "Alice",
            "last_updated", System.currentTimeMillis()
        ));
    }
}
```

## Context（上下文）

通过 `ToolContext` 访问不可变配置和上下文数据，如用户 ID、会话详细信息或应用程序特定配置。

使用 ToolContext 访问上下文示例

```
import org.springframework.ai.chat.model.ToolContext;
import java.util.function.BiFunction;
import java.util.Map;

public class AccountInfoTool implements
    BiFunction<AccountInfoTool.AccountRequest, ToolContext, String> {
    private static final Map<String, Map<String, Object>> USER_DATABASE = Map.of(
        "user123", Map.of(
            "name", "Alice Johnson",
            "account_type", "Premium",
            "balance", 5000,
            "email", "alice@example.com"
        ),
        "user456", Map.of(
            "name", "Bob Smith",
            "account_type", "Standard",
            "balance", 1200,
            "email", "bob@example.com"
        )
    );

    @SuppressWarnings("all")
    public record AccountRequest(
        @ToolParam(description = "查询条件") String input) {
    }

    @Override
    public String apply(AccountInfoTool.AccountRequest request, ToolContext
toolContext) {
        RunnableConfig config = (RunnableConfig) toolContext
            .getContext().get(AGENT_CONFIG_CONTEXT_KEY);
```

```
        String userId = (String) config.metadata("user_id").orElse(null);
        if (userId == null) {
            return "User ID not provided";
        }
        Map<String, Object> user = USER_DATABASE.get(userId);
        if (user != null) {
            return String.format(
                "Account holder: %s\nType: %s\nBalance: $%d",
                user.get("name"),
                user.get("account_type"),
                Long.valueOf(user.get("balance").toString())
            );
        }
        return "User not found";
    }
}

// 创建工具
ToolCallback accountTool = FunctionToolCallback
  .builder("get_account_info", new AccountInfoTool())
  .description("获取当前用户的账户信息")
  .inputType(String.class)
  .build();

// 在 ReactAgent 中使用
ReactAgent agent = ReactAgent.builder()
  .name("financial_assistant")
  .model(chatModel)
  .tools(accountTool)
  .systemPrompt("你是一个财务助手")
  .build();

// 调用时传递上下文
// 注意：需要通过适当的方式传递上下文数据
RunnableConfig config = RunnableConfig.builder()
    .addMetadata("user_id", "user123")
    .build();
AssistantMessage answer = agent.call("查询当前用户信息", config);
System.out.println(answer.getText());
```

## Memory（存储）

使用存储访问跨对话的持久数据。在 Spring AI Alibaba 中，你可以使用 checkpointer 来实现长期记忆。

使用 Memory 存储示例

```
record SaveUser(
    @ToolParam(description = "用户ID") String userId,
    @ToolParam(description = "姓名") String name,
    @ToolParam(description = "年龄") int age,
    @ToolParam(description = "邮箱") String email
) {}
record GetUserRequest(
    @ToolParam(description = "用户ID") String userId
) {}
```

```java

// 存储用户信息
Map<String, SaveUser> userInfo = new HashMap<>();

// 创建保存用户信息工具
ToolCallback createSaveUserInfoTool() {
    return FunctionToolCallback
        .builder("save_user_info", (SaveUser user) -> {
            // 简化的实现
            userInfo.put(user.userId(), user);
            return "用户信息已保存: " + user;
        })
        .description("保存用户信息")
        .inputType(SaveUser.class)
        .build();
}

// 创建获取用户信息工具
ToolCallback createGetUserInfoTool() {
    return FunctionToolCallback
        .builder("get_user_info", (GetUserRequest userId) -> {
            // 简化的实现
            SaveUser user = userInfo.get(userId.userId());
            if (user == null) {
                return "用户不存在";
            }
            return user.toString();
        })
        .description("根据ID获取用户信息")
        .inputType(GetUserRequest.class)
        .build();
}

// 访问内存存储
@Test
void accessingMemoryStore() throws GraphRunnerException {
    // 创建工具
    ToolCallback saveUserInfoTool = createSaveUserInfoTool();
    ToolCallback getUserInfoTool = createGetUserInfoTool();

    // 创建带有持久化记忆的Agent
    ReactAgent agent = ReactAgent.builder()
        .name("my_agent")
        .model(chatModel)
        .tools(saveUserInfoTool, getUserInfoTool)
        .saver(new MemorySaver())
        .build();

    // 第一个会话：保存用户信息
    RunnableConfig config1 = RunnableConfig.builder()
        .threadId("session_1")
        .build();
    AssistantMessage resp1 = agent.call("保存用户：userid: abc123, name: Foo, age: 25, email: foo@example.com", config1);

    // 第二个会话：获取用户信息，注意这里用的是不同的 threadId
    RunnableConfig config2 = RunnableConfig.builder()
        .threadId("session_2")
```

```
        .build();
    AssistantMessage resp2 = agent.call("获取用户ID为 'abc' 的用户信息", config2);

    System.out.println(resp1.getText());
    System.out.println("查询结果：");
    System.out.println(resp2.getText());
}
```

# 在ReactAgent中使用

ReactAgent 提供了多种方式来提供和使用工具。根据你的使用场景，可以选择最适合的方式。

## 工具提供方式

### 1. 直接工具

最直接的方式是使用 `tools()` 方法直接传入 `ToolCallback` 实例。这种方式适合工具数量较少、工具定义明确的场景。

使用 tools() 方法提供工具

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.function.FunctionToolCallback;

// 创建工具
ToolCallback weatherTool = FunctionToolCallback
  .builder("get_weather", new WeatherFunction())
  .description("Get weather for a given city")
  .inputType(WeatherInput.class)
  .build();

ToolCallback searchTool = FunctionToolCallback
  .builder("search", new SearchFunction())
  .description("Search for information")
  .inputType(String.class)
  .build();

// 使用 tools() 方法直接提供工具
ReactAgent agent = ReactAgent.builder()
  .name("my_agent")
  .model(chatModel)
  .tools(weatherTool, searchTool)  // 直接传入 ToolCallback 实例
  .systemPrompt("You are a helpful assistant with access to weather and search
tools.")
  .saver(new MemorySaver())
  .build();
```

**适用场景**:

- 工具数量较少（通常少于 5 个）
- 工具定义在编译时已知
- 需要类型安全的工具定义

### 2. 方法工具

使用 `methodTools()` 方法传入带有 `@Tool` 注解方法的对象。这种方式让工具定义更加简洁，适合将工具逻辑组织在类中。

使用 methodTools() 方法提供工具

```java
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.ai.tool.annotation.ToolParam;

// 定义工具类，使用 @Tool 注解
public class CalculatorTools {
  @Tool(description = "Add two numbers together")
  public String add(
          @ToolParam(description = "First number") int a,
          @ToolParam(description = "Second number") int b) {
      return String.valueOf(a + b);
  }

  @Tool(description = "Multiply two numbers together")
  public String multiply(
          @ToolParam(description = "First number") int a,
          @ToolParam(description = "Second number") int b) {
      return String.valueOf(a * b);
  }
}

// 使用 methodTools() 方法
CalculatorTools calculatorTools = new CalculatorTools();

ReactAgent agent = ReactAgent.builder()
  .name("calculator_agent")
  .model(chatModel)
  .description("An agent that can perform calculations")
  .instruction("You are a helpful calculator assistant.")
  .methodTools(calculatorTools)  // 传入带有 @Tool 注解方法的对象
  .saver(new MemorySaver())
  .build();

// 可以传入多个 methodTools 对象
WeatherTools weatherTools = new WeatherTools();
ReactAgent multiAgent = ReactAgent.builder()
  .name("multi_tool_agent")
  .model(chatModel)
  .methodTools(calculatorTools, weatherTools)  // 多个工具对象
  .build();
```

**适用场景**:

- 工具逻辑组织在类中
- 需要将相关工具分组
- 工具方法需要访问类成员变量

### 3. 工具提供者

使用 `ToolCallbackProvider` 接口动态提供工具。这种方式适合需要根据运行时条件动态决定提供哪些工具的场景。

使用 toolCallbackProviders() 方法提供工具

```java
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.ToolCallbackProvider;
import org.springframework.ai.tool.function.FunctionToolCallback;
import java.util.List;

// 实现 ToolCallbackProvider 接口
public class CustomToolCallbackProvider implements ToolCallbackProvider {
  private final List<ToolCallback> toolCallbacks;

  public CustomToolCallbackProvider(List<ToolCallback> toolCallbacks) {
      this.toolCallbacks = toolCallbacks;
  }

  @Override
  public ToolCallback[] getToolCallbacks() {
      return toolCallbacks.toArray(new ToolCallback[0]);
  }
}

// 创建工具
ToolCallback searchTool = FunctionToolCallback.builder("search", new
SearchToolWithContext())
  .description("Search for information")
  .inputType(String.class)
  .build();

// 创建 ToolCallbackProvider
ToolCallbackProvider toolProvider = new
CustomToolCallbackProvider(List.of(searchTool));

// 使用 toolCallbackProviders() 方法
ReactAgent agent = ReactAgent.builder()
  .name("search_agent")
  .model(chatModel)
  .description("An agent that can search for information")
  .instruction("You are a helpful assistant with search capabilities.")
  .toolCallbackProviders(toolProvider)  // 使用 ToolCallbackProvider
  .saver(new MemorySaver())
  .build();
```

**适用场景**:

- 需要根据运行时条件动态提供工具
- 工具来自外部系统或配置
- 需要实现工具的动态加载和卸载

## 4. 工具名称解析

使用 `toolNames()` 方法指定工具名称，配合 `resolver()` 方法提供的 `ToolCallbackResolver` 来解析工具。这种方式适合工具定义和工具使用分离的场景。

使用 toolNames() 和 resolver() 方法提供工具

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.function.FunctionToolCallback;
import org.springframework.ai.tool.resolution.StaticToolCallbackResolver;
import java.util.List;

// 创建工具（使用复合类型）
ToolCallback searchTool = FunctionToolCallback.builder("search", new
SearchFunctionWithRequest())
  .description("Search for information")
  .inputType(SearchRequest.class)
  .build();

ToolCallback calculatorTool = FunctionToolCallback.builder("calculator", new
CalculatorFunctionWithRequest())
  .description("Perform arithmetic calculations")
  .inputType(CalculatorRequest.class)
  .build();

// 创建 StaticToolCallbackResolver，包含所有工具
StaticToolCallbackResolver resolver = new StaticToolCallbackResolver(
  List.of(calculatorTool, searchTool));

// 使用 toolNames() 指定要使用的工具名称，必须配合 resolver() 使用
ReactAgent agent = ReactAgent.builder()
  .name("multi_tool_agent")
  .model(chatModel)
  .description("An agent with multiple tools")
  .instruction("You are a helpful assistant with access to calculator and search
tools.")
  .toolNames("calculator", "search")  // 使用工具名称而不是 ToolCallback 实例
  .resolver(resolver)  // 必须提供 resolver 来解析工具名称
  .saver(new MemorySaver())
  .build();
```

**重要提示**：`toolNames()` 方法必须与 `resolver()` 方法配合使用，否则会抛出异常。

**适用场景**：

- 工具定义和工具使用分离
- 需要从配置或外部系统读取工具名称
- 工具可能动态变化，但名称保持稳定

### 5. 工具解析器

直接使用 `resolver()` 方法提供 `ToolCallbackResolver`。解析器可以用于工具节点，也可以与 `toolNames()` 配合使用。

使用 resolver() 方法提供工具

```java
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.function.FunctionToolCallback;
import org.springframework.ai.tool.resolution.StaticToolCallbackResolver;
import java.util.List;

// 创建工具
ToolCallback calculatorTool = FunctionToolCallback.builder("calculator", new
CalculatorFunctionWithContext())
  .description("Perform arithmetic calculations")
  .inputType(String.class)
  .build();

// 创建 resolver
StaticToolCallbackResolver resolver = new StaticToolCallbackResolver(
  List.of(calculatorTool));

// 使用 resolver，可以直接在 tools 中使用，也可以仅通过 resolver 提供
ReactAgent agent = ReactAgent.builder()
  .name("resolver_agent")
  .model(chatModel)
  .description("An agent using ToolCallbackResolver")
  .instruction("You are a helpful calculator assistant.")
  .tools(calculatorTool)  // 直接指定工具
  .resolver(resolver)  // 同时设置 resolver 供工具节点使用
  .saver(new MemorySaver())
  .build();
```

**适用场景**:

- 需要自定义工具解析逻辑
- 工具来自多个来源需要统一管理
- 需要实现工具的动态查找和加载

### 6. 组合使用多种方式

你可以同时使用多种工具提供方式，ReactAgent 会将它们合并。

组合使用多种工具提供方式

```java
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.ToolCallbackProvider;
import org.springframework.ai.tool.function.FunctionToolCallback;
import java.util.List;

// Method tools
CalculatorTools calculatorTools = new CalculatorTools();

// Direct tool
ToolCallback searchTool = FunctionToolCallback.builder("search", new
SearchToolWithContext())
  .description("Search for information")
  .inputType(String.class)
  .build();

```

```
// ToolCallbackProvider
ToolCallbackProvider toolProvider = new
CustomToolCallbackProvider(List.of(searchTool));

// 组合使用多种方式
ReactAgent agent = ReactAgent.builder()
  .name("combined_tool_agent")
  .model(chatModel)
  .description("An agent with multiple tool provision methods")
  .instruction("You are a helpful assistant with calculator and search
capabilities.")
  .methodTools(calculatorTools)  // Method-based tools
  .toolCallbackProviders(toolProvider)  // Provider-based tools
  .tools(searchTool)  // Direct tools
  .saver(new MemorySaver())
  .build();
```

**适用场景**:

- 工具来自不同来源
- 需要灵活组合不同类型的工具
- 逐步迁移或扩展现有工具集

## 选择建议

根据你的具体需求选择合适的工具提供方式:

| 方式 | 适用场景 | 优点 | 缺点 |
| --- | --- | --- | --- |
| `tools()` | 工具数量少、定义明确 | 简单直接、类型安全 | 工具多时代码冗长 |
| `methodTools()` | 工具逻辑组织在类中 | 代码组织清晰、易于维护 | 需要创建工具类 |
| `toolCallbackProviders()` | 动态提供工具 | 灵活、支持运行时决策 | 需要实现接口 |
| `toolNames()` + `resolver()` | 工具定义和使用分离 | 解耦、支持配置化 | 必须配合 resolver |
| `resolver()` | 自定义解析逻辑 | 高度灵活 | 需要实现解析器 |
| 组合使用 | 复杂场景 | 最大灵活性 | 可能增加复杂度 |

## 基础使用示例

在 ReactAgent 中使用工具示例

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.function.FunctionToolCallback;

// 创建工具
ToolCallback weatherTool = FunctionToolCallback
  .builder("get_weather", new WeatherFunction())
  .description("Get weather for a given city")
  .inputType(String.class)
  .build();

ToolCallback searchTool = FunctionToolCallback
```

```
  .builder("search", new SearchFunction())
  .description("Search for information")
  .inputType(String.class)
  .build();

// 创建带有工具的 Agent
ReactAgent agent = ReactAgent.builder()
  .name("my_agent")
  .model(chatModel)
  .tools(weatherTool, searchTool)
  .systemPrompt("You are a helpful assistant with access to weather and search tools.")
  .saver(new MemorySaver())
  .build();

// 使用 Agent
AssistantMessage response = agent.call("What's the weather like in San Francisco?");
System.out.println(response.getText());
```

## 远程MCP工具调用示例

在实际的 React Agent 应用中，我们多数情况需要接入三方平台或其他应用所提供的 MCP 工具，且目前已经有众多的 MCP 工具平台，大多数工具不需要自己手动定义，你可以通过构建 MCP Client 来使用远端 MCP Server 平台（ModelScope、百炼等）提供的工具。

参考mcp广场：

- 魔塔：https://modelscope.cn/mcp
- 百炼：https://bailian.console.aliyun.com/cn-beijing#/mcp-market

### 1. 基于 SpringBoot

基于SpringBoot的自动发现，构建 MCPClient为ReactAgent提供工具

在使用这种方式前，你必须引入以下依赖：

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-mcp-client</artifactId>
</dependency>
```

并且注入相关 MCP 的配置项：

```
spring:
  ai:
    # 结合Spring AI使用MCP的必须配置
    mcp:
      client:
        name: saa-mcp-client
        type: async
        # 定义streamableHttp的MCP工具
        streamable-http:
          connections:
```

```
            amap-maps:
              # 从魔搭社区获取调用url，移除尾部endpoint,例如：https://mcp.api-inference.modelscope.net/?????????/
              url: ${MODEL_SCOPE_AMAP_BASE_URL}
              endpoint: mcp
        # 定义基于sse的MCP工具
        sse:
          connections:
            12306-mcp:
              # 从魔搭社区获取调用url，移除尾部endpoint,例如：https://mcp.api-inference.modelscope.net/?????????/
              url: ${MODEL_SCOPE_12306_BASE_URL}
              sse-endpoint: sse
```

在配置完成后，你可以很容易地在 React Agent 中使用这些远程的 MCP 工具。

基于 Spring Boot 的自动发现构建 MCP Client 为 React Agent 提供工具

```
@Service
public class RemoteMcpToolsExample {
    private final ToolCallbackProvider toolCallbackProvider;
    private final ChatModel chatModel;

    public RemoteMcpToolsExample
        (ToolCallbackProvider toolCallbackProvider, ChatModel chatModel) {
        this.toolCallbackProvider = toolCallbackProvider;
        this.chatModel = chatModel;
    }

    /**
     * 执行React Agent
     * @param instruction             指令
     * @param chatModel               聊天模型
     * @param travel_planning_session 会话ID
     * @param message                 对话内容
     * @param toolCallbackProvider    工具回调提供者
     * @param toolCallback            工具回调
     * @return 执行结果
     * @throws GraphRunnerException 运行错误
     */
    public static String executeAgent(String instruction,
                                      ChatModel chatModel,
                                      String travel_planning_session,
                                      String message,
                                      ToolCallbackProvider toolCallbackProvider,
                                      ToolCallback... toolCallback) throws GraphRunnerException {
        // 构建带有MCP工具的ReactAgent
        Builder builder = ReactAgent.builder()
            .name("旅行规划助手")
            .model(chatModel)
            .description("您的旅行助理")
            .instruction(instruction)
            .saver(new MemorySaver());

```

```
        //═══════════════════════════════添加工具═══════════════════════════════
        if (toolCallbackProvider != null) {
            // 从Spring AI ToolCallbackProvider获取工具，这些工具在application.yml中配置
            ToolCallback[] toolCallbacks = toolCallbackProvider.getToolCallbacks();
            System.out.println("═══════从SpringToolCallbackProvider获取工具═══════");
            for (ToolCallback callback : toolCallbacks) {
                System.out.println(JSON.toJSONString(callback));
            }
            System.out.println("═══════从SpringToolCallbackProvider获取工具═══════");
            builder.toolCallbackProviders(toolCallbackProvider);
        } else {
            builder.tools(toolCallback);
        }
        //═══════════════════════════════添加工具═══════════════════════════════

        // 创建React Agent
        ReactAgent agent = builder.build();
        RunnableConfig config = RunnableConfig.builder()
            .threadId(travel_planning_session)
            .build();

        // 通过stream函数进行会话
        Flux<NodeOutput> stream = agent.stream(message, config);
        StringBuffer answerString = new StringBuffer();
        stream
            .doOnNext(output -> {
                if ("_AGENT_MODEL_".equals(output.node())) {
                    answerString
                        .append(((StreamingOutput<?>) output)
                                .message().getText());
                } else if ("_AGENT_TOOL_".equals(output.node())) {
                    answerString.append("\nTool Call:")
                        .append(((ToolResponseMessage)
                                 ((StreamingOutput<?>) output).message())
                                .getResponses().get(0))
                        .append("\n");
                }
            })
            .blockLast();
        return answerString.toString();
    }

    /**
     * 基于Spring Boot使用远端MCP工具 -- React Agent
     */
    public void remoteMcpToolsReactWithSpringBootExample()
        throws GraphRunnerException {
        String answerString = executeAgent(
            "您是一位乐于助人的助手，擅长旅行路线规划和火车票查询。",
            chatModel,
            "travel_planning_session",
            """
            我计划明天从上海前往北京旅行。
            1. 请帮我查询这趟旅程可以在哪些车站下车（即可用的到达/下车站点）。
```

```
            2. 请帮我查询可用的车次及其发车时间。
            3. 请同时查询北京明天的天气预报。
            """,
            toolCallbackProvider);
        try {
            if (!answerString.isEmpty()) {
                String refineInstruction =
                    "请整理一下旅行规划信息，去除重复内容，保持条理清晰：\n" +
                    answerString;
                String refinedAnswer = chatModel.call(refineInstruction);
                System.out.println("\n========== 整理后的最终回答 ==========");
                System.out.println(refinedAnswer);
            }
        } catch (Exception e) {
            System.err.println("整理回答时出错: " + e.getMessage());
            // 如果整理失败，至少保留原始输出
            System.out.println("\n========== 原始回答（未整理） ==========");
            System.out.println(answerString);
        }
    }
}
```

## 2. 使用 MCP SDK

如果项目不依赖于 Spring Boot 脚手架，你也可以通过如下方式自行构建可供 React Agent 使用的工具。

使用 MCP SDK 为 React Agent 提供工具

```
@Slf4j
public class RemoteMcpToolsExampleSdk {
    /**
     * 将MCP工具转换为Spring AI的ToolCallback
     * @param mcpTool    MCP工具定义，包含名称、描述等信息
     * @param mcpClient  MCP同步客户端，用于实际调用远程工具
     * @param serverName 服务器名称，用于日志标识
     * @return 转换后的ToolCallback对象
     */
    private static ToolCallback createToolCallback(
        McpSchema.Tool mcpTool,
        McpSyncClient mcpClient,
        String serverName) {
        return FunctionToolCallback.builder(
            mcpTool.name(),
            (Map<String, Object> functionInput) -> {
                try {
                    // 构建工具请求
                    log.debug("[{}] Call MCP Tool: {} With Input: {}",
                              serverName, mcpTool.name(), functionInput);
                    McpSchema.CallToolRequest callRequest =
                        new McpSchema.CallToolRequest(
                        mcpTool.name(), functionInput);

                    // 调用工具
                    McpSchema.CallToolResult callResult =
                        mcpClient.callTool(callRequest);
```

```
28
29                      // 获取返回结果
30                      StringBuilder resultBuilder = new StringBuilder();
31                      for (McpSchema.Content content : callResult.content()) {
32                          if (content instanceof McpSchema.TextContent
textContent) {
33                              resultBuilder.append(textContent.text());
34                          }
35                      }
36
37                      String result = resultBuilder.toString();
38                      log.debug("[{}] MCP Tool Return: {}", serverName, result);
39                      return result;
40                  } catch (Exception e) {
41                      log.error("[{}] Call MCP Tool Failed: {}",
42                                serverName, mcpTool.name(), e);
43                      return "{\"error\": \"" + e.getMessage() + "\"}";
44                  }
45              })
46              .description(mcpTool.description())
47              .inputType(Map.class)
48              .build();
49      }
50
51      /**
52       * 从指定的 MCP 服务器获取工具并转换为 ToolCallback 列表
53       * @param baseUrl        MCP 服务器的基础 URL
54       * @param endpoint       SSE 端点路径（如 "sse"）
55       * @param httpBuilder    HTTP 客户端构建器
56       * @param clientsToClose 需要关闭的客户端列表（用于资源管理）
57       * @param serverName     服务器名称（用于日志和工具名称前缀）
58       * @return ToolCallback 列表
59       */
60      @SuppressWarnings("all")
61      private static List<ToolCallback> fetchMcpTools(
62          String baseUrl,
63          String endpoint,
64          HttpClient.Builder httpBuilder,
65          List<McpSyncClient> clientsToClose,
66          String serverName,
67          boolean isStreamable) {
68          List<ToolCallback> toolCallbacks = new ArrayList<>();
69          McpSyncClient mcpClient;
70          try {
71              // 创建MCP客户端传输层
72              McpClientTransport mcpClientTransport;
73              if (isStreamable) {
74                  ObjectMapper objectMapper = new ObjectMapper();
75                  mcpClientTransport = HttpClientStreamableHttpTransport
76                      .builder(baseUrl)
77                      .endpoint(endpoint)
78                      .clientBuilder(HttpClient.newBuilder())
79                      .jsonMapper(new JacksonMcpJsonMapper(objectMapper)).build();
80
81              } else {
82                  mcpClientTransport =
HttpClientSseClientTransport.builder(baseUrl)
83                      .clientBuilder(httpBuilder)
```

```
                    .sseEndpoint(endpoint)
                    .build();
            }

            // 构建 MCP 客户端
            mcpClient = McpClient.sync(mcpClientTransport)
                .requestTimeout(Duration.ofSeconds(60))
                .initializationTimeout(Duration.ofSeconds(60))
                .capabilities(McpSchema.ClientCapabilities
                .builder()
                .roots(true)
                .build())
                .build();

            clientsToClose.add(mcpClient);

            // 初始化MCP客户端
            log.info("[{}] Initialize MCP Client ...", serverName);
            McpSchema.InitializeResult initResult = mcpClient.initialize();
            log.info("[{}] MCP Client Initialize "+
                     "Successful: serverInfo={}, capabilities={}",
                     serverName, initResult.serverInfo(),
                     initResult.capabilities());

            // 获取工具列表
            log.info("[{}] Get Tools ...", serverName);
            McpSchema.ListToolsResult toolsResult = mcpClient.listTools();
            List<McpSchema.Tool> mcpTools = toolsResult.tools();

            log.info("[{}] Found {} Tools From MCP Server",
                     serverName, mcpTools.size());

            // 注册为ToolCallback
            for (McpSchema.Tool mcpTool : mcpTools) {
                log.info("[{}] Register MCP Tool: name={}, description={}",
                         serverName, mcpTool.name(), mcpTool.description());
                ToolCallback toolCallback =
                    createToolCallback(mcpTool, mcpClient, serverName);
                toolCallbacks.add(toolCallback);
            }
        } catch (Exception e) {
            log.error("[{}] Fetch MCP Tools Error", serverName, e);
        }
        return toolCallbacks;
    }

    /**
     * 解耦 Spring Boot 使用远端MCP工具 -- React Agent
     *
     * <p>
     * 不使用 Spring 依赖注入的情况下，直接使用 MCP 客户端
     * 获取远程工具，并将其转换为 Spring AI 的 ToolCallback，最后在 ReactAgent 中
使用。
     * </p>
     * <p>关键步骤：</p>
     * <ol>
     *   <li>创建 MCP 客户端传输层 (HttpClientSseClientTransport)</li>
     *   <li>构建并初始化 MCP 同步客户端</li>
```

```
     *   <li>调用 listTools() 获取远程服务器的工具列表</li>
     *   <li>将 MCP 工具转换为 Spring AI 的 ToolCallback</li>
     * </ol>
     * @param chatModel 聊天模型实例
     */
    public static void remoteMcpToolsReactWithoutSpringBootExample
        (ChatModel chatModel) {
        // 定义 MCP 服务器基础 URL
        String modelScope12306BaseUrlSse =
            "https://mcp.api-inference.modelscope.net/c7350a753c1646/";
        String modelScopeAmapBaseUrlSse =
            "https://mcp.api-inference.modelscope.net/7c929e3bb1584e/";

        // 创建可复用的 HTTP 客户端构建器
        HttpClient.Builder httpBuilder = HttpClient.newBuilder()
            .version(HttpClient.Version.HTTP_1_1)
            .connectTimeout(Duration.ofSeconds(60));

        List<ToolCallback> toolCallbacks = new ArrayList<>();
        List<McpSyncClient> clientsToClose = new ArrayList<>();
        try {
            // 获取 12306 MCP 工具
            List<ToolCallback> tools12306 = fetchMcpTools(
                modelScope12306BaseUrlSse,
                "sse",
                httpBuilder,
                clientsToClose,
                "12306", false
            );
            toolCallbacks.addAll(tools12306);

            // 获取高德地图 MCP 工具
            List<ToolCallback> toolsAmap = fetchMcpTools(
                modelScopeAmapBaseUrlSse,
                "sse",
                httpBuilder,
                clientsToClose,
                "amap", false
            );
            toolCallbacks.addAll(toolsAmap);

            // 打印找到的工具数量（也可将这些 ToolCallback 封装为
ToolCallbackProvider）
            System.out.printf("""
                              ==========从 MCP 服务器发现工具==========
                              共从 MCP 服务器找到 %d 个工具
                              """,
                              toolCallbacks.size());

            // 构建包含 MCP 工具的 React Agent 并执行
            String resp = RemoteMcpToolsExample.executeAgent(
                "你是一个拥有旅行路线规划、火车票查询和地图服务能力的得力助手。",
                chatModel,
                "travel_planning_session_no_spring",
                """
                我计划明天从上海去北京旅行。
                1. 请帮我查询行程中可以下车的车站（即可用的到达/下车站点）。
                2. 请查询可用的车次及其发车时间。
```

```
                3. 请同时查询北京明天的天气预报。
                4. 请帮我规划从北京南站到天安门广场的路线。
                """,
                null,
                toolCallbacks.toArray(new ToolCallback[0]));
            if (!resp.isEmpty()) {
                String refineInstruction =
                    "请整理一下旅行规划信息，去除重复内容，保持条理清晰：\n" +
resp;
                String refinedAnswer = chatModel.call(refineInstruction);
                System.out.println("\n======== 整理后的最终回答 ========");
                System.out.println(refinedAnswer);
            }
        } catch (Exception e) {
            log.error("execute MCP Agent error", e);
        } finally {
            // 关闭所有 MCP 客户端连接
            for (McpSyncClient client : clientsToClose) {
                try {
                    if (client != null) {
                        client.close();
                        log.info("MCP Client Is Closed");
                    }
                } catch (Exception e) {
                    log.warn("Close MCP Client Error", e);
                }
            }
        }
    }
}
```

### 3. 为ChatClient提供

结合示例 1 和示例 2，你可以通过 Spring Boot 或 任意一种可以构建 ToolCallbackProvider 的方式来为 ChatClient 提供 MCP 工具。

为 ChatClient 提供 MCP 工具

```
@Service
public class RemoteMcpToolsExample {
    //.....

    /**
     * 基于SpringBoot使用远端MCP工具 -- Only ChatClient
     */
    public void remoteMcpToolsWithChatCliAndSpringBootExample() {
        // 从Spring AI ToolCallbackProvider获取工具，这些工具在application.yml中配
置
        ToolCallback[] toolCallbacks = toolCallbackProvider.getToolCallbacks();
        System.out.println("========从SpringToolCallbackProvider获取工具
========");
        for (ToolCallback callback : toolCallbacks) {
            System.out.println(JSON.toJSONString(callback));
        }
        System.out.println("========从SpringToolCallbackProvider获取工具
========");
```

```
16
17          // 创建ChatClient
18          ChatClient chatClient = ChatClient
19              .builder(chatModel)
20              .build();
21          // 创建ChatClient请求
22          ChatClient.ChatClientRequestSpec doChat =
23              chatClient
24              .prompt("您是一位乐于助人的助手，擅长旅行路线规划和火车票查询。")
25              .user("""
26                    我计划明天从上海前往北京旅行。
27                    1. 请帮我查询这趟旅程可以在哪些车站下车（即可用的到达/下车站点）。
28                    2. 请帮我查询可用的车次及其发车时间。
29                    3. 请同时查询北京明天的天气预报。
30                    """)
31                .toolCallbacks(toolCallbackProvider);
32          // 查看DefaultToolCallingManager的日志
33          String text =
    doChat.call().chatResponse().getResult().getOutput().getText();
34          System.out.println(text);
35      }
36  }
```

# Memory

## 概述

记忆可以让 Agent 记住之前的会话内容。对于 AI Agent，记忆至关重要，因为它让它们能够记住先前的交互、从反馈中学习并适应用户偏好。随着 Agent 处理更复杂的任务和大量用户交互，这种能力对于效率和用户满意度都变得至关重要。

短期记忆让你的应用程序能够在单个线程或会话中记住先前的交互。

> **注意**：会话可以隔离同一个 Agent 实例中的多个不同交互，类似于电子邮件在单个对话中分组消息的方式。

## 理解ReactAgent中的短期记忆

Spring AI Alibaba 将短期记忆作为 Agent 状态的一部分进行管理。

通过将这些存储在 Graph 的状态中，Agent 可以访问给定对话的完整上下文，同时保持不同对话之间的分离。状态使用 checkpointer 持久化到数据库（或内存），以便可以随时恢复线程。短期记忆在调用 Agent 或完成步骤（如工具调用）时更新，并在每个步骤开始时读取状态。

## 记忆带来的上下文过长问题

保留所有对话历史是实现短期记忆最常见的形式。但较长的对话对历史可能会导致大模型 LLM 上下文窗口超限，导致上下文丢失或报错。

即使你在使用的大模型上下文长度足够大，大多数模型在处理较长上下文时的表现仍然很差。因为很多模型会被过时或偏离主题的内容"分散注意力"。同时，过长的上下文，还会带来响应时间变长、Token 成本增加等问题。

在 Spring AI ALibaba 中，ReactAgent 使用 messages 记录和传递上下文，其中包括指令（SystemMessage）和输入（UserMessage）。在 ReactAgent 中，消息（Message）在用户输入和模型响应之间交替，导致消息列表随着时间的推移变得越来越长。由于上下文窗口有限，许多应用程序可以从使用技术来移除或"忘记"过时信息中受益，即 “上下文工程”。

# 使用方法

在 Spring AI Alibaba 中，要向 Agent 添加短期记忆（会话级持久化），你需要在创建 Agent 时指定 `checkpointer`。

配置短期记忆示例

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;

import com.alibaba.cloud.ai.graph.checkpoint.savers.MemorySaver;
import com.alibaba.cloud.ai.graph.RunnableConfig;

// 配置 checkpointer
ReactAgent agent = ReactAgent.builder()
  .name("my_agent")
  .model(chatModel)
  .tools(getUserInfoTool)
  .saver(new MemorySaver())
  .build();

// 使用 thread_id 维护对话上下文
RunnableConfig config = RunnableConfig.builder()
  .threadId("1") // threadId 指定会话 ID
  .build();

agent.call("你好！我叫 Bob。", config);
```

## 在生产环境中

在生产环境中，使用数据库支持的checkpointer：

**示例：使用 Redis Checkpointer**：

使用 Redis Checkpointer 示例

```
import com.alibaba.cloud.ai.graph.checkpoint.savers.RedisSaver;
import org.redisson.api.RedissonClient;

// 配置 Redis checkpointer
RedisSaver redisSaver = new RedisSaver(redissonClient);

ReactAgent agent = ReactAgent.builder()
  .name("my_agent")
  .model(chatModel)
  .tools(getUserInfoTool)
  .saver(redisSaver)
  .build();
```

# 自定义Agent记忆

默认情况下，Agent 使用状态通过 `messages` 键管理短期记忆，特别是对话历史。

你可以通过在工具或 Hook 中访问和修改状态来扩展记忆功能。

自定义记忆 Hook 示例

```java
import com.alibaba.cloud.ai.graph.agent.hook.ModelHook;
import com.alibaba.cloud.ai.graph.agent.hook.HookPosition;
import com.alibaba.cloud.ai.graph.OverAllState;
import com.alibaba.cloud.ai.graph.RunnableConfig;
import org.springframework.ai.chat.messages.Message;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.CompletableFuture;

// 在 Hook 中访问和修改状态
public class CustomMemoryHook extends ModelHook {

  @Override
  public String getName() {
      return "custom_memory";
  }

  @Override
  public HookPosition[] getHookPositions() {
      return new HookPosition[]{HookPosition.BEFORE_MODEL};
  }

  @Override
  public CompletableFuture<Map<String, Object>>
      beforeModel(OverAllState state, RunnableConfig config) {
      // 访问消息历史
      Optional<Object> messagesOpt = state.value("messages");
      if (messagesOpt.isPresent()) {
          List<Message> messages = (List<Message>) messagesOpt.get();
          // 处理消息 ...
      }

      // 添加自定义状态
      return CompletableFuture.completedFuture(Map.of(
          "user_id", "user_123",
          "preferences", Map.of("theme", "dark")
      ));
  }

  @Override
  public CompletableFuture<Map<String, Object>>
      afterModel(OverAllState state, RunnableConfig config) {
      return CompletableFuture.completedFuture(Map.of());
  }
}
```

## 常见模式

启用短期记忆后，长对话可能超过 LLM 的上下文窗口。常见的解决方案包括：

- 修剪消息。在调用 LLM 之前移除前 N 条或后 N 条消息
- 删除消息。从 Graph 状态中永久删除消息
- 总结消息。总结历史中较早的消息并用摘要替换它们
- 自定义策略。自定义策略（例如消息过滤等）

这允许 Agent 在 reasoning-acting 循环中持续跟踪对话而不超过 LLM 的上下文窗口。

## 修剪消息

大多数 LLM 都有最大支持的上下文窗口（以 token 计）。

决定何时截断消息的一种方法是计算消息历史中的 token 数量，并在接近该限制时进行截断。

要在 Agent 中修剪消息历史，请使用 `MessagesModelHook`：

MessageTrimmingHook 修剪消息示例

```
@HookPositions({HookPosition.BEFORE_MODEL})
public class MessageTrimmingHook extends MessagesModelHook {

    private static final int MAX_MESSAGES = 3;

    @Override
    public String getName() {
        return "message_trimming";
    }

    @Override
    public AgentCommand beforeModel
        (List<Message> previousMessages, RunnableConfig config) {
        // 保留第一条 + 最后keepLast条 = MAX_MESSAGES条
        if (previousMessages.size() > MAX_MESSAGES) {
            int keepLast = MAX_MESSAGES - 1;
            List<Message> trimmedMessages = new ArrayList<>();
            // 第一条
            trimmedMessages.add(previousMessages.get(0));
            // 最后keepLast条
            int fromIndex = Math.max(1, previousMessages.size() - keepLast);
            trimmedMessages
                .addAll(previousMessages.subList(fromIndex,
previousMessages.size()));
            return new AgentCommand(trimmedMessages, UpdatePolicy.REPLACE);
        }
        // 否则，无需更改
        return new AgentCommand(previousMessages);
    }
}

// 使用
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
```

```
    .hooks(new MessageTrimmingHook())
    .saver(new MemorySaver())
    .build();

RunnableConfig config = RunnableConfig.builder()
    .threadId("1")
    .build();

agent.call("你好，我叫阿伟", config);
agent.call("写一首关于猫的短诗", config);
agent.call("现在对狗做同样的事情", config);
agent.call("现在对鸡做同样的事情", config);

// 多轮对话后看是否仍然有记忆
AssistantMessage finalResponse = agent.call("我叫什么名字？", config);
System.out.println(finalResponse.getText());
```

## 删除消息

你可以从 Graph 状态中删除消息以管理消息历史。

这在你想要删除特定消息或清除整个消息历史时很有用。

要从 Graph 状态中删除消息，你可以在 Hook 中返回新的消息列表：

MessageDeletionHook 删除消息示例

```
@HookPositions({HookPosition.AFTER_MODEL})
public class MessageDeletionHook extends MessagesModelHook {
    @Override
    public String getName() {
        return "message_deletion";
    }
    @Override
    public AgentCommand afterModel
        (List<Message> previousMessages, RunnableConfig config) {
        // 删除最早的两条消息，只保留剩余的消息
        if (previousMessages.size() > 2) {
            List<Message> remainingMessages = previousMessages
                .subList(2, previousMessages.size());
            return new AgentCommand(remainingMessages, UpdatePolicy.REPLACE);
        }
        // 如果消息数量小于等于2，则不进行删除
        return new AgentCommand(previousMessages);
    }
}

// 使用
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .systemPrompt("请简洁明了。")
    .hooks(new MessageDeletionHook())
    .saver(new MemorySaver())
    .build();
RunnableConfig config = RunnableConfig.builder()
```

```
    .threadId("1")
    .build();
agent.call("你好！我是阿伟", config);
agent.call("给我写一首诗吧", config);
// 多轮对话后看看是否还有我的名字
AssistantMessage resp = agent.call("我叫什么名字？", config);
System.out.println(resp.getText());
```

**删除所有消息**：

ClearAllMessagesHook 删除所有消息示例

```
@HookPositions({HookPosition.AFTER_MODEL})
public class ClearAllMessagesHook extends MessagesModelHook {

  @Override
  public String getName() {
      return "clear_all_messages";
  }

  @Override
  public AgentCommand afterModel
      (List<Message> previousMessages, RunnableConfig config) {
      // 删除所有消息，返回空列表
      return new
          AgentCommand(new ArrayList<>(), UpdatePolicy.REPLACE);
  }
}
```

**警告**：删除消息时，**确保**生成的消息历史有效。检查你使用的 LLM 提供商的限制。例如：

- 某些提供商期望消息历史以 `user` 消息开始
- 大多数提供商要求带有工具调用的 `assistant` 消息后跟相应的 `tool` 结果消息

## 总结消息

如上所示，修剪或删除消息的问题在于你可能会丢失消息队列淘汰的信息。因此，一些应用程序受益于使用聊天模型总结消息历史的更复杂方法。

要在 Agent 中总结消息历史，可以使用自定义 Hook：

MessageSummarizationHook 总结消息示例

```
@HookPositions({HookPosition.BEFORE_MODEL})
public class MessageSummarizationHook extends MessagesModelHook {
    private final ChatModel summaryModel;
    private final int maxTokensBeforeSummary;
    private final int messagesToKeep;

    public MessageSummarizationHook(
            ChatModel summaryModel,
            int maxTokensBeforeSummary,
            int messagesToKeep
    ) {
        this.summaryModel = summaryModel;
```

```
        this.maxTokensBeforeSummary = maxTokensBeforeSummary;
        this.messagesToKeep = messagesToKeep;
    }

    @Override
    public String getName() {
        return "message_summarization";
    }

    @Override
    public AgentCommand beforeModel
        (List<Message> previousMessages, RunnableConfig config) {
        // 估算 token 数量（简化版）
        int estimatedTokens = previousMessages.stream()
                .mapToInt(m -> m.getText().length() / 4)
                .sum();

        if (estimatedTokens < maxTokensBeforeSummary) {
            // 如果 token 数量未超过阈值，无需总结
            return new AgentCommand(previousMessages);
        }

        // 需要总结
        int messagesToSummarize = previousMessages.size()
            - messagesToKeep;
        if (messagesToSummarize ≤ 0) {
            // 如果消息数量不足以总结，无需更改
            return new AgentCommand(previousMessages);
        }

        List<Message> oldMessages =
            previousMessages.subList(0, messagesToSummarize);
        List<Message> recentMessages = previousMessages.subList(
                messagesToSummarize,
                previousMessages.size()
        );

        // 生成摘要
        String summary = generateSummary(oldMessages);

        // 创建摘要消息
        SystemMessage summaryMessage = new SystemMessage(
                "## 之前对话摘要:\n" + summary
        );

        // 构建新的消息列表：摘要消息 + 最近的消息
        List<Message> newMessages = new ArrayList<>();
        newMessages.add(summaryMessage);
        newMessages.addAll(recentMessages);

        // 使用 REPLACE 策略替换所有消息
        return new AgentCommand(newMessages, UpdatePolicy.REPLACE);
    }

    private String generateSummary(List<Message> messages) {
        StringBuilder conversation = new StringBuilder();
        for (Message msg : messages) {
            conversation.append(msg.getMessageType())
```

```
                    .append(": ")
                    .append(msg.getText())
                    .append("\n");
        }
        String summaryPrompt = "请简要总结以下对话:\n\n" + conversation;
        return summaryModel.call(summaryPrompt);
    }
}

// 使用
ChatModel summaryModel = //  ... 用于总结的模型（可以是更便宜的模型）
MessageSummarizationHook summarizationHook =
    new MessageSummarizationHook(
    summaryModel,
    4000,  // 在 4000 tokens 时触发总结
    20     // 总结后保留最后 20 条消息
);

ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .hooks(summarizationHook)
    .saver(new MemorySaver())
    .build();

RunnableConfig config = RunnableConfig.builder()
    .threadId("1")
    .build();

agent.call("你好，我叫阿伟", config);
agent.call("写一首关于猫的短诗", config);
agent.call("现在对狗做同样的事情", config);
agent.call("现在对鸡写一个800字的故事", config);
AssistantMessage finalResponse =
    agent.call("我叫什么名字前面我们聊了什么？ ", config);
System.out.println(finalResponse.getText());
```

# 访问记忆

你可以通过多种方式访问和修改 Agent 的短期记忆（状态）

## 工具

**在工具中读取短期记忆**

使用 `ToolContext` 参数在工具中访问短期记忆（状态）。

`toolContext` 参数从工具签名中隐藏（因此模型看不到它），但工具可以通过它访问状态。

在工具中读取短期记忆示例

```
public class UserInfoTool implements BiFunction<UserInfoTool.UserInfo,
ToolContext, String> {
    @SuppressWarnings("all")
    public record UserInfo(
        @ToolParam(required = false) String name,
```

```
        @ToolParam(required = false) String email) {
    }
    @Override
    public String apply(UserInfoTool.UserInfo request, ToolContext toolContext) {
        // 从上下文中获取用户信息
        RunnableConfig config = (RunnableConfig) toolContext
            .getContext().get(AGENT_CONFIG_CONTEXT_KEY);
        String userId = (String) config.metadata("user_id").orElse("");

        if ("user_123".equals(userId)) {
            return "用户是 John Smith";
        } else {
            return "未知用户";
        }
    }
}

// 创建工具
ToolCallback getUserInfoTool = FunctionToolCallback
    .builder("get_user_info", new UserInfoTool())
    .description("查找用户信息")
    .inputType(UserInfoTool.UserInfo.class)
    .build();

// 使用
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .tools(getUserInfoTool)
    .saver(new MemorySaver())
    .build();

RunnableConfig config = RunnableConfig.builder()
    .threadId("1")
    .addMetadata("user_id", "user_123")
    .build();

AssistantMessage response = agent.call("获取用户信息", config);
System.out.println(response.getText());
```

**从工具写入短期记忆**

要在执行期间修改 Agent 的短期记忆（状态），你可以在 Hook 中更新状态，或者使用工具返回的信息更新状态。

这对于持久化中间结果或使信息对后续工具或提示可访问很有用。

## 提示

在 Hook 中访问短期记忆（状态）以基于对话历史或自定义状态字段创建动态提示。

DynamicPromptInterceptor 动态提示示例

```
public class DynamicPromptInterceptor extends ModelInterceptor {
    @Override
```

```
    public ModelResponse interceptModel(ModelRequest request, ModelCallHandler handler) {
        // 从上下文中获取用户名
        Map<String, Object> context = request.getContext();
        String userName = (String) context.get("user_name");

        // 创建动态系统提示
        String systemPrompt = "你是一个有帮助的助手。称呼用户为 " + userName + "。";

        // 创建修改后的请求（示例），实际使用中需要根据具体 API 进行调整
        SystemMessage enhancedSystemMessage;
        if (request.getSystemMessage() == null) {
            enhancedSystemMessage = new SystemMessage(systemPrompt);
        } else {
            enhancedSystemMessage = new SystemMessage(
                request.getSystemMessage().getText() +
                "\n" + systemPrompt);
        }

        // 创建增强后的请求
        ModelRequest enhancedRequest = ModelRequest.builder(request)
            .systemMessage(enhancedSystemMessage)
            .build();
        return handler.call(enhancedRequest);
    }
    @Override
    public String getName() {
        return "DynamicPromptInterceptor";
    }
}

// 创建Agent
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    .interceptors(new DynamicPromptInterceptor())
    .build();
// 创建RunnableConfig
RunnableConfig config = RunnableConfig.builder()
    .threadId("1")
    .addMetadata("user_name", "阿伟")
    .build();
// 测试对话
AssistantMessage response = agent.call("写一个我的自传，内容你自己编吧", config);
System.out.println(response.getText());
```

## Before Model

在 `beforeModel` Hook 中访问短期记忆（状态）以在模型调用之前处理消息。

代码可以参考前面的修剪消息

### After Model

在 `afterModel` Hook 中访问短期记忆（状态）以在模型调用之后处理消息。

ValidateResponseHook After Model 示例

```java
import com.alibaba.cloud.ai.graph.agent.hook.HookPosition;
import com.alibaba.cloud.ai.graph.agent.hook.HookPositions;
import com.alibaba.cloud.ai.graph.agent.hook.messages.MessagesModelHook;
import com.alibaba.cloud.ai.graph.agent.hook.messages.AgentCommand;
import com.alibaba.cloud.ai.graph.agent.hook.messages.UpdatePolicy;
import com.alibaba.cloud.ai.graph.RunnableConfig;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.ai.chat.messages.Message;
import java.util.ArrayList;
import java.util.List;

@HookPositions({HookPosition.AFTER_MODEL})
public class ValidateResponseHook extends MessagesModelHook {

    private static final List<String> STOP_WORDS =
        List.of("password", "secret", "api_key", "猫");

    @Override
    public String getName() {
        return "validate_response";
    }

    @Override
    public AgentCommand afterModel
        (List<Message> previousMessages, RunnableConfig config) {
        if (previousMessages.isEmpty()) {
            return new AgentCommand(previousMessages);
        }
        Message lastMessage = previousMessages.get(previousMessages.size() - 1);
        String content = lastMessage.getText();
        // 检查是否包含敏感词
        for (String stopWord : STOP_WORDS) {
            if (content.toLowerCase().contains(stopWord)) {
                // 移除包含敏感词的消息，替换为安全消息
                List<Message> filtered = new ArrayList<>(
                    previousMessages.subList(0, previousMessages.size() - 1)
                );
                filtered.add(new AssistantMessage("抱歉，我无法提供该信息。"));
                return new AgentCommand(filtered, UpdatePolicy.REPLACE);
            }
        }
        return new AgentCommand(previousMessages);
    }
}

// 测试使用
ReactAgent agent = ReactAgent.builder()
    .name("secure_agent")
    .model(chatModel)
    .hooks(new ValidateResponseHook())
    .saver(new MemorySaver())
```

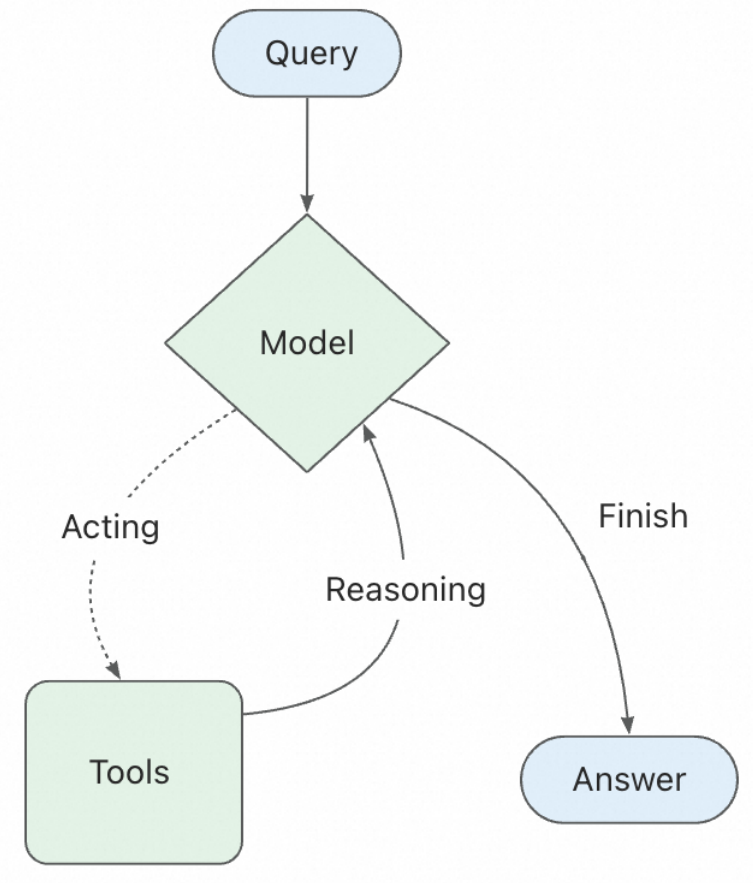
```
    .build();
AssistantMessage resp = agent.call("我们来聊聊猫的话题");
System.out.println(resp.getText());
```

## Hooks和Interceptors

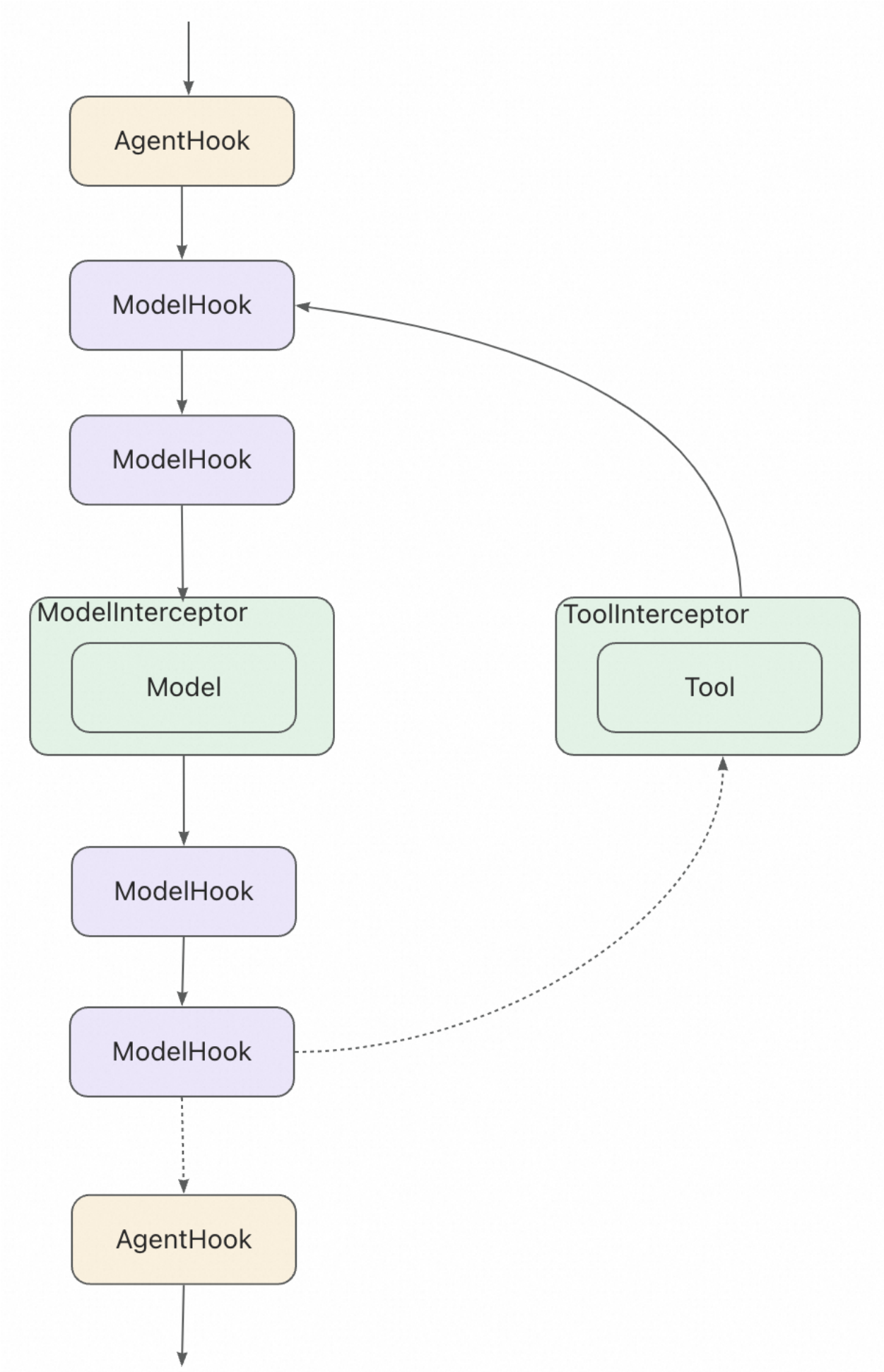
让开发者在每个步骤控制和自定义 Agent 执行。

Hooks 和 Interceptors 提供了一种更精细控制 Agent 内部行为的方式。

核心 Agent 循环涉及调用模型、让其选择要执行的工具，直到不需要调用工具时完成。




Hooks 和 Interceptors 在这些步骤的前后暴露了钩子点，允许你：




- **监控**: 通过日志、分析和调试跟踪 Agent 行为；
- **修改**: 转换提示、工具选择和输出格式；

- **控制**: 添加重试、回退和提前终止逻辑;
- **强制执行**: 应用速率限制、护栏和 PII 检测。

通过将它们传递给 `ReactAgent.builder()` 来添加 Hooks 和 Interceptors:

添加 Hooks 和 Interceptors 到 ReactAgent

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.alibaba.cloud.ai.graph.agent.hook.*;
import com.alibaba.cloud.ai.graph.agent.interceptor.*;

ReactAgent agent = ReactAgent.builder()
  .name("my_agent")
  .model(chatModel)
  .tools(tools)
  .hooks(loggingHook, messageTrimmingHook)
  .interceptors(guardrailInterceptor, retryInterceptor)
  .build();
```

# 内置实现

Spring AI Alibaba 为常见用例提供了预构建的 Hooks 和 Interceptors 实现:

## 消息压缩

当接近 token 限制时自动压缩对话历史。

**适用场景**:

- 超出上下文窗口的长期对话;
- 具有大量历史记录的多轮对话;
- 需要保留完整对话上下文的应用程序。

SummarizationHook 消息压缩示例

```
import com.alibaba.cloud.ai.graph.agent.hook.summarization.SummarizationHook;

// 创建消息压缩 Hook
SummarizationHook summarizationHook = SummarizationHook.builder()
  .model(chatModel)
  .maxTokensBeforeSummary(4000)
  .messagesToKeep(20)
  .build();

// 使用
ReactAgent agent = ReactAgent.builder()
  .name("my_agent")
  .model(chatModel)
  .hooks(summarizationHook)
  .build();
```

**配置选项**:

- `model`: 用于生成摘要的 ChatModel;

- `maxTokensBeforeSummary`: 触发摘要之前的最大 token 数;
- `messagesToKeep`: 摘要后保留的最新消息数。

## 人机协同

暂停 Agent 执行以获得人工批准、编辑或拒绝工具调用。

**适用场景**:

- 需要人工批准的高风险操作（数据库写入、金融交易）;
- 人工监督是强制性的合规工作流程;
- 长期对话，使用人工反馈引导 Agent。

HumanInTheLoopHook 人机协同示例

```
import com.alibaba.cloud.ai.graph.agent.hook.hip.HumanInTheLoopHook;
import com.alibaba.cloud.ai.graph.agent.hook.hip.ToolConfig;

// 创建 Human-in-the-Loop Hook
HumanInTheLoopHook humanReviewHook = HumanInTheLoopHook.builder()
  .approvalOn("sendEmailTool", ToolConfig.builder()
              .description("Please confirm sending the email.").build())
  .approvalOn("deleteDataTool")
  .build();

ReactAgent agent = ReactAgent.builder()
  .name("supervised_agent")
  .model(chatModel)
  .tools(sendEmailTool, deleteDataTool)
  .hooks(humanReviewHook)
  .saver(new RedisSaver())
  .build();
```

**重要提示**：Human-in-the-loop Hook 需要 checkpointer 来维护跨中断的状态。示例中我们演示用了 `RedisSaver`。

更多关于人机协同的使用可以参考官方说明

## 模型调用限制

限制模型调用次数以防止无限循环或过度成本。

**适用场景**:

- 防止失控的 Agent 进行太多 API 调用;
- 在生产部署中强制执行成本控制;
- 在特定调用预算内测试 Agent 行为。

ModelCallLimitHook 模型调用限制示例

```
ReactAgent agent = ReactAgent.builder()
    .name("my_agent")
    .model(chatModel)
    //限制模型调用次数为5次
    .hooks(ModelCallLimitHook.builder().runLimit(5).build())
    .saver(new MemorySaver())
    .build();
```

## PII 检测

检测和处理对话中的个人身份信息。

**适用场景**:

- 具有合规要求的医疗保健和金融应用;
- 需要清理日志的客户服务 Agent;
- 任何处理敏感用户数据的应用程序。

PIIDetectionHook PII 检测示例

```
PIIDetectionHook pii = PIIDetectionHook.builder()
  .piiType(PIIType.EMAIL)
  .strategy(RedactionStrategy.REDACT)
  .applyToInput(true)
  .build();

// 使用
ReactAgent agent = ReactAgent.builder()
  .name("secure_agent")
  .model(chatModel)
  .hooks(pii)
  .build();
```

## 工具重试

自动重试失败的工具调用，具有可配置的指数退避。

**适用场景**:

- 处理外部 API 调用中的瞬态故障;
- 提高依赖网络的工具的可靠性;
- 构建优雅处理临时错误的弹性 Agent。

ToolRetryInterceptor 工具重试示例

```
ReactAgent agent = ReactAgent.builder()
  .name("resilient_agent")
  .model(chatModel)
  .tools(searchTool, databaseTool)
  .interceptors(ToolRetryInterceptor.builder()
      .maxRetries(2)
      .onFailure(ToolRetryInterceptor
                .OnFailureBehavior.RETURN_MESSAGE)
      .build())
  .build();
```

## 规划

在执行工具之前强制执行一个规划步骤，以概述 Agent 将要采取的步骤。

**适用场景**:

- 需要执行复杂、多步骤任务的 Agent;
- 通过在执行前显示 Agent 的计划来提高透明度;
- 通过检查建议的计划来调试错误。

TodoListInterceptor 规划示例

```
ReactAgent agent = ReactAgent.builder()
  .name("planning_agent")
  .model(chatModel)
  .tools(myTool)
  .interceptors(TodoListInterceptor.builder()
              .build())
  .build();
```

## LLM工具选择器

使用一个 LLM 来决定在多个可用工具之间选择哪个工具。

**适用场景**:

- 当多个工具可以实现相似目标时;
- 需要根据细微的上下文差异进行工具选择;
- 动态选择最适合特定输入的工具。

ToolSelectionInterceptor LLM 工具选择器示例

```
ReactAgent agent = ReactAgent.builder()
  .name("smart_selector_agent")
  .model(chatModel)
  .tools(tool1, tool2)
  .interceptors(ToolSelectionInterceptor.builder()
              .build())
  .build();
```

### LLM工具模拟器

在没有实际执行工具的情况下，使用 LLM 模拟工具的输出。

**适用场景**:

- 在演示或测试期间模拟 API;
- 在开发过程中为工具提供占位符行为;
- 在不产生实际成本或副作用的情况下测试 Agent 逻辑。

ToolEmulatorInterceptor LLM 工具模拟器示例

```
ReactAgent agent = ReactAgent.builder()
  .name("emulator_agent")
  .model(chatModel)
  .tools(simulatedTool)
  .interceptors(ToolEmulatorInterceptor.builder()
                .model(chatModel).build())
  .build();
```

### 上下文编辑

在将上下文发送给 LLM 之前对其进行修改，以注入、删除或修改信息。

**适用场景**:

- 向 LLM 提供额外的上下文或指令;
- 从对话历史中删除不相关或冗余的信息;
- 动态修改上下文以引导 Agent 的行为。

ContextEditingInterceptor 上下文编辑示例

```
ReactAgent agent = ReactAgent.builder()
  .name("context_aware_agent")
  .model(chatModel)
  .interceptors(ContextEditingInterceptor.builder()
                .trigger(120000)
                .clearAtLeast(60000)
                .build())
  .build();
```

## 自定义

通过实现在 Agent 执行流程中特定点运行的钩子来构建自定义功能。

你可以通过以下方式创建自定义功能:

1. **MessagesModelHook** - 在模型调用前后执行，专注于消息操作（推荐）;
2. **ModelHook** - 在模型调用前后执行，可访问完整状态;
3. **AgentHook** - 在 Agent 开始和结束时执行;
4. **ModelInterceptor** - 拦截和修改模型请求/响应;
5. **ToolInterceptor** - 拦截和修改工具调用。

## MessagesModelHook

`MessagesModelHook` 是一个专门用于操作消息列表的 Hook，**使用更简单，更推荐**。它直接接收和返回消息列表，无需处理复杂的 `OverAllState`。

**适用场景**：

- 消息修剪、过滤或转换；
- 添加系统提示或上下文消息；
- 消息压缩和摘要；
- 简单的消息操作需求。

代码参考前面的消息修剪

**AgentCommand 和 UpdatePolicy**：

`MessagesModelHook` 通过 `AgentCommand` 返回操作结果：

- **REPLACE策略**：替换所有现有消息；
- **APPEND策略**：将新消息追加到现有消息列表。

使用不同策略的 MessagesModelHook 示例

```
@HookPositions({HookPosition.BEFORE_MODEL})
public class ContextEnhancementHook extends MessagesModelHook {
  @Override
  public String getName() {
      return "context_enhancement";
  }
  @Override
  public AgentCommand beforeModel
      (List<Message> previousMessages, RunnableConfig config) {
      // 示例 1: 使用 REPLACE 策略替换所有消息
      List<Message> newMessages = new ArrayList<>();
      newMessages.add(new SystemMessage("你是一个专业的助手"));
      newMessages.addAll(previousMessages);
      return new AgentCommand(newMessages, UpdatePolicy.REPLACE);

      // 示例 2: 使用 APPEND 策略追加消息
      // List<Message> additionalMessages = List.of(
      //     new UserMessage("请记住：保持友好和专业")
      // );
      // return new AgentCommand(additionalMessages, UpdatePolicy.APPEND);
  }
}
```

**支持跳转控制**：

`MessagesModelHook` 也支持通过 `JumpTo` 实现提前退出：

MessagesModelHook 跳转控制示例

```
@HookPositions({HookPosition.BEFORE_MODEL})
public class EarlyExitHook extends MessagesModelHook {
  @Override
```

```
  public String getName() {
      return "early_exit";
  }
  @Override
  public List<JumpTo> canJumpTo() {
      return List.of(JumpTo.end);
  }
  @Override
  public AgentCommand beforeModel
      (List<Message> previousMessages, RunnableConfig config) {
      // 检查某些条件，如果满足则提前退出
      if (shouldExit(previousMessages)) {
          return new AgentCommand(JumpTo.end, previousMessages);
      }
      return new AgentCommand(previousMessages);
  }
  private boolean shouldExit(List<Message> messages) {
      // 实现你的退出逻辑
      return false;
  }
}
```

## ModelHook

在模型调用前后执行自定义逻辑：

CustomModelHook 自定义 ModelHook 示例

```
import com.alibaba.cloud.ai.graph.agent.hook.ModelHook;
import com.alibaba.cloud.ai.graph.agent.hook.HookPosition;
import com.alibaba.cloud.ai.graph.agent.hook.HookPositions;
import java.util.concurrent.CompletableFuture;

@HookPositions({HookPosition.BEFORE_MODEL, HookPosition.AFTER_MODEL})
public class CustomModelHook extends ModelHook {
  @Override
  public String getName() {
      return "custom_model_hook";
  }
  @Override
  public CompletableFuture<Map<String, Object>> beforeModel
      (OverAllState state, RunnableConfig config) {
      // 在模型调用前执行
      System.out.println("准备调用模型 ... ");
      // 可以修改状态
      // 例如：添加额外的上下文
      return CompletableFuture.completedFuture(Map.of("extra_context", "某些额外信息"));
  }
  @Override
  public CompletableFuture<Map<String, Object>> afterModel
      (OverAllState state, RunnableConfig config) {
      // 在模型调用后执行
      System.out.println("模型调用完成");
      // 可以记录响应信息
      return CompletableFuture.completedFuture(Map.of());
```

```
    }
}
```

**删除消息**:

使用 `ModelHook` 时，可以通过 `RemoveByHash` 来删除 `messages` 中的消息。

**重要提示**：返回的消息列表必须保持原消息列表的顺序，不能打乱顺序。由于 ModelHook 的复杂度，因此我们更推荐直接使用 MessagesModelHook。

使用 RemoveByHash 删除消息示例

```java
@HookPositions({HookPosition.BEFORE_MODEL})
public class MessageDeletionHook extends ModelHook {
  @Override
  public CompletableFuture<Map<String, Object>> beforeModel
      (OverAllState state, RunnableConfig config) {
      Optional<List<Message>> messagesOpt = state.value("messages");
      if (!messagesOpt.isEmpty()) {
          return CompletableFuture.completedFuture(Map.of());
      }
      List<Message> messages = messagesOpt.get();
      // 构建新的消息列表，保持原顺序
      List<Object> newMessages = new ArrayList<>();
      for (Message msg : messages) {
          // 根据条件决定保留或删除
          if (shouldKeep(msg)) {
              // 保留消息
              newMessages.add(msg);
          } else {
              // 标记删除
              newMessages.add(RemoveByHash.of(msg));
          }
      }
      return CompletableFuture
          .completedFuture(Map.of("messages", newMessages));
  }
  private boolean shouldKeep(Message msg) {
      // 实现你的保留逻辑
      return true;
  }
}
```

## 如何选择?

`MessagesModelHook` 和 `ModelHook` 都可以在模型调用前后执行自定义逻辑，但它们有不同的设计目标和适用场景。

**核心区别**

| 特性 | MessagesModelHook | ModelHook |
| --- | --- | --- |
| **易用性** | ⭐⭐⭐⭐⭐ 更简单，直接操作消息列表 | ⭐⭐⭐ 需要理解 `OverAllState` |

| 特性 | MessagesModelHook | ModelHook |
| --- | --- | --- |
| **灵活性** | ⭐⭐⭐ 专注于消息操作 | ⭐⭐⭐⭐⭐ 可访问和修改完整状态 |
| **推荐场景** | 消息修剪、过滤、添加系统提示 | 需要访问全局状态、自定义状态管理 |
| **API 复杂度** | 简单：`AgentCommand` 返回消息列表 | 复杂：返回 `Map<String, Object>` 更新状态 |

**使用建议**

**选择 MessagesModelHook，如果你需要：**

- ✅ 简单的消息操作（修剪、过滤、转换）；
- ✅ 添加或修改系统提示；
- ✅ 消息压缩和摘要；
- ✅ 快速实现消息相关的 Hook。

**选择 ModelHook，如果你需要：**

- ✅ 访问和修改 `OverAllState` 中的其他数据；
- ✅ 在状态中存储自定义信息（如计数器、缓存等）；
- ✅ 基于全局状态做复杂决策；
- ✅ 需要查看 Agent 执行过程中的完整上下文。

**对比示例**

**场景：消息修剪**

使用 `MessagesModelHook`（推荐）：

使用 MessagesModelHook 实现消息修剪

```java
@HookPositions({HookPosition.BEFORE_MODEL})
public class SimpleMessageTrimmingHook extends MessagesModelHook {
  private static final int MAX_MESSAGES = 10;

  @Override
  public String getName() {
      return "simple_message_trimming";
  }

  @Override
  public AgentCommand beforeModel
      (List<Message> previousMessages, RunnableConfig config) {
      if (previousMessages.size() > MAX_MESSAGES) {
          List<Message> trimmed = previousMessages.subList(
              previousMessages.size() - MAX_MESSAGES,
              previousMessages.size()
          );
          return new AgentCommand(trimmed, UpdatePolicy.REPLACE);
      }
```

```
        return new AgentCommand(previousMessages);
    }
}
```

使用 `ModelHook`（更复杂但更灵活）：

使用 ModelHook 实现消息修剪（可访问状态）

```java
@HookPositions({HookPosition.BEFORE_MODEL})
public class AdvancedMessageTrimmingHook extends ModelHook {
    private static final int MAX_MESSAGES = 10;
    private static final String TRIM_COUNT_KEY = "trim_count";

    @Override
    public String getName() {
        return "advanced_message_trimming";
    }

    @Override
    public CompletableFuture<Map<String, Object>> beforeModel
        (OverAllState state, RunnableConfig config) {
        // 可以访问完整状态
        Optional<List<Message>> messagesOpt = state.value("messages");
        if (messagesOpt.isEmpty()) {
            return CompletableFuture.completedFuture(Map.of());
        }
        List<Message> messages = messagesOpt.get();
        // 可以访问和更新自定义状态
        int trimCount = (Integer) state
            .value(TRIM_COUNT_KEY).orElse(0);
        if (messages.size() > MAX_MESSAGES) {
            List<Message> trimmed = messages.subList(
                messages.size() - MAX_MESSAGES,
                messages.size()
            );
            // 可以同时更新消息和自定义状态
            return CompletableFuture.completedFuture
                (Map.of("messages",
                        ReplaceAllWith.of(trimmed),
                        TRIM_COUNT_KEY,
                        trimCount + 1  // 记录修剪次数
            ));
        }
        return CompletableFuture
            .completedFuture(Map.of());
    }
}
```

**总结**：

- **优先使用 `MessagesModelHook`**：对于大多数消息操作场景，它提供了更简洁的 API，代码更易读易维护。
- **使用 `ModelHook`**：当你需要访问全局状态、存储自定义数据或进行复杂的状态管理时。

## AgentHook

在 Agent 整体执行的开始和结束时执行：

CustomAgentHook 自定义 AgentHook 示例

```
@HookPositions
({HookPosition.BEFORE_AGENT, HookPosition.AFTER_AGENT})
public class CustomAgentHook extends AgentHook {
  @Override
  public String getName() {
      return "custom_agent_hook";
  }
  @Override
  public CompletableFuture<Map<String, Object>> beforeAgent
      (OverAllState state, RunnableConfig config) {
      System.out.println("Agent 开始执行");
      // 可以初始化资源、记录开始时间等
      return CompletableFuture
          .completedFuture(
          Map.of("start_time", System.currentTimeMillis())
      );
  }
  @Override
  public CompletableFuture<Map<String, Object>> afterAgent
      (OverAllState state, RunnableConfig config) {
      // 可以清理资源、计算执行时间等
      Optional<Object> startTime = state.value("start_time");
      if (startTime.isPresent()) {
          long duration = System.currentTimeMillis()
              - (Long) startTime.get();
          System.out.println("执行耗时： " + duration + "ms");
      }
      System.out.println("Agent 执行完成");
      return CompletableFuture.completedFuture(Map.of());
  }
}
```

## ModelInterceptor

拦截和修改模型请求和响应：

LoggingInterceptor 自定义 ModelInterceptor 示例

```
public class LoggingInterceptor extends ModelInterceptor {
  @Override
  public ModelResponse interceptModel
      (ModelRequest request, ModelCallHandler handler) {
      // 请求前记录
      System.out.println("发送请求到模型： "
                         + request.getMessages().size()
                         + " 条消息");
      long startTime = System.currentTimeMillis();
      // 执行实际调用
      ModelResponse response = handler.call(request);
```

```
        // 响应后记录
        long duration = System.currentTimeMillis() - startTime;
        System.out.println("模型响应耗时： " + duration + "ms");
        return response;
    }

    @Override
    public String getName() {
        return "LoggingInterceptor";
    }
}
```

**动态工具管理**：

`ModelInterceptor` 支持在模型调用前动态管理工具：

- **`dynamicToolCallbacks`**：动态添加工具回调，可以在运行时根据上下文添加新的工具
- **`tools`**：动态筛选工具，指定本次调用可用的工具名称列表。如果为空，则使用所有默认工具

DynamicToolInterceptor 动态工具管理示例

```
public class DynamicToolInterceptor extends ModelInterceptor {
    // 示例：根据上下文动态创建的工具
    private ToolCallback createContextualTool(String context) {
        return FunctionToolCallback.builder(
            "contextual_tool",
            (String input) →
            "处理上下文： " + context + ", 输入： " + input)
            .description("根据上下文动态创建的工具")
            .build();
    }
    @Override
    public ModelResponse interceptModel
        (ModelRequest request, ModelCallHandler handler) {
        // 从上下文中获取信息，决定添加哪些工具
        Map<String, Object> context = request.getContext();
        String userRole = (String) context
            .getOrDefault("user_role", "default");

        // 构建修改后的请求
        ModelRequest.Builder builder
            = ModelRequest.builder(request);

        // 示例 1: 动态添加工具回调
        List<ToolCallback> dynamicTools = new ArrayList<>();
        if ("premium".equals(userRole)) {
            // 为高级用户添加额外工具
            dynamicTools
                .add(createContextualTool("premium_feature"));
        }
        builder.dynamicToolCallbacks(dynamicTools);
        // 示例 2: 动态筛选工具（只允许使用指定的工具）
        if (shouldRestrictTools(context)) {
            // 只允许使用 search 和 calculator 工具
            builder.tools(List.of("search", "calculator"));
        }
```

```
        // 如果 tools 为空列表，则使用所有默认工具
        ModelRequest modifiedRequest = builder.build();
        return handler.call(modifiedRequest);
    }

    private boolean shouldRestrictTools
        (Map<String, Object> context) {
        // 根据上下文决定是否限制工具
        return context.containsKey("restrict_tools");
    }

    @Override
    public String getName() {
        return "DynamicToolInterceptor";
    }
}
```

**使用场景**:

- 根据用户权限动态添加或移除工具;
- 根据对话上下文临时启用特定工具;
- 实现工具的动态加载和卸载;
- 在特定条件下限制可用的工具集。

## ToolInterceptor

拦截和修改工具调用:

ToolMonitoringInterceptor 自定义 ToolInterceptor 示例

```
public class ToolMonitoringInterceptor extends ToolInterceptor {

  @Override
  public ToolCallResponse interceptToolCall
      (ToolCallRequest request, ToolCallHandler handler) {
      String toolName = request.getToolName();
      long startTime = System.currentTimeMillis();
      System.out.println("执行工具: " + toolName);
      try {
          ToolCallResponse response = handler.call(request);
          long duration = System.currentTimeMillis() - startTime;
          System.out.println("工具 " + toolName
                             + " 执行成功 (耗时: " + duration + "ms)");
          return response;
      } catch (Exception e) {
          long duration = System.currentTimeMillis() - startTime;
          System.err.println("工具 " + toolName
                             + " 执行失败 (耗时: " + duration + "ms): "
                             + e.getMessage());
          return ToolCallResponse.of(
              request.getToolCallId(),
              request.getToolName(),
              "工具执行失败: " + e.getMessage()
          );
      }
  }
```

```
27
28    @Override
29    public String getName() {
30        return "ToolMonitoringInterceptor";
31    }
32  }
```

## 跨调用共享数据

`RunnableConfig` 提供了一个 `context()` 方法，允许你在同一个执行流程中的多个 Hook 调用、多轮模型或工具调用之间共享数据。这对于实现计数器、累积统计信息或跨多次调用维护状态非常有用。

**适用场景**:

- 跟踪模型或工具调用次数;
- 累积性能指标（总耗时、平均响应时间等）;
- 在 before/after Hook 之间传递临时数据;
- 实现基于计数的限流或断路器。

**示例：使用 RunnableConfig.context() 实现调用计数器**

ModelCallCounterHook 调用计数器示例

```
1  @HookPositions
2  ({HookPosition.BEFORE_MODEL, HookPosition.AFTER_MODEL})
3  public class ModelCallCounterHook extends ModelHook {
4    private static final String CALL_COUNT_KEY = "__model_call_count__";
5    private static final String TOTAL_TIME_KEY = "__total_model_time__";
6    private static final String START_TIME_KEY = "__call_start_time__";
7    @Override
8    public String getName() {
9        return "model_call_counter";
10   }
11   @Override
12   public CompletableFuture<Map<String, Object>> beforeModel
13       (OverAllState state, RunnableConfig config) {
14       // 从 context 读取当前计数（如果不存在则默认为 0）
15       int currentCount = config.context()
16           .containsKey(CALL_COUNT_KEY) ?
17           (int)config.context().get(CALL_COUNT_KEY) : 0;
18       System.out.println("模型调用 #" + (currentCount + 1));
19       // 记录开始时间
20       config.context()
21           .put(START_TIME_KEY, System.currentTimeMillis());
22       return CompletableFuture.completedFuture(Map.of());
23   }
24
25   @Override
26   public CompletableFuture<Map<String, Object>> afterModel
27       (OverAllState state, RunnableConfig config) {
28       // 读取当前计数并递增
29       int currentCount = config.context()
30           .containsKey(CALL_COUNT_KEY) ?
31           (int)config.context().get(CALL_COUNT_KEY) : 0;
32       config.context()
33           .put(CALL_COUNT_KEY, currentCount + 1);
```

```
34
35        // 计算本次调用耗时并累加到总耗时
36        if (config.context().containsKey(START_TIME_KEY)) {
37            long startTime = (long) config.context()
38                .get(START_TIME_KEY);
39            long duration = System.currentTimeMillis()
40                - startTime;
41
42            long totalTime = config.context()
43                .containsKey(TOTAL_TIME_KEY) ?
44                (long) config.context()
45                .get(TOTAL_TIME_KEY) : 0L;
46            config.context()
47                .put(TOTAL_TIME_KEY, totalTime + duration);
48            // 输出统计信息
49            int newCount = currentCount + 1;
50            long newTotalTime = totalTime + duration;
51            System.out.println("模型调用完成: " + duration + "ms");
52            System.out.println("累计统计 - 调用次数: "
53                               + newCount
54                               + ", 总耗时: "
55                               + newTotalTime
56                               + "ms, 平均: "
57                               + (newTotalTime / newCount)
58                               + "ms");
59        }
60        return CompletableFuture.completedFuture(Map.of());
61    }
62 }
```

**示例：基于 context 实现调用次数限制**

ModelCallLimiterHook 调用次数限制示例

```
1  @HookPositions
2  ({HookPosition.BEFORE_MODEL, HookPosition.AFTER_MODEL})
3  public class ModelCallLimiterHook extends ModelHook {
4    private static final String CALL_COUNT_KEY = "__model_call_count__";
5    private final int maxCalls;
6    public ModelCallLimiterHook(int maxCalls) {
7        this.maxCalls = maxCalls;
8    }
9    @Override
10   public String getName() {
11       return "model_call_limiter";
12   }
13   @Override
14   public CompletableFuture<Map<String, Object>> beforeModel
15       (OverAllState state, RunnableConfig config) {
16       // 读取当前调用次数
17       int callCount = config.context()
18           .containsKey(CALL_COUNT_KEY) ?
19           (int) config.context().get(CALL_COUNT_KEY) : 0;
20
21       // 检查是否超过限制
22       if (callCount ≥ maxCalls) {
```

```
            System.out.println("达到模型调用次数限制: " + maxCalls);
            // 添加终止消息
            List<Message> messages = new ArrayList<>(
                (List<Message>) state.value("messages")
                .orElse(new ArrayList<>())
            );
            messages.add(new AssistantMessage(
                "已达到模型调用次数限制 (" + callCount + "/"
                + maxCalls + "), Agent 执行终止。"
            ));

            // 返回更新并跳转到结束
            return CompletableFuture
                .completedFuture(Map.of("messages", messages));
        }
        return CompletableFuture.completedFuture(Map.of());
    }
    @Override
    public CompletableFuture<Map<String, Object>> afterModel
        (OverAllState state, RunnableConfig config) {
        // 递增计数器
        int callCount = config.context()
            .containsKey(CALL_COUNT_KEY) ?
            (int) config.context().get(CALL_COUNT_KEY) : 0;
        config.context()
            .put(CALL_COUNT_KEY, callCount + 1);
        return CompletableFuture
            .completedFuture(Map.of());
    }
    @Override
    public List<JumpTo> canJumpTo() {
        return List.of(JumpTo.end);
    }
}
```

**使用示例**:

使用 ModelCallCounterHook 和 ModelCallLimiterHook

```
ReactAgent agent = ReactAgent.builder()
  .name("limited_agent")
  .model(chatModel)
  .tools(tools)
  .hooks(new ModelCallCounterHook())  // 监控调用统计
  .hooks(new ModelCallLimiterHook(5)) // 限制最多调用 5 次
  .build();
```

**关键要点**:

- **context() 是共享的**: 同一个执行流程中的所有 Hook 共享同一个 context;
- **数据持久性**: context 中的数据在整个 Agent 执行期间保持有效;
- **类型安全**: 需要自己管理 context 中数据的类型转换;
- **命名约定**: 建议使用双下划线前缀命名 context key（如 `__model_call_count__`）以避免与用户数据冲突。

## 执行顺序

使用多个 Hooks 和 Interceptors 时，理解执行顺序很重要：

多个 Hooks 和 Interceptors 配置示例

```
ReactAgent agent = ReactAgent.builder()
  .name("my_agent")
  .model(chatModel)
  .hooks(hook1, hook2, hook3)
  .interceptors(interceptor1, interceptor2)
  .interceptors(toolInterceptor1, toolInterceptor2)
  .build();
```

**执行流程**：

1. **Before Agent Hooks**（按顺序）：
    - `hook1.beforeAgent()`
    - `hook2.beforeAgent()`
    - `hook3.beforeAgent()`
2. **Agent 循环开始**
3. **Before Model Hooks**（按顺序）：
    - `hook1.beforeModel()`
    - `hook2.beforeModel()`
    - `hook3.beforeModel()`
4. **Model Interceptors**（嵌套调用）：
    - `interceptor1` → `interceptor2` → 模型调用
5. **After Model Hooks**（逆序）：
    - `hook3.afterModel()`
    - `hook2.afterModel()`
    - `hook1.afterModel()`
6. **Tool Interceptors**（如果有工具调用，嵌套调用）：
    - `toolInterceptor1` → `toolInterceptor2` → 工具执行
7. **Agent 循环结束**
8. **After Agent Hooks**（逆序）：
    - `hook3.afterAgent()`
    - `hook2.afterAgent()`
    - `hook1.afterAgent()`

**关键规则**：

- `before_*` hooks: 从第一个到最后一个
- `after_*` hooks: 从最后一个到第一个（逆序）
- Interceptors: 嵌套调用（第一个拦截器包装所有其他的）

# 实际示例

## 示例 1：内容审核 Interceptor

ContentModerationInterceptor 内容审核示例

```
public class ContentModerationInterceptor extends ModelInterceptor {
    private static final List<String> BLOCKED_WORDS =
        List.of("敏感词1", "敏感词2", "敏感词3");
    @Override
    public ModelResponse interceptModel
        (ModelRequest request, ModelCallHandler handler) {
        // 检查输入
        for (Message msg : request.getMessages()) {
            String content = msg.getText().toLowerCase();
            for (String blocked : BLOCKED_WORDS) {
                if (content.contains(blocked)) {
                    return ModelResponse
                        .of(AssistantMessage
                            .builder()
                            .content("检测到不适当的内容，请修改您的输入")
                            .build());
                }
            }
        }
        // 执行模型调用
        ModelResponse response = handler.call(request);
        // 检查输出
        if (response.getMessage() instanceof
            AssistantMessage assistantMsg) {
            String output = assistantMsg.getText()
                .toLowerCase();
            boolean modified = false;
            for (String blocked : BLOCKED_WORDS) {
                if (output.contains(blocked)) {
                    output = output.replaceAll(blocked, "[已过滤]");
                    modified = true;
                }
            }
            if (modified) {
                // 构建新的AssistantMessage
                AssistantMessage filteredMsg =
                    AssistantMessage.builder()
                    .content(output)
                    .media(assistantMsg.getMedia())
                    .build();
                // 重新构建ModelResponse
                return ModelResponse.of(filteredMsg,
                                        response.getChatResponse());
            }
        }
        return response;
    }
    @Override
    public String getName() {
        return "ContentModerationInterceptor";
```

```
    }
}
```

## 示例 2：性能监控 - 使用 Interceptor

使用 `ModelInterceptor` 和 `ToolInterceptor` 监控模型和工具调用的性能：

性能监控 Interceptor 示例

```
// 模型调用性能监控
public class ModelPerformanceInterceptor extends ModelInterceptor {
  @Override
  public ModelResponse interceptModel
      (ModelRequest request, ModelCallHandler handler) {
      // 请求前记录
      System.out.println("发送请求到模型： "
                         + request.getMessages().size()
                         + " 条消息");
      long startTime = System.currentTimeMillis();
      // 执行实际调用
      ModelResponse response = handler.call(request);
      // 响应后记录
      long duration = System.currentTimeMillis() - startTime;
      System.out.println("模型响应耗时： " + duration + "ms");
      return response;
  }

  @Override
  public String getName() {
      return "ModelPerformanceInterceptor";
  }
}
// 工具调用性能监控
public class ToolPerformanceInterceptor extends ToolInterceptor {
  @Override
  public ToolCallResponse interceptToolCall
      (ToolCallRequest request, ToolCallHandler handler) {
      String toolName = request.getToolName();
      long startTime = System.currentTimeMillis();
      System.out.println("执行工具： " + toolName);
      try {
          ToolCallResponse
              response = handler.call(request);
          long duration
              = System.currentTimeMillis() - startTime;
          System.out.println("工具 " + toolName
                             + " 执行成功 (耗时： "
                             + duration + "ms)");
          return response;
      } catch (Exception e) {
          long duration
              = System.currentTimeMillis() - startTime;
          System.err.println("工具 " + toolName
                             + " 执行失败 (耗时： "
                             + duration + "ms): "
                             + e.getMessage());
```

```
            return ToolCallResponse.of(
                request.getToolCallId(),
                request.getToolName(),
                "工具执行失败: " + e.getMessage()
            );
        }
    }
    @Override
    public String getName() {
        return "ToolPerformanceInterceptor";
    }
}

// 使用示例
ReactAgent agent = ReactAgent.builder()
    .name("monitored_agent")
    .model(chatModel)
    .tools(tools)
    .interceptors(new ModelPerformanceInterceptor())
    .interceptors(new ToolPerformanceInterceptor())
    .build();
```

## 示例 3：工具缓存 Interceptor

ToolCacheInterceptor工具缓存示例

```
public class ToolCacheInterceptor extends ToolInterceptor {
    private Map<String, ToolCallResponse>
        cache = new ConcurrentHashMap<>();
    @Override
    public ToolCallResponse interceptToolCall
        (ToolCallRequest request, ToolCallHandler handler) {
        String cacheKey = generateCacheKey(request);
        // 检查缓存
        ToolCallResponse cached = cache.get(cacheKey);
        if (cached != null && !isExpired(cached)) {
            System.out.println("缓存命中: " + request.getToolName());
            return cached;
        }
        // 执行工具
        ToolCallResponse response = handler.call(request);
        // 缓存结果
        cache.put(cacheKey, response);
        return response;
    }
    @Override
    public String getName() {
        return "ToolCacheInterceptor";
    }
    private String generateCacheKey(ToolCallRequest request) {
        return request.getToolName() + ":" +
                request.getArguments();
    }
    private boolean isExpired(ToolCallResponse response) {
        return false;
    }
```

```
31 }
```

## 二者的区别

在 Spring AI Alibaba 中，Hook 和 Interceptor 都可以用于干预 Agent 执行：

| 特性 | Hook | Interceptor |
| --- | --- | --- |
| **执行位置** | Agent 级别（before/after agent, before/after model） | 模型或工具调用级别 |
| **中断能力** | 支持中断和恢复（如 HumanInTheLoopHook） | 不支持中断，仅拦截和修改 |
| **使用场景** | 人工审批、Agent 间协调 | 日志记录、重试、降级 |
| **配置方式** | `.hooks(List.of( ... ))` | `.interceptors(List.of( ... ))` |

## 结构化输出

结构化输出允许Agent以特定的、可预测的格式返回数据。相比于解析自然语言响应，您可以直接获得 JSON 对象或 Java POJO 形式的结构化数据，应用程序可以直接使用。

Spring AI Alibaba 的 `ReactAgent.Builder` 通过 `outputSchema` 和 `outputType` 方法处理结构化输出。当您设置所需的结构化输出模式时，Agent 会自动在用户消息中增加模式指令，模型会根据指定的格式生成数据。

结构化输出基础配置

```
ReactAgent agent = ReactAgent.builder()
  .name("agent")
  .model(chatModel)
  .outputSchema(schemaString)  // Custom JSON schema as String
  // OR
  .outputType(MyClass.class)   // Java class - auto-converted to schema
  .build();
```

## 输出格式选项

Spring AI Alibaba 支持两种方式控制结构化输出：

- **`outputSchema(String schema)`**: 提供 JSON schema 字符串。推荐使用 `BeanOutputConverter` 从 Java 类自动生成 schema，也可以手动提供自定义的 schema 字符串
- **`outputType(Class<?> type)`**: 提供 Java 类 - 使用 `BeanOutputConverter` 自动转换为 JSON schema（推荐方式，类型安全）
- **不指定**: 返回非结构化的自然语言响应

**推荐做法**：使用 `BeanOutputConverter` 生成 schema，既保证了类型安全，又实现了自动 schema 生成，代码更易维护。

结构化响应在 Agent 的 `AssistantMessage` 中作为 JSON 文本返回，可以解析为您需要的格式。

## 输出Schema策略

您可以使用 `BeanOutputConverter` 从 Java 类自动生成 JSON schema，或者直接提供 JSON schema 字符串。推荐使用 `BeanOutputConverter` 以获得类型安全和自动 schema 生成。

### 方法签名

outputSchema 方法签名

```
Builder outputSchema(String outputSchema)
```

**参数:**

- `outputSchema` (String, 必需): 定义结构化输出格式的 JSON schema 字符串。可以通过 `BeanOutputConverter.getFormat()` 方法从 Java 类自动生成，也可以手动提供自定义的 schema 字符串。

### 示例

**基本 JSON Schema:**

使用 `BeanOutputConverter` 从 Java 类自动生成 JSON Schema：

基本 JSON Schema 示例

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.ai.converter.BeanOutputConverter;
// 定义输出类型
public static class ContactInfo {
    private String name;
    private String email;
    private String phone;
    // Getters and Setters
}

// 使用 BeanOutputConverter 生成 outputSchema
BeanOutputConverter<ContactInfo> outputConverter =
    new BeanOutputConverter<>(ContactInfo.class);
String format = outputConverter.getFormat();
ReactAgent agent = ReactAgent.builder()
    .name("contact_extractor")
    .model(chatModel)
    .outputSchema(format)
    .build();

AssistantMessage result = agent.call(
    "从以下信息提取联系方式：张三，zhangsan@example.com，(555) 123-4567"
);
System.out.println(result.getText());
```

**复杂嵌套 Schema:**

使用 `BeanOutputConverter` 处理复杂嵌套结构：

复杂嵌套 Schema 示例

```
import org.springframework.ai.converter.BeanOutputConverter;

// 定义输出类型（包含嵌套类）
public static class ProductReview {
    private int rating;
    private String sentiment;
    private String[] keyPoints;
    private ReviewDetails details;
    public static class ReviewDetails {
        private String[] pros;
        private String[] cons;
        // Getters and Setters
    }
    // Getters and Setters
}

// 使用 BeanOutputConverter 生成 outputSchema
BeanOutputConverter<ProductReview> outputConverter =
    new BeanOutputConverter<>(ProductReview.class);
String format = outputConverter.getFormat();
ReactAgent agent = ReactAgent.builder()
    .name("review_analyzer")
    .model(chatModel)
    .outputSchema(format)
    .build();

AssistantMessage result = agent.call(
    "分析评价：这个产品很棒，5星好评。配送快速，但价格稍贵。"
);
System.out.println(result.getText());
```

**结构化分析 Schema:**

使用 `BeanOutputConverter` 处理包含实体识别的复杂结构：

结构化分析 Schema 示例

```
import org.springframework.ai.converter.BeanOutputConverter;

// 定义输出类型（包含嵌套实体类）
public static class TextAnalysis {
    private String summary;
    private String[] keywords;
    private String sentiment;
    private Entities entities;
    public static class Entities {
        private String[] persons;
        private String[] locations;
        private String[] organizations;
        // Getters and Setters
    }
    // Getters and Setters
}

```

```java
// 使用 BeanOutputConverter 生成 outputSchema
BeanOutputConverter<TextAnalysis> outputConverter =
    new BeanOutputConverter<>(TextAnalysis.class);
String format = outputConverter.getFormat();
ReactAgent agent = ReactAgent.builder()
    .name("text_analyzer")
    .model(chatModel)
    .outputSchema(format)
    .build();

AssistantMessage result = agent.call(
    "分析这段文字：昨天，李明在北京参加了阿里巴巴公司的技术大会，感受到了创新的力量。"
);
System.out.println(result.getText());
```

使用 `BeanOutputConverter` 生成 schema 提供了类型安全和自动生成的优势。如果您需要完全自定义的 schema 格式，也可以直接提供 JSON schema 字符串。`outputSchema` 方法支持两种方式，为您提供最大的灵活性。

## 输出类型策略

对于类型安全的结构化输出，您可以提供 Java 类，Spring AI Alibaba 将使用 `BeanOutputConverter` 自动将其转换为 JSON schema。这种方法确保了编译时类型安全和自动 schema 生成。

### 方法签名

outputType 方法签名

```java
Builder outputType(Class<?> outputType)
```

**参数:**

- `outputType` (`Class<?>`, 必需): 定义输出结构的 Java 类。该类应该是带有标准 getter 和 setter 的 POJO。

### 示例

使用 `outputType` 是最简洁的方式，框架会自动使用 `BeanOutputConverter` 将 Java 类转换为 JSON schema：

使用 outputType 示例

```java
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.alibaba.cloud.ai.graph.checkpoint.savers.MemorySaver;
import org.springframework.ai.chat.messages.AssistantMessage;

// 定义输出类型
public static class ContactInfo {
    private String name;
    private String email;
    private String phone;
    // Getters and Setters
}
```

```
12
13  // 直接使用 outputType，框架会自动处理 schema 转换
14  ReactAgent agent = ReactAgent.builder()
15      .name("contact_extractor")
16      .model(chatModel)
17      .outputType(ContactInfo.class)
18      .saver(new MemorySaver())
19      .build();
20
21  AssistantMessage result = agent.call(
22      "从以下信息提取联系方式：张三，zhangsan@example.com，(555) 123-4567"
23  );
24  System.out.println(result.getText());
```

**`outputType` vs `outputSchema`**:

- `outputType`: 更简洁，直接传入 Java 类，框架自动处理（**推荐**）
- `outputSchema`: 需要手动使用 `BeanOutputConverter` 生成 schema，或提供自定义字符串，提供更多控制

## 工作原理

当 outputFormat 或 outputType 被指定时，Spring AI Alibaba 会自动选择：

- 当大模型服务支持 “原生结构化输出” 时（目前支持 OpenAiChatModel、DashScopeChatModel），自动使用模型内置的结构化输出能力（这也是目前最稳定、可靠的方式，因为模型服务会自动提供校验支持）。
- 针对其他没有 “原生结构化输出” 的模型，Spring AI Alibaba 会使用内置的 ToolCall策略，通过一个动态的 ToolCall 来格式化模型输出。

结构化响应将在 Agent 的状态对象 OverAllState 中返回，可通过 `structured_output` 读取。

### 模型原生结构化输出

比如，针对 DashScopeChatModel 模型，在配置 outputSchema 或 outputType 后，Spring AI Alibaba 会自动设置如下参数，以启用模型原生结构化输出能力。

DashScope 原生结构化输出配置

```
1  ChatOptions options = DashScopeChatOptions.builder()
2  .withResponseFormat(
3      DashScopeResponseFormat.builder()
4          .type(DashScopeResponseFormat.Type.JSON_OBJECT)
5          .build())
6  .build();
```

同时，Spring AI Alibaba 框架会增强系统 Prompt，引导模型输出格式化内容

增强用户消息方法示例

```
1  // In AgentLlmNode.augmentUserMessage() method
2  public void augmentUserMessage(List<Message> messages, String outputSchema) {
3    if (!StringUtils.hasText(outputSchema)) {
4        return;
```

```
  }
  for (int i = messages.size() - 1; i ≥ 0; i--) {
      Message message = messages.get(i);
      if (message instanceof UserMessage userMessage) {
          messages.set(i, userMessage.mutate()
              .text(userMessage.getText() + System.lineSeparator() +
outputSchema)
              .build());
          break;
      }
  }
}
```

注意，相比于 DashScope 模型是通过增强 Prompt 提示词实现最终的 JSON 格式，实现的是一个尽最大努力的效果，OpenAI 模型则是在模型 API 层面支持 Json 格式，提供格式的严格保证支持。

### ToolCall结构化输出

对于不支持原生结构化输出的模型，Spring AI Alibaba 支持通过调用工具来实现相同效果。此方法适用于所有支持工具调用的模型，即大多数现代模型。

## 错误处理

模型可能不总是返回格式完美的 JSON。以下是处理潜在问题的策略:

### Try-Catch模式

Try-Catch 错误处理示例

```
ReactAgent agent = ReactAgent.builder()
  .name("data_extractor")
  .model(chatModel)
  .outputType(DataOutput.class)
  .build();

try {
  AssistantMessage result = agent.call("提取数据");
  ObjectMapper mapper = new ObjectMapper();
  DataOutput data = mapper.readValue(result.getText(), DataOutput.class);
  // 处理数据
} catch (JsonProcessingException e) {
  System.err.println("JSON解析失败: " + e.getMessage());
  System.err.println("原始输出: " + result.getText());
  // 回退处理
}
```

### 验证模式

验证模式示例

```
public class ValidatedOutput {
  private String title;
  private Integer rating;
```

```
    public void validate() throws IllegalArgumentException {
        if (title == null || title.isEmpty()) {
            throw new IllegalArgumentException("标题不能为空");
        }
        if (rating != null && (rating < 1 || rating > 5)) {
            throw new IllegalArgumentException("评分必须在1-5之间");
        }
    }
    // Getter 和 Setter 方法 ...
}

AssistantMessage result = agent.call("生成评价");
ValidatedOutput output = mapper.readValue(result.getText(),
ValidatedOutput.class);
output.validate();  // 如果无效则抛出异常
```

### 重试模式

重试模式示例

```
int maxRetries = 3;
DataOutput data = null;

for (int i = 0; i < maxRetries; i++) {
    try {
        AssistantMessage result = agent.call("提取数据");
        data = mapper.readValue(result.getText(), DataOutput.class);
        break;  // 成功
    } catch (Exception e) {
        if (i == maxRetries - 1) {
            throw new RuntimeException("多次尝试后仍然失败", e);
        }
        System.out.println("第" + (i + 1) + "次尝试失败，重试中 ...");
    }
}
```

Spring AI Alibaba 专注于简单性和灵活性，允许开发者在显式 schema 字符串（最大控制）和 Java 类（类型安全）之间进行选择。

## Skills

Skills 是可复用的指令与上下文包，智能体在相关任务时会自动发现并使用。通过 **SkillRegistry** 管理技能、**SkillsAgentHook** 注册 `read_skill` 工具并注入技能列表到系统提示，模型在需要时调用 `read_skill(skill_name)` 按需加载完整内容。

## 核心概念

### 渐进式披露

系统提示中先只注入技能列表（name、description、skillPath）；模型判断需要某技能时调用 `read_skill(skill_name)` 加载完整 SKILL.md；再按需访问技能目录下的资源或使用与该技能绑定的工具。

### 目录结构

每个技能一个子目录，必须包含 `SKILL.md`：

```
skill-name/
├── SKILL.md          # 必需
├── references/       # 可选
├── examples/
└── scripts/
```

### SKILL.md 格式规范

```
---
name: skill-name
description: This skill should be used when ...
---

# 技能名称
正文：功能说明、使用方法、可用资源列表等。
```

**必需字段**：`name`（建议小写字母、数字、连字符，最长 64 字符）、`description`（超长会被截断）。

---

## 在Agent中使用Skills

### 使用FileSystemSkillRegistry

智能体支持从本地文件系统中加载 skills 技能，以下示例假设 `skills` 在进程工作目录，如：

```
skills/
├── pdf-extractor/
    ├── SKILL.md
    ├── references/
    └── scripts/
```

FileSystemSkillRegistry + SkillsAgentHook

```
SkillRegistry registry = FileSystemSkillRegistry.builder()
  .projectSkillsDirectory(System.getProperty("user.dir") + "/skills")
  .build();

SkillsAgentHook hook = SkillsAgentHook.builder()
  .skillRegistry(registry)
  .build();

ReactAgent agent = ReactAgent.builder()
  .name("skills-agent")
  .model(chatModel)
  .saver(new MemorySaver())
  .hooks(List.of(hook))
  .build();

```

```
agent.call("请介绍你有哪些技能");
```

目录配置：

`userSkillsDirectory(String|Resource)`、`projectSkillsDirectory(String|Resource)`；

不设置时用户级默认 `~/saa/skills`，项目级默认 `./skills`，同名技能“项目级别”覆盖“用户级别”。

## 使用ClasspathSkillRegistry

技能放在 `src/main/resources/skills` 或随 JAR 打包。可选 `.basePath("/tmp")` 指定JAR内资源复制到的目录（默认 `/tmp`）。

ClasspathSkillRegistry

```
SkillRegistry registry = ClasspathSkillRegistry.builder()
  .classpathPath("skills")
  .build();

SkillsAgentHook hook = SkillsAgentHook.builder()
  .skillRegistry(registry)
  .build();

ReactAgent agent = ReactAgent.builder()
  .name("skills-agent")
  .model(chatModel)
  .hooks(List.of(hook))
  .build();
```

## 完整集成示例

技能常需配合脚本执行（如技能目录下的 Python 脚本）和 Shell 命令。下面示例使用 **ClasspathSkillRegistry** 加载技能、**SkillsAgentHook** 提供 `read_skill`、**ShellToolAgentHook** 提供 Shell 工具、**PythonTool** 提供 Python 执行能力，Agent 可根据技能说明读取并处理技能目录下的文件。

Skills + Python + Shell 完整集成

```
// 1. 技能注册表：从classpath:skills加载
// （如src/main/resources/skills/）
SkillRegistry registry = ClasspathSkillRegistry.builder()
    .classpathPath("skills")
    // 复制skills到家目录，避免权限问题
    .basePath(System.getProperty("user.home"))
    .build();

// 2. Skills Hook：注册 read_skill 工具并注入技能列表到系统提示
SkillsAgentHook skillsHook = SkillsAgentHook.builder()
    .skillRegistry(registry)
    .build();

// 3. Shell Hook：提供 Shell 命令执行
ShellToolAgentHook shellHook = ShellToolAgentHook.builder()
    // 指定工作目录为家目录
```

```
    .shellTool2(ShellTool2
                .builder(System.getProperty("user.home"))
                .build())
    .build();

// 4. 构建 Agent：同时挂载 Skills Hook、Shell Hook 和 Python 工具
ReactAgent agent = ReactAgent.builder()
    .name("skills-integration-agent")
    .model(chatModel)
    .saver(new MemorySaver())
    .tools(PythonTool.createPythonToolCallback(PythonTool.DESCRIPTION))
    .hooks(ModelCallLimitHook.builder().runLimit(3).build())
    .hooks(List.of(skillsHook, shellHook))
    //.enableLogging(true)
    .build();

// 5. 调用示例
System.out.println(agent.call("请介绍你有哪些技能").getText());
String skillFilePath = System.getProperty("user.home")
    + "/skills/pdf-extractor/saa-roadmap.pdf";
AssistantMessage response = agent.call(
    "请从 " + skillFilePath + " 文件中提取关键信息。");
System.out.println("═══════════".repeat(20));
System.out.println(response.getText());
```

- **SkillRegistry**：`FileSystemSkillRegistry` 用 `projectSkillsDirectory(path)` 或 `ClassPathResource("skills")`；`ClasspathSkillRegistry` 用 `classpathPath("skills")`。
- **ShellTool2**：`ShellTool2.builder(workDir).build()`，`workDir` 为 Shell 执行的工作目录（如 `System.getProperty("user.dir")`）。
- **PythonTool**：`PythonTool.createPythonToolCallback(PythonTool.DESCRIPTION)` 即够用，如需自定义描述可传第二个参数。
- 技能列表中会包含每个技能的 `skillPath`，模型可用该路径拼出技能目录下文件的绝对路径并交给 Python/Shell 处理。

---

# 高级用法

## 渐进式工具Tool披露

通过将工具与 Skill 技能名绑定，可以做到工具跟随 Skill 实现渐进式披露：仅当模型对该技能调用了 `read_skill` 后，对应工具才会加入当次请求，实现按需暴露。激活后该技能的工具在会话后续轮次中仍可用。

groupedTools 绑定工具到技能

```java
Map<String, List<ToolCallback>> groupedTools = Map.of(
  "my-skill",   // 与 SKILL.md 的 name 一致，如 'pdf-extractor'
  List.of(myTool)
);

SkillsAgentHook hook = SkillsAgentHook.builder()
  .skillRegistry(registry)
  .groupedTools(groupedTools)
  .build();
```

## 生产环境配置

**自动重载技能**

启用技能自动重载

```java
SkillsAgentHook hook = SkillsAgentHook.builder()
  .skillRegistry(registry)
  .autoReload(true)
  .build();
```

每次 Agent 执行前会调用 `registry.reload()`（若实现支持；不支持则抛 `UnsupportedOperationException`，Hook 会捕获并打 debug 日志）。

注意，每次 Agent 执行可能包含多次模型推理，`registry.reload()` 仅会在第一次推理时执行并加载最新的 skills，这样能保证同一次 Agent 执行时行为的连续性。

**用户级与项目级目录**

用户级与项目级技能目录

```java
SkillRegistry registry = FileSystemSkillRegistry.builder()
  .userSkillsDirectory("/home/user/saa/skills")
  .projectSkillsDirectory("/app/project/skills")
  .build();
```

同名技能项目覆盖用户。

**自定义系统提示模板**

SAA 框架内置了 Skill Prompt 模板，用来引导实现 Skill 的渐进式披露。用户可结合自己系统的 Skill 组织方式定制 Prompt 模板。

自定义技能系统提示模板

```
SystemPromptTemplate customTemplate = SystemPromptTemplate.builder()
  .template("## 可用技能\n{skills_list}\n\n## 加载说明
\n{skills_load_instructions}")
  .build();

FileSystemSkillRegistry registry = FileSystemSkillRegistry.builder()
  .projectSkillsDirectory("./skills")
  .systemPromptTemplate(customTemplate)
  .build();
```

模板变量：`{skills_list}`、`{skills_load_instructions}`。

### 拓展SkillRegistry实现

实现 `SkillRegistry` 接口（`get`、`listAll`、`contains`、`size`、`readSkillContent`、`getSkillLoadInstructions`、`getRegistryType`、`getSystemPromptTemplate`，可选 `reload()`）即可接入现有 Skills 体系。`SkillMetadata` 需包含 `name`、`description`、`skillPath`（及可选 `source`）。可参考 `AbstractSkillRegistry`、`FileSystemSkillRegistry`、`ClasspathSkillRegistry`。

---

## 在Graph中使用Skills

除在**ReactAgent**上通过**SkillsAgentHook**使用Skills外，在基于 **Graph** 或 **ChatClient** 的链路中，可通过 **ChatClient** 配合 **SkillPromptAugmentAdvisor**（`spring-ai-alibaba-graph-core`）将技能列表注入系统提示，实现渐进式披露的「技能发现」部分。

### 使用

`SkillPromptAugmentAdvisor` 是 Spring AI 的 `Advisor`，在每次请求的 `before` 阶段将技能元数据（name、description、skillPath）注入系统提示，使模型知晓可用技能及加载说明；模型需要完整 SKILL.md 时，需配合 `read_skill` 工具（可由 SkillsAgentHook 提供或自行注册）。

ChatClient + SkillPromptAugmentAdvisor

```
import org.springframework.ai.chat.client.ChatClient;
import com.alibaba.cloud.ai.graph.advisors.SkillPromptAugmentAdvisor;

// 方式一：指定技能目录（字符串路径），Advisor 内部创建 FileSystemSkillRegistry
SkillPromptAugmentAdvisor skillAdvisor = SkillPromptAugmentAdvisor.builder()
  .projectSkillsDirectory("./skills")       // 或绝对路径 /path/to/skills
  // .userSkillsDirectory("~/saa/skills")  // 可选，默认 ~/saa/skills
  .lazyLoad(false)                          // 可选，true 则首次请求时再加载技能
  .build();

ChatClient chatClient = ChatClient.builder(chatModel)
  .defaultAdvisors(skillAdvisor)
  .build();

// 调用时系统提示中会包含可用技能列表
String response = chatClient.prompt()
  .user("请介绍你有哪些技能")
  .call()
  .content();
```

使用已有 SkillRegistry 构建 SkillPromptAugmentAdvisor

```
import com.alibaba.cloud.ai.graph.skills.registry.SkillRegistry;
import
com.alibaba.cloud.ai.graph.skills.registry.filesystem.FileSystemSkillRegistry;

SkillRegistry registry = FileSystemSkillRegistry.builder()
  .projectSkillsDirectory("./skills")
  .build();

SkillPromptAugmentAdvisor skillAdvisor = SkillPromptAugmentAdvisor.builder()
  .skillRegistry(registry)
  .build();

ChatClient chatClient = ChatClient.builder(chatModel)
  .defaultAdvisors(skillAdvisor)
  .build();
```

**说明**：

- **SkillPromptAugmentAdvisor** 仅负责在系统提示中注入技能列表与加载说明，不注册 `read_skill` 工具。若需模型按需读取完整 SKILL.md，请在 ChatClient/Graph 中额外注册 `read_skill`（例如使用带 SkillsAgentHook 的 Agent 节点，或单独将 `ReadSkillTool` 注册为工具）。
- **Graph 中的 Agent 节点**：若节点内部使用 `ChatClient`，可在构建该 `ChatClient` 时加入 `SkillPromptAugmentAdvisor`；若节点使用 ReactAgent，则直接使用 **SkillsAgentHook**（会同时注入技能列表并注册 `read_skill`）。
- 技能通常需配合脚本执行（如技能目录下的 Python 脚本）和 Shell 命令才能在生产环境中正常使用。

---

## 最佳实践与性能建议

- **控制 SKILL.md 大小**：单文件建议约 1.5k–2k tokens，长内容放 `references/` 并在正文中列路径。
- **技能名称一致**：`name`、`read_skill` 参数、`groupedTools` 的 key 保持一致。
- **按需使用 groupedTools**：仅需「随技能激活」的工具用 groupedTools，其余用 Agent 的 `.tools()` 即可。
- **常用 API**：`hook.listSkills()`、`hook.hasSkill(name)`、`hook.getSkillCount()`；`registry.reload()`（ClasspathSkillRegistry 支持）。

# AgentsFramework进阶

主要包含：RAG、多智能体、人机协同、上下文工程、A2A、Workflow 等。

***提示：进阶部分官方提供的完整示例代码点击 链接 查看***

## Multi-agent

**Multi-agent** 将复杂的应用程序分解为多个协同工作的专业化Agent。与依赖单个Agent处理所有步骤不同，**Multi-agent架构**允许你将更小、更专注的Agent组合成协调的工作流。

Multi-agent系统在以下情况下很有用：

- 单个Agent拥有太多工具，难以做出正确的工具选择决策
- 上下文或记忆增长过大，单个Agent难以有效跟踪
- 任务需要**专业化**（例如：规划器、研究员、数学专家）

## Multi-agent模式

Spring AI Alibaba支持以下Multi-agent模式：

| 模式 | 工作原理 | 控制流 | 使用场景 |
| --- | --- | --- | --- |
| **Tool Calling** | Supervisor Agent将其他Agent作为*工具*调用。"工具"Agent不直接与用户对话——它们只执行任务并返回结果。 | 集中式：所有路由都通过调用Agent。 | 任务编排、结构化工作流。 |
| **Handoffs** | 当前的Agent决定将控制权转移给另一个Agent。活动Agent随之变更，用户可以继续与新的Agent直接交互。 | 去中心化：Agent可以改变当前由谁来担当活跃Agent。 | 跨领域对话、专家接管。 |

## 选择模式

| 问题 | 工具调用 (Agent Tool) | 交接（Handoffs） |
| --- | --- | --- |
| 需要集中控制工作流程? | ✅ 是 | ❌ 否 |
| 希望Agent直接与用户交互? | ❌ 否 | ✅ 是 |
| 专家之间复杂的、类人对话? | ❌ 有限 | ✅ 强 |

> 你可以混合使用两种模式——使用**交接**进行Agent切换，并让每个Agent**将子Agent作为工具调用**来执行专门任务。

关于工具调用模式的使用请查看 Agent Tool 文档。

## 自定义Agent上下文

Multi-agent设计的核心是**上下文工程**——决定每个Agent看到什么信息。Spring AI Alibaba 为你提供细粒度的控制：

- 将对话或状态的哪些部分传递给每个Agent
- 为子Agent定制专门的提示
- 包含/排除中间推理
- 为每个Agent自定义输入/输出格式

系统的质量**在很大程度上取决于**上下文工程。目标是确保每个Agent都能访问执行任务所需的正确数据，无论它是作为工具还是作为活动Agent。

# Instruction占位符

在 Multi-agent 系统中，`instruction` 支持使用**占位符**来动态引用状态中的数据。这是实现 Agent 之间数据传递的关键机制。

## 支持的占位符

| 占位符 | 说明 | 使用场景 |
| --- | --- | --- |
| `{input}` | 用户输入的原始内容 | 第一个Agent或需要用户输入的 Agent |
| `{outputKey}` | 引用其他Agent通过 `outputKey` 存储的输出 | 顺序执行中，后续Agent引用前面Agent的输出 |
| `{stateKey}` | 引用状态中的任意键值 | 访问状态中的任何数据 |

## 占位符工作原理

1. **自动替换**：系统会在执行 Agent 的 instruction 时，自动将占位符替换为对应的实际值
2. **状态查找**：占位符会从当前状态（`OverAllState`）中查找对应的值
3. **类型安全**：占位符的值会被转换为字符串并插入到 instruction 中

## 使用示例

```
// 第一个Agent：使用 {input} 获取用户输入
ReactAgent writerAgent = ReactAgent.builder()
    .name("writer_agent")
    .instruction("你是一个知名的作家。请根据用户的提问进行回答：{input}。")
    .outputKey("article")
    .build();

// 第二个Agent：使用 {article} 引用第一个Agent的输出
ReactAgent reviewerAgent = ReactAgent.builder()
    .name("reviewer_agent")
    .instruction("请对文章进行评审修正：\n{article}，最终返回评审修正后的文章内容")
    .outputKey("reviewed_article")
    .build();
```

## 最佳实践

1. **明确命名**：使用有意义的 `outputKey`，便于后续Agent引用
2. **占位符格式**：使用 `{keyName}` 格式，确保与 `outputKey` 一致
3. **错误处理**：如果占位符对应的值不存在，系统会保留原始占位符文本
4. **多值引用**：可以在一个 instruction 中使用多个占位符

> 💡 **提示**：占位符机制使得 Agent 之间的数据传递变得简单直观，无需手动管理状态传递逻辑。

# 交接Handoffs

💡 **重要参数说明**：在多 Agent 模式下，以下参数对于控制 Agent 行为和上下文传递至关重要：

1. **`instruction`**：用于在当前 Agent 节点处插入新的问题说明，引导模型和流程运行。支持使用占位符（如 `{input}`、`{outputKey}` 等）来动态引用状态中的数据，实现 Agent 之间的数据传递。
2. **`returnReasoningContent`**：控制子 Agent 的上下文是否返回父流程中。如果设置为 `false`，则其他 Agent 不会有机会看到这个子 Agent 内部的推理过程，它们只能看到这个 Agent 输出的内容（比如通过 `outputKey` 引用）。这对于减少上下文大小、提高效率非常有用。
3. **`includeContents`**：父流程中可能包含非常多子 Agent 的推理过程、每个子 Agent 的输出等。`includeContents` 用来控制当前子 Agent 执行时，是只基于自己的 `instruction` 给到的内容工作，还是会带上所有父流程的上下文。设置为 `false` 可以让子 Agent 专注于自己的任务，不受父流程复杂上下文的影响。
4. **`outputKey`**：指定输出内容的键名，可被后续 Agent 通过占位符引用（如 `{outputKey}`）。使用有意义的 `outputKey` 名称，便于后续 Agent 引用和状态管理。
5. **`systemPrompt` 和 `instruction`（Routing 和 Supervisor）**：`LlmRoutingAgent` 和 `SupervisorAgent` 还支持定制 `systemPrompt` 和 `instruction`，用于覆盖默认实现，控制后续任务流转的行为。`systemPrompt` 定义路由决策的整体框架，`instruction` 提供具体的路由指导。

在**交接**模式中，Agent可以直接将控制权传递给彼此。"活动"Agent会发生变化，用户与当前拥有控制权的Agent进行交互。

流程：

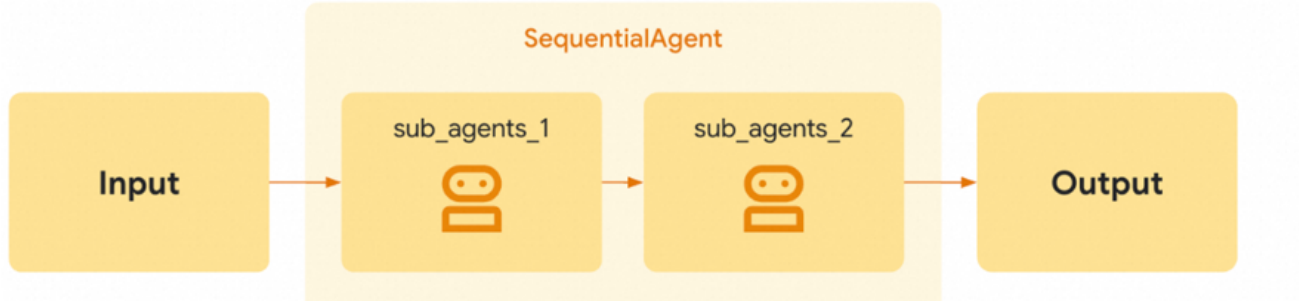
1. **当前Agent**决定它需要另一个Agent的帮助
2. 它将控制权（和状态）传递给**下一个Agent**
3. **新Agent**直接与用户交互，直到它决定再次交接或完成

## 顺序执行

在**顺序执行**模式中，多个Agent按预定义的顺序依次执行。每个Agent的输出成为下一个Agent的输入。

流程：

1. **Agent A**处理初始输入
2. **Agent A**的输出传递给**Agent B**
3. **Agent B**处理并传递给**Agent C**
4. 最后一个Agent返回最终结果




**实现**

SequentialAgent 实现示例

```java
// 创建专业化的子Agent
ReactAgent writerAgent = ReactAgent.builder()
  .name("writer_agent")
  .model(chatModel)
  .description("专业写作Agent")
  .instruction("你是一个知名的作家，擅长写作和创作。请根据用户的提问进行回答：{input}。")
  .outputKey("article")
  .build();
ReactAgent reviewerAgent = ReactAgent.builder()
  .name("reviewer_agent")
  .model(chatModel)
  .description("专业评审Agent")
  .instruction("你是一个知名的评论家，擅长对文章进行评论和修改。" +
               "对于散文类文章，请确保文章中必须包含对于西湖风景的描述。待评论文章：\n {article}"
               + "最终只返回修改后的文章，不要包含任何评论信息。")
  .outputKey("reviewed_article")
  .build();

// 创建顺序Agent
SequentialAgent blogAgent = SequentialAgent.builder()
  .name("blog_agent")
  .description("根据用户给定的主题写一篇文章，然后将文章交给评论员进行评论")
  .subAgents(List.of(writerAgent, reviewerAgent))
  .build();

// 使用
Optional<OverAllState> result = blogAgent.invoke("帮我写一个100字左右的散文");
if (result.isPresent()) {
  OverAllState state = result.get();
  // 访问第一个Agent的输出
  state.value("article").ifPresent(article -> {
      if (article instanceof AssistantMessage) {
          System.out.println("原始文章： " +
                             ((AssistantMessage) article)
                             .getText());
      }
  });
  // 访问第二个Agent的输出
  state.value("reviewed_article").ifPresent(reviewedArticle -> {
      if (reviewedArticle instanceof AssistantMessage) {
          System.out.println("评审后文章： " +
                             ((AssistantMessage) reviewedArticle)
                             .getText());
      }
  });
}
```

**关键特性**

1. **按顺序执行**：Agent按照 `subAgents` 列表中定义的顺序执行
2. **状态传递**：每个Agent的输出通过 `outputKey` 存储在状态中，可被后续Agent访问
3. **消息历史**：默认情况下，所有Agent共享消息历史
4. **推理内容控制**：使用 `returnReasoningContents` 控制是否在消息历史中包含中间推理

---

**控制推理内容**

控制推理内容示例

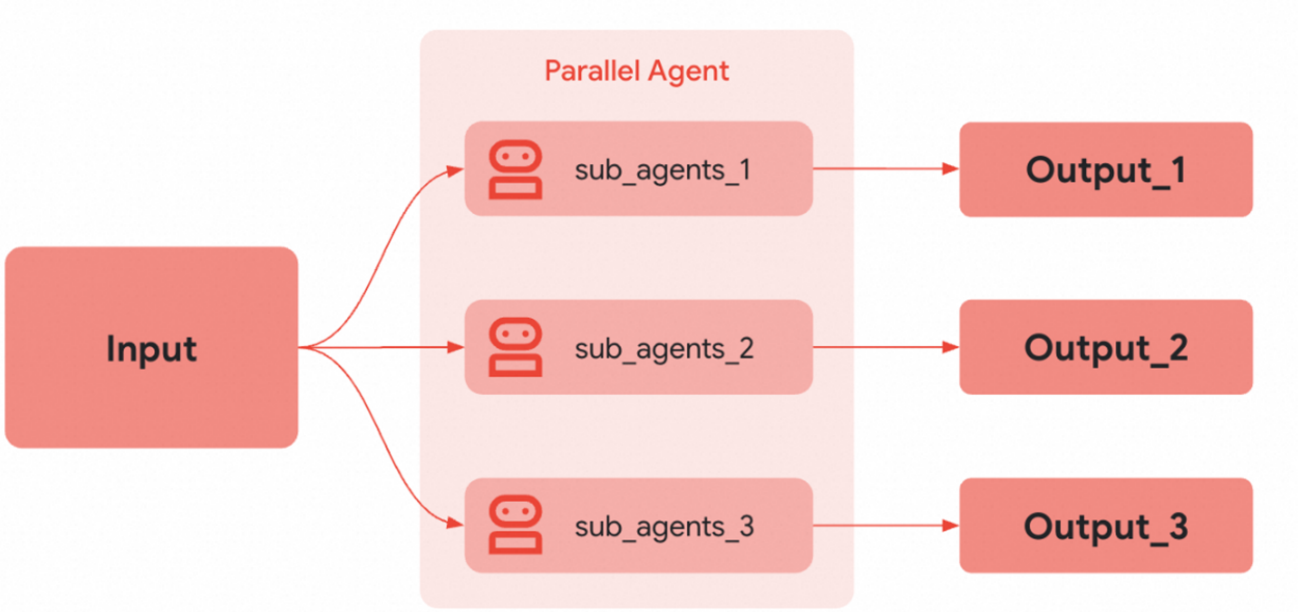
```
ReactAgent writerAgent = ReactAgent.builder()
  .name("writer_agent")
  .model(chatModel)
  .returnReasoningContents(true)
  .outputKey("article")
  .build();
ReactAgent reviewerAgent = ReactAgent.builder()
  .name("reviewer_agent")
  .model(chatModel)
  .instruction("请对文章进行评审修正：\n {article}，最终返回评审修正后的文章内容")
  .includeContents(true) // 包含上一个Agent的推理内容
  .returnReasoningContents(true)
  .outputKey("reviewed_article")
  .build();

SequentialAgent blogAgent = SequentialAgent.builder()
  .name("blog_agent")
  .subAgents(List.of(writerAgent, reviewerAgent))
  .build();

Optional<OverAllState> result = blogAgent.invoke("帮我写一个100字左右的散文");
if (result.isPresent()) {
  // 消息历史将包含所有工具调用和推理过程
  List<Message> messages = (List<Message>) result
      .get().value("messages")
      .orElse(List.of());
  // 包含所有中间步骤
  System.out.println("消息数量：" + messages.size());
}
```

## 并行执行

在**并行执行**模式中，多个Agent同时处理相同的输入。它们的结果被收集并合并。

流程：

1. 输入同时发送给**所有Agent**
2. 所有Agent**并行**处理
3. 结果被**合并**成单一输出




**实现**

ParallelAgent 实现示例

```
// 创建多个专业化Agent
ReactAgent proseWriterAgent = ReactAgent.builder()
  .name("prose_writer_agent")
  .model(chatModel)
  .description("专门写散文的AI助手")
  .instruction("你是一个知名的散文作家，擅长写优美的散文。" +
               "用户会给你一个主题：{input}，你只需要创作一篇100字左右的散文。")
  .outputKey("prose_result")
  .build();

ReactAgent poemWriterAgent = ReactAgent.builder()
  .name("poem_writer_agent")
  .model(chatModel)
  .description("专门写现代诗的AI助手")
  .instruction("你是一个知名的现代诗人，擅长写现代诗。" +
               "用户会给你的主题是：{input}，你只需要创作一首现代诗。")
  .outputKey("poem_result")
  .build();

ReactAgent summaryAgent = ReactAgent.builder()
  .name("summary_agent")
  .model(chatModel)
  .description("专门做内容总结的AI助手")
  .instruction("你是一个专业的内容分析师，擅长对主题进行总结和提炼。" +
               "用户会给你一个主题：{input}，你只需要对这个主题进行简要总结。")
  .outputKey("summary_result")
  .build();

// 创建并行Agent
ParallelAgent parallelAgent = ParallelAgent.builder()
  .name("parallel_creative_agent")
  .description("并行执行多个创作任务，包括写散文、写诗和做总结")
  .mergeOutputKey("merged_results")
  .subAgents(List.of(proseWriterAgent, poemWriterAgent, summaryAgent))
  .mergeStrategy(new ParallelAgent.DefaultMergeStrategy())
```

```
  .build();

// 使用
Optional<OverAllState> result = parallelAgent.invoke("以'西湖'为主题");
if (result.isPresent()) {
  OverAllState state = result.get();
  // 访问各个Agent的输出
  state.value("prose_result").ifPresent(r ->
          System.out.println("散文: " + r));
  state.value("poem_result").ifPresent(r ->
          System.out.println("诗歌: " + r));
  state.value("summary_result").ifPresent(r ->
          System.out.println("总结: " + r));
  // 访问合并后的结果
  state.value("merged_results").ifPresent(r ->
          System.out.println("合并结果: " + r));
}
```

**自定义合并策略**

你可以实现自定义的合并策略来控制如何组合多个Agent的输出:

自定义合并策略完整示例

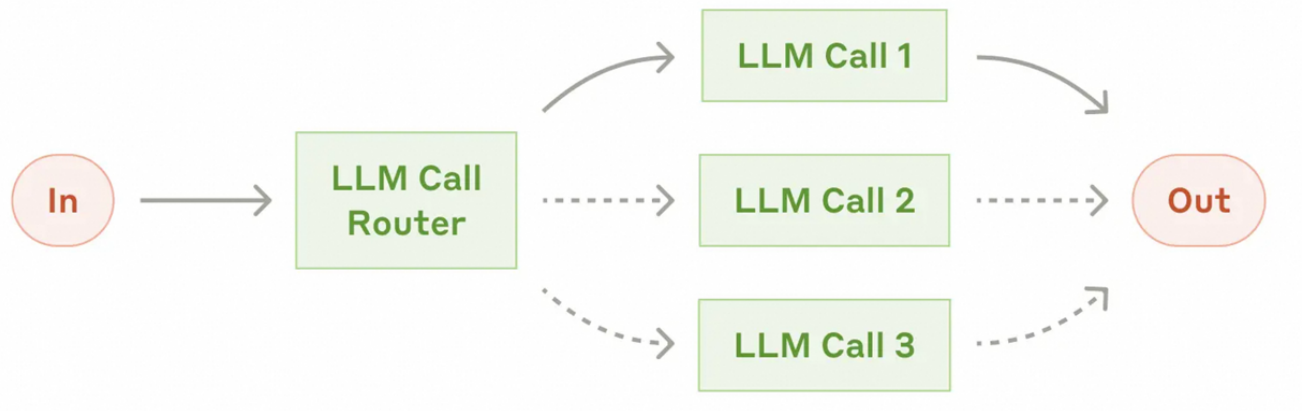
```
public class CustomMergeStrategy implements ParallelAgent.MergeStrategy {
  @Override
  public Map<String, Object> merge(Map<String, Object> mergedState, OverAllState
state) {
      // 从每个Agent的状态中提取输出
      state.data().forEach((key, value) -> {
          if (key.endsWith("_result")) {
              Message message = (Message) value;
              Object existing = mergedState.get("all_results");
              if (existing == null) {
                  mergedState.put("all_results", message.getText());
              }
              else {
                  mergedState.put("all_results",
                                  existing + "\n---\n" + message.getText());
              }
          }
      });
      return mergedState;
  }
}

// 使用自定义合并策略
ParallelAgent parallelAgent = ParallelAgent.builder()
  .name("parallel_agent")
  .subAgents(List.of(agent1, agent2, agent3))
  .mergeStrategy(new CustomMergeStrategy())
  .mergeOutputKey("final_merged_result")
  .build();
```

## 路由

在**路由模式**中，使用大语言模型（LLM）动态决定将请求路由到哪个子Agent。这种模式非常适合需要智能选择不同专家Agent的场景。

流程：

1. **路由Agent**接收用户输入
2. **LLM**分析输入并决定最合适的子Agent
3. **选中的子Agent**处理请求
4. 结果返回给用户




**实现**

LlmRoutingAgent 实现示例

```
// 创建专业化的子Agent
ReactAgent writerAgent = ReactAgent.builder()
  .name("writer_agent")
  .model(chatModel)
  .description("擅长创作各类文章，包括散文、诗歌等文学作品")
  .instruction("你是一个知名的作家，擅长写作和创作。请根据用户的提问进行回答。")
  .outputKey("writer_output")
  .build();
ReactAgent reviewerAgent = ReactAgent.builder()
  .name("reviewer_agent")
  .model(chatModel)
  .description("擅长对文章进行评论、修改和润色")
  .instruction("你是一个知名的评论家，擅长对文章进行评论和修改。" +
               "对于散文类文章，请确保文章中必须包含对于西湖风景的描述。")
  .outputKey("reviewer_output")
  .build();
ReactAgent translatorAgent = ReactAgent.builder()
  .name("translator_agent")
  .model(chatModel)
  .description("擅长将文章翻译成各种语言")
  .instruction("你是一个专业的翻译家，能够准确地将文章翻译成目标语言。")
  .outputKey("translator_output")
  .build();

// 创建路由Agent
LlmRoutingAgent routingAgent = LlmRoutingAgent.builder()
  .name("content_routing_agent")
  .description("根据用户需求智能路由到合适的专家Agent")
```

```
  .model(chatModel)
  .subAgents(List.of(writerAgent, reviewerAgent, translatorAgent))
  .build();

// 使用 - LLM会自动选择最合适的Agent
Optional<OverAllState> result1 = routingAgent
    .invoke("帮我写一篇关于春天的散文");
// LLM会路由到 writerAgent
Optional<OverAllState> result2 = routingAgent
    .invoke("请帮我修改这篇文章：春天来了，花开了。");
// LLM会路由到 reviewerAgent
Optional<OverAllState> result3 = routingAgent
    .invoke("请将以下内容翻译成英文：春暖花开");
// LLM会路由到 translatorAgent
```

### 关键特性

1. **智能路由**：LLM根据输入内容和子Agent的描述自动选择最合适的Agent
2. **灵活扩展**：可以轻松添加新的专家Agent，LLM会自动识别并路由
3. **描述驱动**：子Agent的 `description` 非常重要，它告诉LLM何时应该选择该Agent
4. **单次执行**：每次请求只路由到一个Agent执行

### 优化路由准确性

为了提高路由的准确性，需要注意以下几点：

优化路由准确性示例

```
// 1. 提供清晰明确的Agent描述
ReactAgent codeAgent = ReactAgent.builder()
  .name("code_agent")
  .model(chatModel)
  .description("专门处理编程相关问题，包括代码编写、调试、重构和优化。"
              + "擅长Java、Python、JavaScript等主流编程语言。")
  .instruction("你是一个资深的软件工程师 ...")
  .build();
// 2. 明确Agent的职责边界
ReactAgent businessAgent = ReactAgent.builder()
  .name("business_agent")
  .model(chatModel)
  .description("专门处理商业分析、市场研究和战略规划问题。"
              + "不处理技术实现细节。")
  .instruction("你是一个资深的商业分析师 ...")
  .build();
// 3. 使用不同领域的Agent避免重叠
LlmRoutingAgent routingAgent = LlmRoutingAgent.builder()
  .name("multi_domain_router")
  .model(chatModel)
  .subAgents(List.of(codeAgent, businessAgent, writerAgent))
.build();
```

### 自定义系统提示和指令

`LlmRoutingAgent` 支持通过 `systemPrompt` 和 `instruction` 来自定义路由决策行为，提供更精确的路由控制。

### 使用SystemPrompt

`systemPrompt` 用于设置路由决策的系统提示，会替换默认的系统提示。你可以通过它提供详细的决策规则和上下文：

LlmRoutingAgent 自定义系统提示示例

```
final String ROUTING_SYSTEM_PROMPT = """
你是一个智能的内容路由Agent，负责根据用户需求将任务路由到最合适的专家Agent。

## 你的职责
1. 仔细分析用户输入的意图和需求
2. 根据任务特性，选择最合适的专家Agent
3. 确保路由决策准确、高效

## 可用的子Agent及其职责

### writer_agent
- **功能**：擅长创作各类文章，包括散文、诗歌等文学作品
- **适用场景**：
* 用户需要创作新文章、散文、诗歌等原创内容
* 简单的写作任务
- **输出**：writer_output

### reviewer_agent
- **功能**：擅长对文章进行评论、修改和润色
- **适用场景**：
* 用户需要修改、评审或优化现有文章
* 需要提高文章质量
- **输出**：reviewer_output

### translator_agent
- **功能**：擅长将文章翻译成各种语言
- **适用场景**：
* 用户需要将内容翻译成其他语言
* 多语言转换需求
- **输出**：translator_output

## 决策规则

1. **写作任务**：如果用户需要创作新内容，选择 writer_agent
2. **修改任务**：如果用户需要修改或优化现有内容，选择 reviewer_agent
3. **翻译任务**：如果用户需要翻译内容，选择 translator_agent

## 响应格式
只返回Agent名称（writer_agent、reviewer_agent、translator_agent），不要包含其他解释。
""";

LlmRoutingAgent routingAgent = LlmRoutingAgent.builder()
  .name("content_routing_agent")
  .description("根据用户需求智能路由到合适的专家Agent")
  .model(chatModel)
  .systemPrompt(ROUTING_SYSTEM_PROMPT)
  .subAgents(List.of(writerAgent, reviewerAgent, translatorAgent))
  .build();
```

### 使用Instruction

`instruction` 用于设置路由决策的用户指令，会作为 `UserMessage` 添加到消息列表中。你可以通过它提供额外的上下文信息或特定的路由指导：

LlmRoutingAgent 使用指令示例

```
// 使用 instruction 提供额外的路由指导
final String ROUTING_INSTRUCTION = """
请根据用户的需求，选择最合适的Agent来处理任务。

特别注意：
- 如果用户明确提到"写"、"创作"、"生成"等词汇，优先选择 writer_agent
- 如果用户提到"修改"、"优化"、"评审"等词汇，选择 reviewer_agent
- 如果用户提到"翻译"、"转换语言"等词汇，选择 translator_agent
""";

LlmRoutingAgent routingAgent = LlmRoutingAgent.builder()
  .name("content_routing_agent")
  .description("根据用户需求智能路由到合适的专家Agent")
  .model(chatModel)
  .instruction(ROUTING_INSTRUCTION)
  .subAgents(List.of(writerAgent, reviewerAgent, translatorAgent))
  .build();
```

### 同时使用

你可以同时使用 `systemPrompt` 和 `instruction` 来提供更完整的路由决策上下文：

LlmRoutingAgent 同时使用 SystemPrompt 和 Instruction 示例

```
final String ROUTING_SYSTEM_PROMPT = """
你是一个智能的内容路由Agent，负责根据用户需求将任务路由到最合适的专家Agent。

## 可用的子Agent及其职责

### writer_agent
- **功能**：擅长创作各类文章
- **输出**：writer_output

### reviewer_agent
- **功能**：擅长对文章进行评论、修改和润色
- **输出**：reviewer_output

### translator_agent
- **功能**：擅长将文章翻译成各种语言
- **输出**：translator_output

## 响应格式
只返回Agent名称，不要包含其他解释。
""";

final String ROUTING_INSTRUCTION = """
请仔细分析用户输入，根据以下规则选择最合适的Agent：
1. 创作新内容 → writer_agent
```

```
2. 修改现有内容 → reviewer_agent
3. 翻译内容 → translator_agent
""";

LlmRoutingAgent routingAgent = LlmRoutingAgent.builder()
  .name("content_routing_agent")
  .description("根据用户需求智能路由到合适的专家Agent")
  .model(chatModel)
  .systemPrompt(ROUTING_SYSTEM_PROMPT)
  .instruction(ROUTING_INSTRUCTION)
  .subAgents(List.of(writerAgent, reviewerAgent, translatorAgent))
  .build();
```

**二者的区别**

| 特性 | SystemPrompt | Instruction |
| --- | --- | --- |
| **作用位置** | 系统消息（SystemMessage） | 用户消息（UserMessage） |
| **用途** | 定义路由Agent的角色、职责和决策规则 | 提供具体的路由指导或额外上下文 |
| **优先级** | 更高，影响整体路由行为 | 作为补充信息 |
| **使用场景** | 需要详细定义路由规则和Agent职责时 | 需要提供特定场景的路由指导时 |

💡 **提示**：

- 使用 `systemPrompt` 来定义路由Agent的整体行为和决策框架
- 使用 `instruction` 来提供特定场景的路由指导或额外上下文
- 两者可以配合使用，提供更精确的路由控制

## 混合模式示例

你可以组合不同的模式创建复杂的工作流：

混合模式示例

```
// 1. 创建研究Agent（并行执行）
ReactAgent webResearchAgent = ReactAgent.builder()
  .name("web_research")
  .model(chatModel)
  .description("从互联网搜索信息")
  .instruction("请搜索并收集关于以下主题的信息：{input}")
  .outputKey("web_data")
  .build();
ReactAgent dbResearchAgent = ReactAgent.builder()
  .name("db_research")
  .model(chatModel)
  .description("从数据库查询信息")
  .instruction("请从数据库中查询并收集关于以下主题的信息：{input}")
  .outputKey("db_data")
  .build();
ParallelAgent researchAgent = ParallelAgent.builder()
  .name("parallel_research")
  .description("并行收集多个数据源的信息")
```

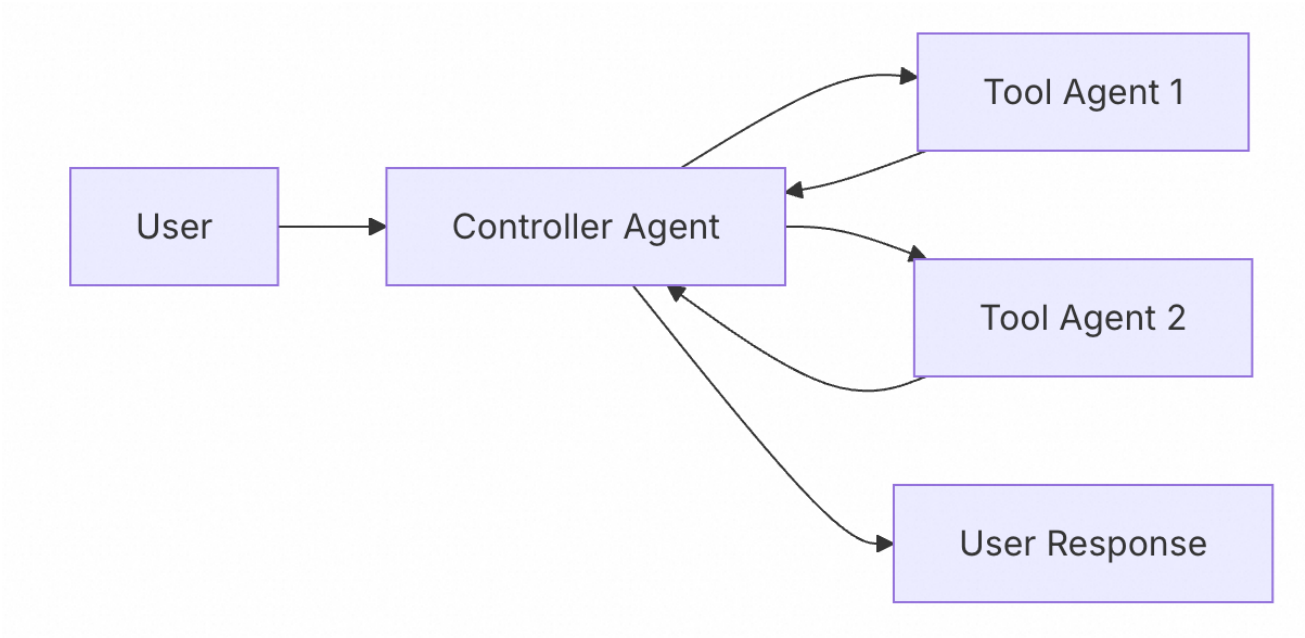
```
  .subAgents(List.of(webResearchAgent, dbResearchAgent))
  .mergeOutputKey("research_data")
  .build();
// 2. 创建分析Agent
ReactAgent analysisAgent = ReactAgent.builder()
  .name("analysis_agent")
  .model(chatModel)
  .description("分析研究数据")
  .instruction("请分析以下收集到的数据并提供见解：{research_data}")
  .outputKey("analysis_result")
  .build();
// 3. 创建报告Agent（路由选择格式）
ReactAgent pdfReportAgent = ReactAgent.builder()
  .name("pdf_report")
  .model(chatModel)
  .description("生成PDF格式报告")
  .instruction("""
              请根据研究结果和分析结果生成一份PDF格式的报告。
              研究结果：{research_data}
              分析结果：{analysis_result}
              """)
  .outputKey("pdf_report")
  .build();
ReactAgent htmlReportAgent = ReactAgent.builder()
  .name("html_report")
  .model(chatModel)
  .description("生成HTML格式报告")
  .instruction("""
              请根据研究结果和分析结果生成一份HTML格式的报告。
              研究结果：{research_data}
              分析结果：{analysis_result}
              """)
  .outputKey("html_report")
  .build();
LlmRoutingAgent reportAgent = LlmRoutingAgent.builder()
  .name("report_router")
  .description("根据需求选择报告格式")
  .model(chatModel)
  .subAgents(List.of(pdfReportAgent, htmlReportAgent))
  .build();
// 4. 组合成顺序工作流
SequentialAgent hybridWorkflow = SequentialAgent.builder()
  .name("research_workflow")
  .description("完整的研究工作流：并行收集 → 分析 → 路由生成报告")
  .subAgents(List.of(researchAgent, analysisAgent, reportAgent))
  .build();
// 使用
Optional<OverAllState> result = hybridWorkflow
              .invoke("研究AI技术趋势并生成HTML报告");
```

## Agent Tool

在**工具调用**中，一个Agent（"**控制器**"）将其他Agent视为*工具*（AgentTool），在需要时调用。控制器管理编排，而工具Agent执行特定任务并返回结果。

流程：

1. **控制器**接收输入并决定调用哪个工具（子Agent）
2. **工具Agent**根据控制器的指令运行其任务
3. **工具Agent**将结果返回给控制器
4. **控制器**决定下一步或完成任务




作为工具使用的Agent通常**不期望**与用户继续对话。它们的角色是执行任务并将结果返回给控制器Agent。如果你需要子Agent能够与用户对话，请改用**交接（HandOff）**模式。

关于工具调用模式的使用请查看 HandOffs 模式文档。

## 实现

下面是一个最小示例，其中主Agent通过工具定义访问单个子Agent：

AgentTool 基础示例

```
// 创建子Agent
ReactAgent writerAgent = ReactAgent.builder()
  .name("writer_agent")
  .model(chatModel)
  .description("可以写文章")
  .instruction("你是一个知名的作家，擅长写作和创作。请根据用户的提问进行回答。")
  .build();

// 创建主Agent，将子Agent作为工具
ReactAgent blogAgent = ReactAgent.builder()
  .name("blog_agent")
  .model(chatModel)
  .instruction("根据用户给定的主题写一篇文章。使用写作工具来完成任务。")
  .tools(AgentTool.getFunctionToolCallback(writerAgent))
  .build();

// 使用
Optional<OverAllState> result = blogAgent
    .invoke("帮我写一个100字左右的散文");
```

在这种模式中：

1. 主Agent在决定任务匹配子Agent的描述时调用工具
2. 子Agent独立运行并返回结果
3. 主Agent接收结果并继续编排

# 自定义点

你可以在几个点控制主Agent和子Agent之间的上下文传递：

1. **子Agent名称**（`"writer_agent"`）：这是主Agent引用子Agent的方式。由于它影响提示，请谨慎选择。
2. **子Agent描述**（`"可以写文章"`）：这是主Agent"知道"的关于子Agent的内容。它直接影响主Agent决定何时调用它。
3. **子Agent的输入**：你可以自定义此输入以更好地塑造子Agent如何解释任务。
4. **子Agent的输出**：这是传递回主Agent的**响应**。你可以调整返回的内容以控制主Agent如何解释结果。

# 控制子Agent的输入

有两个主要杠杆来控制主Agent传递给子Agent的输入：

- **修改提示词**——调整主Agent的提示或工具元数据（即子Agent的名称和描述），以更好地指导何时以及如何调用子Agent。
- **上下文注入**——通过使用 `inputSchema` 或 `inputType` 来定义结构化输入，使子Agent能够接收更丰富的上下文信息。

## 使用inputSchema

使用标准的 JSON Schema 格式定义输入结构，确保子Agent能够接收结构化的输入信息：

使用 inputSchema 示例

```
// 定义子Agent的输入Schema（标准 JSON Schema 格式）
String writerInputSchema = """
  {
      "type": "object",
      "properties": {
          "topic": {
              "type": "string"
          },
          "wordCount": {
              "type": "integer"
          },
          "style": {
              "type": "string"
          }
      },
      "required": ["topic", "wordCount", "style"]
  }
  """;

ReactAgent writerAgent = ReactAgent.builder()
  .name("structured_writer_agent")
  .model(chatModel)
```

```
  .description("根据结构化输入写文章")
  .instruction("你是一个专业作家。请严格按照输入的主题、字数和风格要求创作文章。")
  .inputSchema(writerInputSchema)
  .build();

ReactAgent coordinatorAgent = ReactAgent.builder()
  .name("coordinator_agent")
  .model(chatModel)
  .instruction("你需要调用写作工具来完成用户的写作请求。"
              +"请根据用户需求，使用结构化的参数调用写作工具。")
  .tools(AgentTool.getFunctionToolCallback(writerAgent))
  .build();

Optional<OverAllState> result = coordinatorAgent
    .invoke("请写一篇关于春天的散文，大约150字");
```

## 使用inputType

使用 Java 类型定义输入，框架会自动生成 JSON Schema：

使用 inputType 示例

```
// 定义输入类型
public record ArticleRequest(
  String topic,      // 文章主题
  int wordCount,     // 字数要求
  String style       // 文章风格
) {}

ReactAgent writerAgent = ReactAgent.builder()
  .name("typed_writer_agent")
  .model(chatModel)
  .description("根据类型化输入写文章")
  .instruction("""
              你是一个专业作家。请严格按照输入的
              topic（主题）、
              wordCount（字数）和 style（风格）要求创作文章。
              """)
  .inputType(ArticleRequest.class)
  .build();

ReactAgent coordinatorAgent = ReactAgent.builder()
  .name("coordinator_with_type_agent")
  .model(chatModel)
  .instruction("你需要调用写作工具来完成用户的写作请求。工具接收 JSON 格式的参数。")
  .tools(AgentTool.getFunctionToolCallback(writerAgent))
  .build();

Optional<OverAllState> result = coordinatorAgent
              .invoke("请写一篇关于秋天的现代诗，大约100字");
```

# 控制子Agent的输出

塑造主Agent从子Agent接收的内容的常见策略：

- 修改提示词

  ——优化子Agent的提示以指定应返回的确切内容。

  - 当输出不完整、过于冗长或缺少关键细节时很有用。
  - 常见的失败模式是子Agent执行工具调用或推理但**不在最终消息中包含结果**。提醒它控制器（和用户）只看到最终输出，因此必须在那里包含所有相关信息。
- **自定义输出格式**——使用 `outputSchema` 或 `outputType` 定义结构化输出格式。

## 使用outputSchema

使用 `BeanOutputConverter` 生成输出 Schema，提供类型安全和自动 schema 生成：

使用 outputSchema 示例

```
// 定义输出类型
public static class ArticleOutput {
  private String title;
  private String content;
  private int characterCount;
  // Getters and Setters
}

// 使用 BeanOutputConverter 生成 outputSchema
BeanOutputConverter<ArticleOutput> outputConverter =
    new BeanOutputConverter<>(ArticleOutput.class);
String format = outputConverter.getFormat();
ReactAgent writerAgent = ReactAgent.builder()
  .name("writer_with_output_schema")
  .model(chatModel)
  .description("写文章并返回结构化输出")
  .instruction("你是一个专业作家。请创作文章并严格按照指定的JSON格式返回结果。")
  .outputSchema(format)
  .build();
ReactAgent coordinatorAgent = ReactAgent.builder()
  .name("coordinator_output_schema")
  .model(chatModel)
  .instruction("调用写作工具完成用户请求，工具会返回结构化的文章数据。")
  .tools(AgentTool.getFunctionToolCallback(writerAgent))
  .build();

Optional<OverAllState> result = coordinatorAgent
    .invoke("写一篇关于冬天的短文");
```

## 使用outputType

使用 Java 类型定义输出，框架会自动生成输出 schema：

使用 outputType 示例

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
```

```
import com.alibaba.cloud.ai.graph.agent.AgentTool;
import org.springframework.ai.converter.BeanOutputConverter;

// 定义输出类型
public class ArticleOutput {
  private String title;
  private String content;
  private int characterCount;
  // getters and setters
}

ReactAgent writerAgent = ReactAgent.builder()
  .name("writer_with_output_type")
  .model(chatModel)
  .description("写文章并返回类型化输出")
  .instruction("""
               你是一个专业作家。请创作文章并返回包含
               title、content 和 characterCount 的结构化结果。
               """)
  .outputType(ArticleOutput.class)
  .build();

ReactAgent coordinatorAgent = ReactAgent.builder()
  .name("coordinator_output_type")
  .model(chatModel)
  .instruction("调用写作工具完成用户请求。")
  .tools(AgentTool.getFunctionToolCallback(writerAgent))
  .build();

Optional<OverAllState> result = coordinatorAgent
               .invoke("写一篇关于夏天的小诗");
```

## 完整类型化示例

同时使用 `inputType` 和 `outputType` 进行完整的类型化Agent工具调用：

完整类型化示例

```
// 定义输入和输出类型
public record ArticleRequest(String topic, int wordCount, String style) {}
public class ArticleOutput {
  private String title;
  private String content;
  private int characterCount;
  // getters and setters
}
public class ReviewOutput {
  private String comment;
  private boolean approved;
  private List<String> suggestions;
  // getters and setters
}
// 创建完整类型化的Agent
ReactAgent writerAgent = ReactAgent.builder()
  .name("full_typed_writer")
  .model(chatModel)
```

```
  .description("完整类型化的写作工具")
  .instruction("""根据结构化输入（topic、wordCount、style）
               创作文章，并返回结构化输出
               （title、content、characterCount）。
               """)
  .inputType(ArticleRequest.class)
  .outputType(ArticleOutput.class)
  .build();

ReactAgent reviewerAgent = ReactAgent.builder()
  .name("typed_reviewer")
  .model(chatModel)
  .description("完整类型化的评审工具")
  .instruction("""对文章进行评审，返回评审意见
               （comment、approved、suggestions）。
               """)
  .outputType(ReviewOutput.class)
  .build();

ReactAgent orchestratorAgent = ReactAgent.builder()
  .name("orchestrator")
  .model(chatModel)
  .instruction("""
               协调写作和评审流程。先调用写作工具创作文章，
               然后调用评审工具进行评审。
               """)
  .tools(
      AgentTool.getFunctionToolCallback(writerAgent),
      AgentTool.getFunctionToolCallback(reviewerAgent)
  )
  .build();

Optional<OverAllState> result = orchestratorAgent
               .invoke("请写一篇关于友谊的散文，约200字，需要评审");
```

## 多个子Agent作为工具

在实际应用中，主Agent通常需要访问多个不同的子Agent工具，根据任务需求选择合适的工具进行调用。这种模式允许你构建更灵活、更强大的多Agent系统。

多个子Agent作为工具示例

```
// 创建写作Agent
ReactAgent writerAgent = ReactAgent.builder()
  .name("writer_agent")
  .model(chatModel)
  .description("专门负责创作文章和内容生成")
  .instruction("你是一个专业作家，擅长各类文章创作。")
  .build();

// 创建翻译Agent
ReactAgent translatorAgent = ReactAgent.builder()
  .name("translator_agent")
  .model(chatModel)
  .description("专门负责文本翻译工作")
  .instruction("你是一个专业翻译，能够准确翻译多种语言。")
```

```
  .build();

// 创建总结Agent
ReactAgent summarizerAgent = ReactAgent.builder()
  .name("summarizer_agent")
  .model(chatModel)
  .description("专门负责内容总结和提炼")
  .instruction("你是一个内容总结专家，擅长提炼关键信息。")
  .build();

// 创建主Agent，集成多个工具
ReactAgent multiToolAgent = ReactAgent.builder()
  .name("multi_tool_coordinator")
  .model(chatModel)
  .instruction("你可以访问多个专业工具：写作、翻译和总结。" +
          "根据用户需求选择合适的工具来完成任务。")
  .tools(
      AgentTool.getFunctionToolCallback(writerAgent),
      AgentTool.getFunctionToolCallback(translatorAgent),
      AgentTool.getFunctionToolCallback(summarizerAgent)
  )
  .build();

// 使用 - 主Agent会根据需求自动选择合适的工具
Optional<OverAllState> result = multiToolAgent
    .invoke("请写一篇关于AI的文章，然后翻译成英文，最后给出摘要");
```

在这种模式中：

1. **专业化分工**：每个子Agent专注于特定领域（写作、翻译、总结等）
2. **灵活组合**：主Agent可以根据任务需求调用一个或多个工具
3. **智能路由**：主Agent根据工具的描述和用户需求，自动选择合适的工具
4. **顺序执行**：主Agent可以按顺序调用多个工具，实现复杂的工作流

**提示**：为每个子Agent提供清晰、准确的`description`非常重要，这直接影响主Agent如何选择合适的工具。描述应该简洁地说明Agent的职责和能力。

# A2A Agent

## A2A协议简介

随着智能体应用的广泛落地，智能体应用间的分布式部署与远程通信成为要解决的关键问题，Google 推出的 Agent2Agent（A2A）协议即面向这一落地场景：A2A 解决智能体与其他使用不同框架、部署在不同机器、不同公司的智能体进行有效通信和协作的问题。

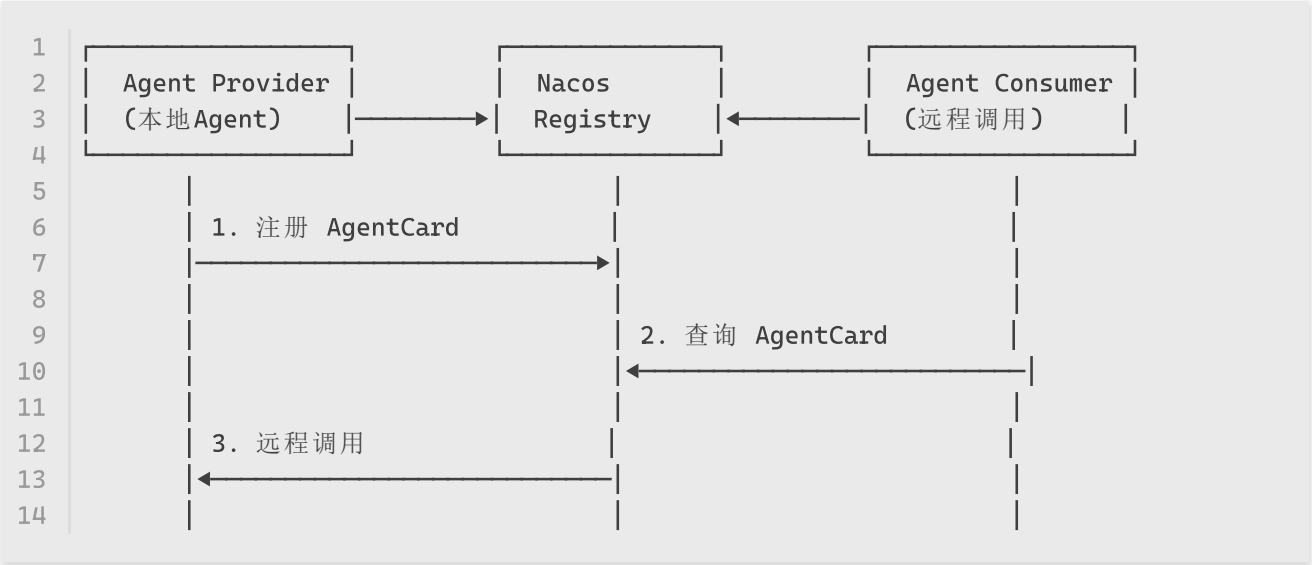
A2A 协议定义了智能体之间通信的标准方式，使得不同框架、不同部署环境的智能体能够无缝协作。

## A2A架构

Spring AI Alibaba 的 A2A 实现包含三个核心组件：

1. **A2A Server**：将本地 ReactAgent 暴露为 A2A 服务
2. **A2A Registry**：Agent 注册中心（支持 Nacos）
3. **A2A Discovery**：Agent 发现机制（支持 Nacos）

### 注册与发现流程




## 发布A2A智能体

要将一个智能体发布为 A2A 服务，需要：

1. 创建 ReactAgent Bean
2. 配置 A2A Server
3. （可选）配置 Nacos Registry 进行自动注册

### 定义本地Agent

定义本地 ReactAgent 示例

```
@Configuration
public class A2AAgentConfig {
  @Bean(name = "dataAnalysisAgent")
  public ReactAgent dataAnalysisAgent(ChatModel chatModel) {
      return ReactAgent.builder()
              .name("data_analysis_agent")
              .model(chatModel)
              .description("专门用于数据分析和统计计算的本地智能体")
              .instruction("你是一个专业的数据分析专家，擅长处理各类数据统计和分析任务。" +
                      "你能够理解用户的数据分析需求，提供准确的统计计算结果和专业的分析建议。")
              .outputKey("messages")
              .build();
  }
}
```

### 配置A2A Server

在 `application.yml` 中配置 A2A Server：

A2A Server 配置示例

```yaml
spring:
  ai:
    alibaba:
      a2a:
        server:
          version: 1.0.0
          card:
            name: data_analysis_agent
            description: 专门用于数据分析和统计计算的本地智能体
            provider:
              name: Spring AI Alibaba Documentation
              organization: Spring AI Alibaba
```

启动应用后，A2A Server 会自动：

- 根据 ReactAgent Bean 生成 AgentCard
- 暴露 REST API 端点：
  - `/.well-known/agent.json` - AgentCard 元数据
  - `/a2a/message` - Agent 调用端点

# 调用A2A远程智能体

## 发现Agent

Spring AI Alibaba 支持通过 `AgentCardProvider` 从注册中心（如 Nacos）发现远程 Agent。

使用 AgentCardProvider 发现并调用远程 Agent

```java
@Component
public class A2AExample {
  private final AgentCardProvider agentCardProvider;
  @Autowired
  public A2AExample(
      @Qualifier("nacosAgentCardProvider")
      AgentCardProvider agentCardProvider) {
      this.agentCardProvider = agentCardProvider;
  }
  public void callRemoteAgent() {
      // 通过 AgentCardProvider 从注册中心发现 Agent
      A2aRemoteAgent remote = A2aRemoteAgent.builder()
              .name("data_analysis_agent")
              // 从 Nacos 自动获取 AgentCard
              .agentCardProvider(agentCardProvider)
              .description("数据分析远程代理")
              .build();
      // 远程调用
      Optional<OverAllState> result = remote
          .invoke("请根据季度数据给出同比与环比分析概要。");
      result.ifPresent(state -> {
          System.out.println("调用成功: " + state.value("output"));
      });
  }
}
```

# 基于Nacos的A2A注册

Nacos3 最新版本提供了 A2A AgentCard 模型的存储与推送支持，因此可以作为 A2A Registry 实现。

Spring AI Alibaba 通过与 Nacos A2A Registry 集成，可以实现：

1. **A2A Server AgentCard 自动注册到 Nacos A2A Registry**
2. **A2a Client 自动订阅发现可用 AgentCard，实现 AgentCard 调用的负载均衡**

## 配置Nacos注册与发现

Nacos A2A 配置示例

```yaml
spring:
  ai:
    alibaba:
      a2a:
        nacos:
          server-addr: 127.0.0.1:8848
          username: nacos
          password: nacos
          discovery:
            enabled: true   # 启用服务发现（查询其他 Agent）
          registry:
            enabled: true   # 启用服务注册（注册本地 Agent）
        server:
          version: 1.0.0
          card:
            name: data_analysis_agent
            description: 专门用于数据分析和统计计算的本地智能体
            provider:
              name: Spring AI Alibaba Documentation
              organization: Spring AI Alibaba
```

**重要配置说明**：

- `registry.enabled: true` - 必须启用才能将 Agent 注册到 Nacos（服务提供者）
- `discovery.enabled: true` - 启用后才能通过 AgentCardProvider 发现其他 Agent（服务消费者）
- `server.card.name` - 必须与 ReactAgent Bean 的 `name` 一致

## 完整示例

A2A 一体化示例：注册 → 发现 → 调用

```java
@Component
public class A2AExample {
  private final AgentCardProvider agentCardProvider;
  private final ReactAgent localDataAnalysisAgent;
  @Autowired
  public A2AExample(
            @Qualifier("nacosAgentCardProvider") AgentCardProvider
agentCardProvider,
            @Qualifier("dataAnalysisAgent") ReactAgent localDataAnalysisAgent) {
      this.agentCardProvider = agentCardProvider;
```

```
        this.localDataAnalysisAgent = localDataAnalysisAgent;
    }
    public void runDemo() {
        // 1. 本地直连：验证本地注册的ReactAgent可用
        Optional<OverAllState> localResult = localDataAnalysisAgent.invoke(
            "请对上月销售数据进行趋势分析，并给出关键结论。"
        );
        localResult.ifPresent(state -> {
            System.out.println("本地调用成功");
        });

        // 2. 发现：通过AgentCardProvider从注册中心获取该Agent的AgentCard
        A2aRemoteAgent remote = A2aRemoteAgent.builder()
                .name("data_analysis_agent")
                 // 从Nacos自动获取AgentCard
                .agentCardProvider(agentCardProvider)
                .description("数据分析远程代理")
                .build();

        // 3. 远程调用：通过A2aRemoteAgent调用
        Optional<OverAllState> remoteResult = remote.invoke(
            "请根据季度数据给出同比与环比分析概要。"
        );
        remoteResult.ifPresent(state -> {
            System.out.println("远程调用成功");
        });
    }
}
```

# 验证和测试

## 验证本地AgentCard

```
curl http://localhost:8080/.well-known/agent.json
```

## 验证Nacos注册

1. 打开 Nacos 控制台：http://localhost:8848/nacos
2. 登录（nacos/nacos）
3. 查看A2A服务注册维度，应该能看到注册的Agent

## 注册与发现的区别

| 功能 | 配置项 | 作用 | 角色 |
| --- | --- | --- | --- |
| **Registry（注册）** | `registry.enabled: true` | 将本地 Agent 注册到 Nacos | 服务提供者 |
| **Discovery（发现）** | `discovery.enabled: true` | 从 Nacos 查询其他 Agent | 服务消费者 |

两者可独立配置，也可同时启用。当同时启用时：

- 作为**提供者**：注册本地 Agent 到 Nacos
- 作为**消费者**：可发现并调用其他已注册的 Agent（包括自己）

## 注意事项

1. **依赖要求**：
   - 需要添加 `spring-ai-alibaba-starter-a2a-nacos` 依赖
   - 确保 Nacos 服务正常运行
2. **AgentCard 元数据**：
   - `server.card.name` 必须与 ReactAgent Bean 的 `name` 一致
   - `server.card.provider` 可选，用于标识 Agent 提供者信息
3. **多 Agent 注册**：
   - 默认情况下，只有一个 Agent Bean 会被注册
   - 如需注册多个 Agent，需运行多个应用实例，每个实例配置不同的 Agent

## 故障排查

### Agent没有注册到Nacos

- 检查 `registry.enabled: true` 是否配置
- 查看应用日志，确认 Nacos Registry AutoConfiguration 是否生效
- 验证 Nacos 连接配置（server-addr、username、password）

### AgentCardProvider无法发现Agent

- 检查 `discovery.enabled: true` 是否配置
- 确认 Agent 已成功注册到 Nacos
- 验证 agent name 是否匹配

### 远程调用失败

- 确认目标 Agent 的 REST API 端点可访问
- 检查网络连接和防火墙配置
- 查看 A2A 消息传输日志

# RAG

大型语言模型（LLM）虽然强大，但有两个关键限制：

- **有限的上下文**——它们无法一次性摄取整个语料库
- **静态知识**——它们的训练数据在某个时间点被冻结

检索通过在查询时获取相关的外部知识来解决这些问题。这是**检索增强生成（RAG）**的基础：使用特定上下文的信息来增强 LLM 的回答。

## 构建知识库

**知识库**是用于检索的文档或结构化数据的存储库。

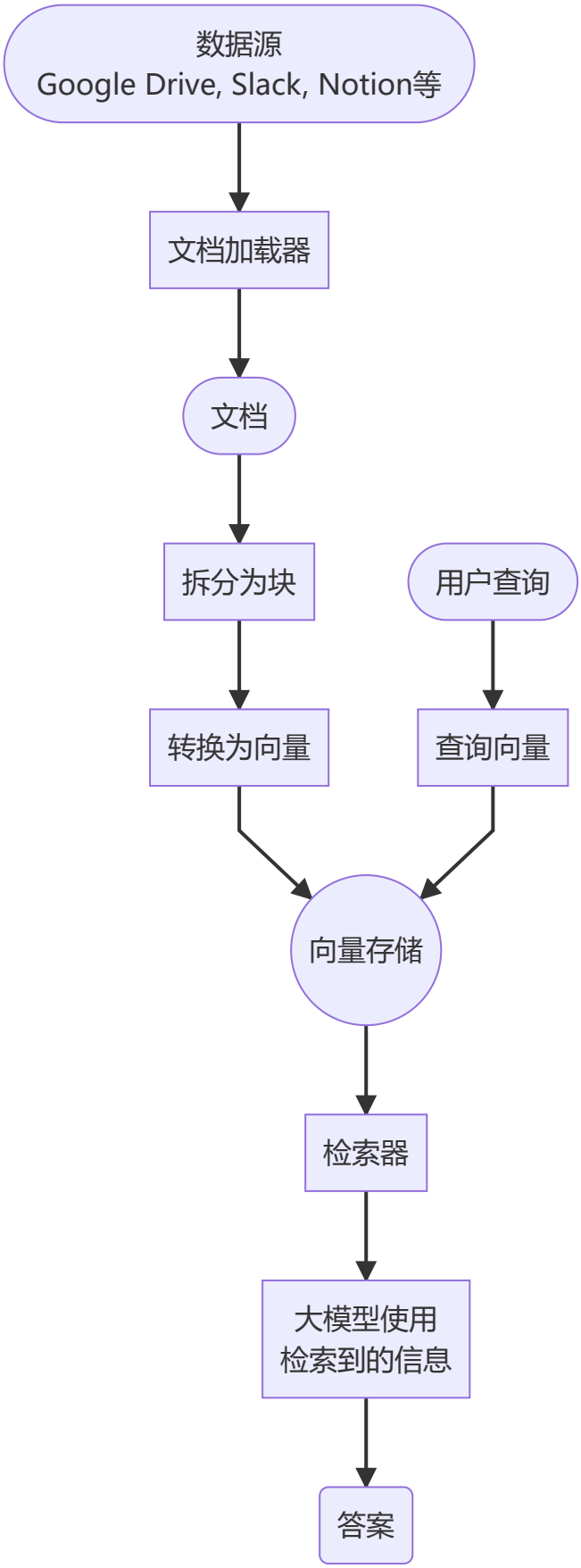
如果你需要自定义知识库，可以使用 Spring AI Alibaba 的文档加载器和向量存储从你自己的数据构建。

> 如果你已经有一个知识库（例如 SQL 数据库、CRM 或内部文档系统），你**不需要**重建它。你可以：
>
> - 将其连接为 Agent 的**工具**用于 Agentic RAG
> - 查询它并将检索到的内容作为上下文提供给 LLM（两步 RAG）

## 从检索到RAG

检索允许 LLM 在运行时访问相关上下文。但大多数实际应用更进一步：它们**将检索与生成集成**以产生基于事实的、上下文感知的答案。

这是**检索增强生成（RAG）**的核心思想。检索管道成为结合搜索和生成的更广泛系统的基础。



每个组件都是模块化的：你可以交换加载器、分割器、嵌入或向量存储，而无需重写应用程序的逻辑。

## 构建模块

在 Spring AI Alibaba 中，你可以使用以下组件构建 RAG 系统：

**文档加载器和解析器**

从外部源（文件、数据库、云存储、在线平台等）摄取数据，返回标准化的文档对象。Spring AI Alibaba 提供了丰富的 Document Reader 和 Document Parser 实现，支持 PDF、Word、Markdown、GitHub、Notion、语雀等多种数据源和格式。

**文本分割器**

将大型文档分解为更小的块，这些块可以单独检索并适合模型的上下文窗口。文本分割是 ETL 管道中的关键步骤，详见 ETL Pipeline。

**嵌入模型**

嵌入模型将文本转换为数字向量，使得具有相似含义的文本在向量空间中靠近在一起。Spring AI Alibaba 支持多种 Embedding Model 实现，包括 DashScope、OpenAI、Ollama 等。

**向量存储**

用于存储和搜索嵌入的专用数据库。Spring AI Alibaba 支持多种向量数据库，包括 Milvus、Pinecone、Redis、Elasticsearch 等。更多实现请查看 向量数据库文档。

**检索器**

检索器是一个接口，给定非结构化查询返回文档。Spring AI 提供了模块化的 RAG 架构，支持查询转换、查询扩展、文档后处理等高级功能，详见 Retrieval Augmented Generation。

## 构建知识库

使用ETL管道（Extract、Transform、Load）可以轻松构建知识库。Spring AI 提供了统一的 ETL 接口，支持链式处理

```
import org.springframework.ai.document.Document;
import org.springframework.ai.reader.TextReader;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.core.io.FileSystemResource;
import org.springframework.core.io.Resource;
import java.util.List;

// 1. 加载文档
Resource resource = new FileSystemResource("path/to/document.txt");
TextReader textReader = new TextReader(resource);
List<Document> documents = textReader.get();

// 2. 分割文档为块
TokenTextSplitter splitter = new TokenTextSplitter();
List<Document> chunks = splitter.apply(documents);

```

```
// 3. 将块添加到向量存储
vectorStore.add(chunks);

// 现在你可以使用向量存储进行检索
List<Document> results = vectorStore.similaritySearch("查询文本");
```

更多关于ETL管道的详细说明和高级用法，请参考 ETL Pipeline 文档。

## RAG架构

RAG 可以以多种方式实现，具体取决于你的系统需求。我们在下面的部分概述每种类型。

| 架构 | 描述 | 控制性 | 灵活性 | 延迟 | 使用场景示例 |
| --- | --- | --- | --- | --- | --- |
| **两步 RAG** | 检索总是在生成之前发生。简单且可预测 | ✅ 高 | ❌ 低 | ⚡ 快 | FAQ、文档机器人 |
| **Agentic RAG** | LLM 驱动的 Agent *决定何时*以及*如何*在推理过程中检索 | ❌ 低 | ✅ 高 | ⏳ 可变 | 具有多工具访问的研究助手 |
| **混合 RAG** | 结合两种方法的特点，包含验证步骤 | ⚖️ 中 | ⚖️ 中 | ⏳ 可变 | 带质量验证的领域特定问答 |

**提示**

**延迟**：延迟在**两步 RAG**中通常更**可预测**，因为 LLM 调用的最大次数是已知且有上限的。这种可预测性假设 LLM 推理时间是主要因素。但是，实际延迟也可能受检索步骤性能的影响——例如 API 响应时间、网络延迟或数据库查询——这些可能因使用的工具和基础设施而异。

### 两步RAG

在**两步 RAG**中，检索步骤总是在生成步骤之前执行。这种架构简单且可预测，适合许多应用，其中检索相关文档是生成答案的明确前提。

Spring AI 提供了开箱即用的 `QuestionAnswerAdvisor` 和 `RetrievalAugmentationAdvisor`，简化两步 RAG 的实现。这些 Advisor 自动处理检索和上下文增强，详见 Retrieval Augmented Generation。

**实现示例**

两步RAG实现示例

```
// 两步RAG：检索 → 生成
String userQuestion = "Spring AI Alibaba支持哪些模型？";
// 步骤1：检索相关文档
List<Document> relevantDocs = vectorStore.similaritySearch(userQuestion);
// 步骤2：从文档中构建上下文
String context = relevantDocs.stream()
    .map(Document::getText)
```

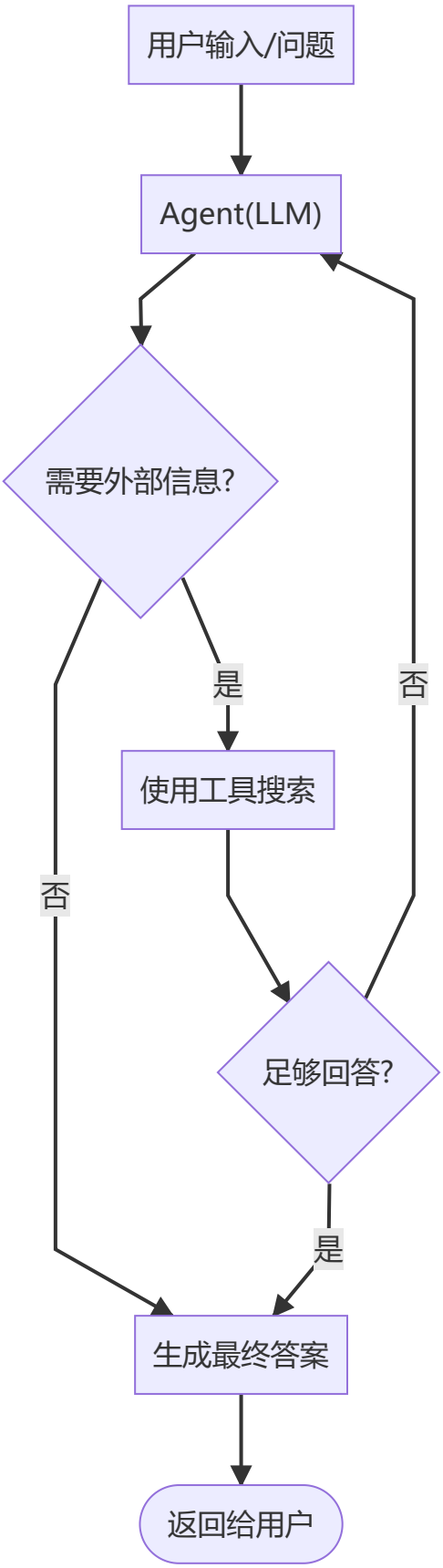
```
    .collect(Collectors.joining("\n\n"));
// 步骤3：结合上下文生成答案
ChatClient chatClient = ChatClient.builder(chatModel).build();
String answer = chatClient.prompt()
    .user(u -> u.text("基于以下上下文回答问题：\n\n上下文：\n"
                      + context + "\n\n问题：" + userQuestion))
    .call()
    .content();
System.out.println("答案：" + answer);
```

## AgenticRAG

**Agentic 检索增强生成（RAG）将检索增强生成的优势与基于 Agent 的推理相结合。Agent（由 LLM 驱动）不是在回答之前检索文档，而是逐步推理并决定在交互过程中何时**以及**如何**检索信息。

**提示**

Agent 启用 RAG 行为所需的唯一条件是访问一个或多个可以获取外部知识的**工具**——例如文档加载器、Web API 或数据库查询。




**实现示例**

AgenticRAG实现示例

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import org.springframework.ai.document.Document;
import org.springframework.ai.tool.ToolCallback;
```

```java
import org.springframework.ai.tool.function.FunctionToolCallback;
import org.springframework.ai.vectorstore.VectorStore;
import java.util.List;
import java.util.function.Function;
import java.util.stream.Collectors;

// 创建文档检索工具
public class DocumentSearchTool {
  private final VectorStore vectorStore;
  public DocumentSearchTool(VectorStore vectorStore) {
      this.vectorStore = vectorStore;
  }
  public record Request(String query) {}
  public record Response(String content) {}
  public Response search(Request request) {
      // 从向量存储检索相关文档
      List<Document> docs = vectorStore.similaritySearch(request.query());
      // 合并文档内容
      String combinedContent = docs.stream()
          .map(Document::getText)
          .collect(Collectors.joining("\n"));
      return new Response(combinedContent);
  }
}

// 使用示例
DocumentSearchTool searchTool = new DocumentSearchTool(vectorStore);
// 创建工具回调
ToolCallback searchCallback = FunctionToolCallback
    .builder("search_documents", searchTool::search)
    .description("搜索文档以查找相关信息")
    .inputType(DocumentSearchTool.Request.class)
    .build();

// 创建带有检索工具的Agent
ReactAgent ragAgent = ReactAgent.builder()
  .name("rag_agent")
  .model(chatModel)
  .instruction("你是一个智能助手。当需要查找信息时，使用search_documents工具。" +
            "基于检索到的信息回答用户的问题，并引用相关片段。")
  .tools(searchCallback)
  .build();

// Agent会自动决定何时调用检索工具
ragAgent.invoke("Spring AI Alibaba支持哪些向量数据库？");
```

在这个例子中：

1. Agent 接收用户问题
2. Agent 推理并决定是否需要检索文档
3. 如果需要，Agent调用 `search_documents` 工具
4. Agent使用检索到的信息生成答案
5. 如果信息不足，Agent 可以再次调用工具

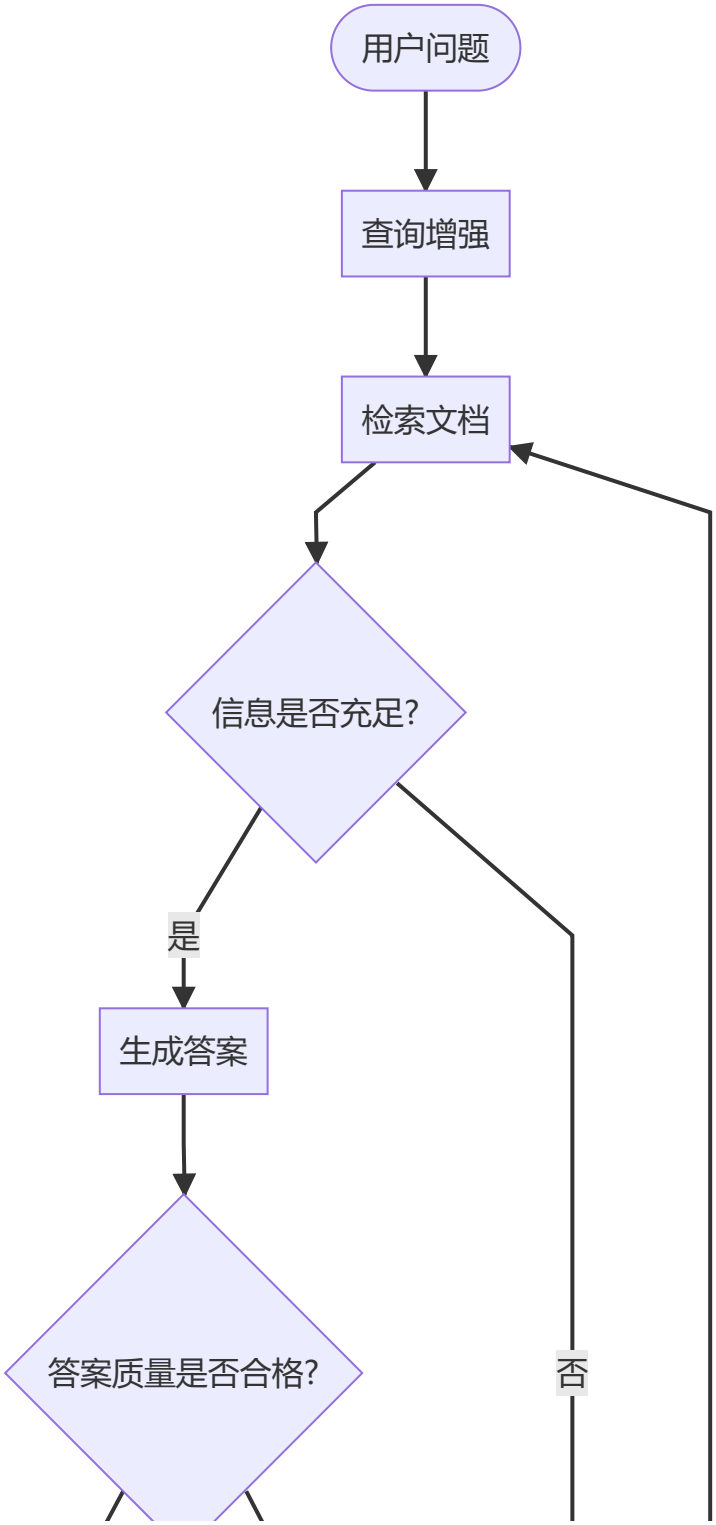
## 混合RAG

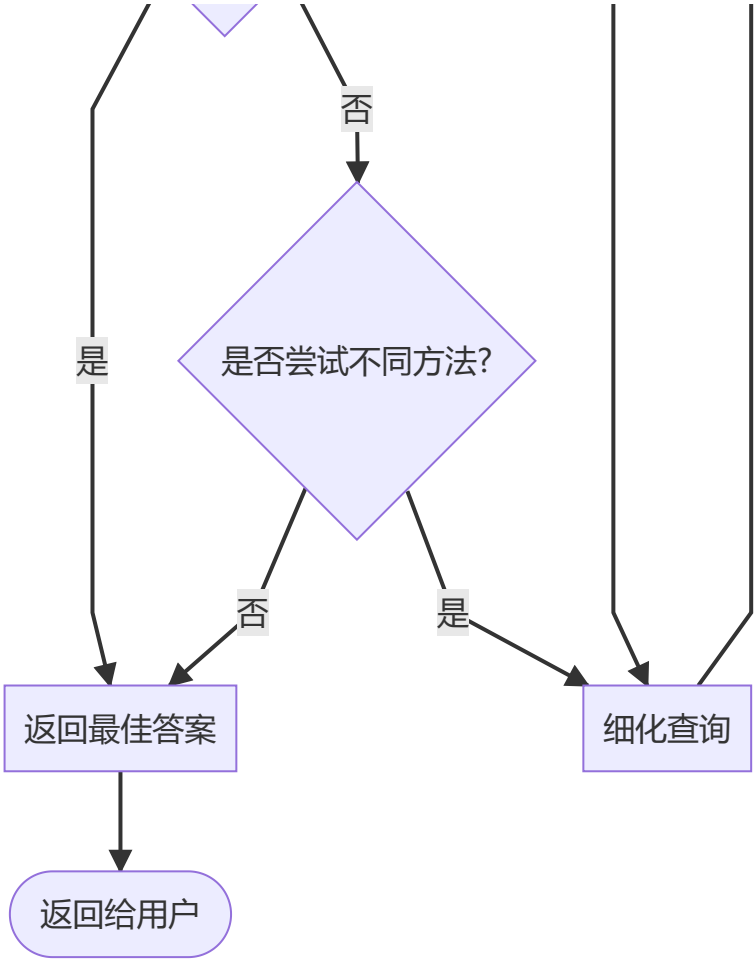
混合 RAG 结合了两步RAG 和 AgenticRAG 的特点。它引入了中间步骤，如查询预处理、检索验证和生成后检查。这些系统比固定管道提供更多灵活性，同时保持对执行的一定控制。

典型组件包括：

- **查询增强**：修改输入问题以提高检索质量。这可能涉及重写不清晰的查询、生成多个变体或用额外上下文扩展查询。
- **检索验证**：评估检索到的文档是否相关且充分。如果不够，系统可能会优化查询并再次检索。
- **答案验证**：检查生成的答案的准确性、完整性以及与源内容的一致性。如果需要，系统可以重新生成或修订答案。

架构通常支持这些步骤之间的多次迭代






**混合 RAG 的特点**：

1. **多工具检索（Agentic RAG）**：Agent 可以自主选择使用文档搜索、网络搜索等工具
2. **查询增强（两步 RAG）**：在 Agent 开始时通过 `AgentHook` 增强查询（**只执行一次**，避免每次 reasoning 循环都调用，降低成本），提高检索质量
3. **答案验证（两步 RAG）**：在生成后通过 Interceptor 验证答案质量，必要时重新生成
4. **灵活组合**：结合了 Agentic RAG 的灵活性和两步 RAG 的质量控制

**性能优化**：使用 `AgentHook` 进行查询增强，只在 Agent 开始时执行一次，而不是每次模型调用前都执行，显著降低了 LLM 调用成本。

这种架构适用于：

- 具有模糊或不明确查询的应用
- 需要验证或质量控制步骤的系统
- 领域特定的问答系统，要求高准确性

## 最佳实践

1. **选择合适的架构**：
    - 简单 FAQ → 两步 RAG
    - 复杂研究任务 → Agentic RAG
    - 需要质量保证 → 混合 RAG
2. **优化检索质量**：
    - 使用合适的文本分割策略
    - 选择高质量的嵌入模型

    - 实现查询重写和扩展
3. **控制上下文大小**：
    - 限制检索到的文档数量
    - 使用文档排序和过滤
    - 考虑模型的上下文窗口限制
4. **监控和评估**：
    - 跟踪检索质量指标
    - 评估答案准确性
    - 收集用户反馈
5. **性能优化**：
    - 缓存常见查询的检索结果
    - 使用异步检索
    - 批量处理文档嵌入

## Spring AI Alibaba RAG组件

Spring AI Alibaba 提供了构建 RAG 系统的核心组件和模块化架构：

RAG核心组件导入

```
// ETL Pipeline 组件
import org.springframework.ai.document.Document;
import org.springframework.ai.reader.DocumentReader;// 文档读取器
import org.springframework.ai.transformer.DocumentTransformer;// 文档转换器（如文本分割）
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.VectorStore;// 向量存储（DocumentWriter）

// 嵌入模型
import org.springframework.ai.embedding.EmbeddingModel;
import org.springframework.ai.embedding.EmbeddingRequest;
import org.springframework.ai.embedding.EmbeddingResponse;

// 模块化 RAG 组件 - Pre-Retrieval
import org.springframework.ai.rag.query.transformer.QueryTransformer;
import org.springframework.ai.rag.query.transformer.RewriteQueryTransformer;
import org.springframework.ai.rag.query.transformer.CompressionQueryTransformer;
import org.springframework.ai.rag.query.transformer.TranslationQueryTransformer;
import org.springframework.ai.rag.query.expander.QueryExpander;
import org.springframework.ai.rag.query.expander.MultiQueryExpander;

// 模块化 RAG 组件 - Retrieval
import org.springframework.ai.rag.retriever.DocumentRetriever;
import org.springframework.ai.rag.retriever.VectorStoreDocumentRetriever;
import org.springframework.ai.rag.document.join.DocumentJoiner;
import org.springframework.ai.rag.document.join.ConcatenationDocumentJoiner;

// 模块化 RAG 组件 - Post-Retrieval
import org.springframework.ai.rag.postprocessor.DocumentPostProcessor;

// 模块化 RAG 组件 - Generation
import org.springframework.ai.rag.query.augmenter.QueryAugmenter;
import org.springframework.ai.rag.query.augmenter.ContextualQueryAugmenter;
```

```
33
34  // Advisor API（开箱即用的 RAG 流程）
35  import org.springframework.ai.chat.advisor.vectorstore.QuestionAnswerAdvisor;
36  import org.springframework.ai.rag.RetrievalAugmentationAdvisor;
37
38  // ChatModel 和 Agent
39  import org.springframework.ai.chat.model.ChatModel;
40  import org.springframework.ai.chat.client.ChatClient;
41  import com.alibaba.cloud.ai.graph.agent.ReactAgent;
42
43  // 工具（用于 Agentic RAG）
44  import org.springframework.ai.tool.ToolCallback;
45  import org.springframework.ai.tool.function.FunctionToolCallback;
```

### 模块化RAG架构

Spring AI 实现了模块化RAG架构，支持灵活的组件组合：

- **Pre-Retrieval（检索前）**：查询转换（重写、压缩、翻译）、查询扩展（多查询扩展）
- **Retrieval（检索）**：文档搜索、文档连接
- **Post-Retrieval（检索后）**：文档后处理（重排序、去重、压缩）
- **Generation（生成）**：查询增强、上下文注入

这种模块化设计允许你根据需求灵活组合不同的组件，构建适合特定场景的 RAG 流程。详细说明请参考 Retrieval Augmented Generation 模块文档。

## 相关文档

### Agent Framework

- Tools - 创建检索工具
- Agents - 构建AgenticRAG
- Memory - 对话记忆管理
- Multi-Agent - 多Agent协作

### RAG 组件

- Retrieval Augmented Generation - RAG API 和模块化架构
- ETL Pipeline - 数据提取、转换和加载
- Document Readers - 文档加载器实现
- Document Parsers - 文档解析器实现
- Embeddings - 嵌入模型 API
- Vector Databases - 向量数据库集成

# 生态集成-ToolCalling

Spring AI Alibaba 提供了丰富的 Tool Calling 扩展实现，允许 AI 模型与外部 API 和工具进行交互，增强模型的能力。本文档将详细介绍 Tool Calling 的使用方法，并列出所有支持的扩展实现。

## 概述

Tool Calling（工具调用，也称为 Function Calling）是 AI 应用中的常见模式，允许模型与一组 API 或工具进行交互，增强模型的能力。

工具主要用于：

- **信息检索**：从外部数据源检索信息，如数据库、Web 服务、文件系统或 Web 搜索引擎。例如，获取当前天气、检索最新新闻、查询数据库等。
- **执行操作**：在软件系统中执行特定操作，如发送电子邮件、在数据库中创建新记录、提交表单或触发工作流。例如，预订航班、填写表单、生成代码等。

Spring AI Alibaba 扩展了 Spring AI 的 Tool Calling 功能，提供了多种预构建的工具实现，包括：

- 搜索引擎（百度搜索、Google Scholar、Tavily Search 等）
- 翻译服务（阿里翻译、百度翻译、Google 翻译等）
- 地图服务（高德地图、百度地图、腾讯地图等）
- 数据服务（天气、快递、新闻等）
- 开发工具（GitHub、JSON 处理、正则表达式等）
- 其他工具（时间、Python 执行、敏感词过滤等）

## 详细使用说明

# Python Tool 示例

`PythonTool` 使用 GraalVM polyglot 在沙箱环境中执行 Python 代码。这个工具允许 AI 代理执行 Python 代码片段并获取结果。

### 依赖配置

使用 Maven 添加依赖：

```
<dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-starter-tool-calling-python</artifactId>
    <version>${version}</version>
</dependency>
<!-- GraalVM Polyglot for Python execution -->
<dependency>
    <groupId>org.graalvm.polyglot</groupId>
    <artifactId>polyglot</artifactId>
    <version>${graalvm.polyglot.version}</version>
</dependency>
<dependency>
    <groupId>org.graalvm.polyglot</groupId>
    <artifactId>python-community</artifactId>
    <version>${graalvm.polyglot.version}</version>
    <type>pom</type>
</dependency>
```

**注意**：Python Tool 需要 GraalVM polyglot 依赖，这些依赖已作为可选依赖包含。如果需要使用，请确保 GraalVM polyglot 在 classpath 中。

## 自动配置

Python Tool 默认启用，会自动注册为 `ToolCallback` 并可供 AI 代理使用。如果需要禁用，可以在配置文件中设置：

```yaml
spring:
  ai:
    alibaba:
      python:
        tool:
          enabled: false
```

## 基本使用

自动装配后的tool名称是`python`

**示例 1：ChatClient中使用**

```java
// 创建 ChatClient 并添加 Python Tool
ChatClient chatClient = ChatClient.builder(chatModel)
    .defaultToolNames("python")
    .build();
// 使用工具进行对话
String question = "请计算1+1";
String response = chatClient
    .prompt(question)
    .call()
    .content();
System.out.println(response);
```

**示例 2：直接使用**

```java
PythonService pythonService = new PythonService();
// 创建请求
PythonService.Request request = new PythonService.Request("2 + 2");
// 执行Python代码
PythonService.Response result = pythonService
    .apply(request, new ToolContext(new HashMap<>()));
System.out.println("Result: " + result);
```

**示例 3：执行复杂计算**

```java
// 创建 ChatClient 并添加 Python Tool
ChatClient chatClient = ChatClient.builder(chatModel)
    .defaultToolNames("python")
    .build();

// 构建提示信息
String prompt = String
    .format("""
    请使用 Python 工具计算以下数据的统计信息：
    %s
```

```
    请计算平均值、最大值、最小值和总和。
    """,
    "40 90 100 300");

// 使用工具进行对话
String content = chatClient.prompt(prompt)
    .call()
    .content();
System.out.println(content);
```

### 示例 4：字符串处理

```
PythonService pythonService = new PythonService();
ToolContext toolContext = new ToolContext(new HashMap<>());
// 字符串连接
PythonService.Request request1 = new PythonService.Request(
    "'Hello, ' + 'World'"
);
PythonService.Response result1 = pythonService.apply(request1, toolContext);
System.out.println(result1);
// 列表操作
PythonService.Request request2 = new PythonService.Request(
    "[1, 2, 3, 4, 5][:3]"
);
PythonService.Response result2 = pythonService.apply(request2, toolContext);
System.out.println(result2);
// 字典操作
PythonService.Request request3 = new PythonService.Request(
    "{'name': 'Alice', 'age': 30}.get('name')"
);
PythonService.Response result3 = pythonService.apply(request3, toolContext);
System.out.println(result3);
```

### 示例 5：SpringBoot中使用

```
@Component
public class PythonToolAgent {
    private final ChatClient chatClient;
    public PythonToolAgent(ChatModel chatModel) {
        this.chatClient = ChatClient.builder(chatModel)
                .defaultToolNames("python")
                .build();
    }
    public String askWithPython(String question) {
        return chatClient.prompt(question)
                .call()
                .content();
    }
}
// 测试使用
@Resource
PythonToolAgent pythonToolAgent;
String s = pythonToolAgent.askWithPython("请计算1+1");
System.out.println(s);
```

**示例 6：自定义ToolCallback**

```java
@Configuration
public class PythonToolConfiguration {
    @Resource
    private PythonService pythonService;
    // 函数名为pythonToolCallback可以阻止自动装配
    @Bean
    public ToolCallback pythonToolCallback() {
        return FunctionToolCallback.builder("python", pythonService)
            .description("执行Python代码并返回结果。支持数学计算、字符串处理、列表操作等。")
            .inputType(PythonService.Request.class)
            .build();
    }
}
```

**示例 7：多工具组合使用**

```java
ChatClient client = ChatClient.builder(chatModel)
    .defaultToolNames("python", TimeConstants.TOOL_NAME)
    .build();
String prompt = "请使用工具计算明年高考还有多少天\n\n" +
    "工具使用说明:\n" +
    "1. 使用 " + TimeConstants.TOOL_NAME + "工具，获取当前时间。\n" +
    "2. 使用 python 工具，执行Python代码并返回结果。\n";
String response = client.prompt(prompt)
    .call()
    .content();
System.out.println(response);
```

## 安全特性

Python Tool 在沙箱环境中运行，具有以下安全限制：

- **文件 I/O 已禁用**：无法读取或写入文件
- **本地访问已禁用**：无法访问本地系统资源
- **进程创建已禁用**：无法创建新进程
- **默认限制所有访问**：所有访问默认受限

这些限制确保了 Python 代码的执行安全性，防止恶意代码对系统造成损害。

## 支持的数据类型

Python Tool 支持以下 Python 数据类型的返回：

- **字符串**：直接返回字符串值
- **数字**：转换为字符串返回
- **布尔值**：转换为字符串返回
- **数组/列表**：转换为字符串表示形式（如 `[1, 2, 3]`）
- **其他类型**：使用 `toString()` 方法转换

### 注意事项

1. **GraalVM 依赖**：确保 GraalVM polyglot 依赖在 classpath 中
2. **性能考虑**：每次执行都会创建新的 Context，对于频繁调用可能需要优化
3. **错误处理**：代码执行错误会被捕获并返回错误消息
4. **代码限制**：由于安全限制，某些 Python 功能可能不可用
5. **资源管理**：Context 会自动关闭，无需手动管理

## 支持的扩展实现

下表列出了 Spring AI Alibaba 提供的所有 Tool Calling 扩展实现：

| 模块名称 | ArtifactId | 功能说明 |
| --- | --- | --- |
| **阿里翻译** | `spring-ai-alibaba-starter-tool-calling-alitranslate` | 阿里云翻译服务 |
| **阿里云 AI 搜索** | `spring-ai-alibaba-starter-tool-calling-aliyunaisearch` | 阿里云 AI 搜索 |
| **高德地图** | `spring-ai-alibaba-starter-tool-calling-amap` | 高德地图 AP |
| **百度地图** | `spring-ai-alibaba-starter-tool-calling-baidumap` | 百度地图 AP |
| **百度搜索** | `spring-ai-alibaba-starter-tool-calling-baidusearch` | 百度搜索 AP |
| **百度翻译** | `spring-ai-alibaba-starter-tool-calling-baidutranslate` | 百度翻译 AP |
| **Brave Search** | `spring-ai-alibaba-starter-tool-calling-bravesearch` | Brave 搜索引擎 |
| **Common** | `spring-ai-alibaba-starter-tool-calling-common` | 通用工具基础 |
| **钉钉** | `spring-ai-alibaba-starter-tool-calling-dingtalk` | 钉钉 API |
| **DuckDuckGo** | `spring-ai-alibaba-starter-tool-calling-duckduckgo` | DuckDuckGo 搜索 |
| **Firecrawl** | `spring-ai-alibaba-starter-tool-calling-firecrawl` | Firecrawl 网页抓取 |
| **GitHub Toolkit** | `spring-ai-alibaba-starter-tool-calling-githubtoolkit` | GitHub API |

| 模块名称 | Artifactid | 功能说明 |
| --- | --- | --- |
| **Google Scholar** | `spring-ai-alibaba-starter-tool-calling-googlescholar` | Google Scholar |
| **Google 翻译** | `spring-ai-alibaba-starter-tool-calling-googletranslate` | Google 翻译 API |
| **Google Trends** | `spring-ai-alibaba-starter-tool-calling-googletrends` | Google Trends |
| **Jina Crawler** | `spring-ai-alibaba-starter-tool-calling-jinacrawler` | Jina 爬虫 |
| **JSON 处理** | `spring-ai-alibaba-starter-tool-calling-jsonprocessor` | JSON 数据处理 |
| **快递 100** | `spring-ai-alibaba-starter-tool-calling-kuaidi100` | 快递 100 A |
| **飞书** | `spring-ai-alibaba-starter-tool-calling-larksuite` | 飞书 API |
| **Memcached** | `spring-ai-alibaba-starter-tool-calling-memcached` | Memcache |
| **Metaso** | `spring-ai-alibaba-starter-tool-calling-metaso` | Metaso 搜索 |
| **Microsoft 翻译** | `spring-ai-alibaba-starter-tool-calling-microsofttranslate` | Microsoft 翻译 |
| **MinIO** | `spring-ai-alibaba-starter-tool-calling-minio` | MinIO 对象存储 |
| **Ollama Search Model** | `spring-ai-alibaba-starter-tool-calling-ollamasearchmodel` | Ollama 搜索模型 |
| **OpenAlex** | `spring-ai-alibaba-starter-tool-calling-openalex` | OpenAlex 学术 |
| **OpenTripMap** | `spring-ai-alibaba-starter-tool-calling-opentripmap` | OpenTripMa |
| **Python** | `spring-ai-alibaba-starter-tool-calling-python` | Python 代码执行 |

| | | |
| --- | --- | --- |
| **正则表达式** | `spring-ai-alibaba-starter-tool-calling-regex` | 正则表达式处理 |
| **Searches** | `spring-ai-alibaba-starter-tool-calling-searches` | 搜索工具集合 |
| **敏感词过滤** | `spring-ai-alibaba-starter-tool-calling-sensitivefilter` | 敏感词过滤 |

| 模块名称 | ArtifactId | 功能说明 |
| --- | --- | --- |
| **SerpAPI** | `spring-ai-alibaba-starter-tool-calling-serpapi` | SerpAPI |
| | | |
| **新浪新闻** | `spring-ai-alibaba-starter-tool-calling-sinanews` | 新浪新闻 AP |
| **Tavily Search** | `spring-ai-alibaba-starter-tool-calling-tavilysearch` | Tavily 搜索 |
| **腾讯地图** | `spring-ai-alibaba-starter-tool-calling-tencentmap` | 腾讯地图 AP |
| **时间工具** | `spring-ai-alibaba-starter-tool-calling-time` | 时间处理 |
| **头条新闻** | `spring-ai-alibaba-starter-tool-calling-toutiaonews` | 头条新闻 AP |
| **TripAdvisor** | `spring-ai-alibaba-starter-tool-calling-tripadvisor` | TripAdvisor API |
| **Tushare** | `spring-ai-alibaba-starter-tool-calling-tushare` | Tushare 金融数据 |
| **天气** | `spring-ai-alibaba-starter-tool-calling-weather` | 天气 API |
| **Wikipedia** | `spring-ai-alibaba-starter-tool-calling-wikipedia` | Wikipedia API |
| **世界银行数据** | `spring-ai-alibaba-starter-tool-calling-worldbankdata` | 世界银行数据 |
| **有道翻译** | `spring-ai-alibaba-starter-tool-calling-youdaotranslate` | 有道翻译 AP |
| **语雀** | `spring-ai-alibaba-starter-tool-calling-yuque` | 语雀 API |

## 使用说明

所有 Tool Calling 实现都遵循相同的使用模式：

1. **添加依赖**：在 `pom.xml` 或 `build.gradle` 中添加相应的依赖
2. **自动配置**：大多数工具都支持 Spring Boot 自动配置，会自动注册为 `ToolCallback`
3. **在 ChatClient 中使用**：通过 `ChatClient.builder().defaultToolNames( ... )` 添加工具
4. **AI 模型调用**：AI 模型会根据对话内容自动决定是否调用工具

## 通用接口

所有 Tool 都实现了 `BiFunction<Request, ToolContext, String>` 接口，并通过 `FunctionToolCallback` 包装为 `ToolCallback`：

```
public interface ToolCallback {
    String getName();
    String getDescription();
    ToolResponse call(ToolRequest request);
}
```

## 配置说明

大多数工具都支持通过 Spring Boot 配置属性进行配置：

```
spring:
  ai:
    alibaba:
      tool-name:
        enabled: true  # 启用/禁用工具
        # 其他工具特定配置
```

## 认证和API Key

许多工具需要 API Key 或认证信息：

- **搜索引擎**：通常需要 API Key
- **翻译服务**：需要服务提供商的 API Key
- **地图服务**：需要地图服务商的 Key
- **其他服务**：根据具体服务要求配置

请参考各工具的文档了解具体的配置方法。

## 最佳实践

1. **工具选择**：根据实际需求选择合适的工具，避免添加不必要的工具
2. **错误处理**：工具调用可能失败，确保有适当的错误处理机制
3. **性能优化**：对于频繁调用的工具，考虑缓存结果
4. **安全性**：确保 API Key 等敏感信息的安全存储
5. **成本控制**：某些工具（如搜索、翻译）可能产生费用，注意使用量

## 更多信息

- 每个工具的具体使用方法和配置选项，请参考各模块的文档
- 对于需要认证的工具，请确保正确配置访问凭证
- 某些工具可能需要额外的依赖或运行时环境
- 建议查看 Spring AI 官方文档了解 Tool Calling 的更多细节

# 生态集成-MCPs

**提示：初次使用 MCP?** 请从我们的 MCP 入门指南 开始，快速了解并查看实践示例。

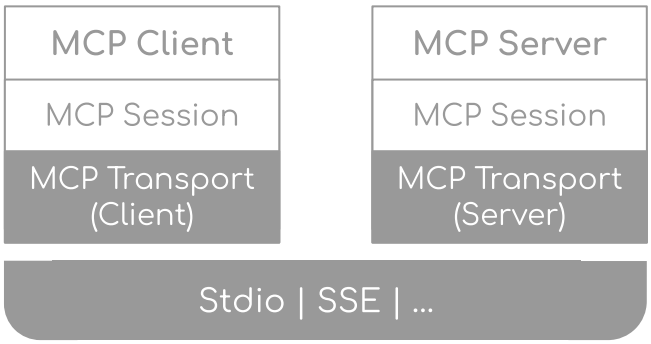
Model Context Protocol (MCP) 是一个标准化协议，使 AI 模型能够以结构化的方式与外部工具和资源交互。 可以将其视为 AI 模型与现实世界之间的桥梁——允许它们通过一致的接口访问数据库、API、文件系统和其他外部服务。 它支持多种传输机制，以在不同环境中提供灵活性。

MCP Java SDK 提供了 Model Context Protocol 的 Java 实现，通过同步和异步通信模式实现与 AI 模型和工具的标准化交互。

Spring AI 通过专用的 Boot Starters 和 MCP Java Annotations 全面支持 MCP，使构建能够无缝连接到外部系统的复杂 AI 驱动应用程序变得前所未有的简单。 这意味着 Spring 开发者可以参与 MCP 生态系统的两个方面——构建消费 MCP 服务器的 AI 应用程序，以及创建将基于 Spring 的服务暴露给更广泛 AI 社区的 MCP 服务器。 使用 Spring Initializer 引导支持 MCP 的 AI 应用程序。

## JavaSDK架构

Java MCP 实现遵循三层架构，将关注点分离以提高可维护性和灵活性：




### Client/Server Layer (Top)

顶层处理主要应用程序逻辑和协议操作：

- *McpClient* - 管理客户端操作和服务器连接
- *McpServer* - 处理服务器端协议操作和客户端请求
- 两个组件都利用下面的会话层进行通信管理

### Session Layer (Middle)

中间层管理通信模式并维护连接状态：

- *McpSession* - 核心会话管理接口
- *McpClientSession* - 客户端特定的会话实现
- *McpServerSession* - 服务器端特定的会话实现

## Transport Layer (Bottom)

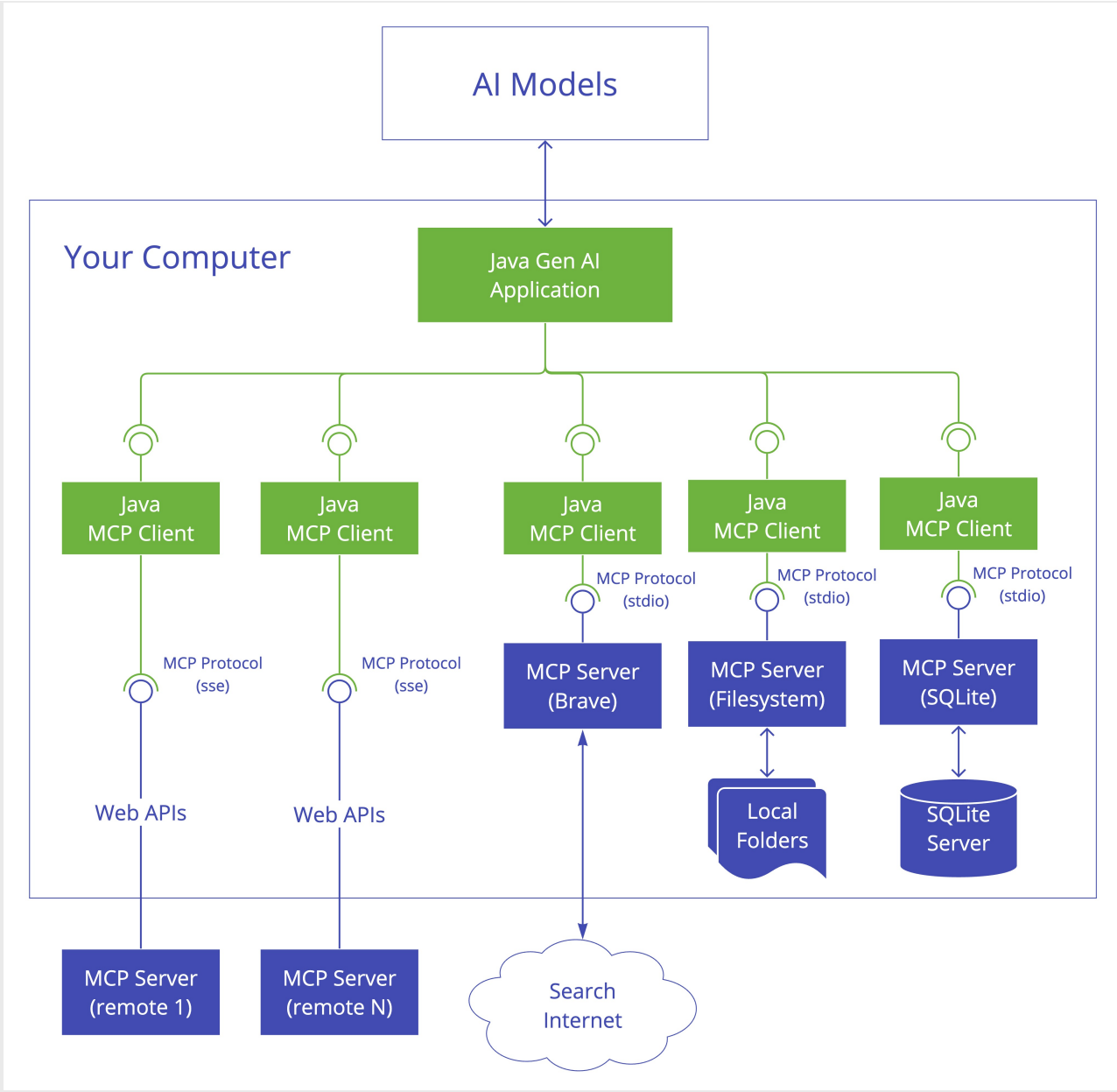
底层处理实际的消息传输和序列化：

- *McpTransport* - 管理 JSON-RPC 消息序列化和反序列化
- 支持多种传输实现（STDIO、HTTP/SSE、Streamable-HTTP 等）
- 为所有更高级别的通信提供基础

**MCP Client**

MCP Client 是 Model Context Protocol (MCP) 架构中的关键组件，负责建立和管理与 MCP 服务器的连接。它实现协议的客户端，处理：

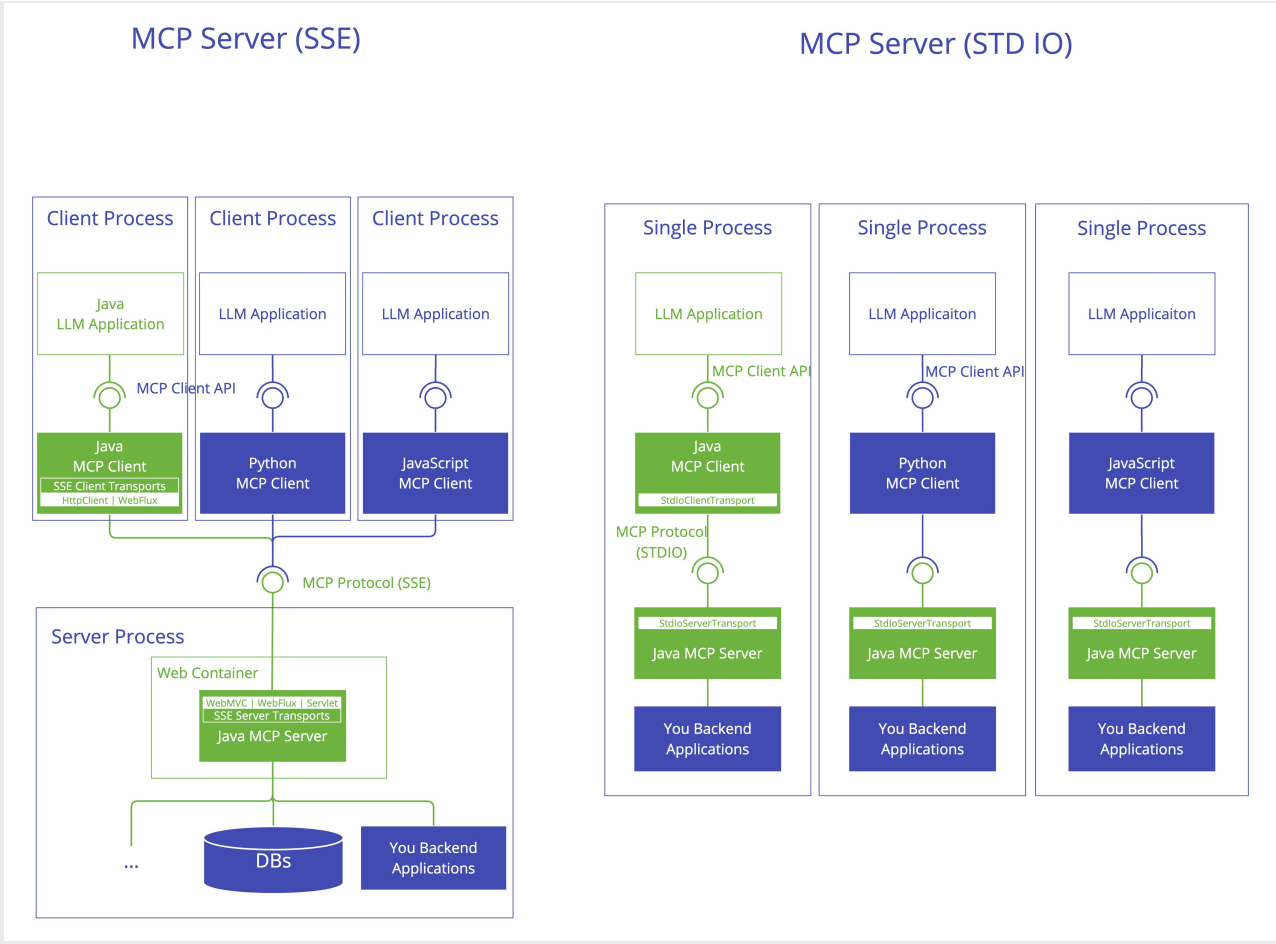
* 协议版本协商以确保与服务器的兼容性
* 能力协商以确定可用功能
* 消息传输和 JSON-RPC 通信
* 工具发现和执行
* 资源访问和管理
* Prompt 系统交互
* 可选功能：
** Roots 管理
** Sampling 支持
* 同步和异步操作
* 传输选项：
** 基于 Stdio 的传输，用于基于进程的通信
** 基于 Java HttpClient 的 SSE 客户端传输
** WebFlux SSE 客户端传输，用于响应式 HTTP 流式传输




**MCP Server**

MCP Server 是 Model Context Protocol (MCP) 架构中的基础组件，向客户端提供工具、资源和能力。它实现协议的服务器端，负责：

* 服务器端协议操作实现
** 工具暴露和发现
** 基于 URI 的资源管理
** Prompt 模板提供和处理
** 与客户端的能力协商
** 结构化日志记录和通知
* 并发客户端连接管理
* 同步和异步 API 支持
* 传输实现：
** Stdio、Streamable-HTTP、Stateless Streamable-HTTP、SSE




# SpringAI MCP集成

Spring AI 通过以下 Spring Boot starters 提供 MCP 集成:

## Client Starters

- `spring-ai-starter-mcp-client` - 核心 starter，提供 `STDIO`、基于 Servlet 的 `Streamable-HTTP`、`Stateless Streamable-HTTP` 和 `SSE` 支持
- `spring-ai-starter-mcp-client-webflux` - 基于 WebFlux 的 `Streamable-HTTP`、`Stateless Streamable-HTTP` 和 `SSE` 传输实现

## Server Starters

### STDIO

| Server Type | Dependency | Property |
| --- | --- | --- |
| Standard Input/Output (STDIO) | `spring-ai-starter-mcp-server` | `spring.ai.mcp.server.stdio=true` |

### WebMVC

| Server Type | Dependency | Property |
| --- | --- | --- |
| SSE WebMVC | `spring-ai-starter-mcp-server-webmvc` | `spring.ai.mcp.server.protocol=SS` empty |

| Streamable-HTTP WebMVC | `spring-ai-starter-mcp-server-webmvc` | `spring.ai.mcp.server.protocol=ST` |
|---|---|---|

| Stateless Streamable-HTTP WebMVC | `spring-ai-starter-mcp-server-webmvc` | `spring.ai.mcp.server.protocol=ST` |
|---|---|---|

### WebMVC (Reactive)

| Server Type | Dependency | Property |
|---|---|---|
| SSE WebFlux | `spring-ai-starter-mcp-server-webflux` | `spring.ai.mcp.server.protocol=S` empty |
| Streamable-HTTP WebFlux | `spring-ai-starter-mcp-server-webflux` | `spring.ai.mcp.server.protocol=S` |
| Stateless Streamable-HTTP WebFlux | `spring-ai-starter-mcp-server-webflux` | `spring.ai.mcp.server.protocol=S` |

## SpringAI MCP注解

除了程序化的 MCP 客户端和服务器配置外，Spring AI 还通过 MCP Annotations 模块为 MCP 服务器和客户端提供基于注解的方法处理。 这种方法使用简洁的声明式编程模型和 Java 注解简化了 MCP 操作的创建和注册。

MCP Annotations 模块使开发者能够：

- 使用简单的注解创建 MCP 工具、资源和 prompts
- 以声明方式处理客户端通知和请求
- 减少样板代码并提高可维护性
- 自动为工具参数生成 JSON schemas
- 访问特殊参数和上下文信息

主要功能包括：

- Server Annotations: `@McpTool`、`@McpResource`、`@McpPrompt`、`@McpComplete`
- Client Annotations: `@McpLogging`、`@McpSampling`、`@McpElicitation`、`@McpProgress`
- Special Parameters: `McpSyncServerExchange`、`McpAsyncServerExchange`、`McpTransportContext`、`McpMeta`
- *Automatic Discovery*: 具有可配置包包含/排除的注解扫描
- *Spring Boot Integration*: 与 MCP Boot Starters 无缝集成

---

# MCP Client Boot Starter

Spring AI MCP (Model Context Protocol) Client Boot Starter 为 Spring Boot 应用程序中的 MCP 客户端功能提供自动配置。 它支持同步和异步客户端实现，并提供多种传输选项。

MCP Client Boot Starter 提供：

- 多个客户端实例的管理
- 自动客户端初始化（如果启用）
- 支持多个命名传输（STDIO、Http/SSE 和 Streamable HTTP）
- 与 Spring AI 的工具执行框架集成
- 工具过滤功能，用于选择性包含/排除工具
- 可自定义的工具名称前缀生成，以避免命名冲突
- 适当的生命周期管理，在应用程序上下文关闭时自动清理资源
- 通过 customizers 自定义客户端创建

## Starters

### Standard Client

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-mcp-client</artifactId>
</dependency>
```

标准 starter 通过 `STDIO`（进程内）、`SSE`、`Streamable-HTTP` 和 `Stateless Streamable-HTTP` 传输同时连接到一个或多个 MCP 服务器。 SSE 和 Streamable-Http 传输使用基于 JDK HttpClient 的传输实现。 每个到 MCP 服务器的连接都会创建一个新的 MCP 客户端实例。 您可以选择 `SYNC` 或 `ASYNC` MCP 客户端（注意：不能混合使用同步和异步客户端）。 对于生产部署，我们建议使用基于 WebFlux 的 SSE 和 StreamableHttp 连接，使用 `spring-ai-starter-mcp-client-webflux`。

### WebFlux Client

WebFlux starter 提供与标准 starter 类似的功能，但使用基于 WebFlux 的 Streamable-Http、Stateless Streamable-HTTP 和 SSE 传输实现。

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-mcp-client-webflux</artifactId>
</dependency>
```

## 配置属性

### 通用配置

通用属性以 `spring.ai.mcp.client` 为前缀：

| 属性 | 描述 | 默认值 |
| --- | --- | --- |
| `enabled` | 启用/禁用MCP客户端 | `true` |

| 属性 | 描述 | 默认值 |
| --- | --- | --- |
| `name` | MCP客户端实例的名称 | `spring-ai-mcp-client` |
| `version` | MCP客户端实例的版本 | `1.0.0` |
| `initialized` | 是否在创建时初始化客户端 | `true` |
| `request-timeout` | MCP客户端请求的超时时间 | `20s` |
| `type` | 客户端类型（`SYNC`或`ASYNC`）。所有客户端必须保持同步或异步，不支持混合使用 | `SYNC` |
| `root-change-notification` | 为所有客户端启用/禁用根变更通知 | `true` |
| `toolcallback.enabled` | 启用/禁用与Spring AI工具执行框架集成的MCP工具回调 | `true` |

### 注解属性

MCP客户端注解提供了一种使用Java注解实现 MCP 客户端处理器的声明式方法。客户端 `mcp-annotations` 属性以 `spring.ai.mcp.client.annotation-scanner` 为前缀：

| 属性 | 描述 | 默认值 |
| --- | --- | --- |
| `enabled` | 启用/禁用MCP客户端注解自动扫描 | `true` |

### Stdio传输属性

标准I/O传输的属性以 `spring.ai.mcp.client.stdio` 为前缀：

| 属性 | 描述 | 默认值 |
| --- | --- | --- |
| `servers-configuration` | 包含MCP服务器配置（JSON 格式）的资源 | - |
| `connections` | 命名的 stdio 连接配置映射 | - |
| `connections.[name].command` | 为 MCP 服务器执行的命令 | - |
| `connections.[name].args` | 命令参数列表 | - |
| `connections.[name].env` | 服务器进程的环境变量映射 | - |

示例配置

```yaml
spring:
  ai:
    mcp:
      client:
        stdio:
          root-change-notification: true
          connections:
            server1:
              command: /path/to/server
              args:
                - --port=8080
                - --mode=production
```

```
            env:
              API_KEY: your-api-key
              DEBUG: "true"
```

或者，您可以使用外部JSON文件配置stdio连接，使用 Claude Desktop format：

```yaml
spring:
  ai:
    mcp:
      client:
        stdio:
          servers-configuration: classpath:mcp-servers.json
```

Claude Desktop格式如下所示：

```json
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/username/Desktop",
        "/Users/username/Downloads"
      ]
    }
  }
}
```

## Windows标准IO配置

**重要提示：** 在 Windows 上，像 `npx`、`npm` 和 `node` 这样的命令是作为**批处理文件**（`.cmd`）实现的，而不是原生可执行文件。Java 的 `ProcessBuilder` 无法直接执行批处理文件，需要 `cmd.exe /c` 包装器。

**为什么Windows需要特化**

当 Java 的 `ProcessBuilder`（由 `StdioClientTransport` 内部使用）尝试在 Windows 上生成进程时，它只能执行：

- 原生可执行文件（`.exe` 文件）
- `cmd.exe` 可用的系统命令

像 `npx.cmd`、`npm.cmd` 甚至 `python.cmd`（来自 Microsoft Store）这样的 Windows 批处理文件需要 `cmd.exe` shell 来执行它们。

**方案:cmd.exe包装**

使用 `cmd.exe /c` 包装批处理文件命令：

**Windows配置:**

```
{
  "mcpServers": {
    "filesystem": {
      "command": "cmd.exe",
      "args": [
        "/c",
        "npx",
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "C:\\Users\\username\\Desktop"
      ]
    }
  }
}
```

**Linux/macOS配置:**

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/username/Desktop"
      ]
    }
  }
}
```

**跨平台程序化配置**

对于需要在不同平台上工作而无需单独配置文件应用程序，在 Spring Boot 应用程序中使用 OS 检测：

```
@Bean(destroyMethod = "close")
@ConditionalOnMissingBean(McpSyncClient.class)
public McpSyncClient mcpClient() {
    ServerParameters stdioParams;
    if (isWindows()) {
        // Windows: cmd.exe /c npx approach
        var winArgs = new ArrayList<>(Arrays.asList(
            "/c", "npx", "-y", "@modelcontextprotocol/server-filesystem",
"target"));
        stdioParams = ServerParameters.builder("cmd.exe")
            .args(winArgs)
            .build();
    } else {
        // Linux/Mac: direct npx approach
        stdioParams = ServerParameters.builder("npx")
            .args("-y", "@modelcontextprotocol/server-filesystem", "target")
            .build();
    }
    McpSyncClient client = McpClient.sync(
        new StdioClientTransport(stdioParams, McpJsonMapper.createDefault()))
        .requestTimeout(Duration.ofSeconds(10))
```

```
        .build();
    client.initialize();
    return client;
}

private static boolean isWindows() {
    return System.getProperty("os.name").toLowerCase().contains("win");
}
```

**注意：** 使用 `@Bean` 进行程序化配置时，添加 `@ConditionalOnMissingBean(McpSyncClient.class)` 以避免与来自 JSON 文件的自动配置冲突。

### 路径相关注意事项

**相对路径**（推荐用于提高可移植性）：

```
{
  "command": "cmd.exe",
  "args": ["/c", "npx", "-y", "@modelcontextprotocol/server-filesystem", "target"]
}
```

MCP 服务器根据应用程序的工作目录解析相对路径。

**绝对路径**（Windows 中需要使用反斜杠或转义的正斜杠）：

```
{
  "command": "cmd.exe",
  "args": ["/c", "npx", "-y",
           "@modelcontextprotocol/server-filesystem",
           "C:\\Users\\username\\project\\target"]
}
```

### 常见批处理文件

Windows常见需要使用cmd.exe包装的批处理文件

- `npx.cmd`, `npm.cmd` — Node 包管理器
- `python.cmd` — Python（来自 Microsoft Store 安装）
- `pip.cmd` — Python 包管理器
- `mvn.cmd` — Maven 包装器（Maven Wrapper）
- `gradle.cmd` — Gradle 包装器（Gradle Wrapper）
- 自定义的 `.cmd` 或 `.bat` 脚本

### 实现参考

查看 Spring AI Examples - Filesystem 以获取完整的跨平台 MCP 客户端实现，该实现自动检测 OS 并适当配置客户端。

### Streamable-HTTP传输属性

用于连接到Streamable-HTTP和Stateless Streamable-HTTP MCP服务器。

Streamable-HTTP传输的属性以 `spring.ai.mcp.client.streamable-http` 为前缀：

| 属性 | 描述 | 默认值 |
| --- | --- | --- |
| `connections` | 命名的Streamable-HTTP连接配置映射 | - |
| `connections.[name].url` | 与MCP服务器进行Streamable-HTTP通信的基础URL端点 | - |
| `connections.[name].endpoint` | 连接使用的Streamable-HTTP端点（作为URL后缀） | `/mcp` |

配置示例

```yaml
spring:
  ai:
    mcp:
      client:
        streamable-http:
          connections:
            server1:
              url: http://localhost:8080
            server2:
              url: http://otherserver:8081
              endpoint: /custom-sse
```

### SSE传输属性

Server-Sent Events (SSE) 传输的属性以 `spring.ai.mcp.client.sse` 为前缀：

| 属性 | 描述 | 默认值 |
| --- | --- | --- |
| `connections` | 命名的SSE连接配置映射 | - |
| `connections.[name].url` | 与MCP服务器进行SSE通信的基础URL端点 | - |
| `connections.[name].sse-endpoint` | 连接使用的SSE端点（作为URL后缀） | `/sse` |

配置示例:

```yaml
spring:
  ai:
    mcp:
      client:
        sse:
          connections:
            # Simple configuration using default /sse endpoint
            server1:
              url: http://localhost:8080
            # Custom SSE endpoint
            server2:
```

```
            url: http://otherserver:8081
            sse-endpoint: /custom-sse
          # Complex URL with path and token (like MCP Hub)
          mcp-hub:
            url: http://localhost:3000
            sse-endpoint: /mcp-hub/sse/cf9ec4527e3c4a2cbb149a85ea45ab01
          # SSE endpoint with query parameters
          api-server:
            url: https://api.example.com
            sse-endpoint: /v1/mcp/events?token=abc123&format=json
```

**URL拆分指南**

当您有一个完整的 SSE URL 时，将其拆分为基础 URL 和端点路径：

| 完整的URL | 配置映射 |
| --- | --- |
| `http://localhost:3000/mcp-hub/sse/token123` | `url: http://localhost:3000` `sse-endpoint: /mcp-hub/sse/token123` |
| `https://api.service.com/v2/events?key=secret` | `url: https://api.service.com` `sse-endpoint: /v2/events?key=secret` |
| `http://localhost:8080/sse` | `url: http://localhost:8080` `sse-endpoint: /sse` (或省略使用默认值) |

**SSE 连接故障排查**

*404 Not Found Errors:*

- 验证 URL 拆分：确保基础 `url` 仅包含 scheme、host 和 port
- 检查 `sse-endpoint` 以 `/` 开头并包含完整路径和查询参数
- 在浏览器或 curl 中直接测试完整 URL 以确认其可访问

# 特性

## 同步/异步客户端类型

starter支持两种类型的客户端：

- Synchronous - 默认客户端类型（`spring.ai.mcp.client.type=SYNC`），适用于具有阻塞操作的传统请求-响应模式

  **注意：**SYNC 客户端将仅注册同步 MCP 带注解的方法。异步方法将被忽略。

- Asynchronous - 适用于具有非阻塞操作的响应式应用程序，使用 `spring.ai.mcp.client.type=ASYNC` 配置

  **注意：**ASYNC 客户端将仅注册异步 MCP 带注解的方法。同步方法将被忽略。

## 客户端自定义

自动配置通过回调接口提供广泛的客户端规范自定义功能。这些 customizers 允许您配置 MCP 客户端行为的各个方面，从请求超时到事件处理和消息处理。

**自定义类型**

以下自定义选项可用：

- *Request Configuration* - 设置自定义请求超时
- *Custom Sampling Handlers* - 服务器通过客户端从 LLM 请求 LLM sampling（`completions` 或 `generations`）的标准化方式。此流程允许客户端保持对模型访问、选择和权限的控制，同时使服务器能够利用 AI 能力——无需服务器 API 密钥。
- *File system (Roots) Access* - 客户端向服务器暴露文件系统 `roots` 的标准化方式。 Roots 定义了服务器可以在文件系统内操作的边界，允许它们了解可以访问哪些目录和文件。 服务器可以从支持客户端请求 roots 列表，并在该列表更改时接收通知。
- *Elicitation Handlers* - 服务器在交互期间通过客户端从用户请求额外信息的标准化方式。
- Event Handlers

  - 客户端处理器，在发生某些服务器事件时被通知：

    - Tools change notifications - 当可用服务器工具列表更改时
    - Resources change notifications - 当可用服务器资源列表更改时
    - Prompts change notifications - 当可用服务器 prompts 列表更改时
    - *Logging Handlers* - 服务器向客户端发送结构化日志消息的标准化方式。
    - *Progress Handlers* - 服务器向客户端发送结构化进度消息的标准化方式。

客户端可以通过设置最小日志级别来控制日志详细程度

**自定义示例**

您可以根据应用程序的需求实现 `McpSyncClientCustomizer`（用于同步客户端）或 `McpAsyncClientCustomizer`（用于异步客户端）。

**Sync:**

```
@Component
public class CustomMcpSyncClientCustomizer implements McpSyncClientCustomizer {
    @Override
    public void customize(String serverConfigurationName, McpClient.SyncSpec spec) {
        // 自定义请求超时配置
        spec.requestTimeout(Duration.ofSeconds(30));
        // 设置此客户端可以访问的根URIs
        spec.roots(roots);
        // 设置用于处理消息创建请求的自定义采样处理器
        spec.sampling((CreateMessageRequest messageRequest) -> {
            // 处理采样
            CreateMessageResult result = ...
            return result;
        });
        // 设置用于处理elicitation请求的自定义elicitation处理器
        spec.elicitation((ElicitRequest request) -> {
          // 处理elicitation
```

```
            return new ElicitResult(ElicitResult.Action.ACCEPT,
                                    Map.of("message", request.message()));
        });
        // 添加一个消费者，用于在收到进度通知时接收通知
        spec.progressConsumer((ProgressNotification progress) -> {
         // 处理进度通知
        });
        // 添加一个消费者，用于在可用工具发生变化时（例如工具被添加或移除）接收通知
        spec.toolsChangeConsumer((List<McpSchema.Tool> tools) -> {
            // 处理工具变化
        });
        // 添加一个消费者，用于在可用资源发生变化时（例如资源被添加或移除）接收通知
        spec.resourcesChangeConsumer((List<McpSchema.Resource> resources) -> {
            // 处理资源变化
        });
        // 添加一个消费者，用于在可用提示发生变化时（例如提示被添加或移除）接收通知
        spec.promptsChangeConsumer((List<McpSchema.Prompt> prompts) -> {
            // 处理提示变化
        });
        // 添加一个消费者，用于在收到来自服务器的日志消息时接收通知
        spec.loggingConsumer((McpSchema.LoggingMessageNotification log) -> {
            // 处理日志消息
        });
    }
}
```

**Async:**

```
@Component
public class CustomMcpAsyncClientCustomizer implements McpAsyncClientCustomizer {
    @Override
    public void customize(String serverConfigurationName, McpClient.AsyncSpec
spec) {
        // 自定义请求超时配置
        spec.requestTimeout(Duration.ofSeconds(30));
    }
}
```

`serverConfigurationName` 参数是应用 customizer 并为其创建 MCP 客户端的服务器配置名称。

MCP 客户端自动配置会自动检测并应用在应用程序上下文中找到的任何 customizers。

## 传输支持

自动配置支持多种传输类型：

- Standard I/O (Stdio)（由 `spring-ai-starter-mcp-client` 和 `spring-ai-starter-mcp-client-webflux` 激活）
- (HttpClient) HTTP/SSE 和 Streamable-HTTP（由 `spring-ai-starter-mcp-client` 激活）
- (WebFlux) HTTP/SSE 和 Streamable-HTTP（由 `spring-ai-starter-mcp-client-webflux` 激活）

## 工具过滤

MCP Client Boot Starter 通过 `McpToolFilter` 接口支持对发现的工具进行过滤。这允许您根据自定义条件（如 MCP 连接信息或工具属性）选择性地包含或排除工具。

要实现工具过滤，创建一个实现 `McpToolFilter` 接口的 bean：

```
@Component
public class CustomMcpToolFilter implements McpToolFilter {

    @Override
    public boolean test(McpConnectionInfo connectionInfo, McpSchema.Tool tool) {
        // 基于连接信息和工具属性的过滤逻辑
        // 返回true表示包含该工具，false表示排除该工具

        // 示例：排除来自特定客户端的工具
        if ("restricted-client".equals(connectionInfo.clientInfo().name())) {
            return false;
        }

        // 示例：只包含名称以 "allowed_" 开头的工具
        if (tool.name().startsWith("allowed_")) {
            return true;
        }

        // 示例：基于工具描述或其他属性进行过滤
        if (tool.description() != null &&
            tool.description().contains("experimental")) {
            return false;
        }
        // 默认包含所有其他工具
        return true;
    }
}
```

`McpConnectionInfo` record 提供对以下内容的访问：

- `clientCapabilities` - MCP 客户端的能力
- `clientInfo` - 关于 MCP 客户端的信息（名称和版本）
- `initializeResult` - 来自 MCP 服务器的初始化结果

过滤器会自动检测并应用于同步和异步 MCP 工具回调提供程序。 如果未提供自定义过滤器，默认情况下会包含所有发现的工具。

**注意：** 在应用程序上下文中应只定义一个 `McpToolFilter` bean。 如果需要多个过滤器，请将它们组合到单个复合过滤器实现中。

## 工具名称前缀生成

MCP Client Boot Starter 通过 `McpToolNamePrefixGenerator` 接口支持可自定义的工具名称前缀生成。此功能通过在工具名称中添加唯一前缀，帮助在集成来自多个 MCP 服务器的工具时避免命名冲突。

默认情况下，如果未提供自定义 `McpToolNamePrefixGenerator` bean，starter 使用 `DefaultMcpToolNamePrefixGenerator`，它确保所有 MCP 客户端连接中的唯一工具名称。默认生成器：

- 跟踪所有现有连接和工具名称以确保唯一性
- 通过将非字母数字字符替换为下划线来格式化工具名称（例如，`my-tool` 变为 `my_tool`）
- 当在不同连接中检测到重复的工具名称时，添加计数器前缀（例如，`alt_1_toolName`、`alt_2_toolName`）
- 是线程安全的并保持幂等性 - 相同的（客户端、服务器、工具）组合始终获得相同的唯一名称
- 确保最终名称不超过 64 个字符（必要时从开头截断）

例如：

- 工具 `search` 的第一次出现 → `search`
- 来自不同连接的工具 `search` 的第二次出现 → `alt_1_search`
- 带有特殊字符的工具 `my-special-tool` → `my_special_tool`

您可以通过提供自己的实现来自定义此行为：

```
@Component
public class CustomToolNamePrefixGenerator implements McpToolNamePrefixGenerator
{

    @Override
    public String prefixedToolName
        (McpConnectionInfo connectionInfo, McpSchema.Tool tool) {
        // 自定义生成带前缀工具名称的逻辑

        // 示例：使用服务器名称和版本作为前缀
        String serverName =
connectionInfo.initializeResult().serverInfo().name();
        String serverVersion =
connectionInfo.initializeResult().serverInfo().version();
        return serverName + "_v" + serverVersion.replace(".", "_") + "_" +
tool.name();
    }
}
```

`McpConnectionInfo` record 提供有关 MCP 连接的全面信息：

- `clientCapabilities` - MCP 客户端的能力
- `clientInfo` - 关于 MCP 客户端的信息（名称、标题和版本）
- `initializeResult` - 来自 MCP 服务器的初始化结果，包括服务器信息

**内置前缀生成器**

框架提供几个内置的前缀生成器：

- `DefaultMcpToolNamePrefixGenerator` - 通过跟踪重复项并在需要时添加计数器前缀来确保唯一工具名称（如果未提供自定义 bean，则默认使用）
- `McpToolNamePrefixGenerator.noPrefix()` - 返回没有任何前缀的工具名称（如果多个服务器提供同名工具，可能会导致冲突）

要完全禁用前缀并使用原始工具名称（如果使用多个 MCP 服务器，不推荐），将 no-prefix 生成器注册为 bean：

```java
@Configuration
public class McpConfiguration {
    @Bean
    public McpToolNamePrefixGenerator mcpToolNamePrefixGenerator() {
        return McpToolNamePrefixGenerator.noPrefix();
    }
}
```

前缀生成器通过 Spring 的 `ObjectProvider` 机制自动检测并应用于同步和异步 MCP 工具回调提供程序。 如果未提供自定义生成器 bean，则自动使用 `DefaultMcpToolNamePrefixGenerator`。

**警告：**当使用 `McpToolNamePrefixGenerator.noPrefix()` 与多个 MCP 服务器时，重复的工具名称将导致 `IllegalStateException`。默认的 `DefaultMcpToolNamePrefixGenerator` 通过自动向重复工具名称添加唯一前缀来防止这种情况。

## 上下文到MCP元数据转换器

MCP Client Boot Starter 通过 `ToolContextToMcpMetaConverter` 接口支持将 Spring AI 的 ToolContext 转换为 MCP 工具调用元数据的可自定义转换。 此功能允许您将额外的上下文信息（例如用户 ID、密钥令牌）作为元数据与 LLM 生成的调用参数一起传递。

例如，您可以将 MCP `progressToken` 传递到工具上下文中的 MCP Progress Flow 以跟踪长时间运行操作的进度：

```java
ChatModel chatModel = ...

String response = ChatClient.create(chatModel)
        .prompt("Tell me more about the customer with ID 42")
        .toolContext(Map.of("progressToken", "my-progress-token"))
        .call()
        .content();
```

默认情况下，如果未提供自定义转换器 bean，starter 使用 `ToolContextToMcpMetaConverter.defaultConverter()`，它：

- 过滤掉 MCP 交换键（`McpToolUtils.TOOL_CONTEXT_MCP_EXCHANGE_KEY`）
- 过滤掉具有 null 值的条目
- 将所有其他上下文条目作为元数据传递

您可以通过提供自己的实现来自定义此行为：

```java
@Component
public class CustomToolContextToMcpMetaConverter
    implements ToolContextToMcpMetaConverter {

    @Override
    public Map<String, Object> convert(ToolContext toolContext) {
        if (toolContext == null || toolContext.getContext() == null) {
            return Map.of();
        }

        // 自定义将工具上下文转换为 MCP 元数据的逻辑
        Map<String, Object> metadata = new HashMap<>();
```

```
13
14          // 示例：为所有键添加自定义前缀
15          for (Map.Entry<String, Object> entry :
    toolContext.getContext().entrySet()) {
16              if (entry.getValue() != null) {
17                  metadata.put("app_" + entry.getKey(), entry.getValue());
18              }
19          }
20
21          // 示例：添加额外的元数据
22          metadata.put("timestamp", System.currentTimeMillis());
23          metadata.put("source", "spring-ai");
24
25          return metadata;
26      }
27  }
```

**内置转换器**

框架提供内置转换器：

- `ToolContextToMcpMetaConverter.defaultConverter()` - 过滤掉 MCP 交换键和 null 值（如果未提供自定义 bean，则默认使用）
- `ToolContextToMcpMetaConverter.noOp()` - 返回空映射，有效地禁用上下文到元数据的转换

要完全禁用上下文到元数据的转换：

```
1  @Configuration
2  public class McpConfiguration {
3
4      @Bean
5      public ToolContextToMcpMetaConverter toolContextToMcpMetaConverter() {
6          return ToolContextToMcpMetaConverter.noOp();
7      }
8  }
```

转换器通过 Spring 的 `ObjectProvider` 机制自动检测并应用于同步和异步 MCP 工具回调。 如果未提供自定义转换器 bean，则自动使用默认转换器。

### 禁用MCP工具回调自动配置

MCP ToolCallback 自动配置默认启用，但可以使用 `spring.ai.mcp.client.toolcallback.enabled=false` 属性禁用。

禁用后，不会从可用的MCP工具创建 `ToolCallbackProvider` bean。

## MCP Client注解

MCP Client Boot Starter 自动检测并注册带注解的方法以处理各种 MCP 客户端操作：

- *@McpLogging* - 处理来自 MCP 服务器的日志消息通知
- *@McpSampling* - 处理来自 MCP 服务器的 LLM completions 的 sampling 请求
- *@McpElicitation* - 处理用于从用户收集额外信息的 elicitation 请求
- *@McpProgress* - 处理长时间运行操作的进度通知

- *@McpToolListChanged* - 处理服务器工具列表更改时的通知
- *@McpResourceListChanged* - 处理服务器资源列表更改时的通知
- *@McpPromptListChanged* - 处理服务器 prompt 列表更改时的通知

使用示例

```
@Component
public class McpClientHandlers {

    @McpLogging(clients = "server1")
    public void handleLoggingMessage(LoggingMessageNotification notification) {
        System.out.println("Received log: " + notification.level() +
                           " - " + notification.data());
    }

    @McpSampling(clients = "server1")
    public CreateMessageResult handleSamplingRequest(CreateMessageRequest
request) {
        // Process the request and generate a response
        String response = generateLLMResponse(request);

        return CreateMessageResult.builder()
            .role(Role.ASSISTANT)
            .content(new TextContent(response))
            .model("gpt-4")
            .build();
    }

    @McpProgress(clients = "server1")
    public void handleProgressNotification(ProgressNotification notification) {
        double percentage = notification.progress() * 100;
        System.out.println(String.format("Progress: %.2f%% - %s",
            percentage, notification.message()));
    }

    @McpToolListChanged(clients = "server1")
    public void handleToolListChanged(List<McpSchema.Tool> updatedTools) {
        System.out.println("Tool list updated: " +
                           updatedTools.size() + " tools available");
        // Update local tool registry
        toolRegistry.updateTools(updatedTools);
    }
}
```

注解支持同步和异步实现，可以使用 `clients` 参数为特定客户端配置：

```
@McpLogging(clients = "server1")
public void handleServer1Logs(LoggingMessageNotification notification) {
    // Handle logs from specific server
    logToFile("server1.log", notification);
}

@McpSampling(clients = "server1")
public Mono<CreateMessageResult> handleAsyncSampling(CreateMessageRequest
request) {
```

```
    return Mono.fromCallable(() -> {
        String response = generateLLMResponse(request);
        return CreateMessageResult.builder()
            .role(Role.ASSISTANT)
            .content(new TextContent(response))
            .model("gpt-4")
            .build();
    }).subscribeOn(Schedulers.boundedElastic());
}
```

有关所有可用注解及其使用模式的详细信息，请参阅 MCP Client Annotations 文档。

## 使用示例

将适当的 starter 依赖项添加到项目中，并在 `application.properties` 或 `application.yml` 中配置客户端：

```yaml
spring:
  ai:
    mcp:
      client:
        enabled: true
        name: my-mcp-client
        version: 1.0.0
        request-timeout: 30s
        type: SYNC  # or ASYNC for reactive applications
        sse:
          connections:
            server1:
              url: http://localhost:8080
            server2:
              url: http://otherserver:8081
        streamable-http:
          connections:
            server3:
              url: http://localhost:8083
              endpoint: /mcp
        stdio:
          root-change-notification: false
          connections:
            server1:
              command: /path/to/server
              args:
                - --port=8080
                - --mode=production
              env:
                API_KEY: your-api-key
                DEBUG: "true"
```

MCP 客户端 beans 将自动配置并可用于注入：

```java
@Autowired
private List<McpSyncClient> mcpSyncClients;  // For sync client
// OR
@Autowired
private List<McpAsyncClient> mcpAsyncClients;  // For async client
```

当启用工具回调时（默认行为），所有 MCP 客户端注册的 MCP 工具作为 `ToolCallbackProvider` 实例提供：

```java
@Autowired
private SyncMcpToolCallbackProvider toolCallbackProvider;
ToolCallback[] toolCallbacks = toolCallbackProvider.getToolCallbacks();
```

## 示例应用

- Brave Web Search Chatbot - 一个使用 Model Context Protocol 与 Web 搜索服务器交互的聊天机器人。
- Default MCP Client Starter - 使用默认 `spring-ai-starter-mcp-client` MCP Client Boot Starter 的简单示例。
- WebFlux MCP Client Starter - 使用 `spring-ai-starter-mcp-client-webflux` MCP Client Boot Starter 的简单示例。

# MCP Server Boot Starter

Model Context Protocol (MCP) Servers 是通过标准化协议接口向 AI 应用程序暴露特定功能的程序。每个服务器为特定领域提供聚焦的功能。

Spring AI MCP Server Boot Starters 为在 Spring Boot 应用程序中设置 MCP Servers 提供自动配置。它们使 MCP 服务器功能与 Spring Boot 的自动配置系统无缝集成。

MCP Server Boot Starters 提供：

- MCP 服务器组件的自动配置，包括工具、资源和 prompts
- 支持不同的 MCP 协议版本，包括 STDIO、SSE、Streamable-HTTP 和无状态服务器
- 支持同步和异步操作模式
- 多种传输层选项
- 灵活的工具、资源和 prompt 规范
- 变更通知功能
- 基于注解的服务器开发，具有自动 bean 扫描和注册

## Starters

MCP 服务器支持多种协议和传输机制。 使用专用的 starter 和正确的 `spring.ai.mcp.server.protocol` 属性来配置您的服务器：

### STDIO

| Server Type | Dependency | Property |
| --- | --- | --- |

| Server Type | Dependency | Property |
| --- | --- | --- |
| Standard Input/Output (STDIO) | `spring-ai-starter-mcp-server` | `spring.ai.mcp.server.stdio=true` |

### WebMVC

| Server Type | Dependency | Property |
| --- | --- | --- |
| SSE WebMVC | `spring-ai-starter-mcp-server-webmvc` | `spring.ai.mcp.server.protocol=SS` empty |
| Streamable-HTTP WebMVC | `spring-ai-starter-mcp-server-webmvc` | `spring.ai.mcp.server.protocol=ST` |
| Stateless WebMVC | `spring-ai-starter-mcp-server-webmvc` | `spring.ai.mcp.server.protocol=ST` |

### WebMVC (Reactive)

| Server Type | Dependency | Property |
| --- | --- | --- |
| SSE WebFlux | `spring-ai-starter-mcp-server-webflux` | `spring.ai.mcp.server.protocol=S` empty |
| Streamable-HTTP WebFlux | `spring-ai-starter-mcp-server-webflux` | `spring.ai.mcp.server.protocol=S` |
| Stateless WebFlux | `spring-ai-starter-mcp-server-webflux` | `spring.ai.mcp.server.protocol=S` |

## 服务器能力

根据服务器和传输类型，MCP 服务器可以支持各种功能，例如：

- **Tools** - 允许服务器暴露可由语言模型调用的工具
- **Resources** - 为服务器向客户端暴露资源提供标准化方式
- **Prompts** - 为服务器向客户端暴露 prompt 模板提供标准化方式
- **Utility/Completions** - 为服务器为 prompts 和资源 URI 提供参数自动完成建议提供标准化方式
- **Utility/Logging** - 为服务器向客户端发送结构化日志消息提供标准化方式
- **Utility/Progress** - 通过通知消息对长时间运行的操作进行可选的进度跟踪
- **Utility/Ping** - 服务器报告其状态的可选健康检查机制

默认情况下，所有功能都已启用。禁用功能将阻止服务器向客户端注册和暴露相应的功能。

# 服务协议

MCP 提供多种协议类型，包括：

- **STDIO** - 进程内（例如，服务器在主机应用程序内运行）协议。通信通过标准输入和标准输出进行。要启用 `STDIO`，请设置 `spring.ai.mcp.server.stdio=true`。
- **SSE** - 用于实时更新的 Server-sent events 协议。服务器作为独立进程运行，可以处理多个客户端连接。
- **Streamable-HTTP** - Streamable HTTP transport 允许 MCP 服务器作为独立进程运行，可以使用 HTTP POST 和 GET 请求处理多个客户端连接，并可选地使用 Server-Sent Events (SSE) 流式传输多个服务器消息。它取代了 SSE 传输。要启用 `STREAMABLE` 协议，请设置 `spring.ai.mcp.server.protocol=STREAMABLE`。
- **Stateless** - 无状态 MCP 服务器专为简化部署而设计，在请求之间不维护会话状态。 它们非常适合微服务架构和云原生部署。要启用 `STATELESS` 协议，请设置 `spring.ai.mcp.server.protocol=STATELESS`。

# 同步/异步服务API选项

MCP Server API 支持命令式（即同步）和响应式（例如异步）编程模型。

- **Synchronous Server** - 使用 `McpSyncServer` 实现的默认服务器类型。 它专为应用程序中的简单请求-响应模式而设计。 要启用此服务器类型，请在配置中设置 `spring.ai.mcp.server.type=SYNC`。 激活后，它会自动处理同步工具规范的配置。

  **注意：** SYNC 服务器将仅注册同步 MCP 带注解的方法。异步方法将被忽略。

- **Asynchronous Server** - 异步服务器实现使用 `McpAsyncServer`，并针对非阻塞操作进行了优化。 要启用此服务器类型，请使用 `spring.ai.mcp.server.type=ASYNC` 配置应用程序。 此服务器类型自动设置异步工具规范，并内置 Project Reactor 支持。

  **注意：** ASYNC 服务器将仅注册异步 MCP 带注解的方法。同步方法将被忽略。

更多配置参考：

- mcp-stateless-server-boot-starter-docs
- mcp-stdio-sse-server-boot-starter-docs

# MCP Server注解

MCP Server Boot Starters 为基于注解的服务器开发提供全面支持，允许您使用声明式 Java 注解而不是手动配置来创建 MCP 服务器。

## Key Annotations

- **@McpTool** - 将方法标记为 MCP 工具，自动生成 JSON schema
- **@McpResource** - 通过 URI 模板提供资源访问
- **@McpPrompt** - 为 AI 交互生成 prompt 消息
- **@McpComplete** - 为 prompts 提供自动完成功能

## Special Parameters

注解系统支持特殊参数类型，提供额外的上下文：

- `McpMeta` - 访问来自 MCP 请求的元数据
- `@McpProgressToken` - 接收长时间运行操作的进度令牌

- `McpSyncServerExchange/McpAsyncServerExchange` - 用于高级操作的完整服务器上下文
- `McpTransportContext` - 用于无状态操作的轻量级上下文
- `CallToolRequest` - 用于灵活工具的动态 schema 支持

### 简单示例

```
@Component
public class CalculatorTools {

    @McpTool(name = "add", description = "Add two numbers together")
    public int add(
            @McpToolParam(description = "First number", required = true) int a,
            @McpToolParam(description = "Second number", required = true) int b)
{
        return a + b;
    }

    @McpResource(uri = "config://{key}", name = "Configuration")
    public String getConfig(String key) {
        return configData.get(key);
    }
}
```

## Auto-Configuration

通过 Spring Boot 自动配置，带注解的 beans 会自动检测并注册：

```
@SpringBootApplication
public class McpServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(McpServerApplication.class, args);
    }
}
```

自动配置将：

1. 扫描带有 MCP 注解的 beans
2. 创建适当的规范
3. 将它们注册到 MCP 服务器
4. 根据配置处理同步和异步实现

## 配置属性

配置服务器注解扫描器：

```
spring:
  ai:
    mcp:
      server:
        type: SYNC  # or ASYNC
        annotation-scanner:
          enabled: true
```

## 示例应用

- Weather Server (SSE WebFlux) - 带有 WebFlux 传输的 Spring AI MCP Server Boot Starter
- Weather Server (STDIO) - 带有 STDIO 传输的 Spring AI MCP Server Boot Starter
- Weather Server Manual Configuration - 不使用自动配置但使用 Java SDK 手动配置服务器的 Spring AI MCP Server Boot Starter

# Nacos

## 介绍

Spring AI Alibaba MCP 结合 Nacos 服务注册中心，为企业级智能体应用提供了强大的基础架构支持。这一组合解决方案主要围绕三条核心技术线展开，实现了从服务注册、工具代理到服务发现的完整闭环，为企业级 AI 应用部署提供了坚实基础。

### 一、MCP服务注册到Nacos

Spring AI Alibaba MCP Nacos Registry 服务注册是整个系统的基础环节，通过将 MCP Server 信息与 Tools 注册到 Nacos，实现了服务能力的中央化管理。

#### 核心组件

- NacosMcpRegister
- 实现了`ApplicationListener<WebServerInitializedEvent>`接口，在Web服务器初始化后自动触发注册
- 将MCP服务器信息（名称、版本、协议类型）注册到 Nacos
- 注册工具定义列表，包含每个工具的名称、描述、输入模式和元数据
- NacosMcpRegistryProperties
- 控制注册行为的配置参数
- 配置项包括服务组名、服务名称、SSE 导出路径等
- 支持临时实例和持久实例的选择
- NacosMcpOperationService
- 提供与 Nacos 通信的核心能力
- 实现 MCP 服务的创建、查询和订阅
- 处理服务端点和工具规范的注册与更新

#### 注册流程

- MCP服务器启动后，收集本地工具规范和服务器信息
- 通过NacosMcpOperationService检查Nacos中是否已存在相同服务
- 如存在，验证兼容性并更新工具定义；如不存在，创建新服务
- 注册服务实例（IP、端口、协议类型等）到 Nacos
- 订阅自身服务变更，实现工具元数据的动态更新

#### 技术优势

- 版本管理：支持服务版本控制，确保兼容性
- 协议适配：支持SSE和STDIO两种协议模式
- 元数据同步：工具描述和参数说明可动态更新
- 兼容性检查：确保工具接口变更不破坏现有调用

## 二、Gateway代理MCP服务

Spring AI Alibaba MCP Gateway是将注册到Nacos中的MCP Server信息动态转换为MCP工具，实现了服务能力到 AI 工具的转化。

### 核心组件

- NacosMcpGatewayToolsWatcher
- 监控 Nacos 中 MCP 服务信息变化
- 检测工具新增、更新和删除事件
- 自动更新本地工具注册表
- NacosMcpGatewayToolDefinition
- 定义 Gateway 工具的结构规范
- 支持工具名称、描述、输入模式的定义
- 包含远程服务配置和协议信息
- DynamicNacosToolCallback
- 处理工具调用的核心组件
- 将工具参数转换为 HTTP 请求
- 处理响应并返回结果
- RequestTemplateParser/ResponseTemplateParser
- 处理 HTTP 请求模板解析和参数映射
- 支持 URL 参数、表单数据、JSON 请求体
- 提供响应转换和格式化能力

### 工作流程

- NacosMcpGatewayToolsWatcher 通过 NacosMcpOperationService 订阅服务变化
- 检测到新服务或工具更新时，创建 NacosMcpGatewayToolDefinition
- 通过 NacosMcpGatewayToolsProvider 注册工具到 MCP 服务器
- 当工具被调用时，DynamicNacosToolCallback 处理请求参数
- 基于模板构建 HTTP 请求并发送到目标服务
- 接收服务响应并通过模板转换为工具返回结果

### 技术优势

- 动态适配：无需修改原始 HTTP 服务
- 模板系统：灵活的请求和响应处理
- 多协议支持：支持 HTTP/HTTPS 协议
- 自动同步：与 Nacos 服务注册表实时同步
- 智能路由：基于服务健康状态选择最佳实例

## 三、实现MCP集群发现

Spring AI Alibaba MCP Client结合Nacos实现了MCP服务的集群发现和负载均衡能力。

### 核心组件

- LoadbalancedMcpAsyncClient/LoadbalancedMcpSyncClient
- 提供同步和异步两种调用模式
- 通过 Nacos 发现 MCP 服务实例
- 维护实例连接池并实现负载均衡
- LoadbalancedMcpToolCallback
- 封装工具调用逻辑
- 透明处理负载均衡和故障转移

- 标准化工具调用接口
- LoadbalancedMcpToolCallbackProvider
- 管理工具回调集合
- 支持工具过滤和筛选
- 验证工具定义一致性

**工作流程**

- 初始化时，从 Nacos 获取 MCP 服务端点信息
- 创建并维护与每个端点的连接
- 监控端点变化，动态更新连接池
- 工具调用时，通过轮询算法选择实例
- 处理调用结果，支持异常重试和实例切换

**技术优势**

- 负载均衡：内置轮询策略分散请求
- 高可用：实例故障自动切换
- 动态发现：实时感知服务变化
- 连接池管理：优化连接资源利用
- 统一接口：兼容 Spring AI 标准工具规范

**技术亮点与应用价值**

- 架构优势
- 松耦合设计：三个模块可独立部署和使用
- 可扩展性：支持水平扩展和集群部署
- 统一管理：通过 Nacos 集中管理所有组件
- 容错能力：多层次的故障处理机制
- 兼容性：与现有 HTTP 服务无缝集成

**企业应用价值**

- 快速赋能：迅速将企业现有服务能力赋予 AI 模型
- 降低成本：无需重构现有服务，直接复用
- 中心化治理：统一管理 AI 工具和能力
- 弹性伸缩：根据负载动态调整资源
- 平滑演进：支持服务的迭代升级而不影响调用方

### 总结

Spring AI Alibaba MCP 结合 Nacos 的解决方案通过三个核心模块的协同工作，构建了完整的企业级智能体应用基础架构。通过 Registry 实现服务注册，Gateway 实现工具代理，Client 实现服务发现，形成了闭环系统，使企业能够轻松将现有服务能力转化为 AI 模型可调用的工具。

这一解决方案不仅简化了 AI 应用的开发部署流程，还提供了企业级的高可用性和可扩展性，为企业构建复杂的智能体应用提供了坚实的技术基础，是实现企业服务智能化转型的理想选择。

## Nacos配置MCP

你需要自己搭建Nacos服务，并且要求Nacos的版本是3.1.0+。

配置好Nacos服务后，你可以为MCP单独建立一个命名空间。

后续都以public-mcp为基准，记住对应的命名空间ID，如（c4068cdd-9ffe-4011-8cd3-daad57c433f0）。

有关mcp相关的配置信息可在配置管理/配置列表中查看

# MCP的注册

## 背景

如果你的场景需要新构建MCP Server，那么你可以注册到Nacos MCP Registry，不仅可以统一进行发现MCP服务，还可以帮助你构建的MCP Server具备动态调整治理的能力；

针对新建的 MCP 服务，Nacos提供多语言支持与自动化注册能力，过程中无代码侵入，或者是更换注解，无需写代码集成Nacos就可以自动注册，并且还有主要的特点：

- **跨语言生态适配：** 支持 Java（Spring AI）、Python、TypeScript（进行中）等主流框架，通过标准 SDK 或配置声明快速接入。
- **管理配置自动生效：** 服务注册后，可以通过 Nacos 产品化管理 MCP 元数据，如更新 MCP 描述、Tools 工具列表，更新信息后对应 MCP Server 会自动生效更改。
- **统一管理MCP发现：** 可以通过放在 Nacos 统一管理，可以通过网关或者 Nacos-Mcp-Router进行统一的配置发现；

能动态管理 MCP 信息、工具描述和列表等，而无需进行繁琐的系统重启或运维。这样不仅降低了系统维护的复杂性，还大大提高了调试的效率和便捷性。

注册一个stream-http的示例代码：

## pom依赖

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.ai</groupId>
        <artifactId>spring-ai-starter-mcp-server-webflux</artifactId>
    </dependency>
    <dependency>
        <groupId>com.alibaba.cloud.ai</groupId>
        <artifactId>spring-ai-alibaba-starter-mcp-registry</artifactId>
    </dependency>
</dependencies>
```

## application.yml

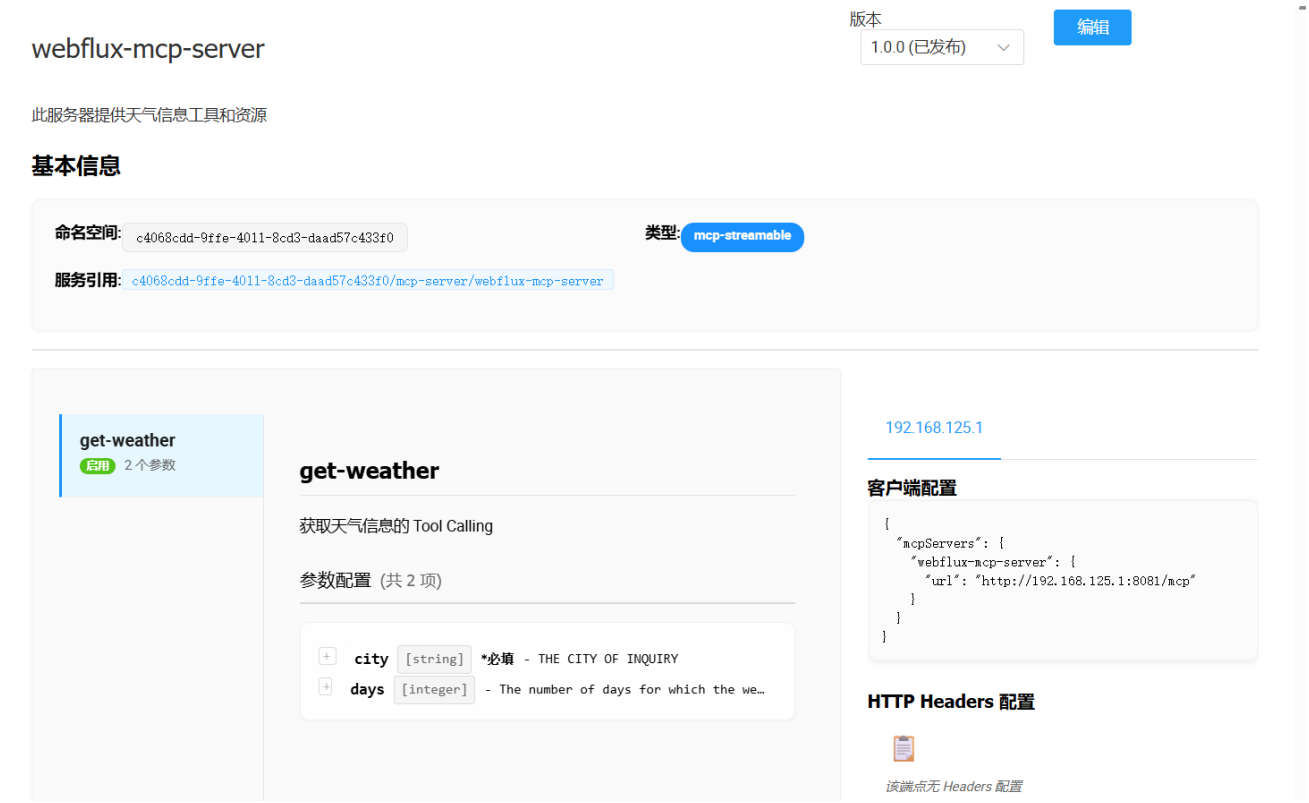
下面列举Nacos相关的配置，其他配置省略

```yaml
spring:
  ai:
    alibaba:
      mcp:
        nacos:
          namespace: c4068cdd-9ffe-4011-8cd3-daad57c433f0
          server-addr: 127.0.0.1:8848
          username: nacos
          password: nacos
          register:
            enabled: true
            # 指定分组名称
            service-group: mcp-server
            # 指定服务名称
            service-name: webflux-mcp-server
```

## 验证

配置好启动你的MCP服务器应用程序，启动好后可以在Nacos控制台MCP列表中看到如下所示的信息

点击详情后，可查看当前 MCP 服务提供的工具信息




在服务管理/服务列表处的 public-mcp 命名空间下，能看到当前节点实例信息

在配置管理/配置列表处的 public-mcp 命名空间下，能看到当前 mcp 服务对应的配置详情信息

点击每个配置文件的详情可以看具体配置内容

# MCP的分布式

## 背景

Spring AI 通过集成 MCP 官方的 java sdk，让 Spring Boot 开发者可以非常方便的开发自己的 MCP 服务，把自己企业内部的业务系统通过标准 MCP 形式发布为 AI Agent 能够接入的工具；另一方面，开发者也可以使用 Spring AI 开发自己的 AI Agent，去接入提供各种能力的 MCP 服务。

在企业级 AI Agent 的应用与落地场景，只是能发布或者调通 MCP 服务是远远不够的，其中一个非常重要的原因就是企业级的系统部署往往都是分布式的，不论是 Agent 还是 MCP Server，作为企业内部的一个个应用，它们都是要部署在多个机器上，要支持分布式的调用（这包括流量的负载均衡、节点变更动态感知等）。

描述：现阶段 MCP Client 和 MCP Server 是一对一的连接方式，若当前 MCP Server 挂掉了，那么 MCP Client 便不能使用 MCP Server 提供的工具能力。工具稳定性的提供得不到保证

解决：做了一些分布式 Client 连接的探索，一个 MCP Client 端可以连接多个 MCP Server（分布式部署），目前采用的方案如下：

1. 新建一个包含服务名和对应连接的类
2. 另外实现监听机制，可以动态的应对 MCP Server 节点上下线，去动态调整 MCP Server 列表

3. （读操作）获取 MCP Server 相关信息的，采用从 MCP Server 列表中随机中获取一个去发起请求，比如获取工具列表信息
4. （写操作）对应 MCP Server 需要更改的信息，由 MCP Client 端发起，需要修改所有的 MCP Server

分布式调用注册至nacos的mcp服务，示例代码：

## pom依赖

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.ai</groupId>
        <artifactId>spring-ai-starter-mcp-client-webflux</artifactId>
    </dependency>
    <dependency>
        <groupId>com.alibaba.cloud.ai</groupId>
        <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
    </dependency>
    <dependency>
        <groupId>com.alibaba.cloud.ai</groupId>
        <artifactId>spring-ai-alibaba-starter-mcp-distributed</artifactId>
    </dependency>
</dependencies>
```

## application.yml

下面列举分布式相关的配置，其他配置省略

```yaml
spring:
  ai:
    alibaba:
      mcp:
        nacos:
          client:
            enabled: true
            streamable:
              connections:
                server1:
                  service-name: webflux-mcp-server
                  version: 1.0.0
            configs:
              server1:
                namespace: 0908ca08-c382-404c-9d96-37fe1628b183
                server-addr: 127.0.0.1:8848
                username: nacos
                password: nacos
```

1. 支持发现 sse、streamable 类型 MCP 服务
2. 支持发现不同命名空间下的 MCP 服务

## 启动

```java
@SpringBootApplication
public class DistributedApplication {
    public static void main(String[] args) {
        SpringApplication.run(DistributedApplication.class, args);
    }
    @Bean
    public CommandLineRunner predefinedQuestions(
        ChatClient.Builder chatClientBuilder,
        // 使用ToolCallbackProvider需要指定使用哪一个
        @Qualifier("distributedAsyncToolCallback") ToolCallbackProvider tools,
        ConfigurableApplicationContext context) {
        ToolCallback[] toolCallbacks = tools.getToolCallbacks();
        System.out.println(">>> Available tools: ");
        for (int i = 0; i < toolCallbacks.length; i++) {
            System.out.println("[" + i + "] " +
                               toolCallbacks[i]
                               .getToolDefinition().name());
        }
        return args -> {
            var chatClient = chatClientBuilder
                    .defaultToolCallbacks(toolCallbacks)
                    .build();
            Scanner scanner = new Scanner(System.in);
            while (true) {
                System.out.print("\n>>> QUESTION: ");
                String userInput = scanner.nextLine();
                if (userInput.equalsIgnoreCase("exit")) {
                    break;
                }
                if (userInput.isEmpty()) {
                    userInput = "北京时间现在几点钟";
                }
                System.out.println("\n>>> ASSISTANT: " +
                                   chatClient.prompt(userInput)
                                   .call().content());
            }
            scanner.close();
            context.close();
        };
    }
}
```

- ASYNC：导入 Bean 名称为 distributedAsyncToolCallback
- SYNC：导入 Bean 名称为 distributedSyncToolCallback

## 验证

为 web-mcp-server服务注册两个实例（演示是以本地启动，如：分别暴露 8081、8082端口号）

MCP Client 侧触发两次工具，将负载均衡的去调用 MCP 服务，你可以通过观察两个服务端控制台的日志情况来验证

# MCP的网关

## 背景

Spring AI Alibaba MCP Gateway 基于 Nacos 提供的 MCP server registry 实现，为普通应用建立一个中间代理层 Java MCP 应用。一方面将 Nacos 中注册的服务信息转换成 MCP 协议的服务器信息，以便 MCP 客户端可以无缝调用这些服务；另一方面可以实现协议转化，将 MCP 协议转换为对后端 HTTP、Dubbo 等服务的调用。基于 Spring AI Alibaba MCP Gateway，您无需对原有业务代码进行改造，新增或者删除 MCP 服务（在 Nacos 中）无需重启代理应用。

示例代码可见：

## pom依赖

```xml
<dependencies>
    <dependency>
        <groupId>com.alibaba.cloud.ai</groupId>
        <artifactId>spring-ai-alibaba-starter-mcp-gateway</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.ai</groupId>
        <artifactId>spring-ai-starter-mcp-server-webmvc</artifactId>
    </dependency>
</dependencies>
```

## application.yml

下面列举网关相关的配置，其他配置省略

```yaml
spring:
  ai:
    mcp:
      server:
        name: mcp-nacos-gateway-example
        version: 1.0.0
        enabled: true
        protocol: sse
        type: sync
        instructions: "Nacos MCP Gateway - 聚合多个 MCP Server 的工具网关"
        sse-message-endpoint: /mcp/messages
    alibaba:
      mcp:
```

```yaml
        # Nacos注册中心配置
        nacos:
          namespace: c4068cdd-9ffe-4011-8cd3-daad57c433f0
          server-addr: 192.168.28.132:8848
          username: nacos
          password: nacos
        # MCP Gateway配置
        gateway:
          enabled: true
          registry: nacos
          # 配置要发现和聚合的MCP Server服务名称
          nacos:
            service-names:
              - web-mcp-server
```

## 测试控制器

在网关服务器中写一个测试控制器，来访问mcp服务

```java
/**
 * <p>
 * 描述：MCP Gateway 测试控制器
 * 提供以下功能：
 * 1. 列出从 Nacos 发现的所有 MCP 工具
 * 2. 通过 ChatClient 调用聚合的 MCP 工具
 * </p>
 * <p>版权：&copy;01星球</p>
 * <p>地址：01星球总部</p>
 * @author 阿伟学长
 * @version 1.0.0
 */
@RestController
@RequestMapping("/api/gateway")
public class GatewayController {
    private final ChatModel chatModel;
    private final ToolCallbackProvider toolCallbackProvider;
    private final static Logger log =
LoggerFactory.getLogger(GatewayController.class);
    public GatewayController(ChatModel chatModel,
                             ToolCallbackProvider toolCallbackProvider) {
        this.chatModel = chatModel;
        this.toolCallbackProvider = toolCallbackProvider;
    }
    /**
     * 列出所有从Nacos发现并聚合的MCP工具
     */
    @GetMapping("/tools")
    public List<Map<String, String>> listTools() {
        ToolCallback[] toolCallbacks = toolCallbackProvider.getToolCallbacks();
        return Arrays.stream(toolCallbacks)
                .map(tool -> Map.of(
                        "name", tool.getToolDefinition().name(),
                        "description", tool.getToolDefinition().description()
                ))
                .collect(Collectors.toList());
    }
```

```java
    /**
     * 使用 AI 模型调用 MCP 工具
     * <p>
     * 示例请求：
     * - /api/gateway/chat?message=现在几点了
     * - /api/gateway/chat?message=杭州的天气怎么样
     * </p>
     */
    @GetMapping("/chat")
    public String chat(@RequestParam String message) {
        log.info("收到用户消息：{}", message);
        ChatClient chatClient = ChatClient.builder(chatModel)
                .defaultToolCallbacks(toolCallbackProvider.getToolCallbacks())
                .build();
        String response = chatClient.prompt()
                .user(message)
                .call()
                .content();
        log.info("AI 响应：{}", response);
        return response;
    }
}
```

启动你的网关，再把前面的web-mcp-server服务也启动起来，访问网关中的接口进行验证。

# Agent Chat UI

参看官方说明。

# Graph Core

这个可以选学，重点要会使用AgentFramework，内容参考官方链接。